
STM32CubeIDE 用户手册

简介

STM32CubeIDE是一款一体化的多操作系统开发工具，是STM32Cube软件生态系统的一部分。它包含一个先进的C/ C++开发平台，支持基于stm32的产品的软件开发

本文档详细介绍了STM32CubeIDE的功能和用法，包括如何开始、创建和构建项目、使用标准和高级技术进行调试，以及许多其他软件分析解决方案。STM32CubeIDE基于Eclipse C/ c++开发工具™(CDT™)和GCC工具链，在本用户手册中无法完全描述。有关Eclipse®的更多信息可从STM32CubeIDE嵌入式帮助系统获得。该产品中包含了涵盖工具链和GDB服务器细节的特殊文档。

1 入门指南

STM32CubeIDE支持基于Arm®Cortex®处理器的STM32产品。详细信息请参考第11节参考文献中列出的意法半导体文档。

注意:

Arm是Arm有限公司(或其子公司)在美国和/或其他地方的注册商标。



1.1 产品信息

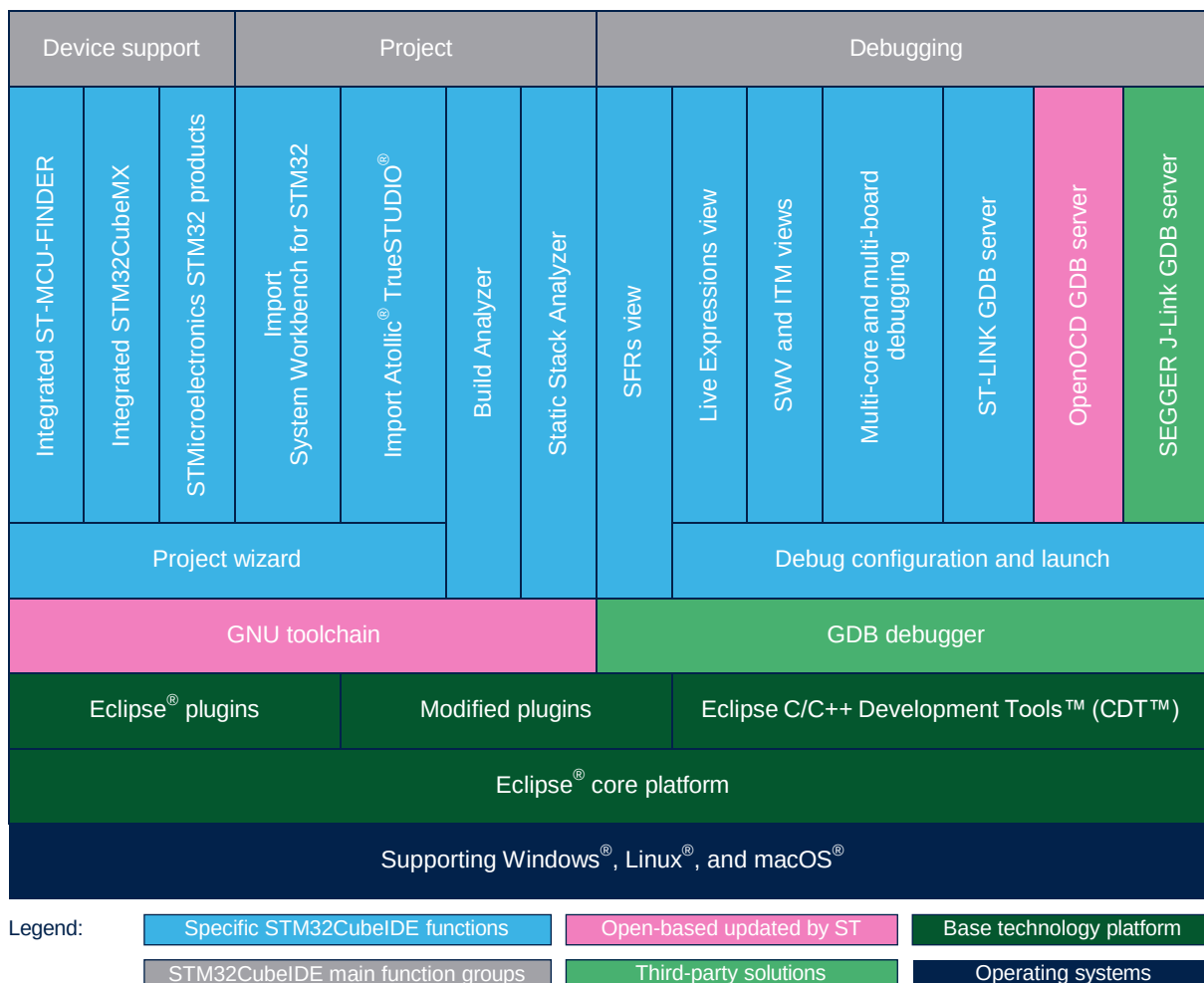
STM32CubeIDE是一个先进的C/ c++开发平台，具有外围配置、代码生成、代码编译、链接和调试功能。它基于Eclipse®/CDT™框架和GCC工具链进行开发，基于GDB进行调试。它允许集成数百个现有插件，以完成Eclipse®IDE的功能。

STM32CubeIDE集成了ST MCUFinder (ST- mcu - finder - pc)和STM32CubeMX功能提供一体化工具体验。它可以很容易地创建新的STM32 MCU或板项目，并使用包含的GCC工具链构建它们。

STM32CubeIDE包括一个构建分析器和一个静态堆栈分析器，它们为用户提供有关项目状态和内存需求的有用信息。

STM32CubeIDE还包括标准和高级调试功能，包括CPU核心寄存器，内存和外围寄存器的视图，以及实时变量监视和串行线查看器接口。如果在调试会话期间由STM32处理器触发错误，则故障分析器将显示错误信息。

Figure 1. STM32CubeIDE key features



1.1.1 操作系统要求

STM32CubeIDE在Microsoft®Windows®、Linux®和macOS®操作系统上进行了测试和验证

重点:

STM32CubeIDE仅支持64位操作系统版本。有关操作系统支持版本的详细信息，请参见[ST-02]。

注意:

macOS®是苹果公司在美国和其他国家注册的商标。

Linux®是Linus Torvalds的注册商标。

1.1.2 下载最新 STM32CubeIDE 版本

最新版本的STM32CubeIDE可从www.st.com/stm32softwaretools网站免费下载。

1.1.3 安装 STM32CubeIDE

STM32CubeIDE 安装指南[ST-04]提供了如何在受支持的 Windows®、Linux®和 macOS®版本上安装的说明。可以同时安装多个版本的 STM32CubeIDE。如果 STM32CubeIDE 尚未安装或必须安装新版本，请阅读安装指南。本手册中的安装更新和其他 Eclipse 插件还提供了关于如何安装更新的信息

1.1.4 许可证

STM32CubeIDE是根据Mix Ultimate Liberty+OSS+第三方V1软件许可协议(SLA0048)交付的。关于各组件的许可协议，请参见[ST-02]。

1.1.5 支持

意法半导体提供了几种不同的支持选项。例如，ST社区提供了随时与世界各地具有类似心态的人见面的场所。访问my.st.com/content/my_st_com/en/support/support-home.html选择支持选项。

1.2 使用 STM32CubeIDE

1.2.1 基础概念和术语

本节将概述使用STM32CubeIDE和Eclipse®术语的基本概念。

工作区

当启动STM32CubeIDE时，将选择一个工作区。工作区包含要使用的开发环境。从技术上讲，工作区是一个可以保存项目的目录。用户可以访问活动工作区中的任何项目。

一个项目包含文件，这些文件可以被组织成子目录。计算机上其他地方存在的文件也可以链接到该项目。

一台计算机可能在文件系统的不同位置拥有多个工作区。用户可以在工作空间之间切换，但一次只能激活一个工作空间。切换工作区是一种从一组项目切换到另一组项目的快速方法。

在实践中，工作空间和项目模型促进了一个结构良好的工作空间层次结构，其中包含项目，而项目又包含文件。

信息中心

第一次启动STM32CubeIDE并选择工作空间时，将打开信息中心。信息中心提供了启动新项目、访问视频、阅读STM32CubeIDE文档或访问ST支持和社区的快速访问。可以随时通过信息中心工具栏按钮或帮助菜单轻松访问信息中心。

透视图，菜单栏，工具栏

当信息中心关闭时，STM32CubeIDE将显示一个透视图，其中包含菜单栏、工具栏、视图和编辑器。每个透视图都针对特定类型的工作进行了优化。例如，C/C++透视图用于创建、编辑和构建项目。Debug透视图用于在硬件上调试代码。

每个透视图都可以根据用户的需要进行定制。例如，如果打开的视图太多，或者视图被重新排序，可以**随时重置透视图**。也可以创建新的透视图。

查看与编辑

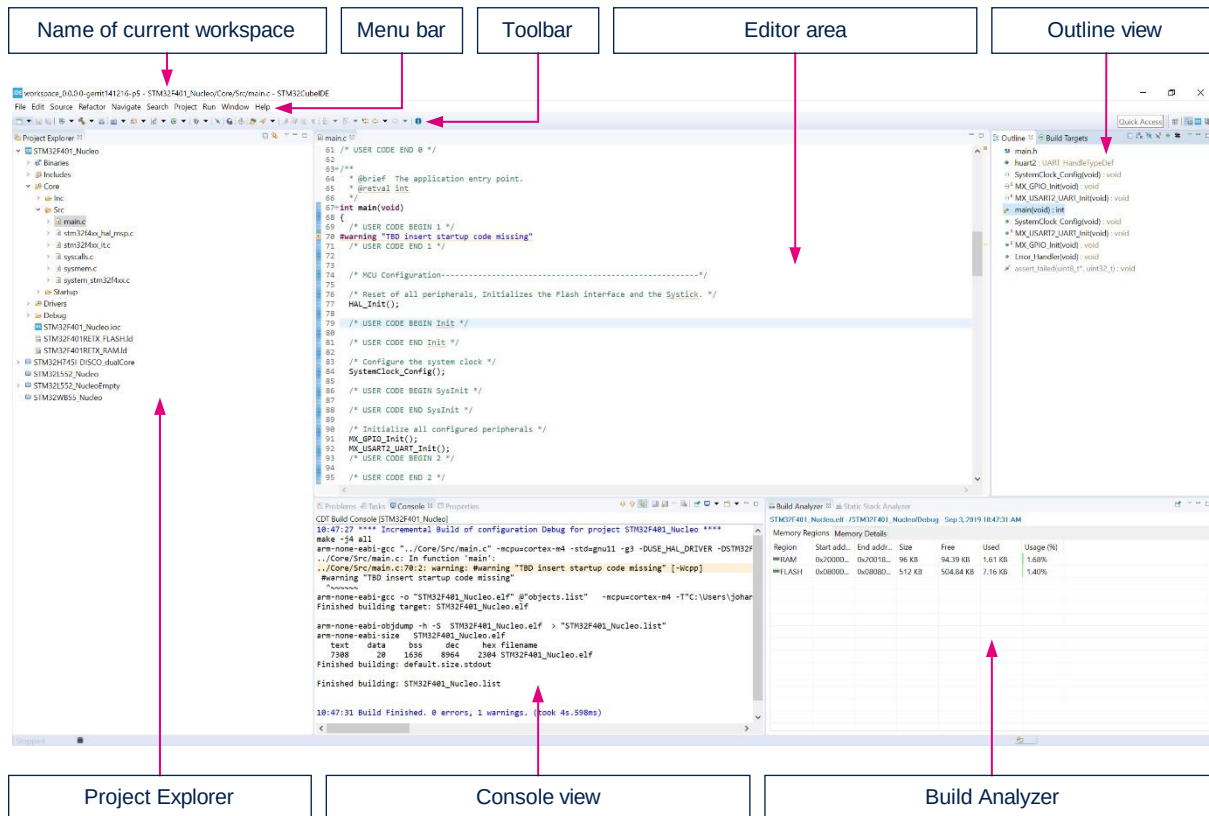
透视图通常显示许多视图。开发每个视图是为了表示特定的信息，例如，这些信息可以从项目或

调试中的嵌入式系统中收集。

透视图有一个编辑器区域。该编辑器可用于编辑项目文件。许多文件可以在编辑器的不同选项卡中编辑

STM32CubeIDE window

Figure 2. STM32CubeIDE window



1.2.2

开始 STM32CubeIDE

根据使用的操作系统执行以下步骤启动STM32CubeIDE。

Windows®

如果在产品安装过程中创建了桌面快捷方式，则可以使用该快捷方式启动STM32CubeIDE。该产品也可以从意法半导体程序下的Windows®开始菜单启动。

以其他方式:

1. Locate where STM32CubeIDE is installed, for instance in C:\ST\STM32CubeIDE_1.0.2
2. Open the STM32CubeIDE folder
3. Start the stm32cubeide.exe program

Linux® or macOS®

当使用Linux®或macOS®时，可以以类似的方式打开产品安装所在的STM32CubeIDE文件夹来启动程序。

STM32CubeIDE Launcher

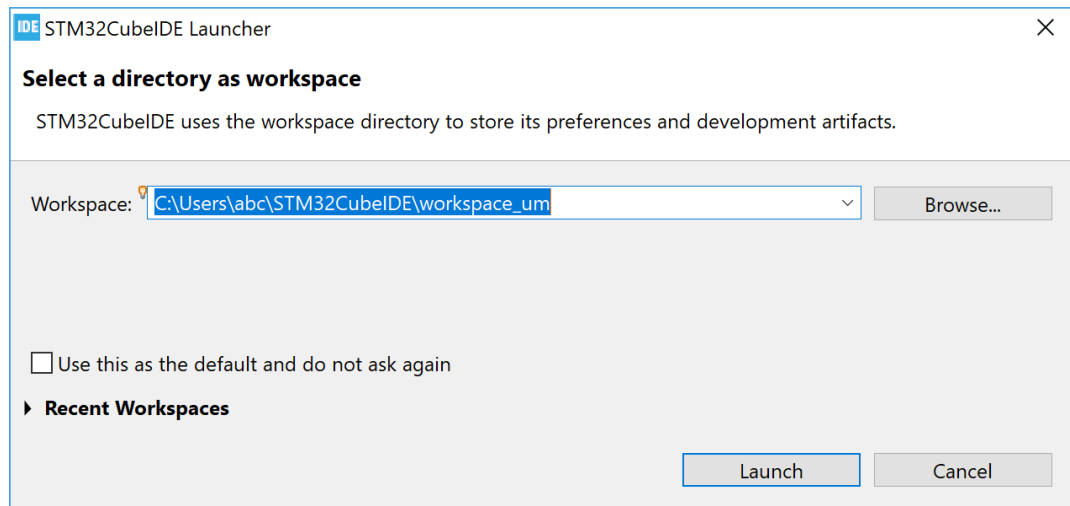
当项目启动时，它将显示带有工作区选择的STM32CubeIDE Launcher对话框。产品第一次启动时，它会显示一个默认位置和工作区名称。该对话框允许用户选择活动工作区的名称和位置，以保存用户当前可访问的所有项目。任何新创建的项目都存储在这个工作区中。如果工作区还不存

在，则创建该工作区。

注意

如果使用Windows®, 请避免将工作区文件夹定位在文件系统根目录以下太多级别, 以避免超出Windows®路径长度字符限制。如果文件路径超过Windows®可以处理的长度, 则会发生生成错误。

Figure 3. STM32CubeIDE Launcher – Workspace selection

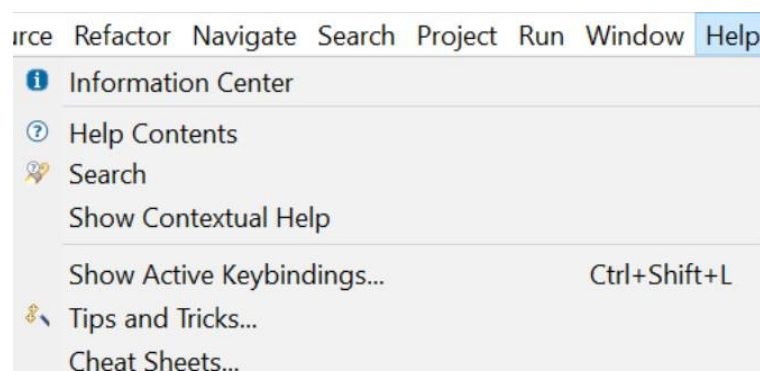


单击[Launch]按钮启动STM32CubeIDE。第一次, 它打开信息中心, 在1.3节信息中心中有描述。

1.2.3 Help system

Help菜单提供了几个不同的帮助系统, 如图4所示。信息中心包含所有可用STM32CubeIDE文档的链接。还建议新用户尝试不同的Eclipse®内置帮助系统, 以了解Eclipse®的基础知识。

Figure 4. Help menu



1.3 Information Center

信息中心提供快速访问:

1. Start a new project
2. Import an existing project
3. Get access to videos
4. Read [STM32CubeIDE documentation](#)
5. Get access to *Getting Started with STM32CubeIDE (STM32CubeIDE quick start guide [ST-03])*
6. Explore the STM32 MPU and MCU wikis

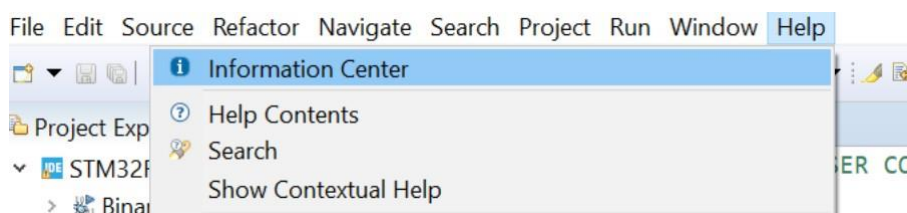
7. Get access to STMicroelectronics support and community on Twitter™, Facebook™, YouTube™, or ST community at community.st.com
8. Explore the STMicroelectronics application tools

在第一次使用本产品之前，不要求阅读所有材料。相反，建议将信息中心视为参考信息的集合，以便在需要时返回。

1.3.1 访问信息中心

使用[信息中心]工具栏按钮，可以在任何时间、从任何角度轻松访问信息中心。该图标位于工具栏的右侧。也可以从[帮助]>[信息中心]菜单命令打开信息中心。

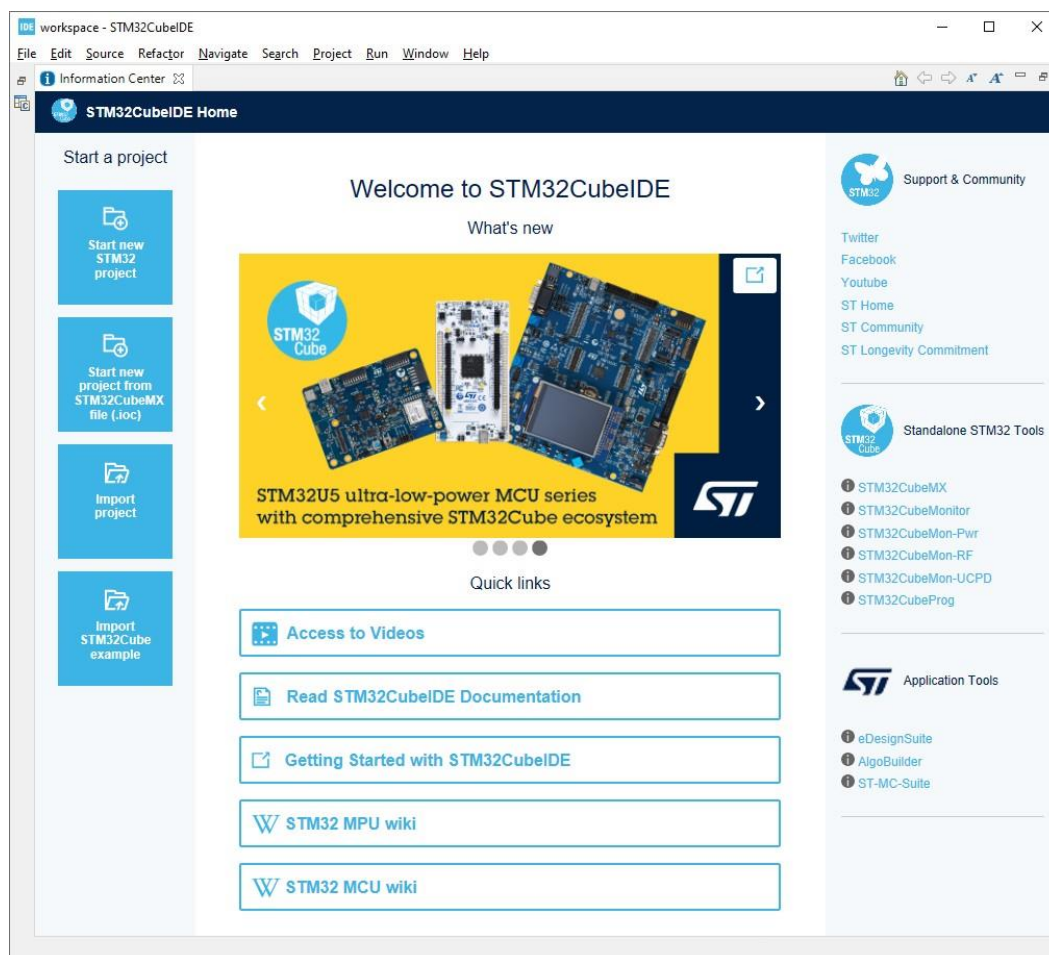
Figure 5. Help - Information Center menu



1.3.2 Home page

打开信息中心后，显示“首页”界面。它包含启动新项目、导入项目、阅读文档和访问意法半导体支持和社区的链接。

Figure 6. Information Center – Home page

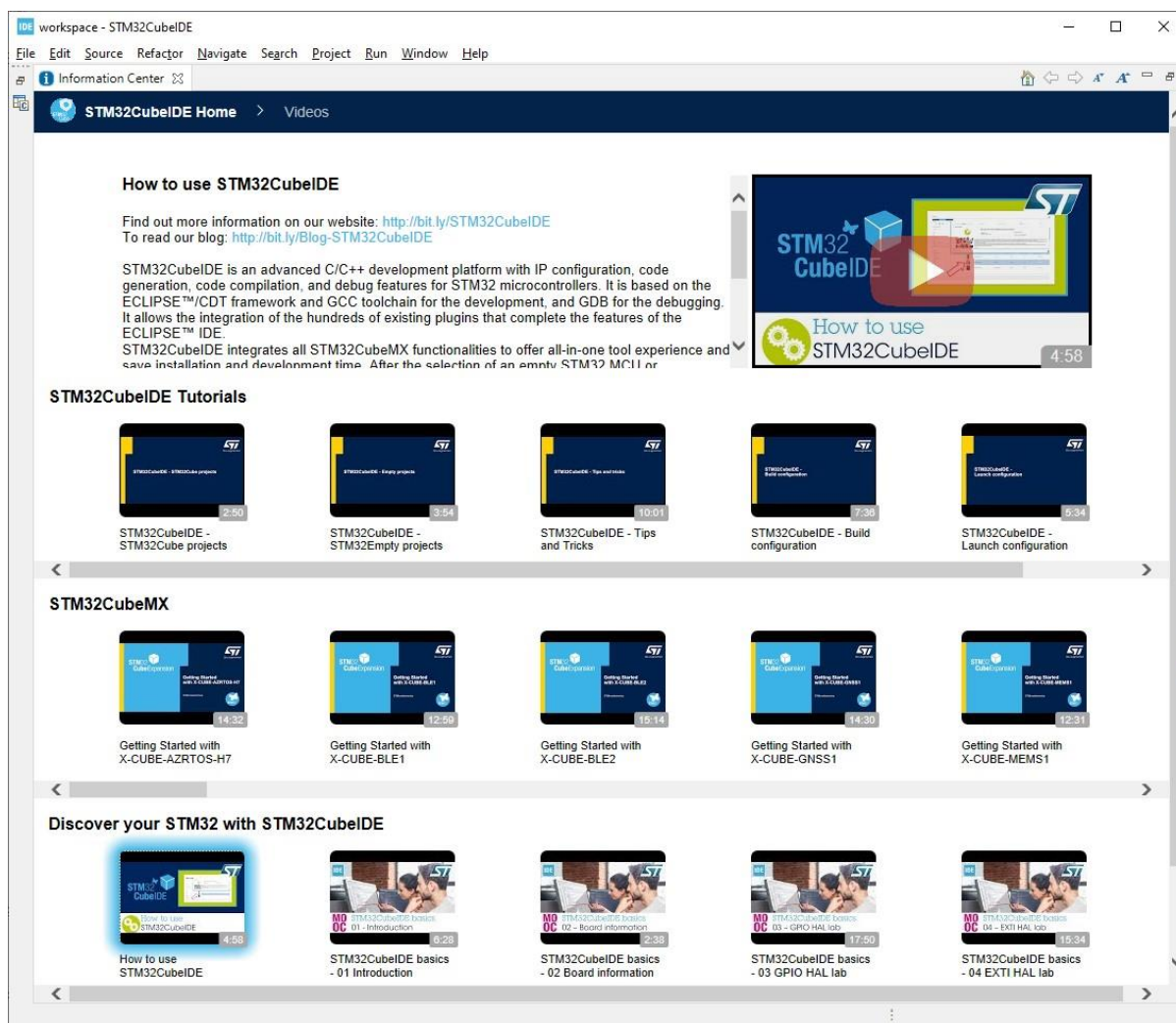


使用旧工作区时，信息中心可能无法显示有效信息，显示“无法显示此页”或在访问文档时打开旧手册。在这种情况下，单击Information Center窗口右上角的[Home] 按钮，重新加载页面。

1.3.3 Videos

信息中心还包含一个视频浏览器页面，当单击访问视频链接时，该页面将从主页打开。

Figure 7. Information Center – Video browser page



滚动视频页面，单击列表中的视频缩略图，在web浏览器中打开它。这些视频按组列出：

- *STM32CubeIDE Tutorials*
- *STM32CubeMX*
- *Discover your STM32 with STM32CubeIDE*

要导航回主页，请按信息中心左上方的STM32CubeIDE主页。

1.4 透视图、编辑器和视图

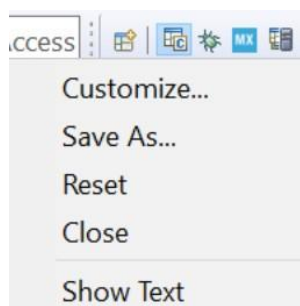
STM32CubeIDE是一个功能强大的产品，具有许多视图，加载了各种特性。同时显示所有视图会给用户带来与手头任务不相关的信息。

为了克服这种情况，可以在透视图组织中组织视图，其中透视图包含许多预定义的视图和默认可见的编辑器区域。透视图通常处理一个开发任务，例如C/ c++代码编辑或调试。

1.4.1 透视图

透视图可根据用户需要进行定制;视图可以移动、调整大小，还可以打开新视图。例如，如果打开的视图太多，或者视图被重新排序，可以随时重置透视图。通过右键单击工具栏中的透视图图标并从列表中选择[reset]来重置透视图。这将重置视图;透视图中添加的视图将被关闭，默认视图将被移动到我原始位置。

Figure 8. Reset perspective



如图8所示，还可以定制透视图并用新名称保存透视图。

从一个透视图切换到另一个透视图是隐藏一些视图和显示其他视图的快速方法。要切换透视图，请选择工具栏右侧的[打开透视图]工具栏按钮。

Figure 9. Toolbar buttons for switching perspective

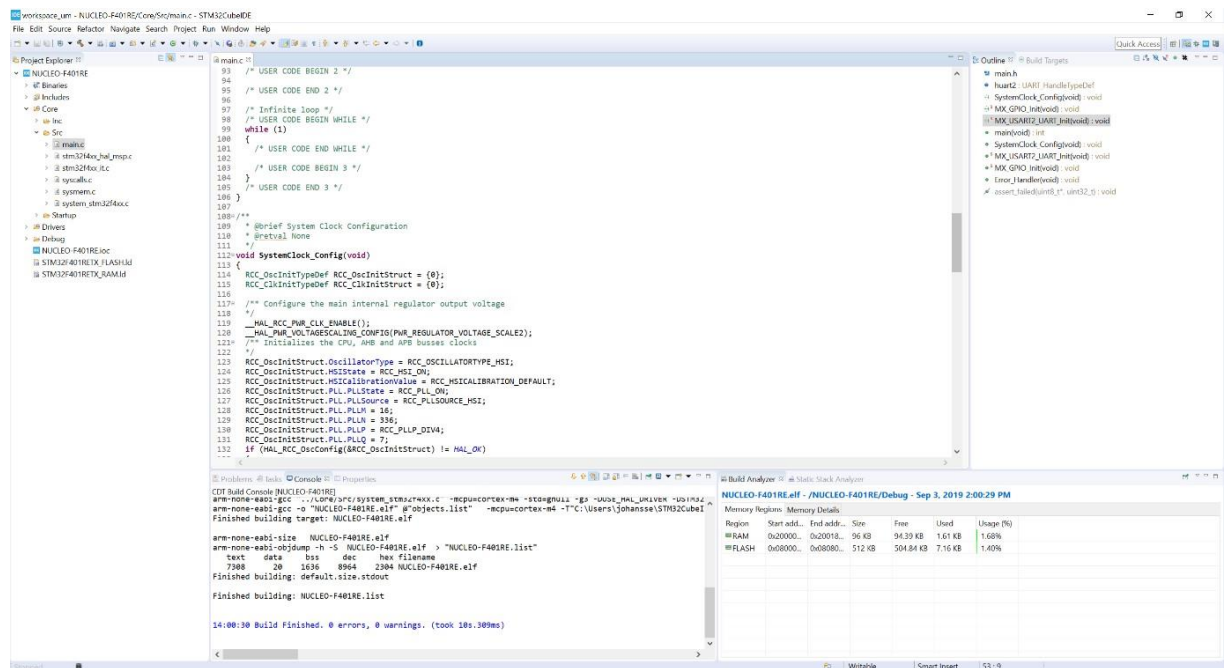


切换透视图的另一种方法是使用菜单命令[Window]>[Perspective]>[Open Perspective]>[Other...], 选择要使用的透视图。

1.4.1.1 C/C++ perspective

C/ C++透视图用于创建新项目、编辑文件和构建项目。透视图的左侧包含Project Explorer视图。编辑器位于中间。右边的部分包含项目的一些视图(Outline和Build Targets视图)。在图10所示示例的底部，有Problems、Tasks、Console和Properties视图。在右下角，显示Build分析器和Static堆栈分析器视图。

Figure 10. C/C++ perspective

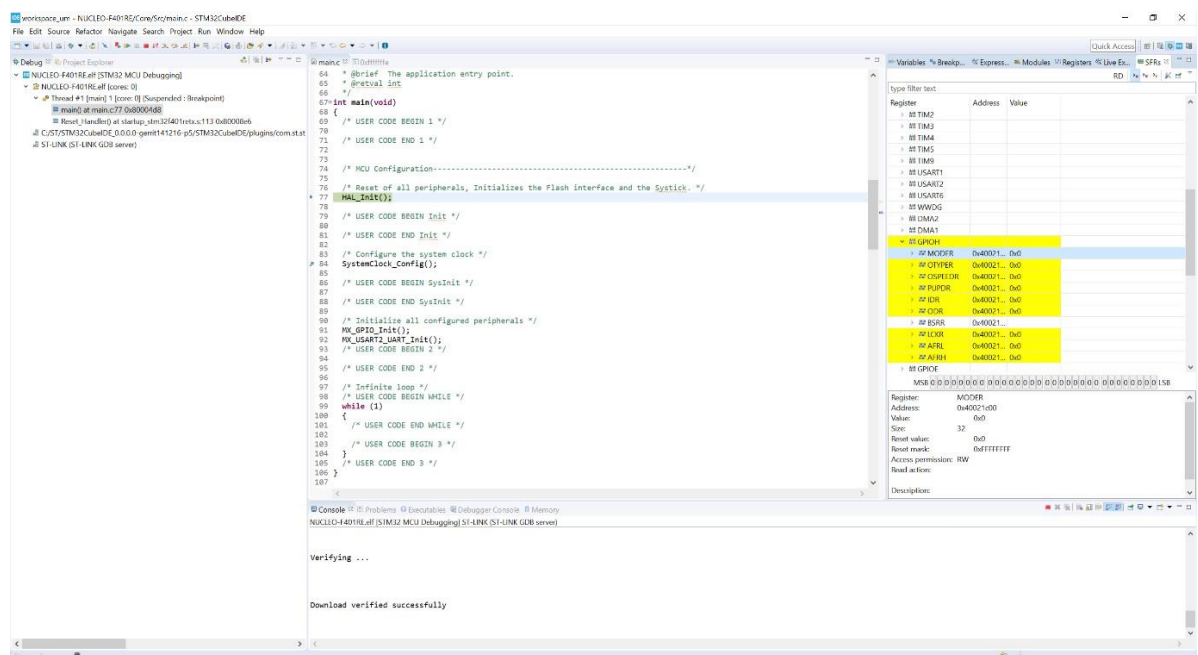


1.4.1.2

Debug perspective

Debug透视图用于调试代码。Debug透视图通常在启动新的调试会话时自动打开。稍后，当调试会话关闭时，透视图将切换回C/ c++透视图

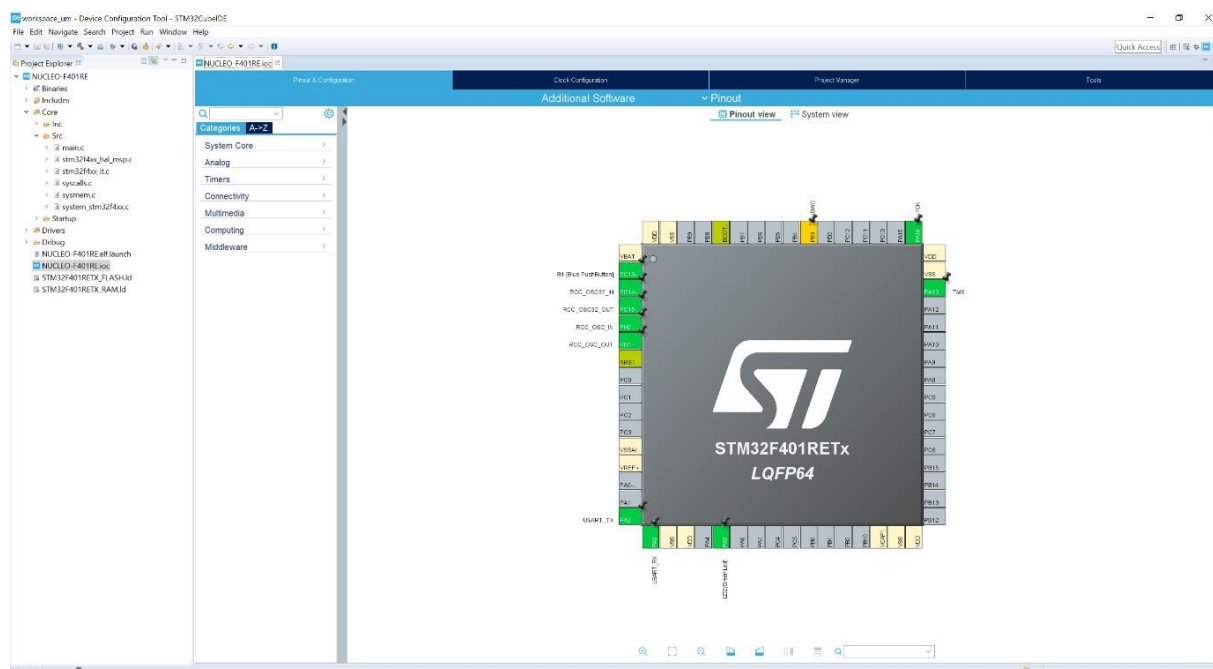
Figure 11. Debug perspective



1.4.1.3 Device Configuration Tool perspective

设备配置工具透视图包含集成在STM32CubeIDE中的STM32CubeMX设备配置工具。此透视图用于设备配置。当一个.ioc文件在编辑器中打开，并使用设备配置工具透视图，则可以在此透视图配置设备。设备的配置方法见[ST-14]。

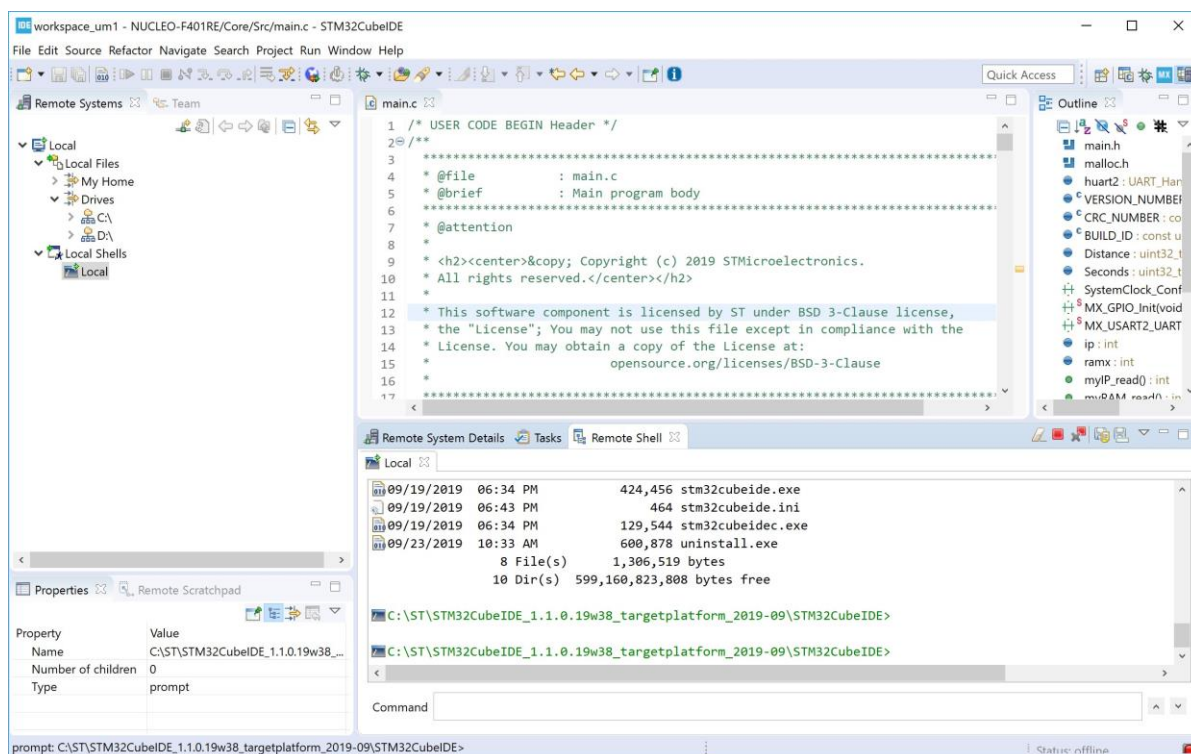
Figure 12. Device Configuration Tool perspective



1.4.1.4 Remote System Explorer perspective

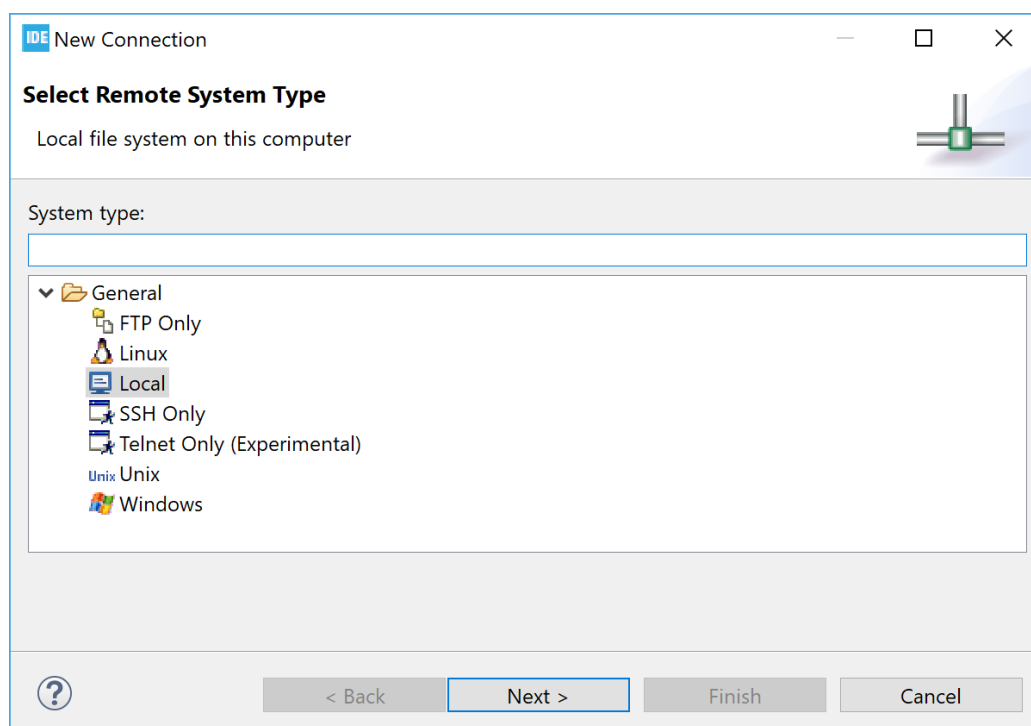
远程系统资源管理器视角主要用于开发基于STM32 Arm®Cortex®mpu的系统。Remote Systems视图用于查看文件，Remote Shell视图用于执行命令。

Figure 13. Remote System Explorer perspective



Remote Systems视图包含通过FTP、Linux®、Local、SSH、Telnet和其他方式打开新连接的按钮。

Figure 14. New connection



1.4.2 编辑器

透视图中的编辑器区域由编辑器使用。可以同时打开任意数量的编辑器，但一次只能打开一个。不同的编辑器可以与不同的文件扩展名相关联。编辑器的例子有;c-编辑器，链接器脚本编辑器，ioc-文件编辑器用于STM32CubeMX设备配置。

要在编辑器中打开一个文件，在Project Explorer视图中双击该文件或通过[File]菜单打开该文件。在编辑器中修改文件时，会显示一个星号(*)，表示该文件有未保存的更改。

1.4.3 视图

默认情况下，只显示与透视图关联的最常见视图。产品中有更多支持不同特性的视图。其中一些视图仅在调试会话正在进行时提供有效数据，而其他视图始终显示数据。

通过选择列表中的一个视图，可以从[Window]>[Show View]菜单打开视图。

Figure 15. [Show View] menu

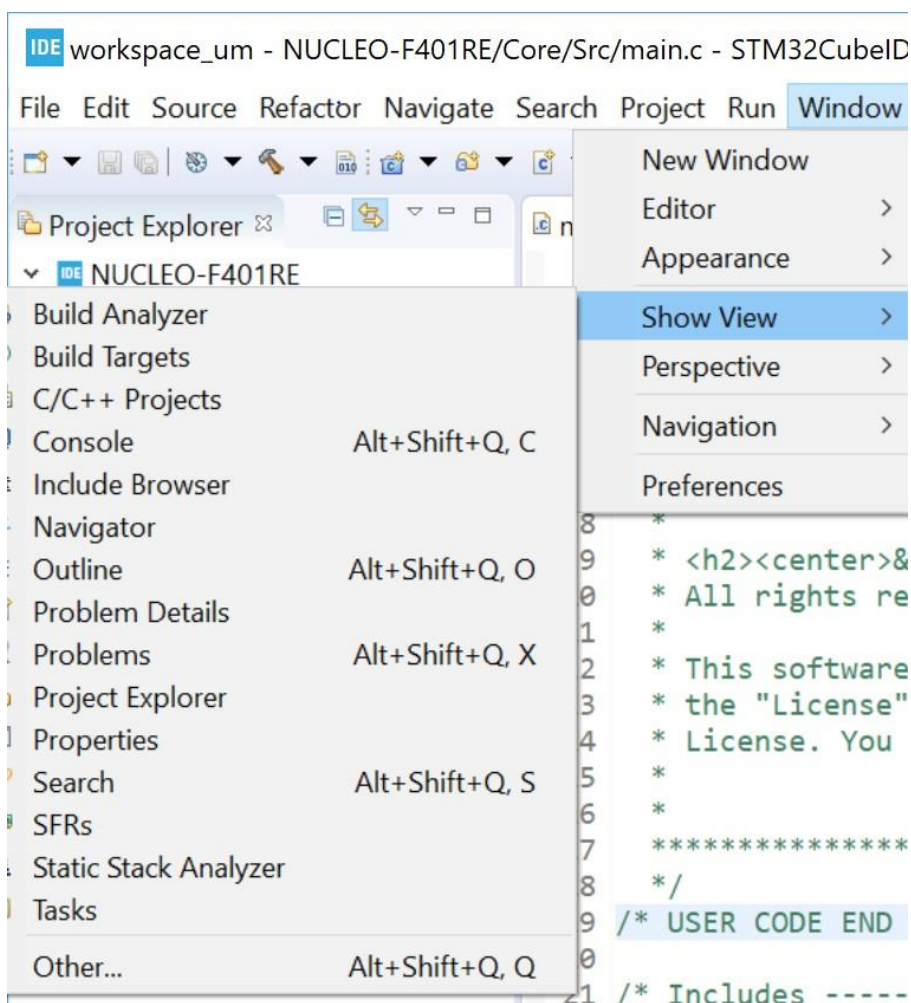
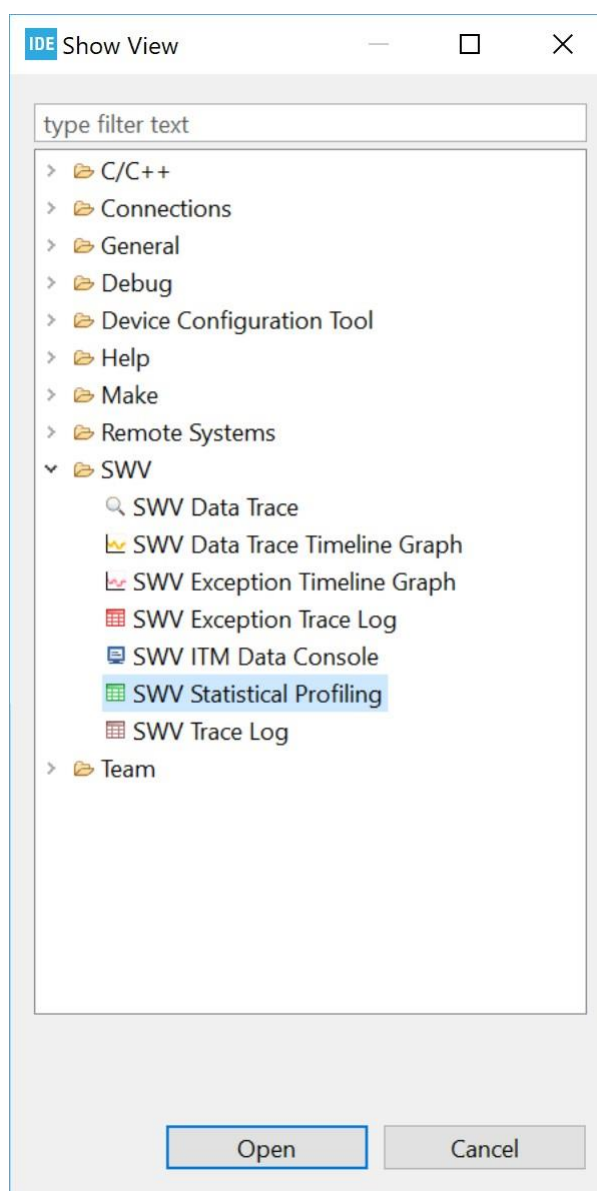


图15中的上述视图列表仍然不完整。它仅包含与当前所选透视图相关的工作任务的最常见视图。要访问更多视图，请从列表中选择[Other...]。这将打开“显示视图”对话框。双击任意视图以打开它并访问其附加功能。

Figure 16. Show View dialog



可以调整视图的大小并更改其位置:只需将视图拖到STM32CubeIDE中的新位置即可。还可以将视图拖到屏幕上的STM32CubeIDE窗口之外。这种分离的视图显示在单独的窗口中。分离视图与其他视图类似，但总是显示在工作台前面。通过将分离视图中的选项卡拖到STM32CubeIDE窗口中，可以再次附加分离视图。

要将透视图恢复到原始状态，右键单击工具栏中的透视图图标，并从列表中选择[Reset]。重置透视图的另一种方法是使用菜单[Window]>[perspective]>[reset perspective]。

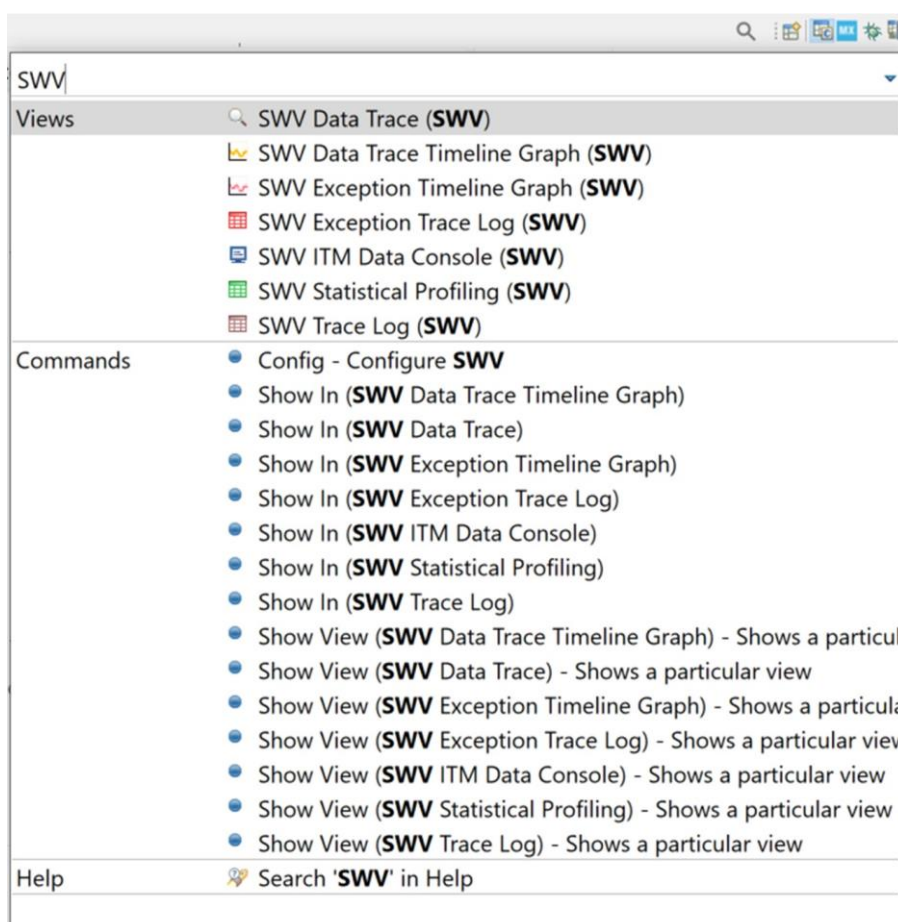
1.4.4 Quick Access edit field

工具栏中的放大镜将打开“快速访问”文本框，可以在其中输入任何搜索短语或关键字。可以使用文本框找到菜单命令、工具栏按钮、首选项设置或视图等GUI对象。当键入任何搜索字符串时，Quick Access将实时显示与条件匹配的所有GUI对象。输入两个或多个字符，然后查看结果列表是如何相应地动态优化的。

Quick Access可以在查找无法快速找到的特定GUI对象时节省时间，例如深埋在配置对话框中的首选项设置。检索隐藏在当前活动透视图中的菜单命令或工具栏按钮也很方便。

例如，在图17中，在Quick Access中输入的搜索字符串“SWV”立即提供了匹配视图、GUI命令和首选项设置的列表。要打开视图或首选项设置，请单击搜索结果列表中的GUI对象。

Figure 17. Quick access

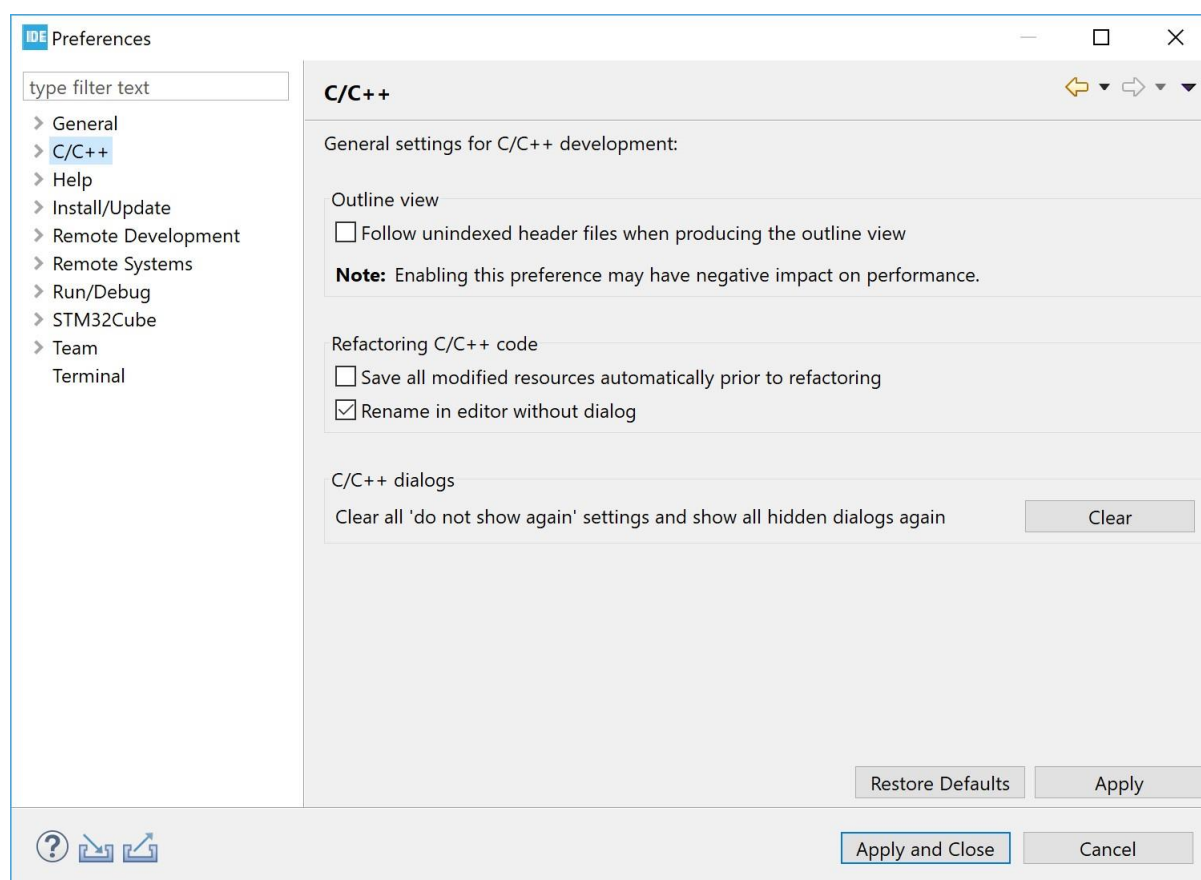


1.5 Configuration - Preferences

STM32CubeIDE 可以以多种方式定制。菜单[Window]>[Preferences]用于打开Preferences对话框。在这个对话框中，左窗格用于导航到某些首选项页面。还有一个筛选字段，可用于缩小显示内容的范围。对话框右上方的箭头控件可用于在页面之间来回导航。右侧窗格包含所显示的首选项的设置。做任何喜欢的改变，并按[应用]更新设置

[Restore Defaults]重置所有修改。首选项设置存储在应用程序工作区中的元数据文件夹中。本用户手册中的第1.7节管理现有工作区提供了关于如何跨工作区备份首选项和复制首选项的信息。

Figure 18. Preferences



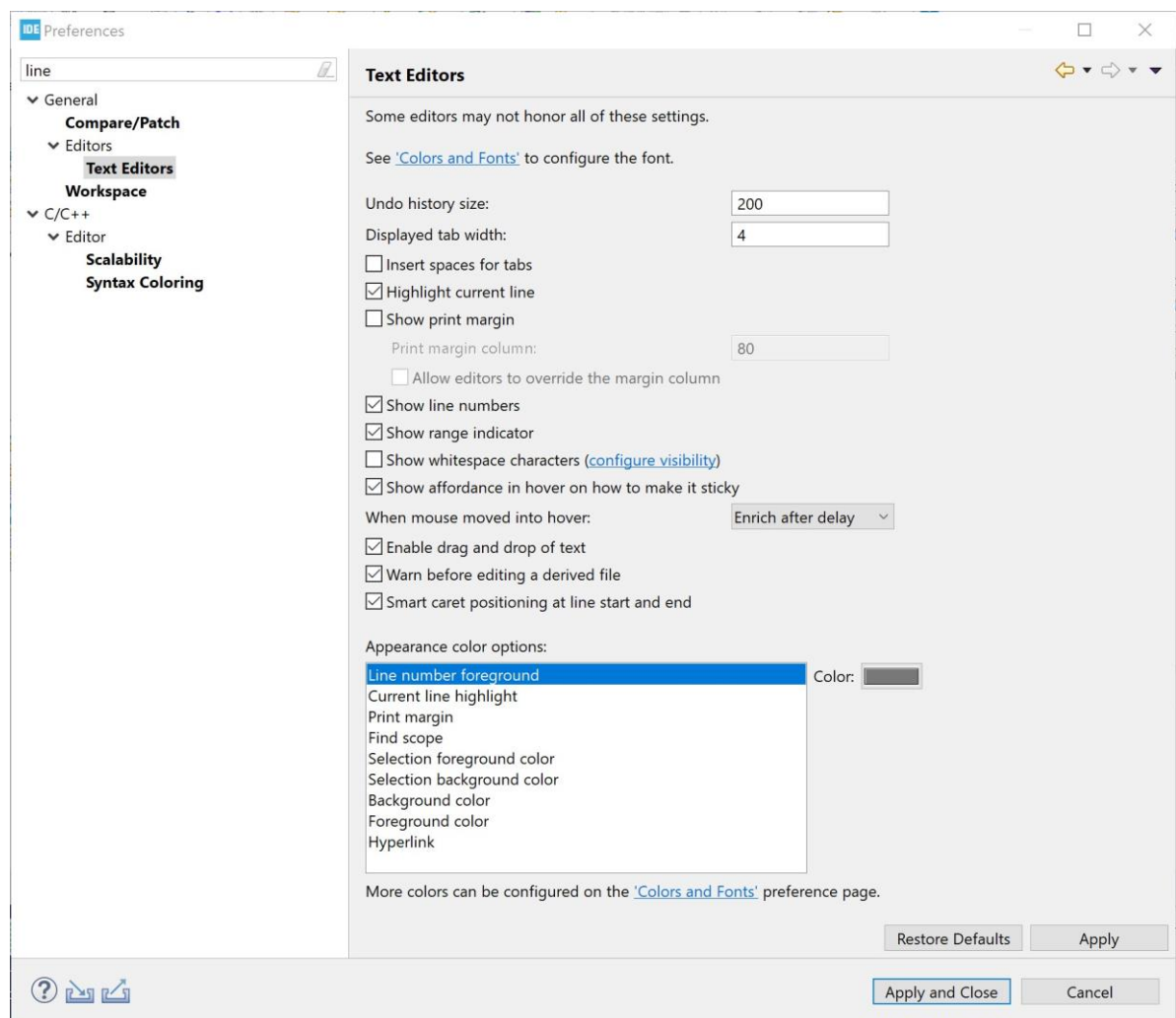
建议浏览首选项页面，了解可能的配置选项。下面几节将介绍其中的一些。

1.5.1 Preferences - Editors

编辑器可以通过多种方式进行配置。例如，菜单选择[General]>[Editors]>[Text Editors]提供了一个包含常规编辑器设置的首选项窗格：

- Displayed tab width
- Insert spaces for tabs
- Highlight current tab
- Show line numbers
- Others

Figure 19. Preferences - Text Editors

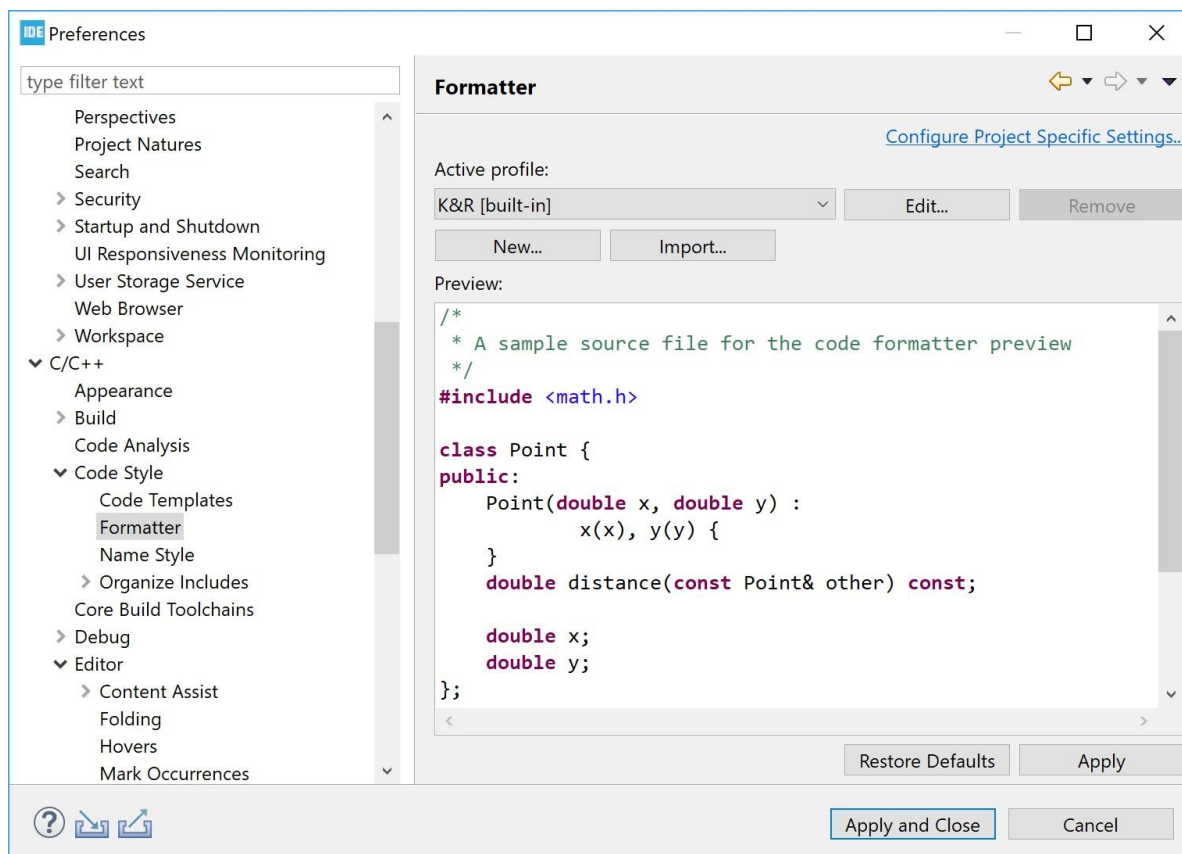


1.5.2 Preferences - Code style formatter

可以将编辑器配置为使用特殊格式。

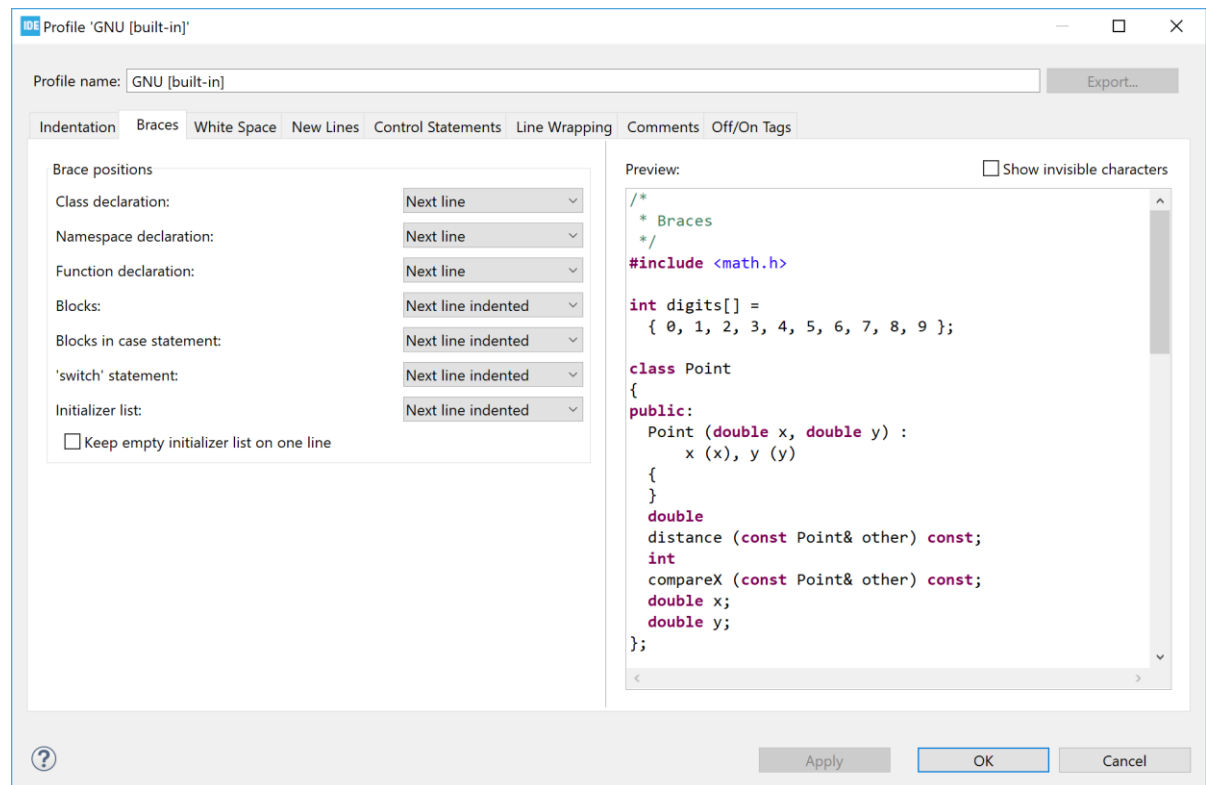
菜单选择[C/C++]>[Code Style]>[Formatter]提供了一个Preferences窗格，其中包含设置活动配置文件。

Figure 20. Preferences - Formatter



此时，如果按下[Edit...], 将打开一个新的对话框，其中可以根据特定的编码规则更新所选的概要文件。如图21所示。

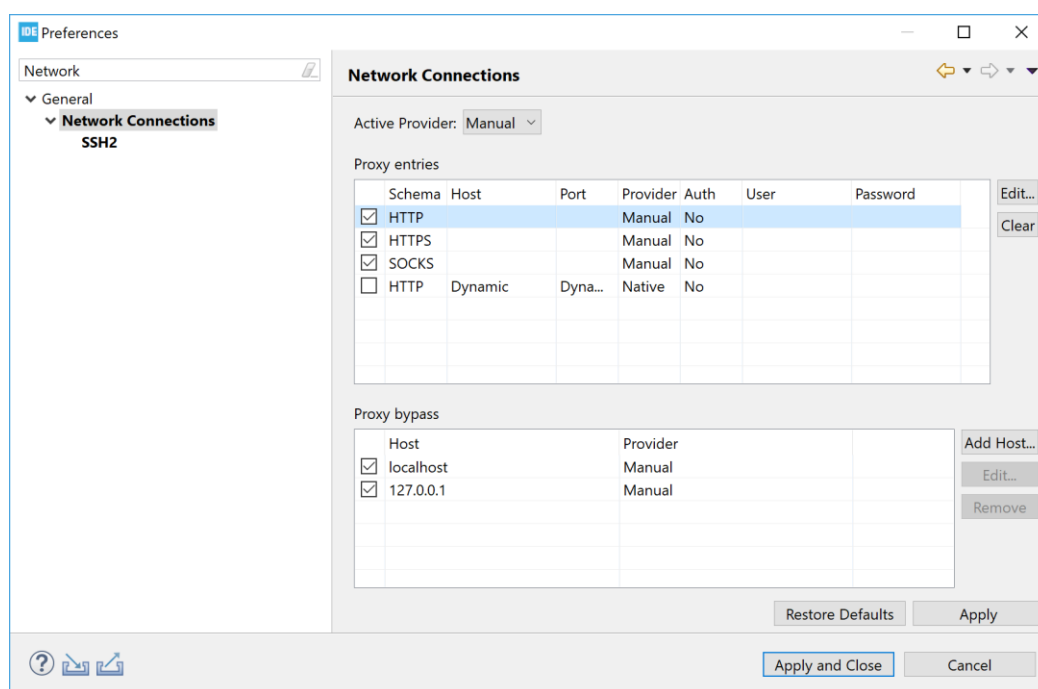
Figure 21. Preferences - Code style edit



1.5.3 Preferences - Network proxy settings

例如，STM32CubeIDE使用Internet访问STM32设备信息。如果使用代理服务器访问Internet，则需要在STM32CubeIDE中进行一些配置设置。代理设置在通过[General]>[Network Connections]获取的“首选项”窗格中进行。要更改设置，请将[Active provider]设置为手动，并使用[Edit...]按钮更新具有特定主机、端口、用户和密码的HTTP和HTTPS代理条目。

Figure 22. Preferences - Network Connections



Note: 如果保存代理设置有问题，原因可能是损坏的secure_storage文件。按照以下步骤来解决问题:

1. Close all running STM32CubeIDE applications
2. Rename file `C:\Users\user_name\.eclipse\org.eclipse.equinox.security\secure_storage` to a new name
3. Restart STM32CubeIDE
4. Update the proxy network settings, with user and password information, and save them to create a new secure_storage file

1.5.4 Preferences - Build variables

STM32CubeIDE首选项提供了仅在IDE中可见的构建变量

菜单选择[C/C++]>[Build]>[Build Variables]提供了一个带有构建变量的首选项窗格，可以在STM32CubeIDE中用作\${VAR}。使能[Show system variables]显示所有可用变量。

Figure 23. Preferences – Build variables

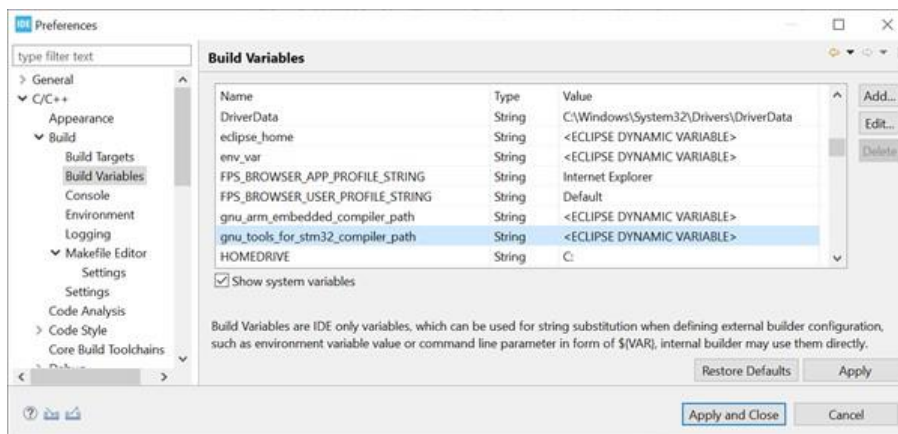
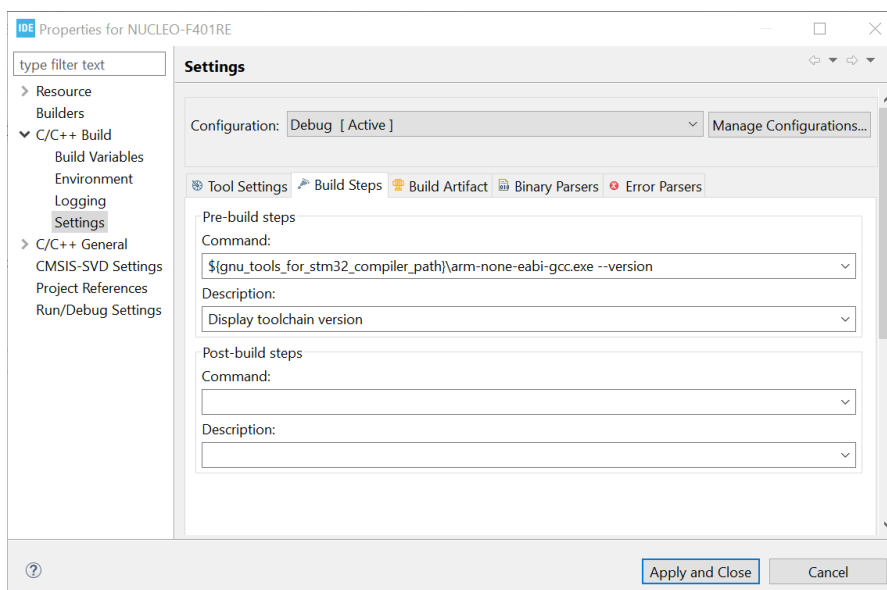


Table 1. Examples of toolchain build variables

Variable	Description
<i>gnu_tools_for_stm32_compiler_path</i>	Path to GNU Tools for STM32 toolchain.
<i>gnu_arm_embedded_compiler_path</i>	Path to GNU Arm Embedded toolchain.
<i>stm32cubeide_make_path</i>	Path to make and BusyBox.

图24给出了一个使用构建变量显示工具链版本的预构建步骤示例。

Figure 24. Pre-build step using build variables



1.6 工作区和项目

工作空间和项目的基本概念比较如下：

- 工作区包含项目。从技术上讲，工作区是一个包含项目目录或对它们的引用的目录。
- 项目包含文件。从技术上讲，项目是一个包含可以在子目录中组织的文件的目录。
- 一台计算机可能在文件系统的不同位置拥有多个工作区。每个工作区可能包含几个项目。
- 用户可以在工作空间之间切换，但一次只能激活一个工作空间。
- 用户可以访问活动工作区中的任何项目。位于另一个工作空间中的项目不能被访问，除非用户切换到该工作空间。
- 项目中包含的文件不需要物理地位于项目的文件夹中，但可以位于其他地方并链接到项目中。
- 切换工作区是一种从一组项目切换到另一组项目的快速方法。它触发了产品的快速重新启动。

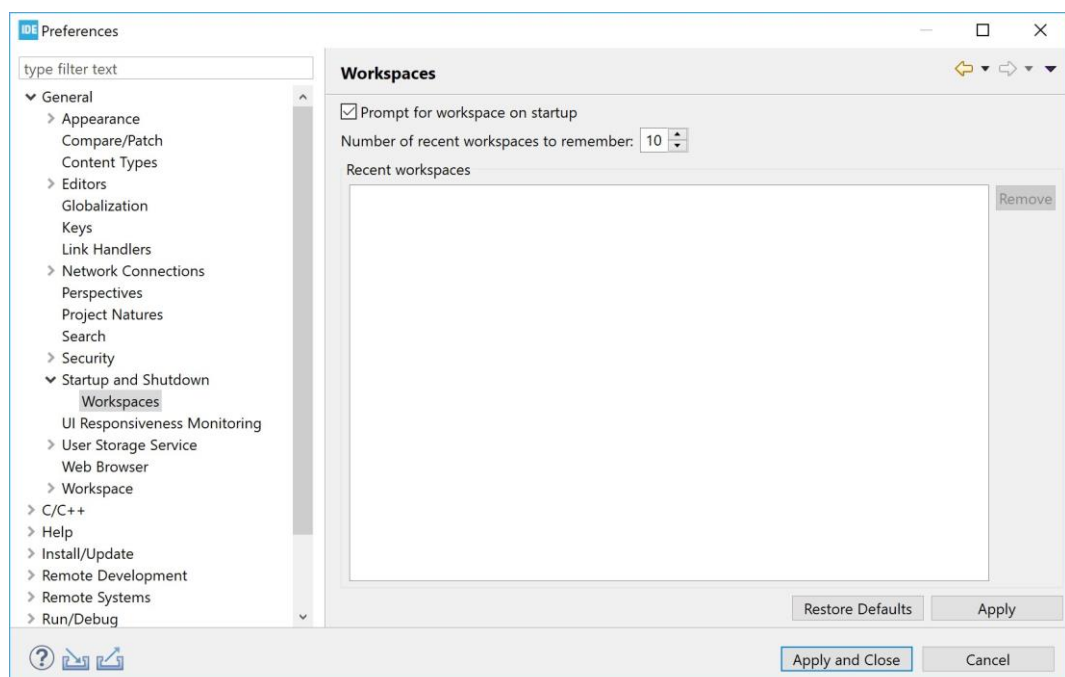
在实践中，项目和工作空间模型促进了工作空间的结构良好的层次结构，包含项目，包含文件。

1.7 Managing existing workspaces管理现有工作区

可以在启动STM32CubeIDE时选择工作空间。在使用STM32CubeIDE期间，也可以切换到另一个工作空间。在本例中，STM32CubeIDE在选择新的工作空间后重新启动。要使用新的工作空间重新启动STM32CubeIDE，请选择菜单[File]>[Switch workspace]。

可以通过选择[Window]>[Preferences]来管理STM32CubeIDE已知的工作空间，然后在首选项对话框中选择[General]>[Startup and Shutdown]>[Workspaces]。在右侧窗格中，可以启用[Prompt for workspace on startup]，并将[Number of recent workspaces to remember]设置为所需的值。

Figure 25. Preferences - Workspaces



还可以从最近工作区列表中选择和删除最近的工作区。但是，从该列表中删除工作空间并不会删除文件。它也不会从文件系统中删除文件。

1.7.1 备份工作区的首选项

为工作空间获取现有首选项的副本通常是一个很好的实践。它可以在崩溃后重新创建工作空间，而无需手动重做设置的耗时过程。

在菜单中选择[File]>[Export]。然后，在面板中选择[General]>[Preferences]。按下[Next]按钮，在下一页中，启用[Export All]以及正确的文件名。

1.7.2 在工作空间之间复制首选项

要将工作空间首选项从一个工作空间复制到另一个工作空间，必须首先创建现有的首选项导出，如备份工作空间首选项中所述。

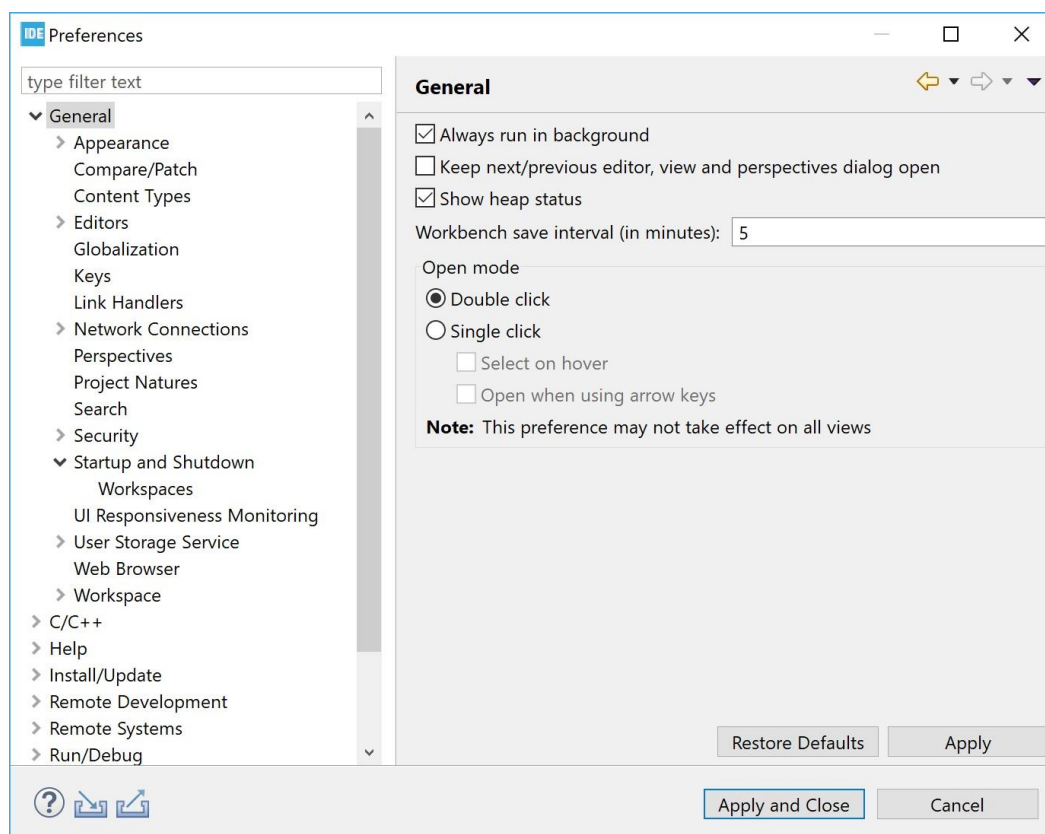
然后选择[File]>[Switch Workspace]和新的工作空间。STM32CubeIDE重新启动并打开新的工作空间。

在菜单中选择[File]>[Import]，在面板中选择[General]>[Preferences]。按[Next]按钮，在下一页启用[Import All]并输入文件名。现在两个工作区中的首选项是相同的。

1.7.3 跟踪Java堆空间

要跟踪使用了多少Java堆空间，请选择[Window]>[Preferences]菜单。在Preferences页面中，选择[General]节点，然后启用[Show heap status]。然后在STM32CubeIDE状态栏中显示当前使用的和可用的Java堆空间。还可以从状态栏手动触发垃圾收集器。

Figure 26. Display of Java heap space status

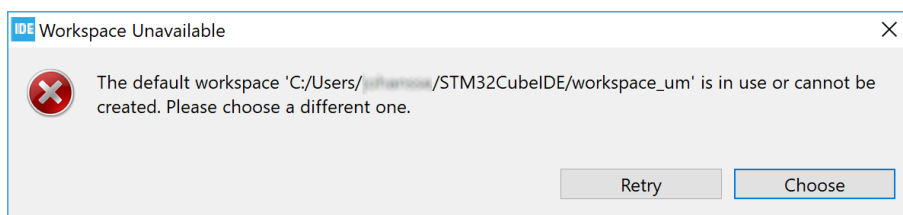


1.7.4 不可用工作区

一次只能有一个STM32CubeIDE实例访问一个工作空间。这是为了防止工作空间中发生冲突的更

改。如果STM32CubeIDE启动时使用的工作空间已经被程序的另一个实例使用，则会显示以下错误消息。

Figure 27. Workspace unavailable



如果显示此消息，请选择不同的工作空间，或返回到已经运行的STM32CubeIDE。

1.8 stm32cubeide和Eclipse® 基础知识

STM32CubeIDE包含如此多的特性，以至于很容易忽略一些真正有用的功能。值得注意的功能包括C/ c++注释的拼写检查、单词和代码补全、内容辅助、参数提示和代码模板。该编辑器还包括一个包含文件依赖浏览器，使用超文本链接的代码导航，书签和待办事项列表，以及强大的搜索机制。接下来的部分将介绍一些很容易被忽略的有用工具。

1.8.1 快捷键

用键盘快捷键代替鼠标很方便。要知道的一个重要捷径就是捷径CTRL + Shift + L。此快捷方式将打开包含所有可用快捷方式的备忘单

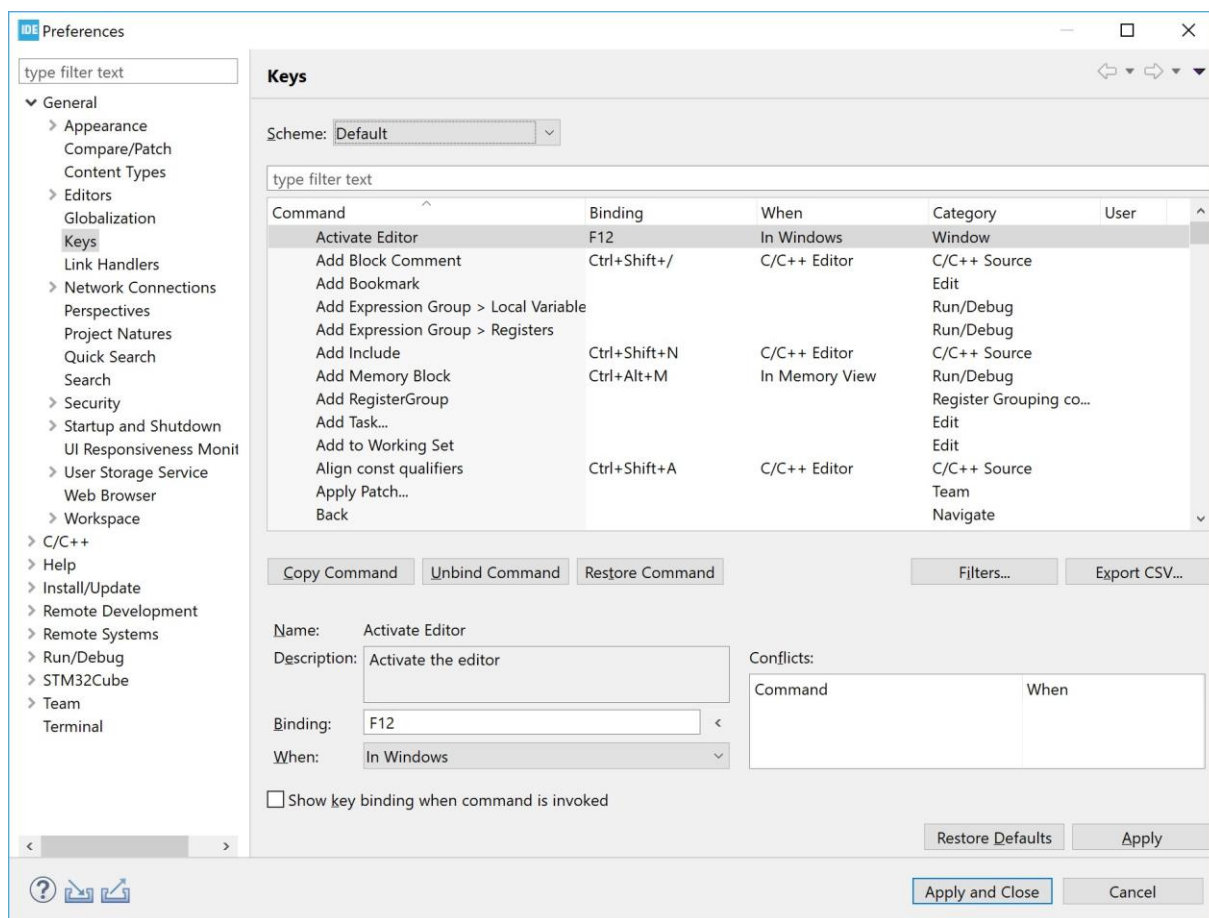
Figure 28. Shortcut keys

Activate Editor	F12
Backward History	Alt+Left
Build All	Ctrl+B
Build Target Build	Shift+F9
Close	Ctrl+F4
Close All	Ctrl+Shift+F4
Collapse All	Ctrl+Shift+Numpad_Divide
Content Assist	Ctrl+Space
Context Information	Ctrl+Shift+Space
Copy	Ctrl+Insert
Cut	Shift+Delete
Debug	F11
Delete	Delete
Expand All	Ctrl+Shift+Numpad_Multiply
Find Text in Workspace	Ctrl+Alt+G
Find and Replace	Ctrl+F
Forward History	Alt+Right
Last Edit Location	Ctrl+Q
Maximize Active View or Editor	Ctrl+M

Press 'Ctrl+Shift+L' to open the preference page

在此工作表中按CTRL+Shift+L打开Preferences对话框中的Keys窗格。

Figure 29. Shortcut preferences



Keys窗格提供了详细检查快捷键和更改方案(Default、Emacs或Microsoft®Visual Studio)、重新配置快捷键等的可能性。

表2给出了一些需要提及的键及其默认绑定。

Table 2. Key shortcut examples

Command	Binding	Where
Copy	Ctrl+C	In dialogs and windows
Cut	Ctrl+X	In dialogs and windows
Paste	Ctrl+V	In dialogs and windows
Debug	F11	In windows
Open declaration	F3	In C/C++ editor
References	Ctrl+Shift+G	In C/C++ editor/views
Find and open files	Ctrl+Shift+R	In C/C++ editor/views
Toggle selection mode normal/block	Alt+Shift+A	In C/C++ editor/views
Zoom In	Ctrl++	Editing text
Zoom Out	Ctrl+-	Editing text

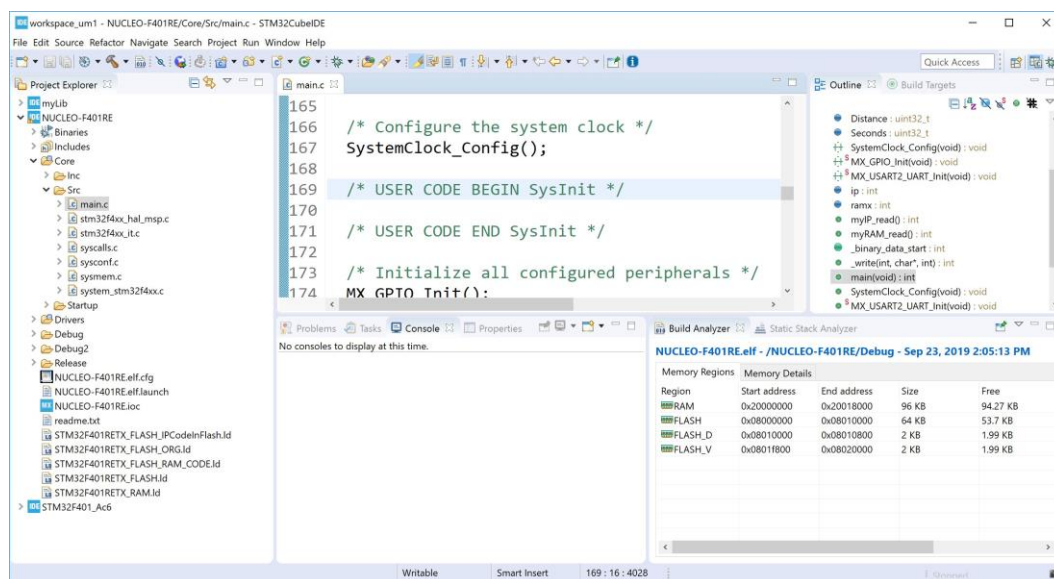
1.8.2 编辑器放大和缩小

可以通过按Ctrl++和Ctrl+-来增加或减少文本编辑器的默认字体大小:

- **Ctrl++** : zoom in text
- **Ctrl+-** : zoom out text

注意: 如果使用带有数字键盘的键盘, 并且在数字键盘上按下了“+”或“-”键, 则使用Shift键除了使缩放工作(Ctrl+Shift++或Ctrl+Shift--).

Figure 30. Editor with text zoomed in



1.8.3 快速找到并打开文件

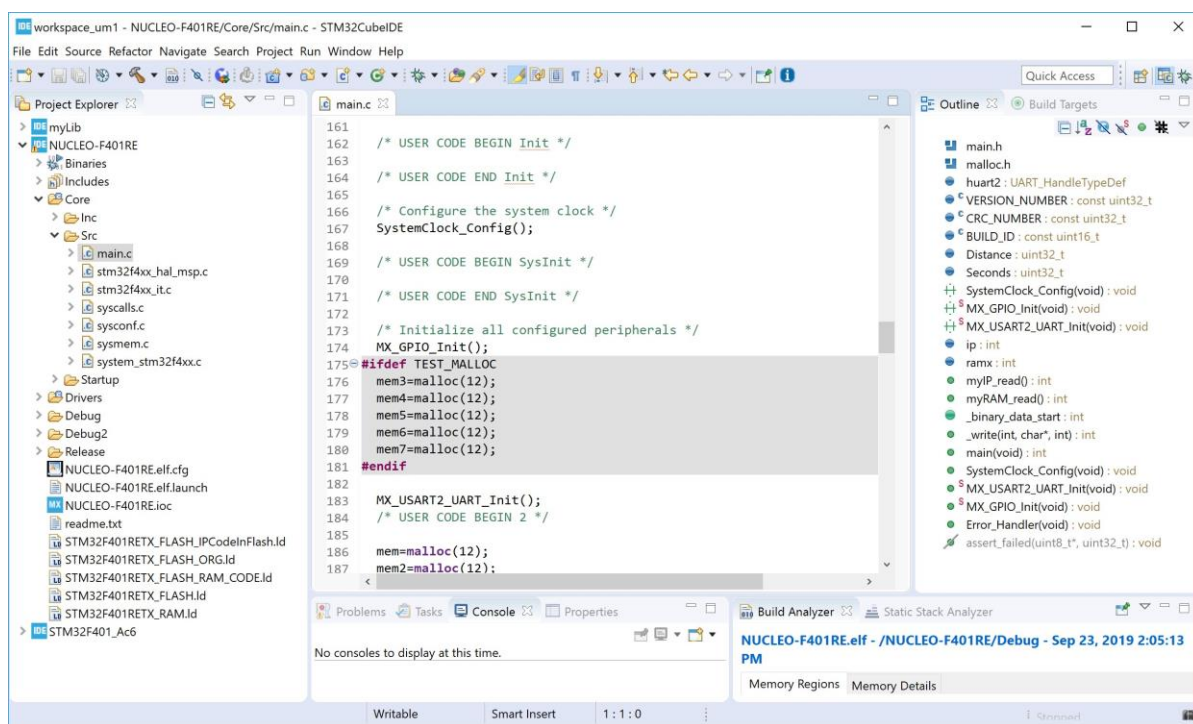
按Ctrl+Shift+R快速找到并打开文件是一个很容易错过的功能。键入要打开的文件名称的一部分的几个字符。可以加上*和?适当地搜索通配符。编辑器然后列出匹配的文件名。在搜索结果列表中选择所需的文件, 并使用以下三种方式之一打开该文件:

- **[Show In]:** 将文件发送到下拉列表中选择的一个视图(例如#include文件依赖项浏览器视图)
- **[Open With]:** 在下拉列表中选择编辑器中打开文件
- **[Open]:** 可能是最常用的选项, 在标准C/C++编辑器中打开文件

1.8.4 分支折叠

包含在`#if`和`#endif`中的代码块可以折叠。要激活该功能，请转到[Window]>[Preferences]，然后转到[C/C++]>[Editor]>[Folding]，并选中[Enable folding of preprocessor branches (`#if`/`#endif`)]复选框。选中复选框后，必须重新启动编辑器。关闭文件，再次打开它，编辑器左侧空白处的小图标显示该功能已被激活。

Figure 31. Editor folding



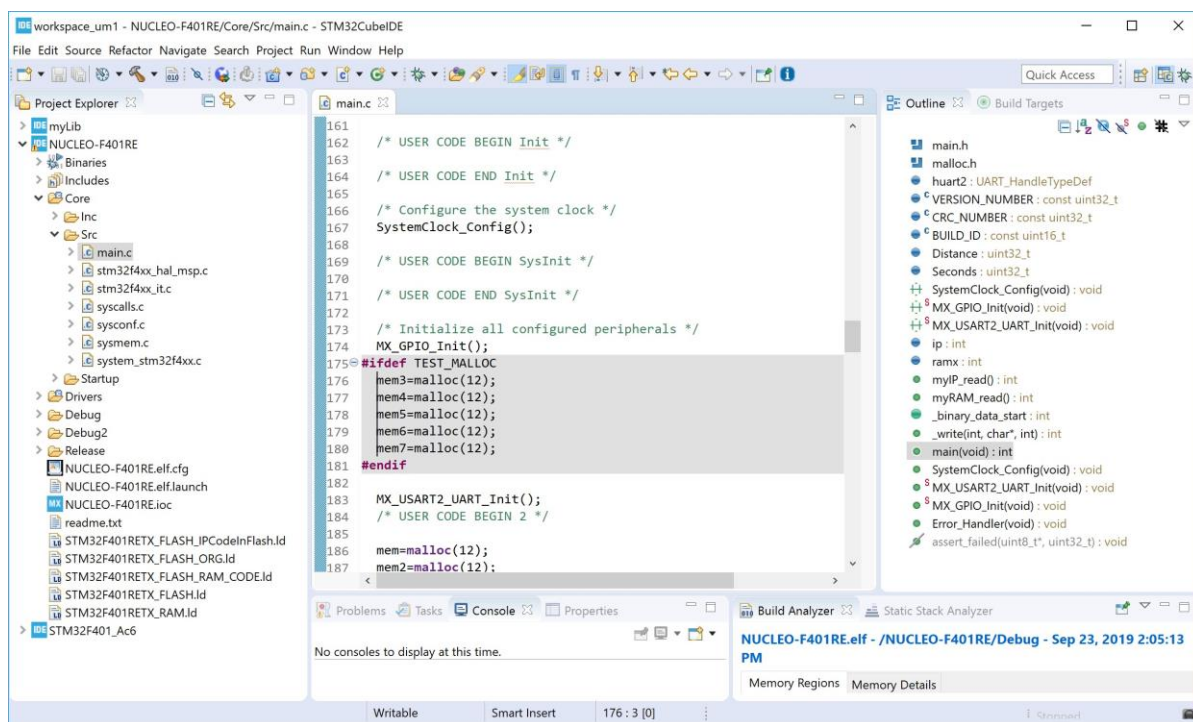
1.8.5 块选择模式

Alt+Shift+A切换选择模式。当块模式被启用时，一个文本块可以由鼠标或键盘使用SHIFT+ARROW按钮选择。

块选择模式的使用

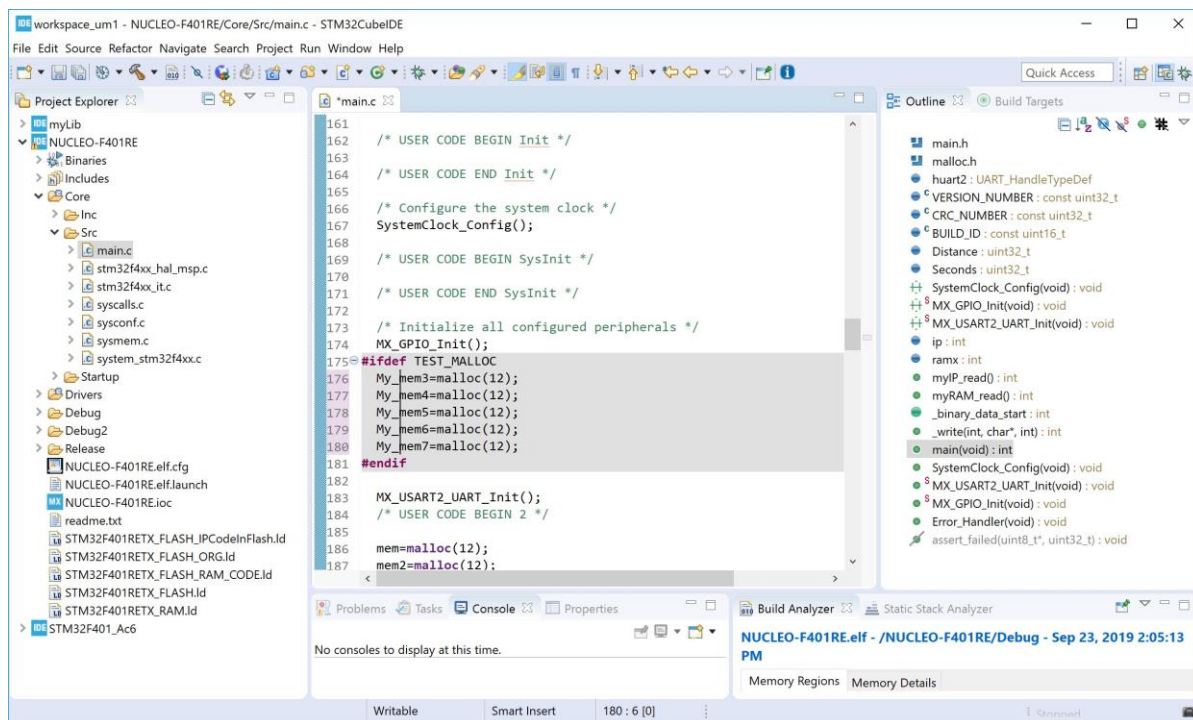
开始使用方块选择模式，按Alt+Shift+A。单击文本中的某处并向下拖动。然后标记一个列，如图32所示。

Figure 32. Editor block selection



添加一些文本，并确保该文本被输入到所有标记的行中。例如，文本“My_”被添加并显示在图33中。

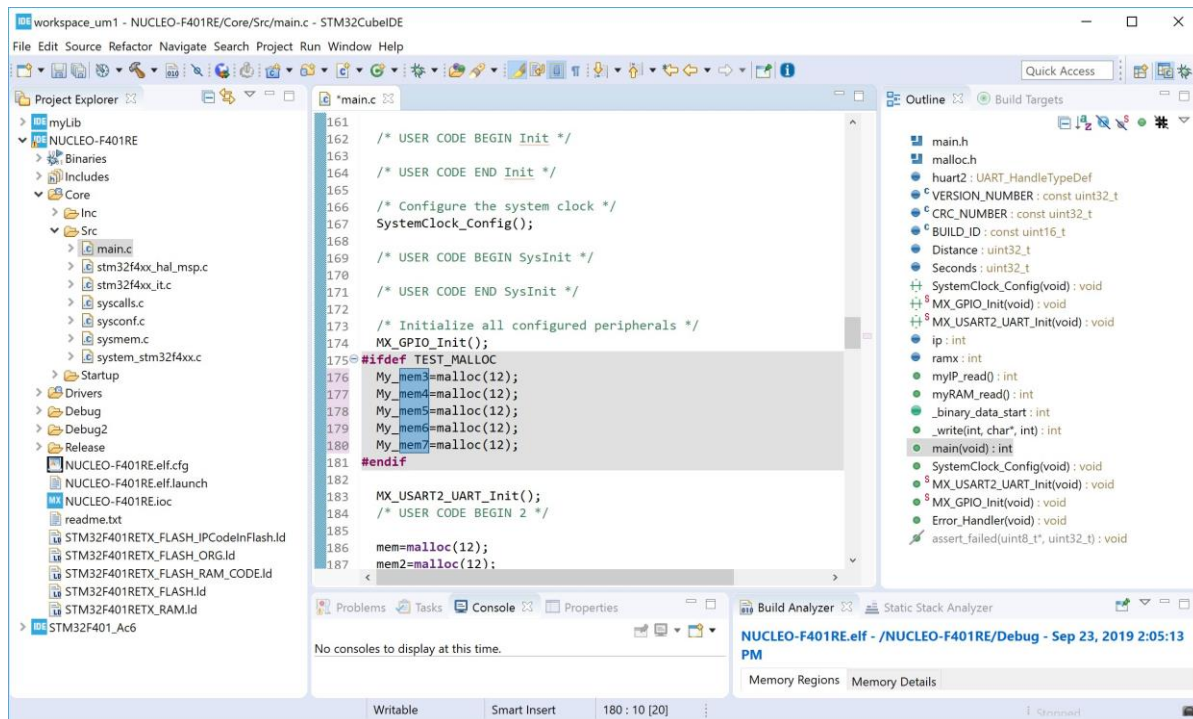
Figure 33. Editor text block addition



区域的选择和编辑

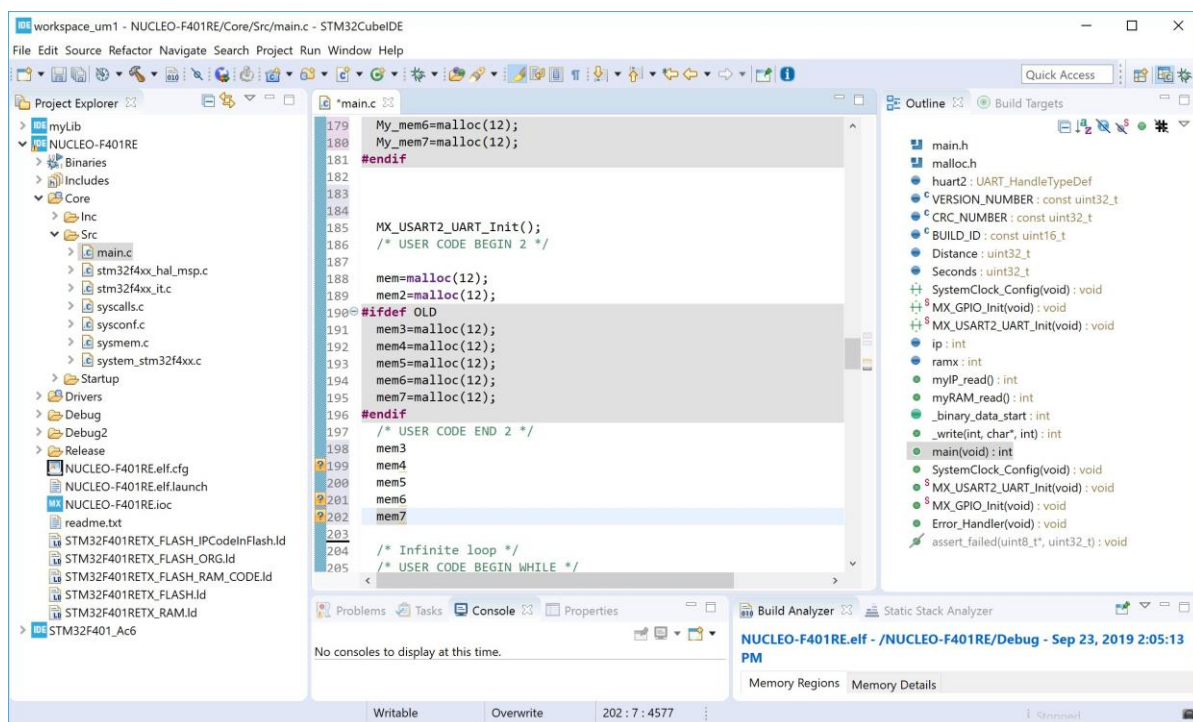
选择一个块。在图34中，选择了以“mem3”到“mem7”开头的块。

Figure 34. Editor column block selection



使用Ctrl+C复制所选的块。然后可以将复制的文本插入到其他地方。为此，输入Alt+Shift+A将选择模式切换回正常模式，将光标移动到另一行，并输入Ctrl+V将复制的列粘贴到新行。

Figure 35. Editor column block paste



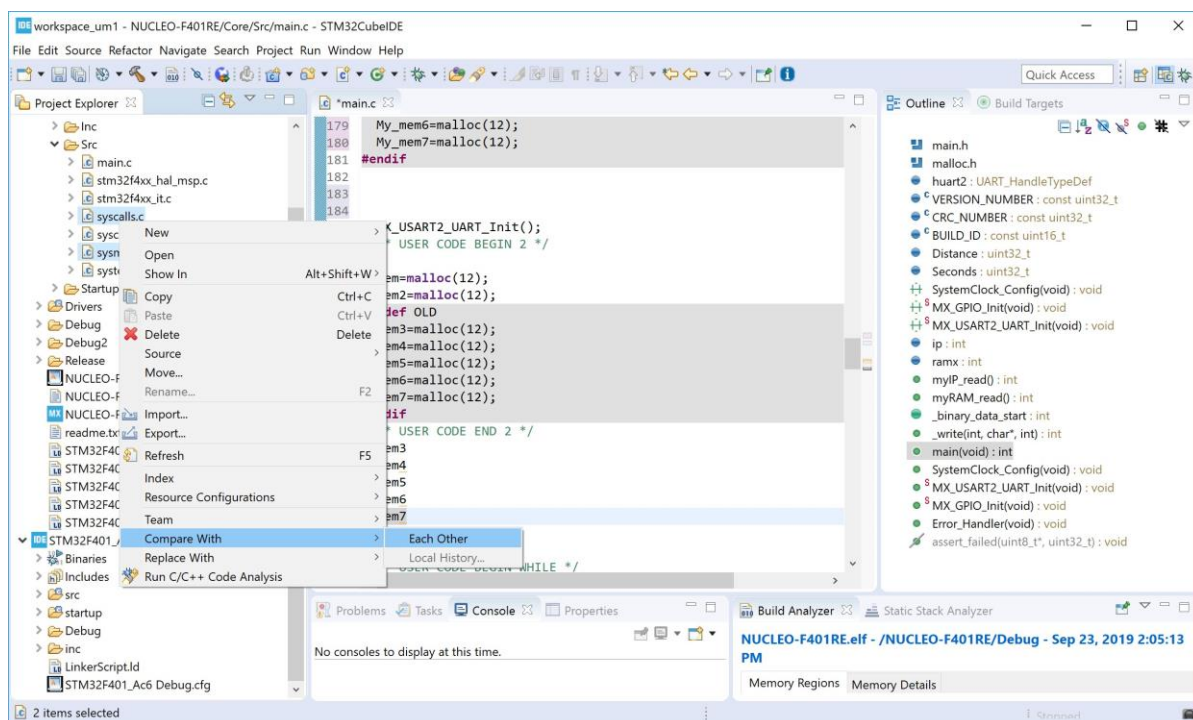
1.8.6 比较文件

在STM32CubeIDE中比较两个文件：

1. Select the two files in the *Project Explorer* view
2. Click on one file
3. Press **CTRL**
4. Click on the other file
Both files are now marked in the *Project Explorer* view
5. Right-click and select **[Compare With]>[Each Other]**

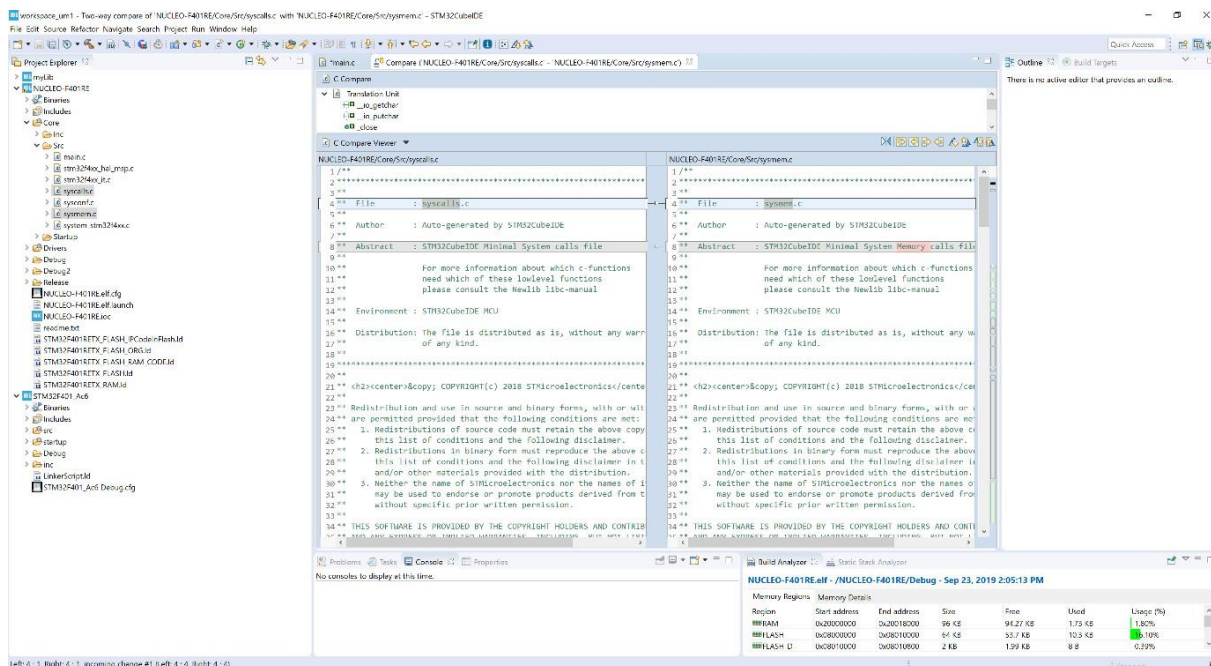
Note: 可以配置如何管理文件的比较。例如，可以在首选项中启用忽略空白。使用[**Window**]>[**Preferences**]打开 Preferences 页面，选择[**General**]>[**Compare/Patch**]，并启用[**Ignore white space**]。

Figure 36. Editor - Compare files



文件差异编辑器打开并比较两个文件。

Figure 37. Editor - File differences

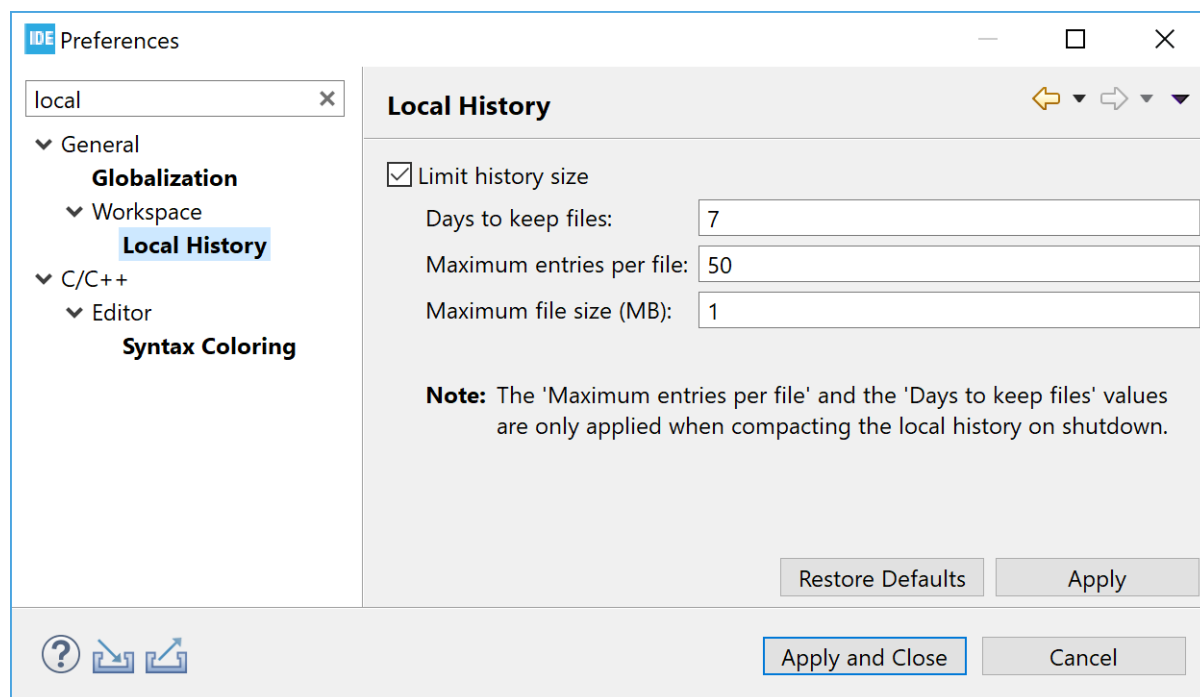


使用导航按钮在差异之间导航，或者简单地使用滚动条在视图中导航以查看文件差异。

1.8.7 本地文件历史记录

建议使用Apache®Subversion®(SVN)或Git™等版本控制系统维护项目。不过，STM32CubeIDE包含一个本地文件，其中包含已编辑文件的历史记录，如果在某个文件失去功能后需要进行一些调查，那么这个文件将非常有用。工作区首选项包含一个Local History页面。

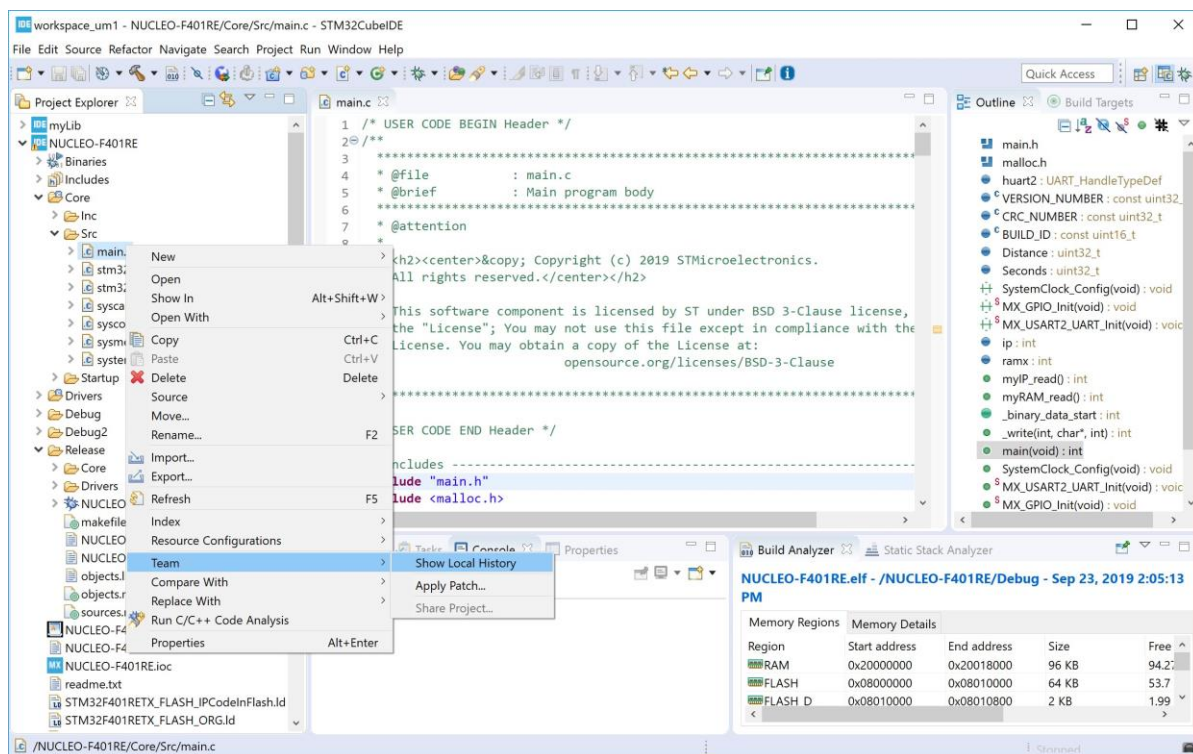
Figure 38. Local history



查询文件的本地历史记录:

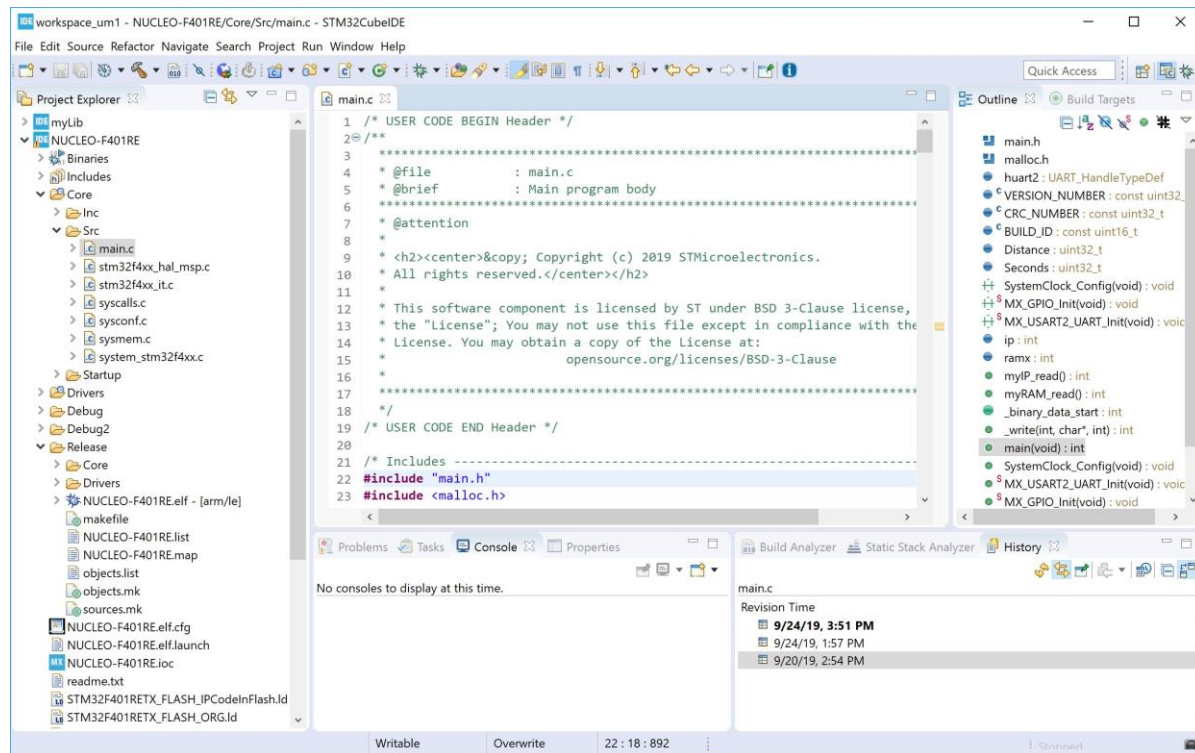
1. Select the file in the *Project Explorer* view
2. Right-click
3. Select **[Team]>[Show local History]**

Figure 39. Show local history



History视图打开并显示文件历史。

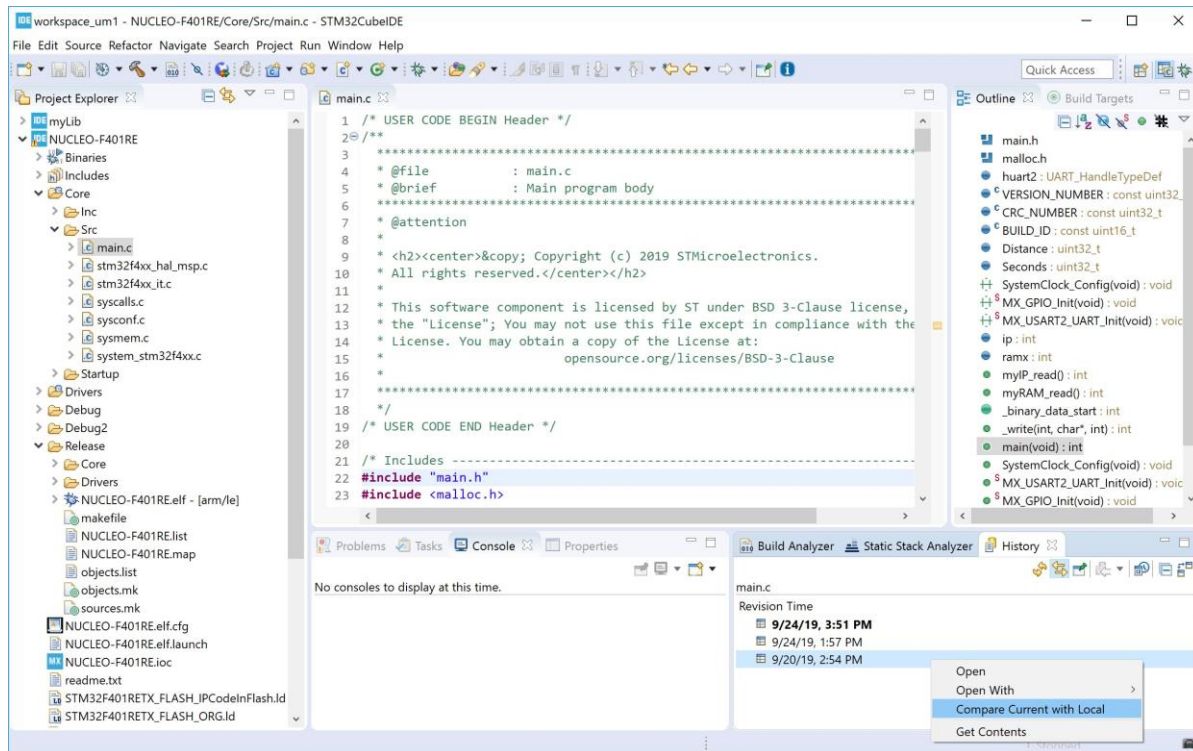
Figure 40. File history



在图40所示的情况中，main.c有三次修订。双击History视图中的文件，在编辑器中打开它。

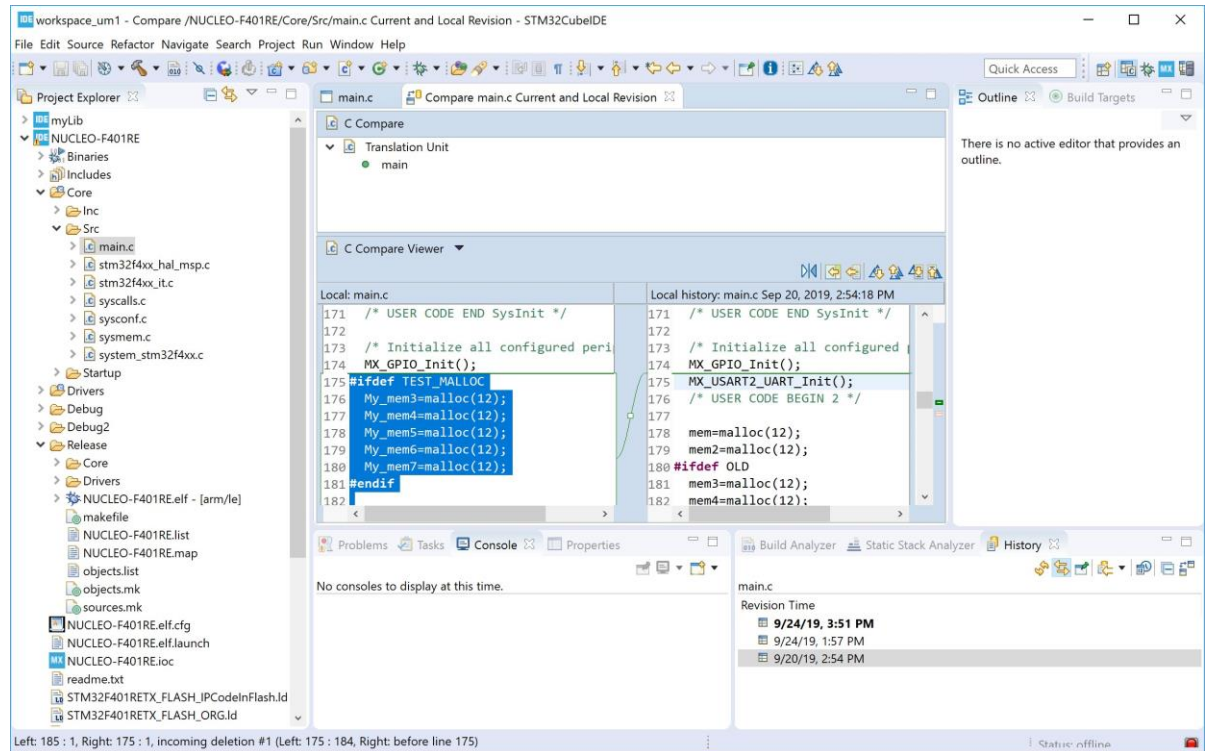
右键单击历史记录中的一个文件，选择[Compare Current with Local]将其与文件的当前版本进行比较

Figure 41. Compare current history with local history



这将打开File Differences编辑器并显示文件更改。

Figure 42. Compare local file differences



2 Creating and building C/C++ projects

正如在1.6节中提到的，工作区是一个包含项目的目录。第一次创建工作区时，它是空的，没有任何项目。需要在工作区中创建或导入项目。本节包含有关如何在工作区中创建项目和构建项目的信息。它还涵盖了如何导入和导出项目。

2.1 项目介绍

项目是工作区中的一个目录，其中包含可以在子目录中组织的文件。可以访问活动工作区中的任何项目。项目中包含的文件不需要物理地位于项目中的文件夹中，但可以位于其他地方并链接到项目中。位于另一个工作空间中的项目不能被访问，除非用户切换到该工作空间或将其中一些项目导入正在使用的工作空间。

可以重命名和删除项目。如果工作空间包含许多项目，也可以关闭其中一些项目以使工作更容易。关闭的项目可以在任何时候重新开放。

本节主要讨论STM32CubeIDE支持的两种类型的STM32项目：

- Executable programs
- Static library projects

但是，STM32CubeIDE所基于的Eclipse®C/ c++开发工具包(CDT™)也包含基本的项目向导，可用于创建C托管构建、c++托管构建和makefile项目：

The STM32 projects can be:

- C or C++
- Generated executable or library file
- Based on [STM32Cube](#) (using STM32 firmware library package) or empty projects

STM32项目还支持先进的伞形项目结构，其中一个项目包含多个项目，例如，对于多核设备，每个核一个项目。

2.2 Creating a new STM32 project

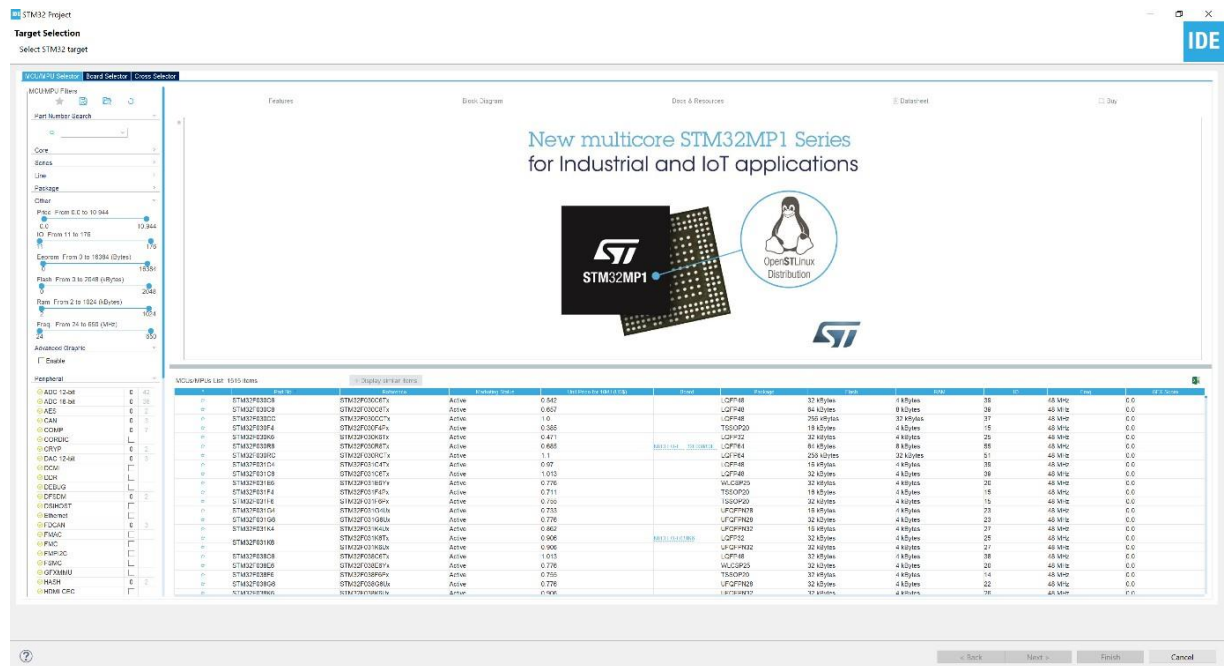
2.2.1 Creating a new STM32 executable project

创建一个新的STM32 C/ c++项目最简单的方法是使用STM32项目向导。通过菜单[File]>[New STM32 Project]选择。

另一种创建新C/ c++项目的方法是打开信息中心并按[启动新的STM32项目]。如1.3信息中心所述，信息中心可以通过工具栏上的按钮  打开，也可以通过菜单[Help]>[Information Center]打开。

这两种方法都初始化和启动STM32项目目标选择工具。

Figure 43. STM32 target selection

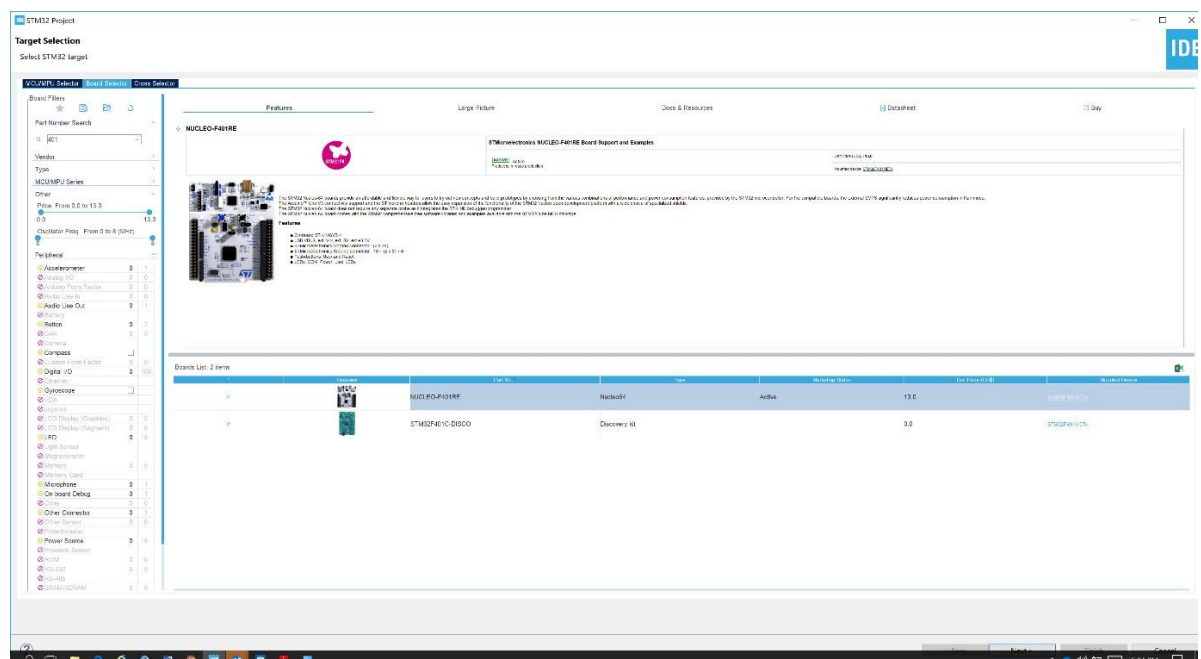


在窗口的顶部可以选择MCU/MPU选择器和Board选择器选项卡。使用第一个选项卡为特定设备创建项目，如果需要为特定板创建项目，则使用第二个选项卡。

本节介绍如何使用板选择器为NUCLEO-F401RE板创建项目。

在窗口左侧可用的不同过滤器中，在Part Number Search字段中键入“401”以过滤名称包含此字符串的板。在图44中，列出了两块板，一个Nucleo板和一个Discovery板。选中“nuclear - f401re”单板。

Figure 44. STM32 board selection



五个选项卡，功能，大图片，文档和资源，数据表和购买，提供了显示所选单板或设备的详细信息的可能性。例如，当选择“文档和资源”时，可用于该板的文档将显示出来并可以打开。选择“Datasheet”时，单板数据表从STMicroelectronics网站下载。

当选择nuclear - f401re板时，按[Next]将打开项目设置页面。

在对话框中输入项目名称并为项目选择所需的设置。图45中填写了名为“nuclear - f401re”的项目作为示例。

Figure 45. Project setup

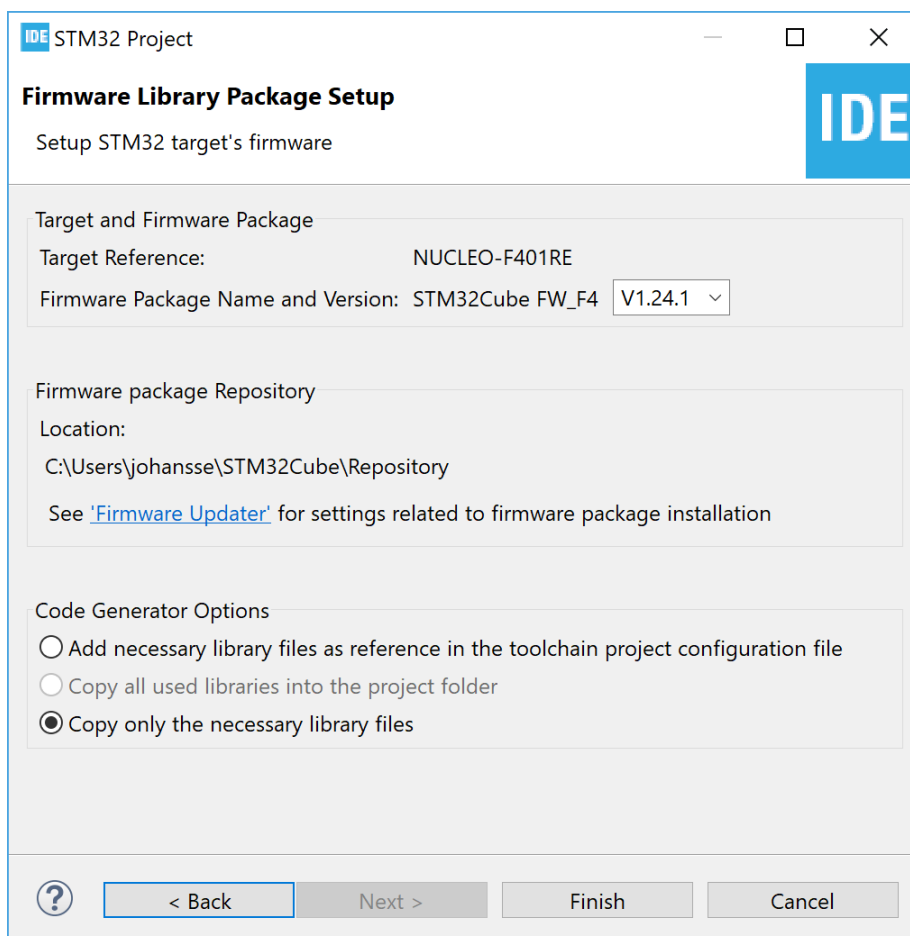
The screenshot shows the 'Project Setup' dialog box in the STM32CubeIDE. The 'Project Name' field is filled with 'NUCLEO-F401RE'. The 'Use default location' checkbox is checked. The 'Location' field shows the default path 'C:/Users/johansse/STM32CubeIDE/workspace_um'. Under the 'Options' section, 'Targeted Language' is set to 'C', 'Targeted Binary Type' is set to 'Executable', and 'Targeted Project Type' is set to 'STM32Cube'. The 'Finish' button is highlighted.

根据图45中的设置，该项目将存储在默认位置，并设置以下选项：

- C project
- Executable binary type
- STM32Cube targeted project type

按[Next]进入Firmware Library Package Setup界面。

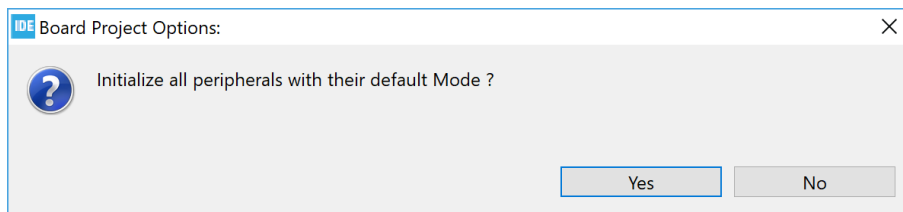
Figure 46. Firmware library package setup



在此页中，可以选择创建项目时要使用的STM32Cube固件包。在这种情况下，使用默认设置。按[Finish]创建项目。

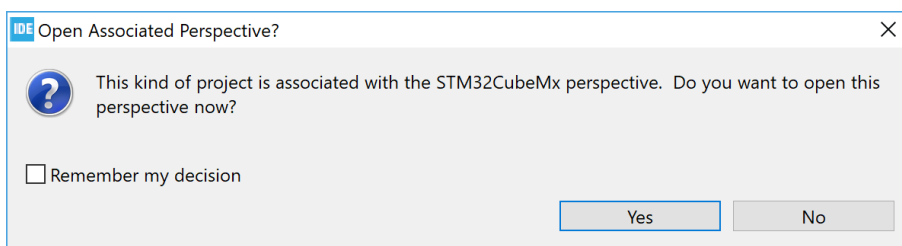
结果显示如下对话框。

Figure 47. Initialization of all peripherals



按[Yes]，因为获得初始化外围设备所需的软件是一个很好的实践。这将打开如图48所示的新对话框。

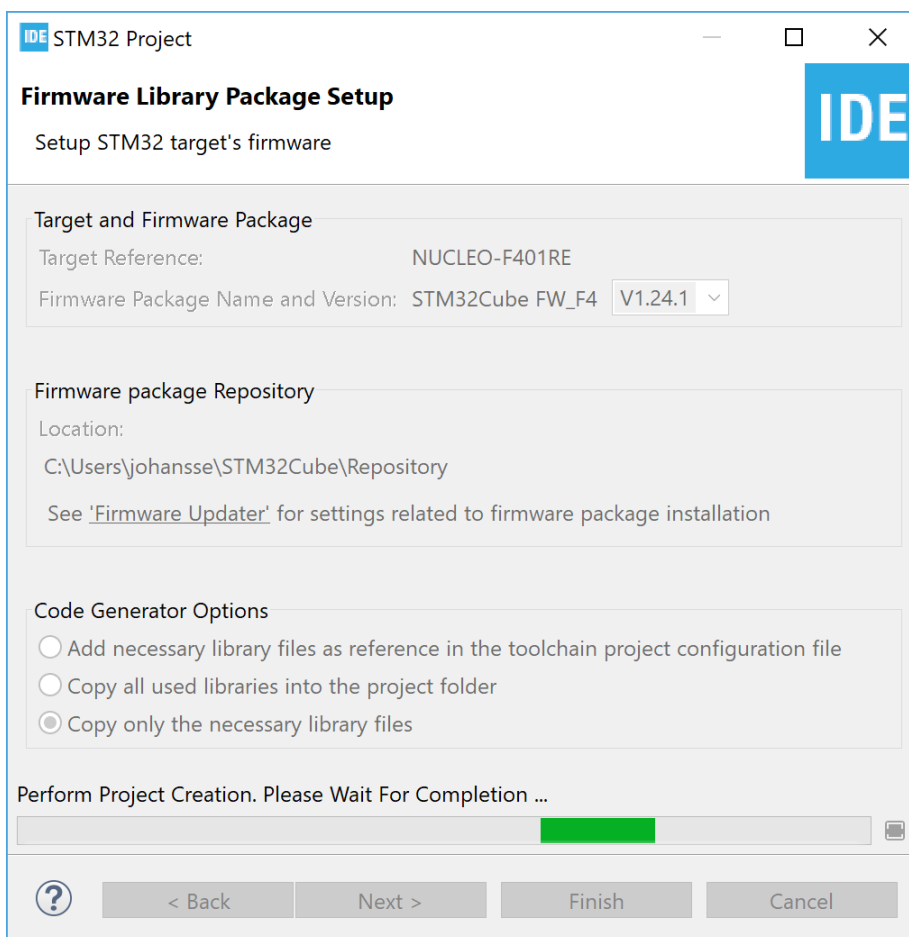
Figure 48. STM32CubeMX perspective opening



如果需要配置设备，打开STM32CubeMX透视图是一个很好的决定。如果下次创建新项目时必须不询问这个问题，启用[Remember my decision]。按[Yes]继续。

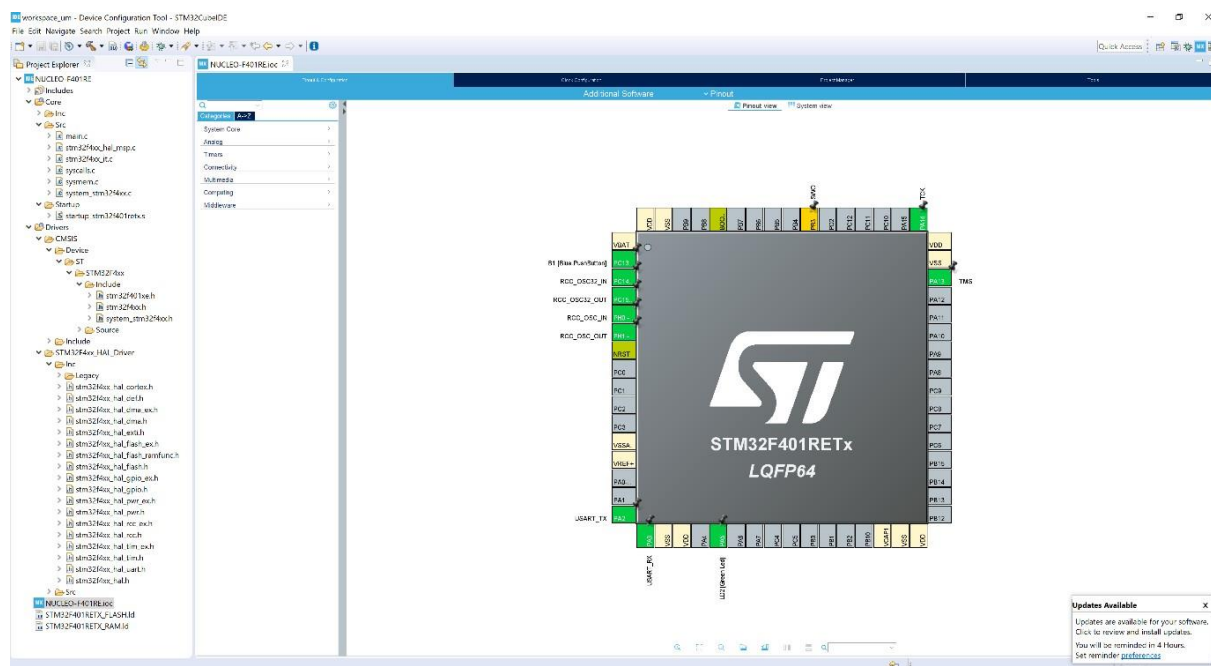
此时，项目创建开始。所需的时间取决于创建项目所需下载的文件数量。

Figure 49. Project creation started



在创建项目时，打开STM32CubeMX透视图，并打开一个用于配置外设、时钟、中间件和功耗的窗口。

Figure 50. STM32CubeMX



新项目在project Explorer视图中列出，其中包含一些文件夹和文件。

NUCLEO-F401RE.ioc文件包含配置设置，在STM32CubeMX编辑器中打开。这个编辑器包含Pinout &配置，时钟配置，项目管理和工具选项卡。当在STM32CubeMX编辑器中进行更改时，选项卡中的.ioc文件被标记为已更改。如果保存了文件，将打开一个对话框，询问“您想生成代码吗？”，这样就很容易在项目中生成支持新设备配置的新代码。有关如何使用STM32CubeMX编辑器的更多信息，请参阅[ST-14]。

通过选择目标项目类型[Empty]而不是[STM32Cube]，可以创建具有更少文件和文件夹的STM32项目（参见图45）。项目设置）。当选择[Empty]时，生成的项目只包含一些文件夹，一个带有Reset_Handler代码和向量的设备启动文件，main.c文件，以及其他一些c文件和链接器脚本文件。STM32头文件、系统文件和CMSIS文件必须手动添加。例如，这些文件可以从其他STM32Cube目标项目或STM32示例项目复制。

Note: 对于空项目，请确保根据应用需求配置浮点单元设置为使用软件FPU或硬件FPU。使用硬件FPU时，需要初始化FPU。对于非空项目，FPU的初始化通常在system_stm32fxxx.c文件中的SystemInit函数中完成。为了通知可能需要FPU配置，在空项目中创建的主文件包含一个编译器警告，声明#warning "FPU is not initialized, but the project is compiling for an FPU. Please initialize the FPU before use."

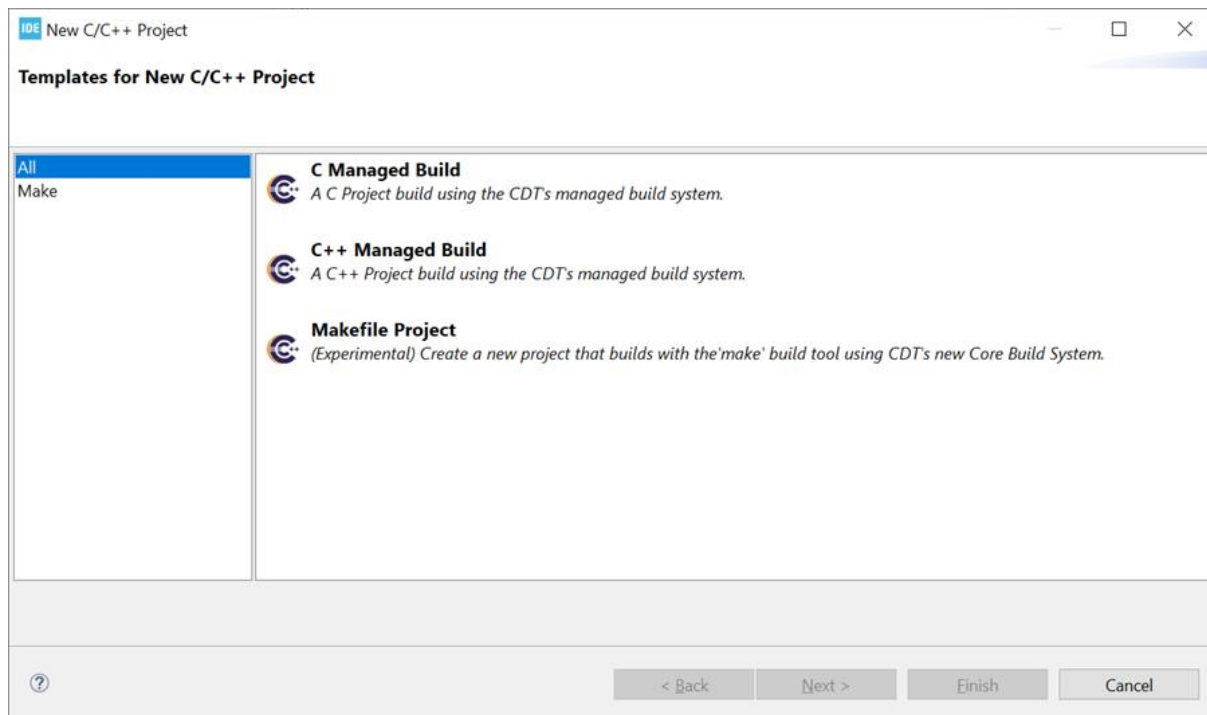
2.2.2 Creating a new STM32 static library project

第2.2.1节中描述的创建一个新的STM32可执行项目的方法也可以用于创建一个静态库项目。然而，静态库经常在多个应用程序项目中重用，可能针对不同的STM32产品。STM32项目向导不支持修改MCU设备。带有一些STM32相关扩展的标准Eclipse®/CDT™项目解决方案支持在每次构建配置时更改MCU设备。

因此，创建静态库项目的推荐方法是依赖于Eclipse®/CDT™项目，如下所述。

要创建Eclipse®/CDT™，请转到[File]>[New]>[C/C++ Project]。这将打开如图51所示的窗口。

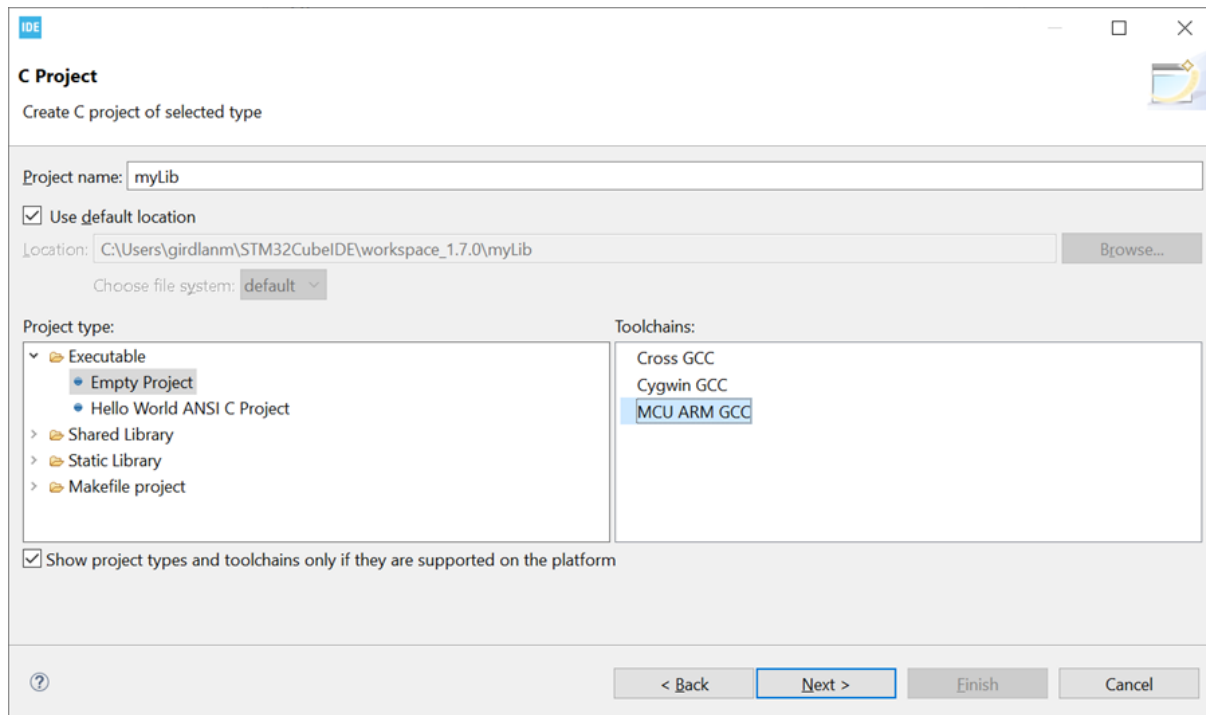
Figure 51. New C/C++ project



根据项目需要选择C Managed Build或c++ Managed Build，然后单击[Next]。这将调出项目类型选择器。空项目类型是唯一支持MCU ARM GCC工具链的类型。确保在可执行文件夹下选择Empty Project，然后选择MCU ARM GCC工具链，如图52所示。

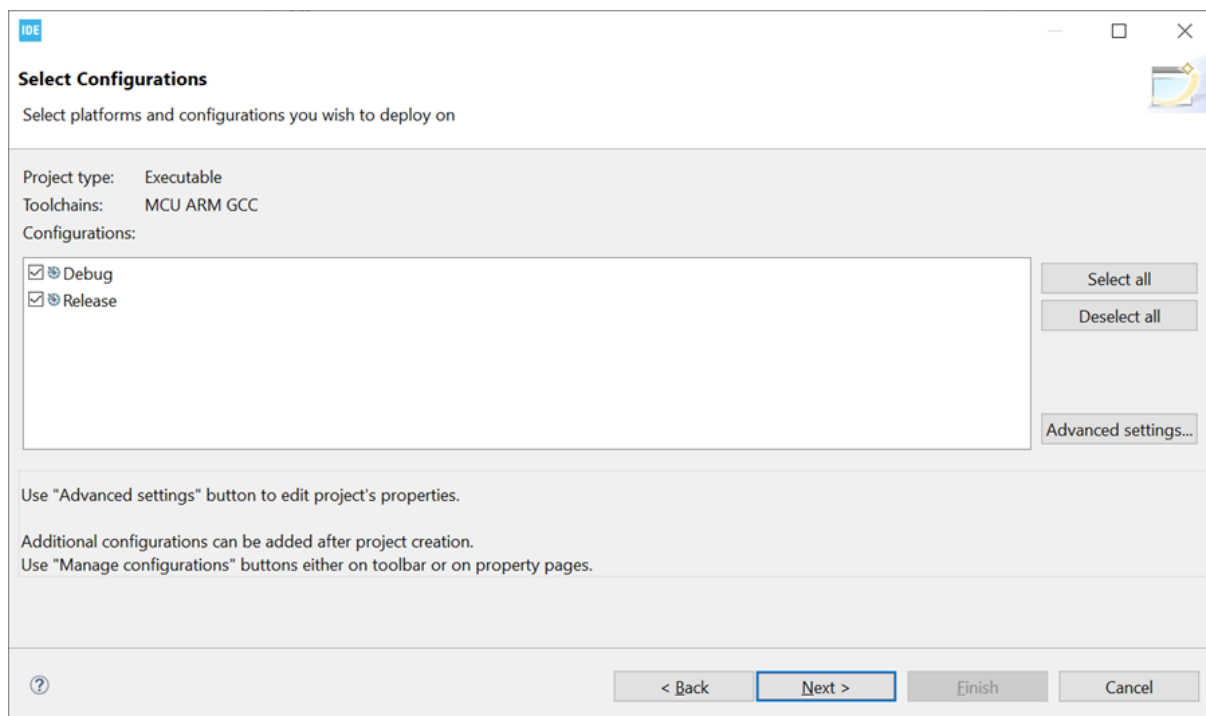
一旦项目命名和类型选择完成，单击[Next]

Figure 52. Project type



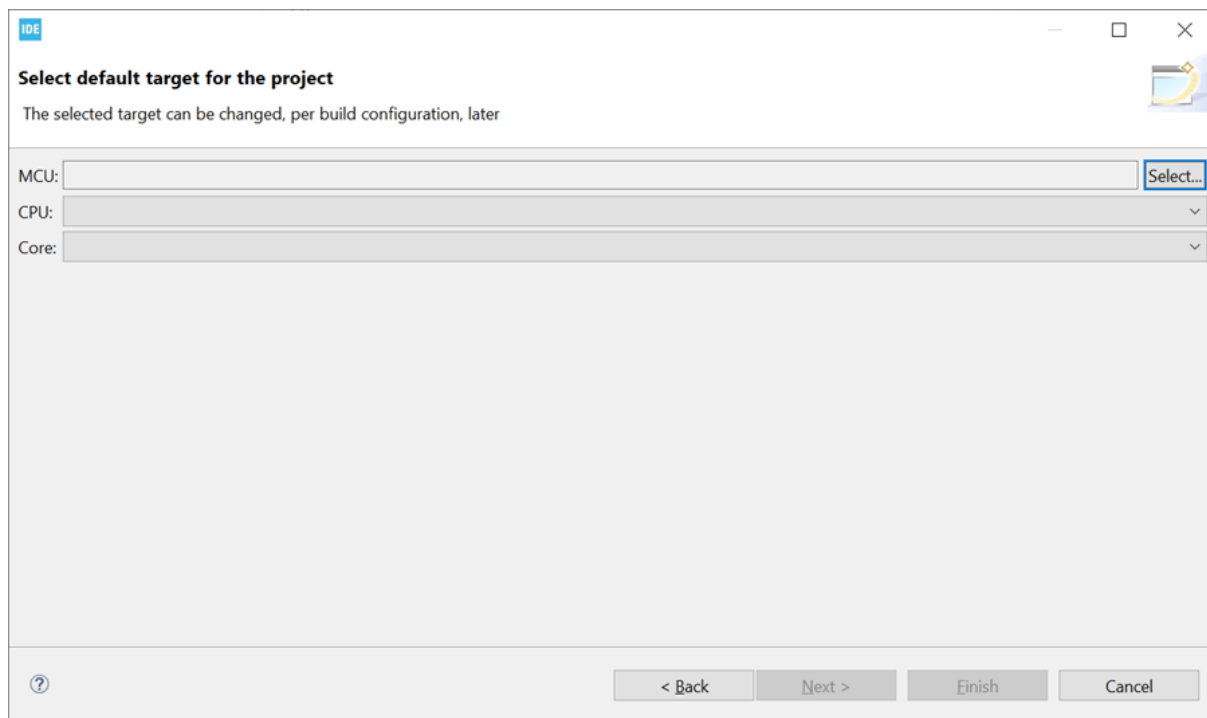
这将启动一个标准的Eclipse®项目配置窗口，如图53所示，单击[Next]。

Figure 53. Project configuration selection



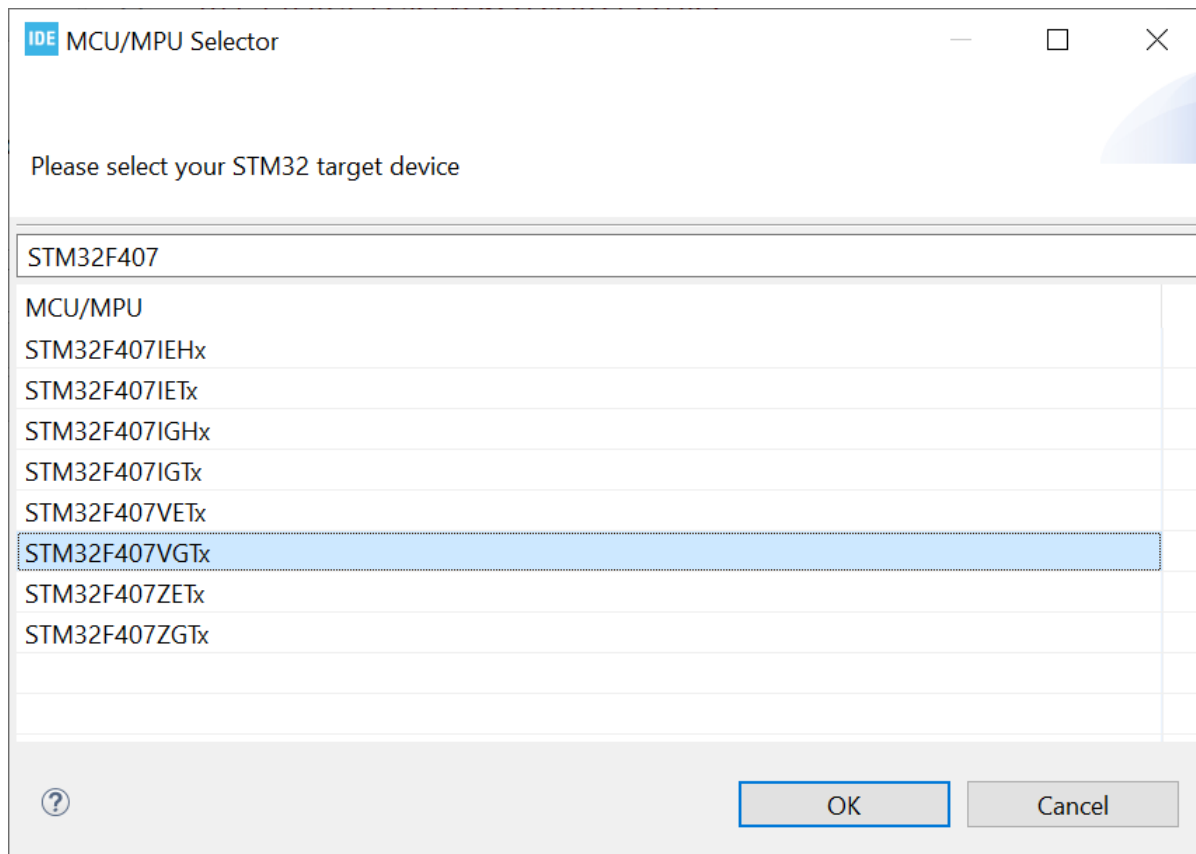
在图54所示的目标选择器屏幕中，通过单击[select...]按钮并为项目过滤正确的目标，确保选择了适当的目标。目标选择器有助于在前面步骤中已经定义的构建配置中正确地设置-mcpu=cortex-mX工具链标志(参见图53)。

Figure 54. Project default target selector



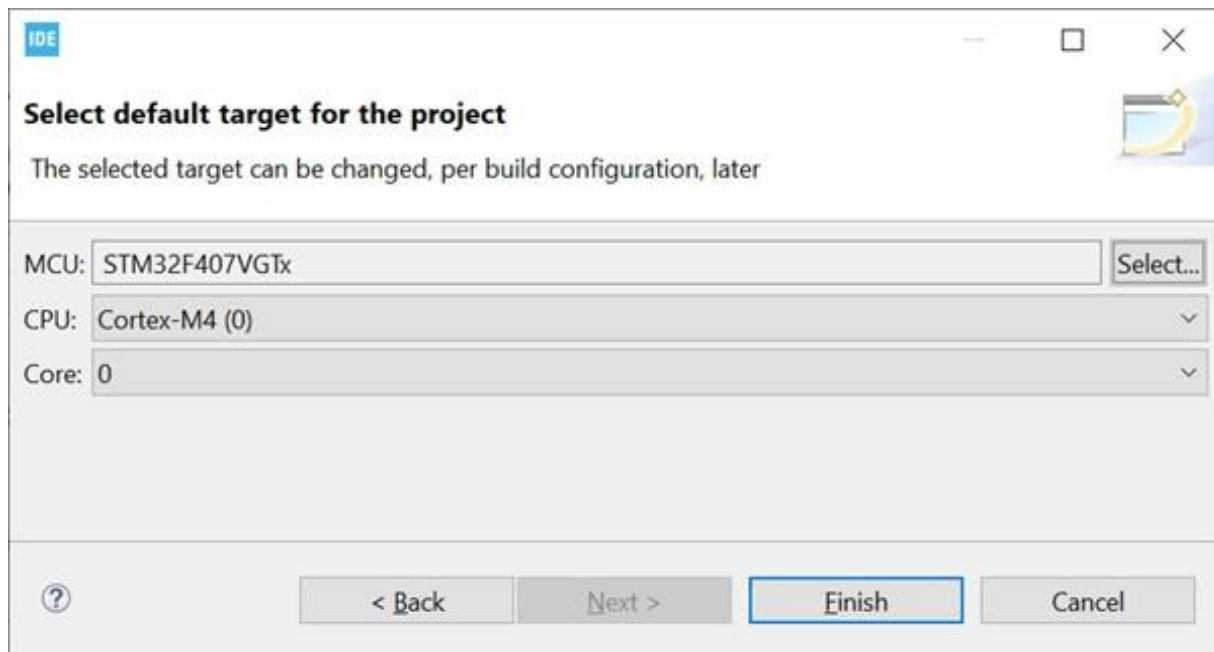
当点击[Select...], 就会出现如图55所示的筛选对话框, 允许用户筛选并选择正确的设备。

Figure 55. Project MCU/MPU selector



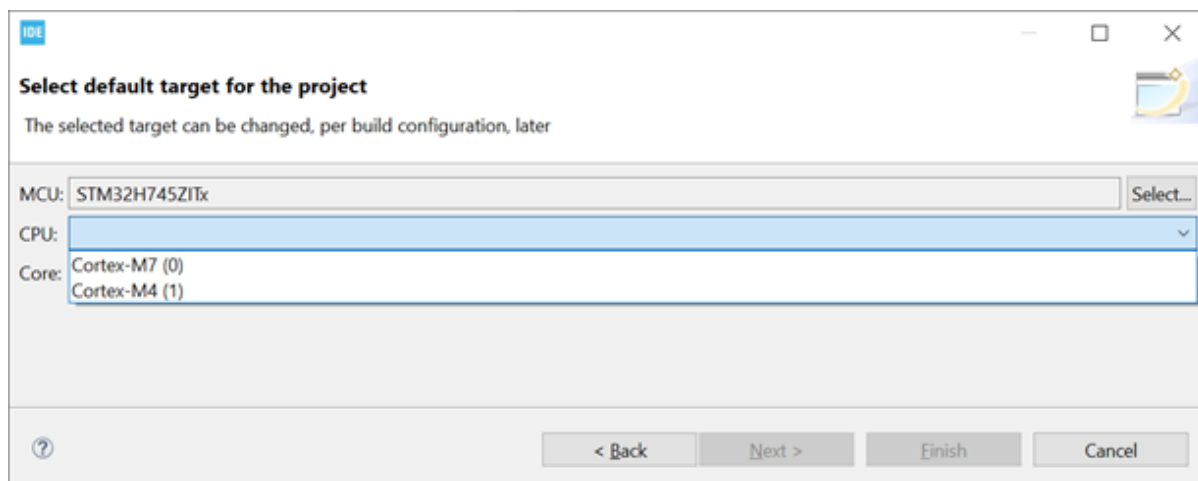
在目标选择(MCU)之后，CPU和Core字段将在简单的单核案例中自动填充，如图56所示。

Figure 56. Project target selection



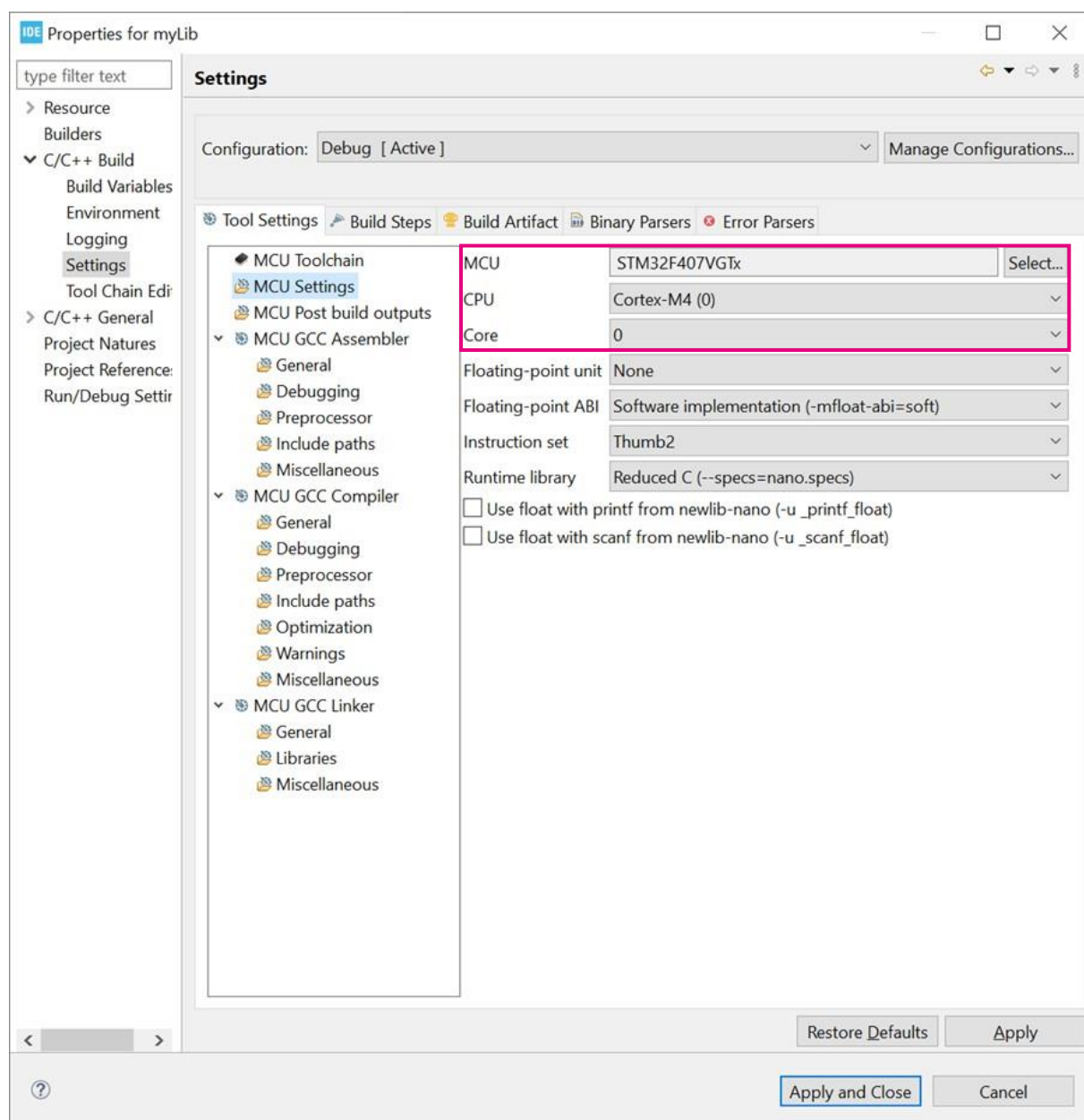
对于更高级的设备，如多核STM32H7微控制器，用户必须选择项目目标的CPU和Core，以确保正确构建代码。稍后还将使用这些设置正确地设置调试配置。确保设置是项目所需的，然后单击[Finish]。

Figure 57. Project target selection (advanced)



在项目创建之后，可以为不同的目标创建不同的构建配置，如2.3.1.2节所述。创建新的构建配置后，在项目资源管理器中右键单击项目，进入[Properties]>[C/C++ Build]>[Settings]>[Tool Settings]>[MCU Settings]，然后单击[Select...]为特定的生成配置选择一个新目标。

Figure 58. Project target change



Tip: CDT™项目还可以用于生成目标独立应用程序或可执行项目。

2.3 Configure the project build setting

创建STM32项目时，它包含该项目的默认C/ c++生成设置。但是GCC可以使用很多不同的选项，每个嵌入式系统都有自己的需求。因此，可以进一步配置项目生成设置，而不是默认生成设置。

在项目发展的不同阶段，通常对建造环境有不同的要求；例如在调试和发布阶段。为了解决这个问题，STM32CubeIDE支持每个项目的不同构建配置。本节首先介绍生成配置，然后是项目生成设置。

2.3.1 Project build configuration

每个生成配置允许项目的不同变体，并包含特定的生成设置。在STM32CubeIDE中创建STM32项目时，默认会创建两个构建配置，Debug和Release。Debug配置使用调试信息构建项目，而不进行任何优化。发布配置使项目优化为更小的代码大小和没有调试信息。默认情况下，在创建项目时，Debug配置被设置为活动生成配置。

可以在任何时候为项目创建新的构建配置。这种新的生成配置可以基于以前可用的生成配置。

在构建项目时，将使用活动构建配置，在构建期间将生成的文件写入与活动构建配置同名的文件夹中。

Note: 生成配置只处理生成设置。如何配置调试设置将在本手册后面介绍。

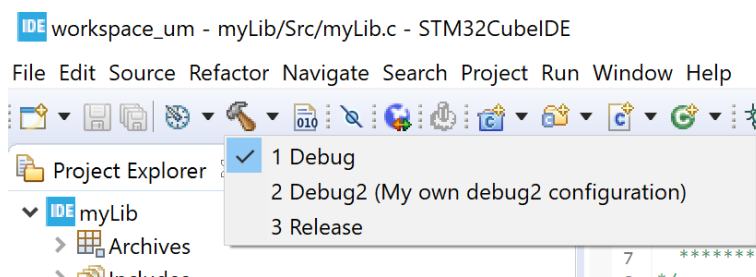
2.3.1.1 Change the active build configuration

To change the active build configuration:

1. Select the project name in the *Project Explorer*
2. Use the toolbar in the C/C++ perspective and click on the arrow to the right of the **[Build]** toolbar button

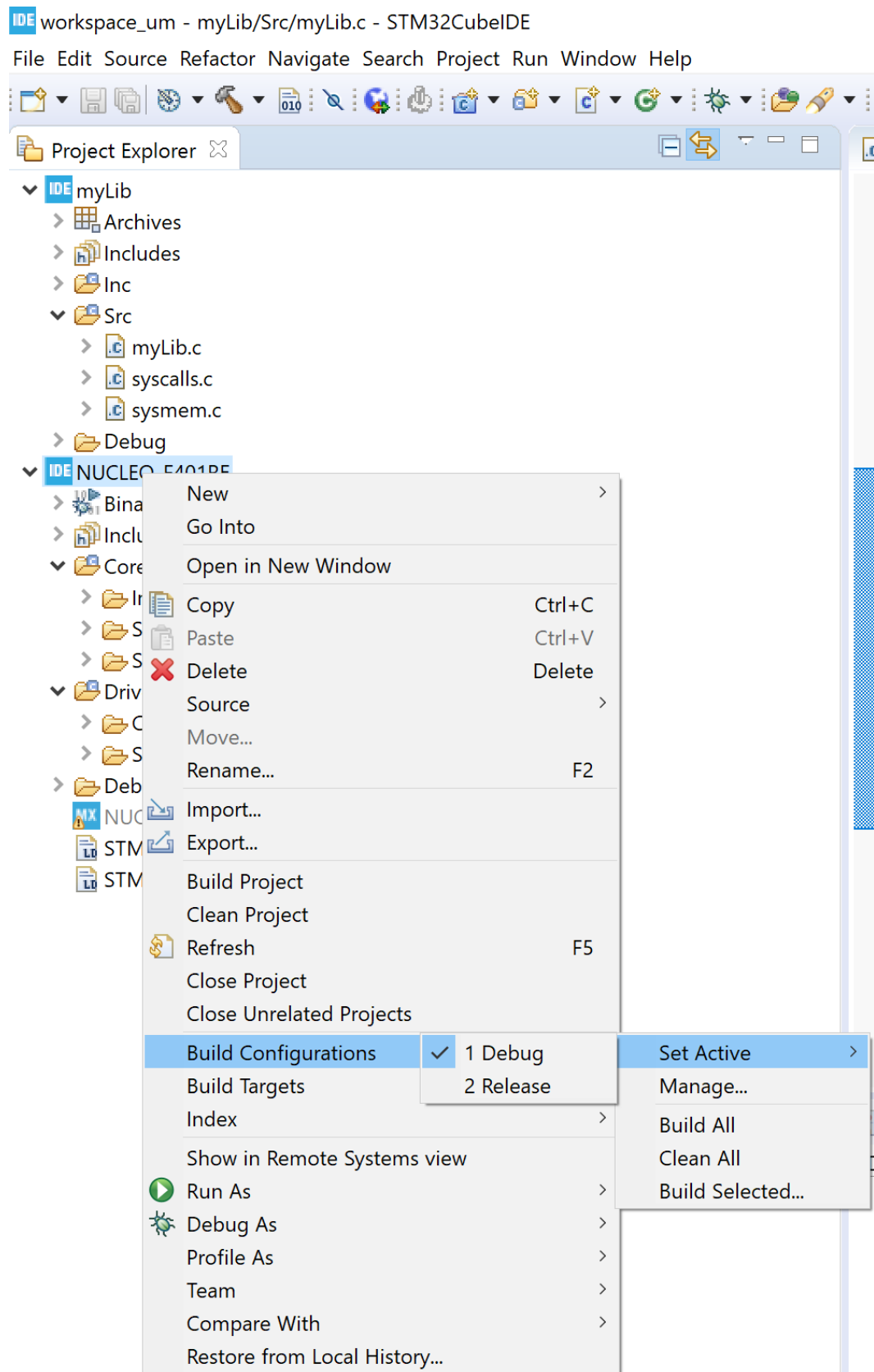
3. The build configurations are listed
Select the build configuration to use from the list.

Figure 59. Set the active build configuration using the toolbar



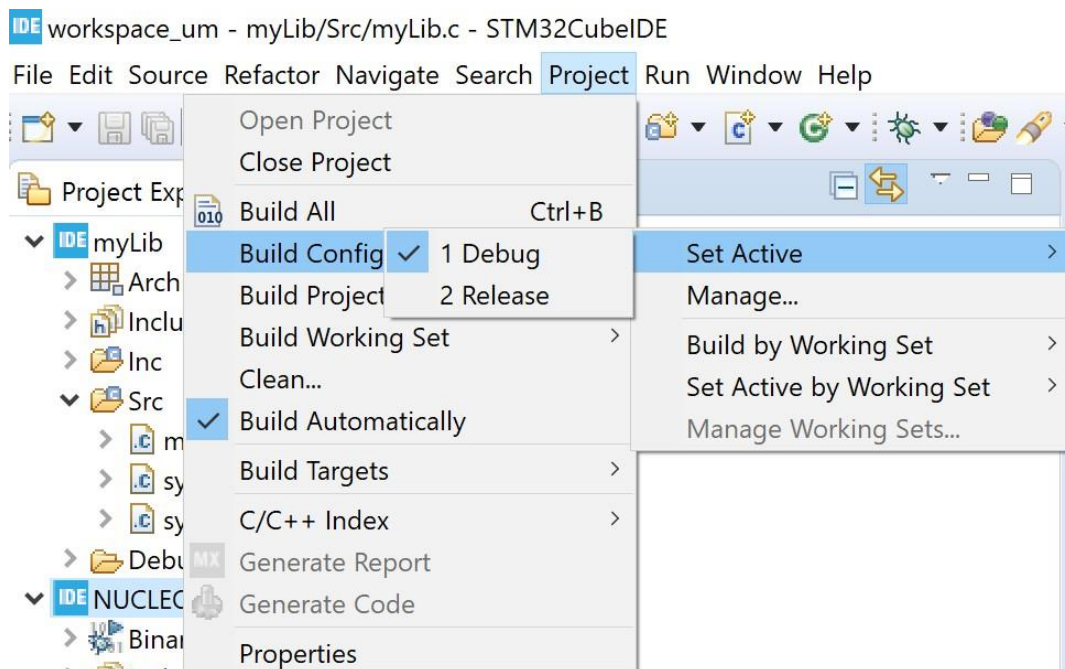
更改活动生成配置的另一种方法是在project Explorer视图中右键单击项目名称，选择[Build Configurations]>[Set Active]，并选择首选的生成配置。

Figure 60. Set active build configuration using right-click



也可以使用菜单[Project]>[Build Configurations]>[Set Active]选择激活的构建配置，并选择所选的构建配置。

Figure 61. Set active build configuration using menu



2.3.1.2

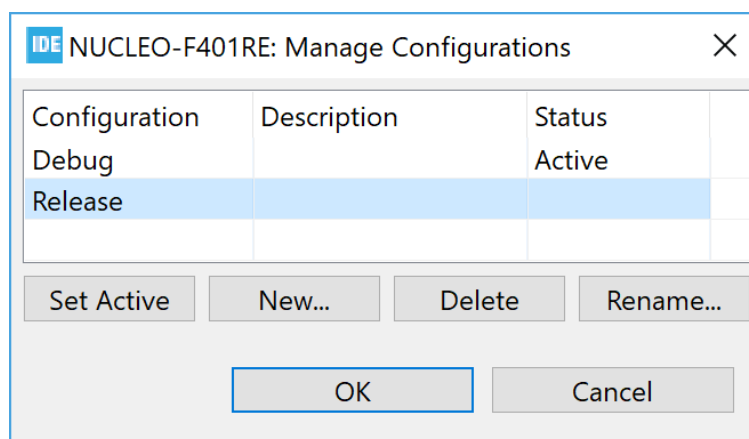
Create a new build configuration

To create a new build configuration:

1. Right-click on the project name in the *Project Explorer* view
2. Either:
 - Select [Build Configurations]>[Manage...]
 - Use the menu [Project]>[Build Configurations]>[Manage...]

Both methods open the *Manage Configurations* dialog.

Figure 62. Manage Configurations dialog



如图62所示，对话框中的一些按钮用于管理配置：

- **[Set Active]** is used to change and select another configuration to be active
- **[New...]** is used to create a new build configuration
- **[Delete]** is used to delete an existing build configuration
- **[Rename...]** is used to rename the build configuration

要创建一个新的构建配置，请按[new...]按钮。这将打开Create New Configuration对话框。在此对话框中，将输入名称和描述。该名称必须是有效的目录名称，因为在使用新配置构建项目时将其用作目录名称。

Figure 63. Create a new build configuration

如图63所示，新的构建配置基于现有的构建配置。在本例中，新配置基于现有的Debug配置。设置完成后按[OK]。

Manage Configurations对话框打开并显示新的调试配置。

Figure 64. Updated Manage Configurations dialog

Configuration	Description	Status
Debug		Active
Debug2	My own debug con...	
Release		

如果需要，将激活配置更改为其他配置，完成配置管理后，按[OK]保存并关闭配置对话框。

2.3.1.3

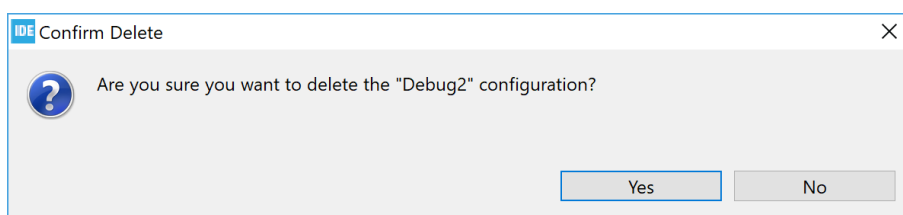
Delete a build configuration

To delete a build configuration:

1. Open the *Manage Configurations* dialog
2. Select the configuration to be deleted
3. Press the **[Delete]** button

For instance, if the *Debug2* configuration is selected and **[Delete]** button is pressed, the following confirmation dialog opens.

Figure 65. Configuration deletion dialog



In this case, select **[No]** to keep the *Debug2* configuration.

2.3.1.4

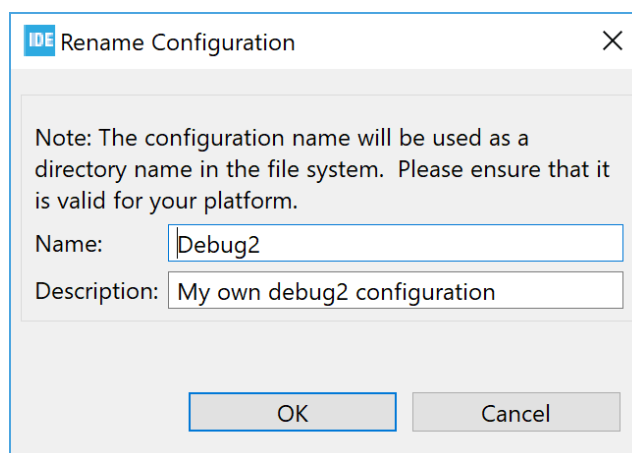
Rename a build configuration

To rename a build configuration:

1. Open the *Manage Configurations* dialog
2. Select the configuration to be renamed
3. Press the **[Rename...]** button

For instance, if the *Debug2* configuration is selected and **[Rename...]** button is pressed, the following confirmation dialog opens.

Figure 66. Configuration renaming dialog



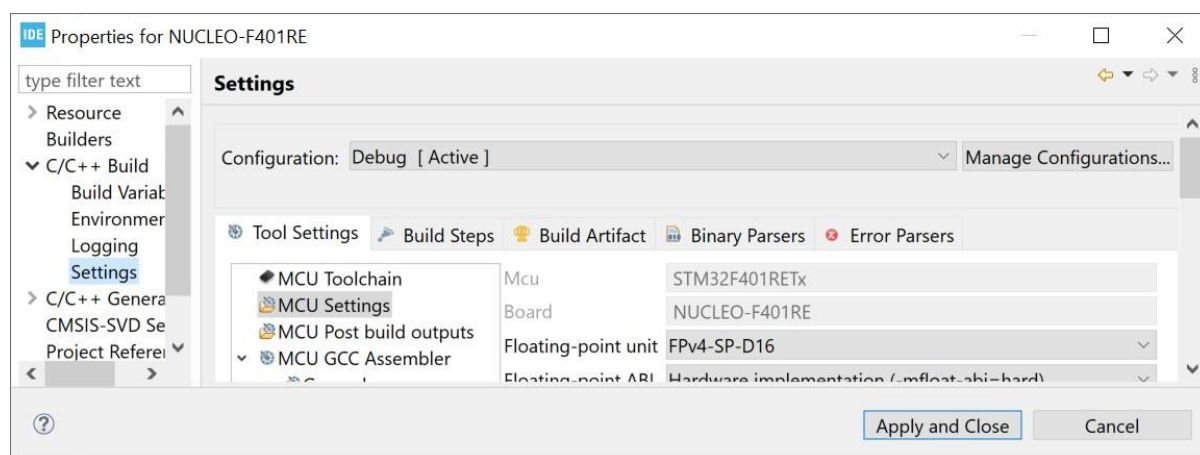
Update the name, description, or both and press **[OK]** to rename the *Debug2* configuration. In this case, press **[Cancel]** and keep the name.

2.3.2 Project C/C++ build settings

每个生成配置包含一个项目C/ c++生成设置。在项目属性中更新项目C/ c++生成设置。要更新构建设置，在project Explorer视图中右键单击项目名称并选择[Properties]或使用菜单[project]>[Properties]。这两种方法都可以打开项目的Properties窗口。

在左边的属性窗格中选择[C/C++ Build]>[Settings]。右边的部分是工具设置、构建步骤、构建工件、二进制解析器和错误解析器选项卡。前两个选项卡是最有用的。

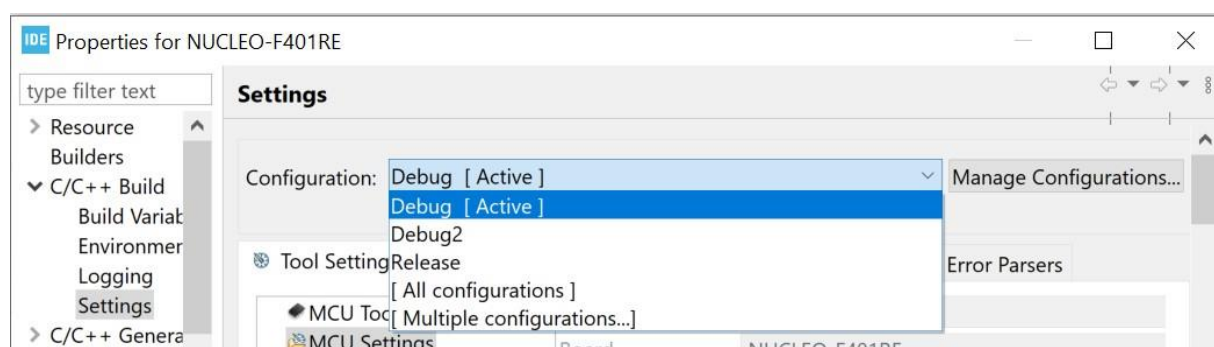
Figure 67. Properties tabs



Note: 调整对话框窗口的大小，如果所有选项卡都不可见，则使用右上角的箭头按钮。

“设置”窗格包含一个[Configuration]选项，用于决定新选项是否仅在活动配置中使用，在另一个配置中使用，在所有配置中使用，还是在多个配置中使用。2.按[Manage Configurations]进入“Manage Configurations”对话框。

Figure 68. Properties configurations

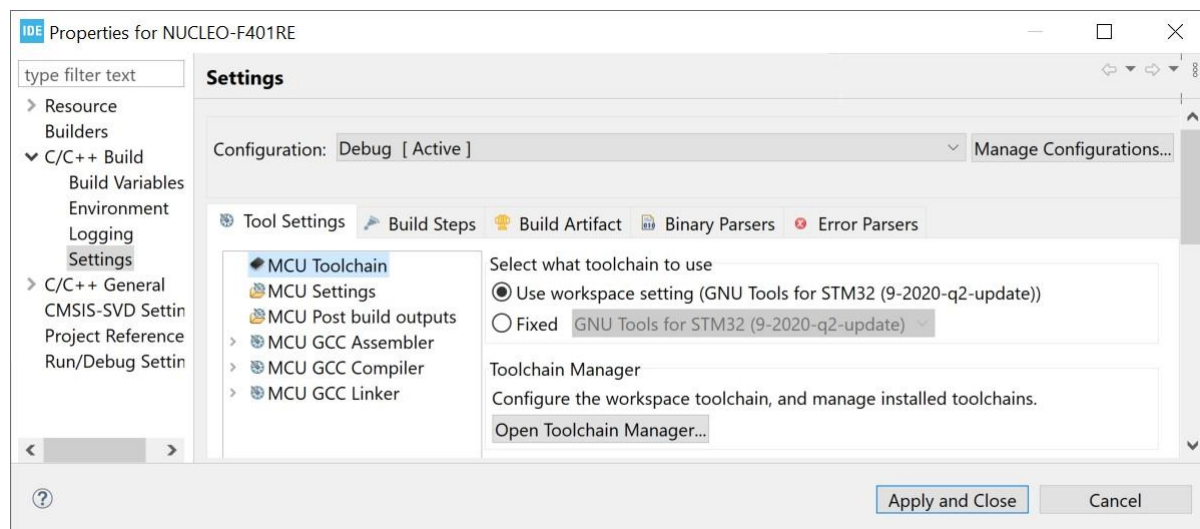


工具设置选项卡进一步分为MCU工具链，MCU设置，MCU后构建输出，MCU GCC汇编器，MCU GCC编译器和MCU GCC链接器。

MCU工具链用于更改工具链。STM32CubeIDE包含STM32工具链的GNU工具的一个版本。工具链管理器用于下载其他GNU ARM嵌入式工具链，并配置为使用本地GNU ARM嵌入式工具链。

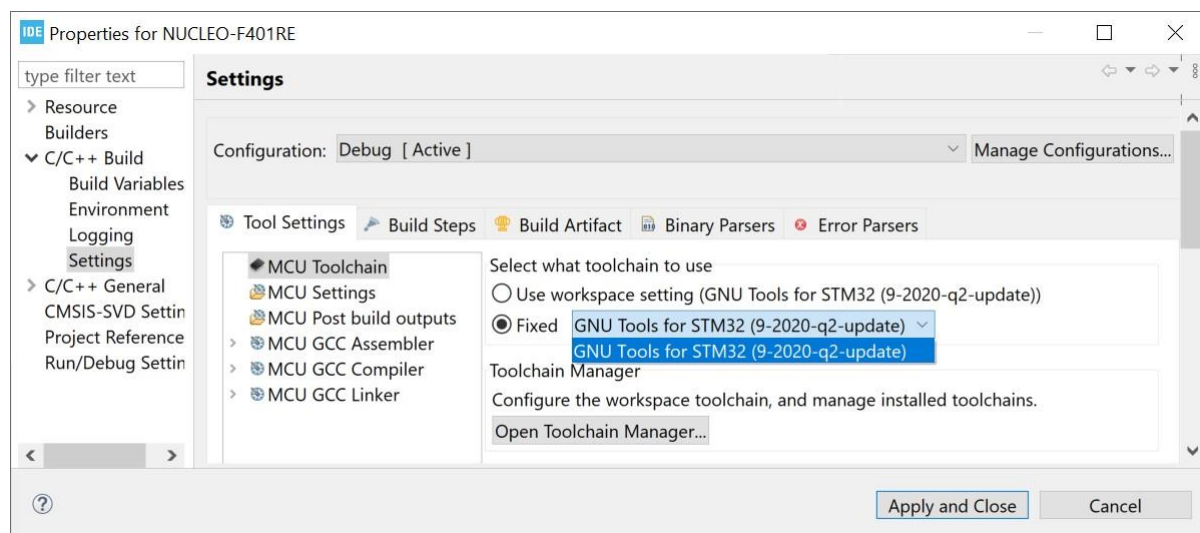
关于在STM32的GNU工具中制作的补丁的信息可以在[EXT-12]中阅读。您可以从信息中心的技术文档页面打开该文档。

Figure 69. Properties toolchain version



选择[Fixed]启用工具链选择。

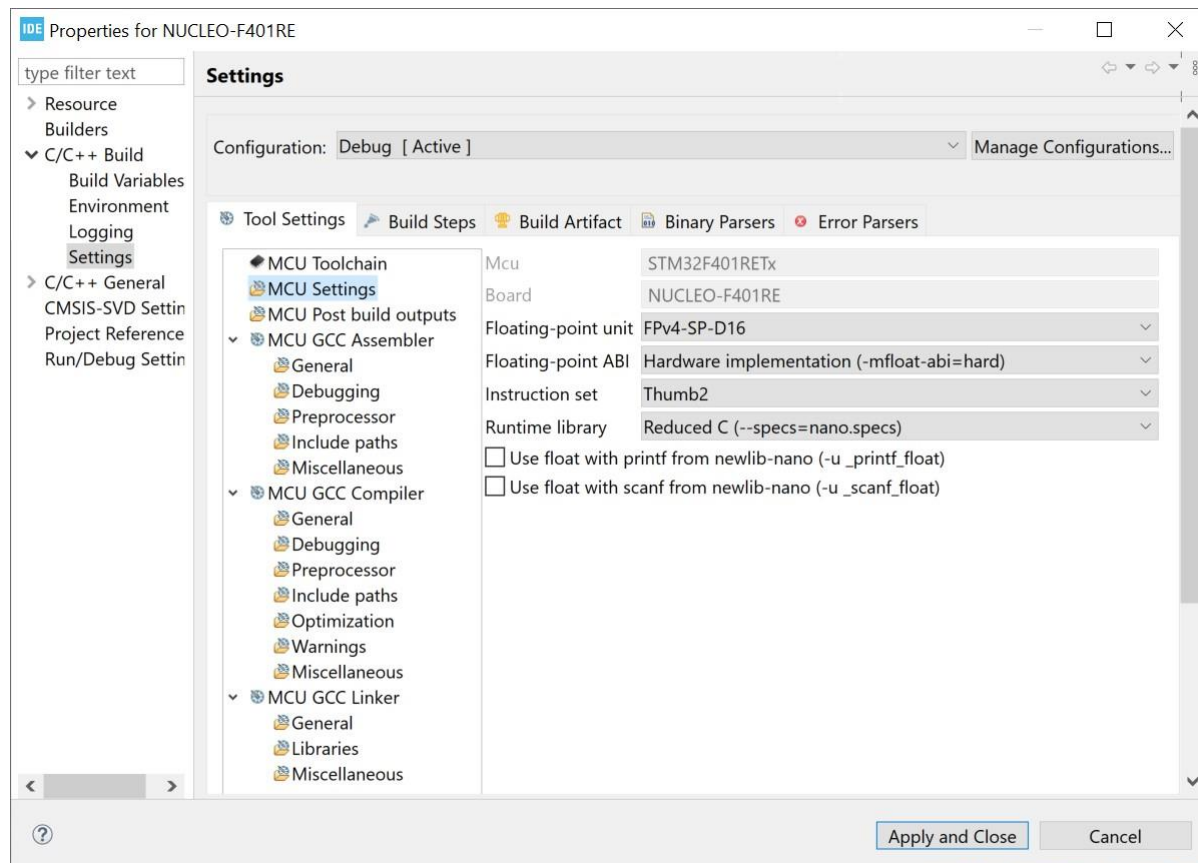
Figure 70. Properties toolchain selection



如图70所示，默认情况下只有STM32的默认工具链GNU Tools是可用的。要安装其他工具链，请单击[Open Toolchain Manager...]按钮打开工具链管理器。第2.11节工具链管理器包含关于如何安装、卸载工具链和选择默认工作区工具链的详细信息。

MCU Settings显示项目所选择的MCU和单板，并提出如何处理浮点数、指令集和运行时库的选择。

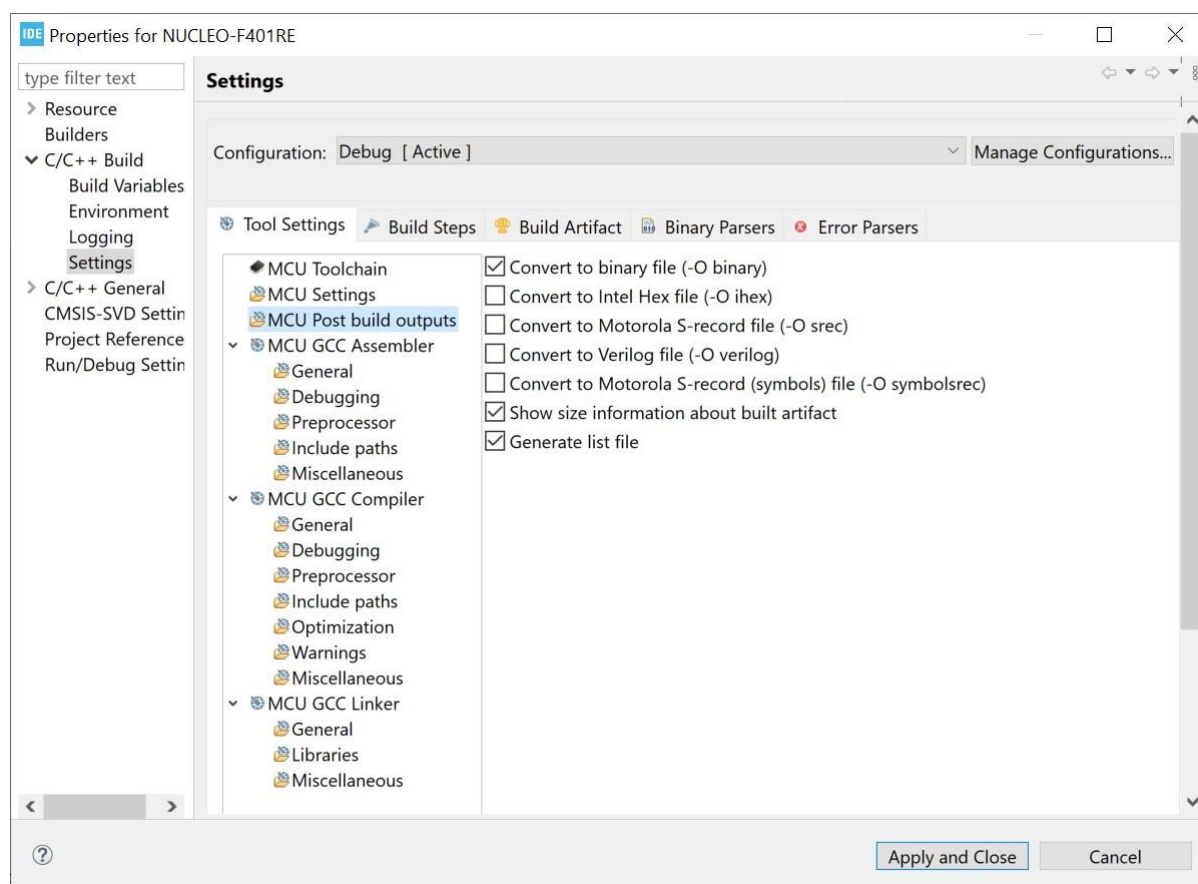
Figure 71. Properties tool MCU settings



单片机后期构建输出提出将elf文件转换为另一种文件格式，显示构建大小信息，生成列表文件。输出文件可以转换为：

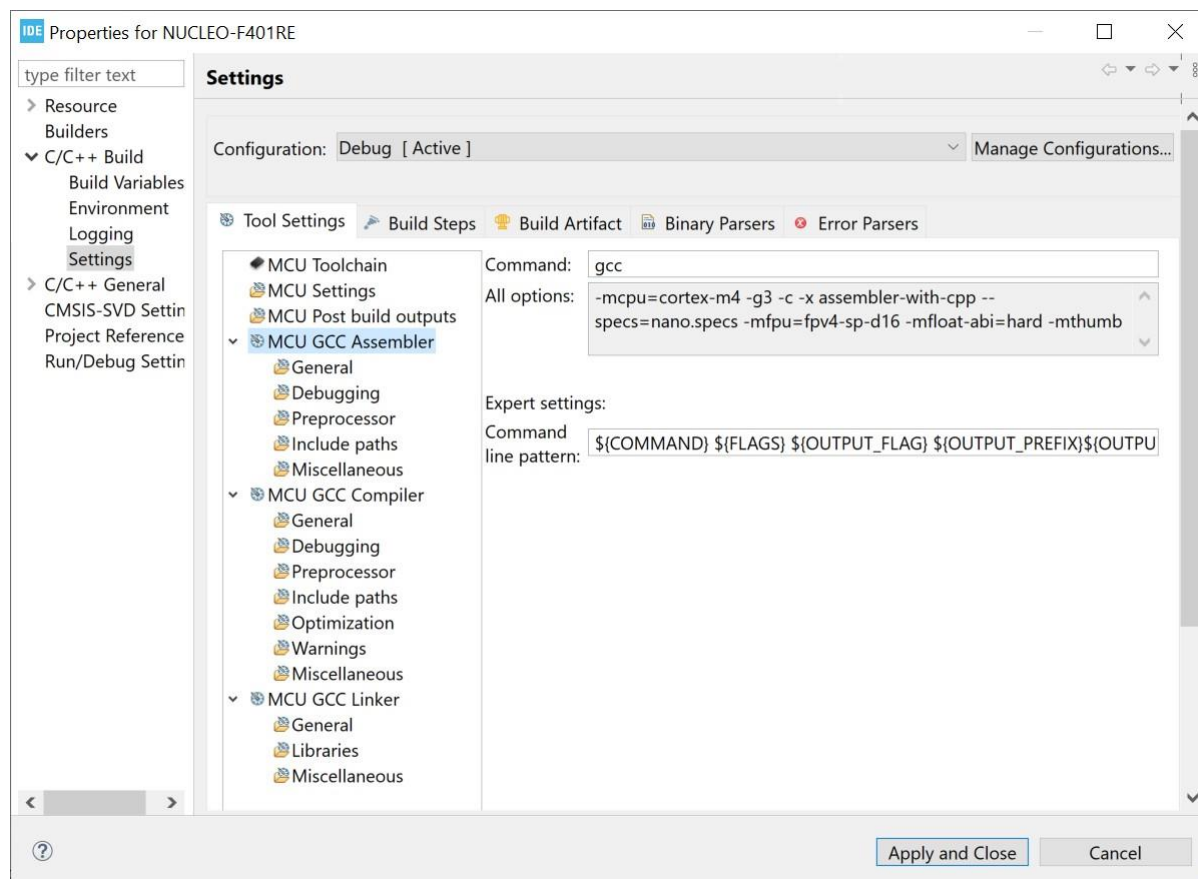
- Binary file
- Intel Hex file
- Motorola S-record file
- Motorola S-record symbols file
- Verilog file

Figure 72. Properties tool MCU post-build settings



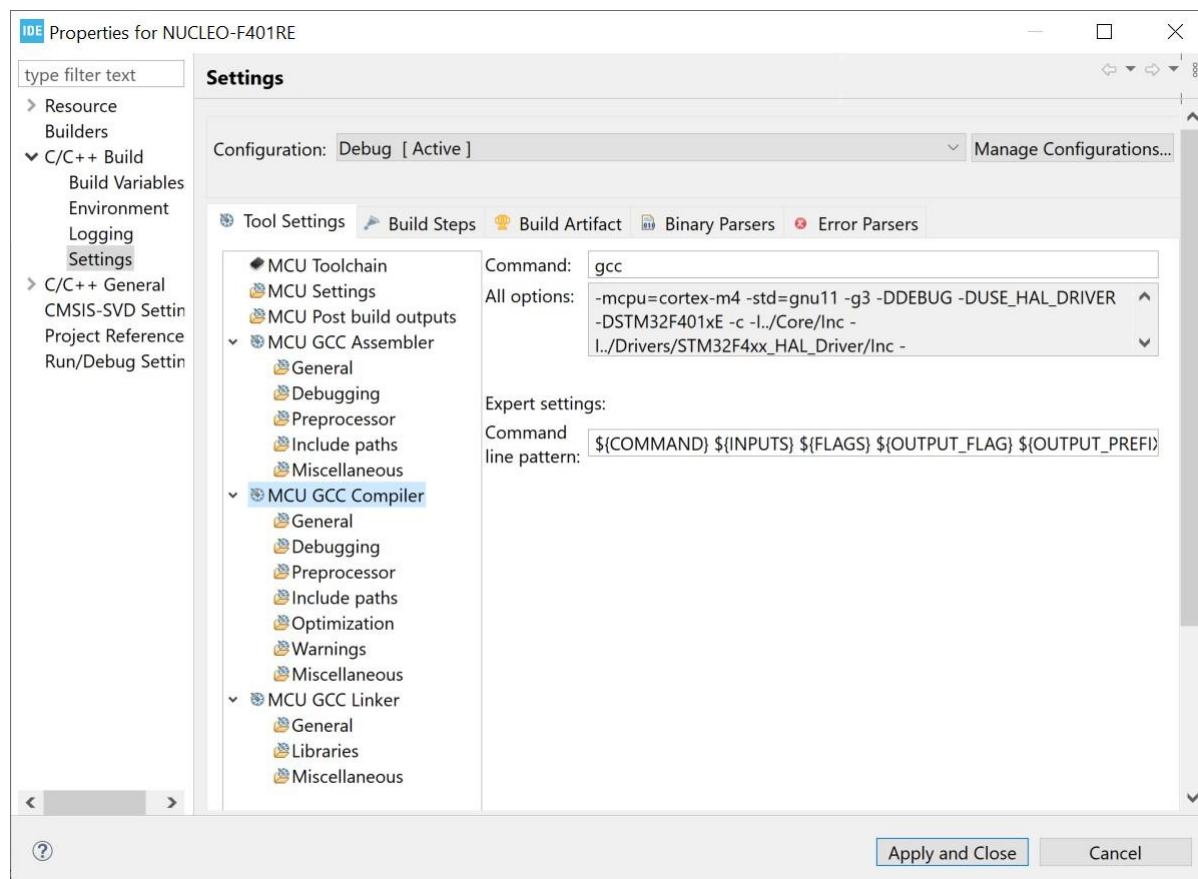
MCU GCC汇编器设置包含汇编程序的选择。主节点显示子节点设置中当前启用的所有汇编命令行选项。子节点用于查看汇编程序的当前设置或更改任何设置。

Figure 73. Properties tool GCC assembler settings



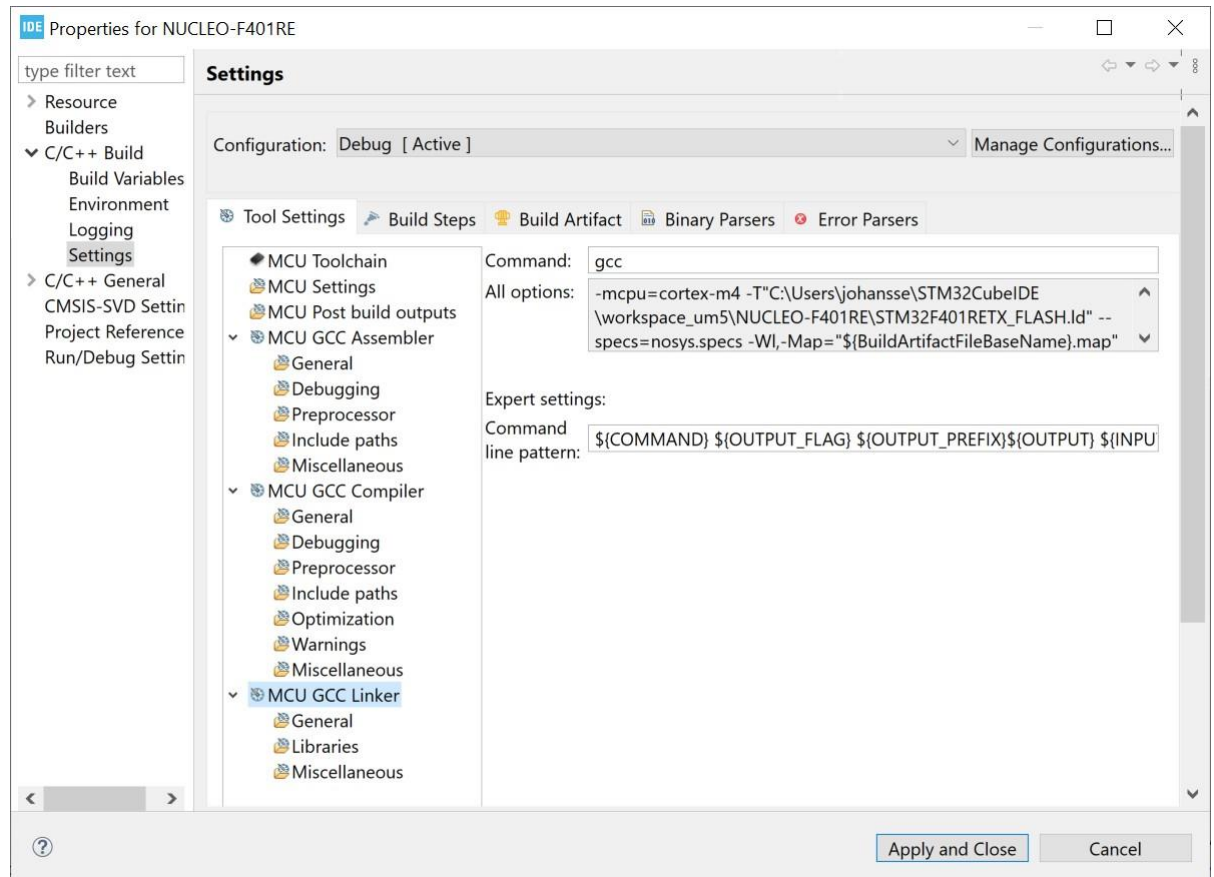
MCU GCC编译器设置包含编译器的选择。主节点显示子节点设置中当前启用的所有编译器命令行选项。子节点用于查看编译器的当前设置或更改任何设置。

Figure 74. Properties tool GCC compiler settings



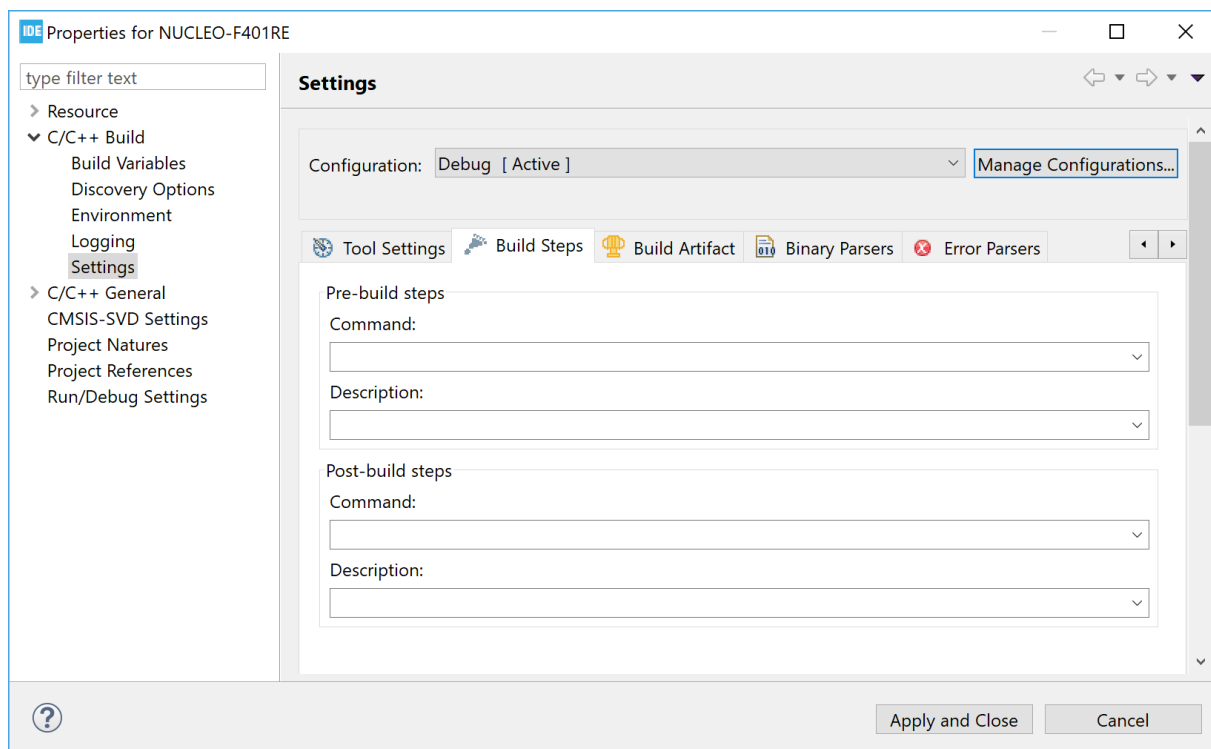
MCU GCC链接器设置包含连接器的选择。主节点显示子节点设置中当前启用的所有链接器命令行选项。子节点用于查看链接器的当前设置或更改任何设置。

Figure 75. Properties tool GCC linker settings



“构建步骤”设置包含用于提供构建前步骤和构建后步骤的字段，这些步骤在构建项目之前和之后运行。编辑字段以运行任何预构建或后构建步骤。

Figure 76. Properties build steps settings

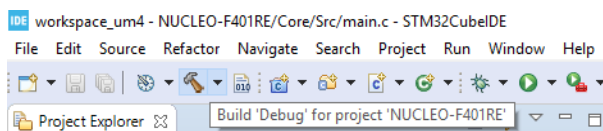


Note: 可以使用makefile目标添加更高级的构建后操作，如第2.4.7节所述。

2.4 Building the project

要开始构建，在project Explorer视图中选择相应的项目，并单击[build]  工具栏按钮。

Figure 77. Project build toolbar



构建也可以从菜单[Project]>[Build Project]开始。[Project]菜单还包含一些其他可用的构建命令，如[Build All]、[Build Project] or [Clean]。

另一种开始构建的方法是在project Explorer视图中右键单击项目。这将打开上下文菜单，使用[Build]命令和其他一些构建选项。

在构建过程中，Console视图列出了构建过程。最后，当elf文件正常创建时，它会列出大小信息。

Figure 78. Project build console

```

CDT Build Console [NUCLEO-F401RE]
arm-none-eabi-gcc -I../Core/Src/main.c -mcpu=cortex-m4 -std=gnu11 -g3 -DUSE_HAL_DRIVER -DSTM32F401xE -DDEBUG -c -I.
arm-none-eabi-gcc -I../Core/Src/stm32f4xx_hal_msp.c -mcpu=cortex-m4 -std=gnu11 -g3 -DUSE_HAL_DRIVER -DSTM32F401xE -I.
arm-none-eabi-gcc -I../Core/Src/stm32f4xx_it.c -mcpu=cortex-m4 -std=gnu11 -g3 -DUSE_HAL_DRIVER -DSTM32F401xE -DDEBUG -c -I.
arm-none-eabi-gcc -I../Core/Src/syscalls.c -mcpu=cortex-m4 -std=gnu11 -g3 -DUSE_HAL_DRIVER -DSTM32F401xE -DDEBUG -c -I.
arm-none-eabi-gcc -I../Core/Src/system.c -mcpu=cortex-m4 -std=gnu11 -g3 -DUSE_HAL_DRIVER -DSTM32F401xE -DDEBUG -c -I.
arm-none-eabi-gcc -I../Core/Src/system_stm32f4xx.c -mcpu=cortex-m4 -std=gnu11 -g3 -DUSE_HAL_DRIVER -DSTM32F401xE -DDEBUG -c -I.
arm-none-eabi-gcc -o "NUCLEO-F401RE.elf" @"objects.list" -mcpu=cortex-m4 -T"C:\Users\johansse\STM32CubeIDE\worksp
Finished building target: NUCLEO-F401RE.elf

arm-none-eabi-size NUCLEO-F401RE.elf
arm-none-eabi-objdump -h -S NUCLEO-F401RE.elf > "NUCLEO-F401RE.list"
   text    data    bss     dec     hex filename
   7308      20    1636    8964    2304 NUCLEO-F401RE.elf
Finished building: default.size.stdout

Finished building: NUCLEO-F401RE.list

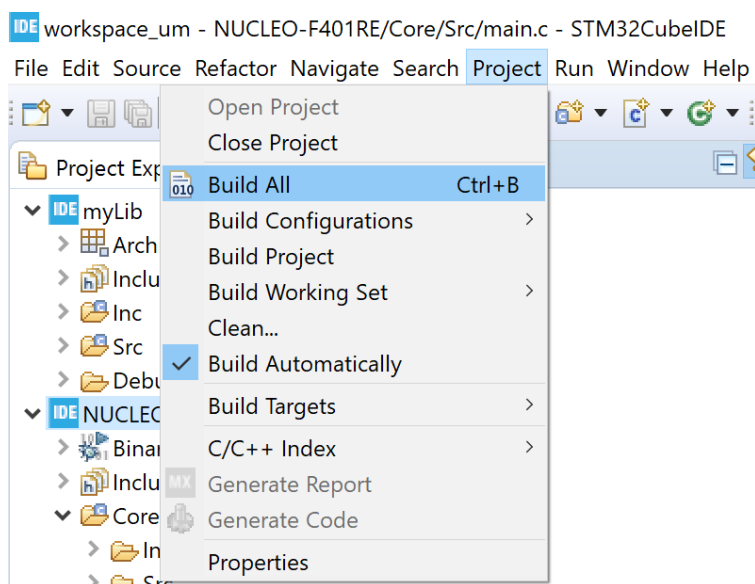
12:42:04 Build Finished. 0 errors, 0 warnings. (took 5s.932ms)

```

2.4.1 Building all projects

工具栏包含[Build all] 按钮，该按钮用于为工作区中所有打开的项目生成活动生成配置。也可以使用菜单[Project]>[Build All]来启动所有项目的构建。

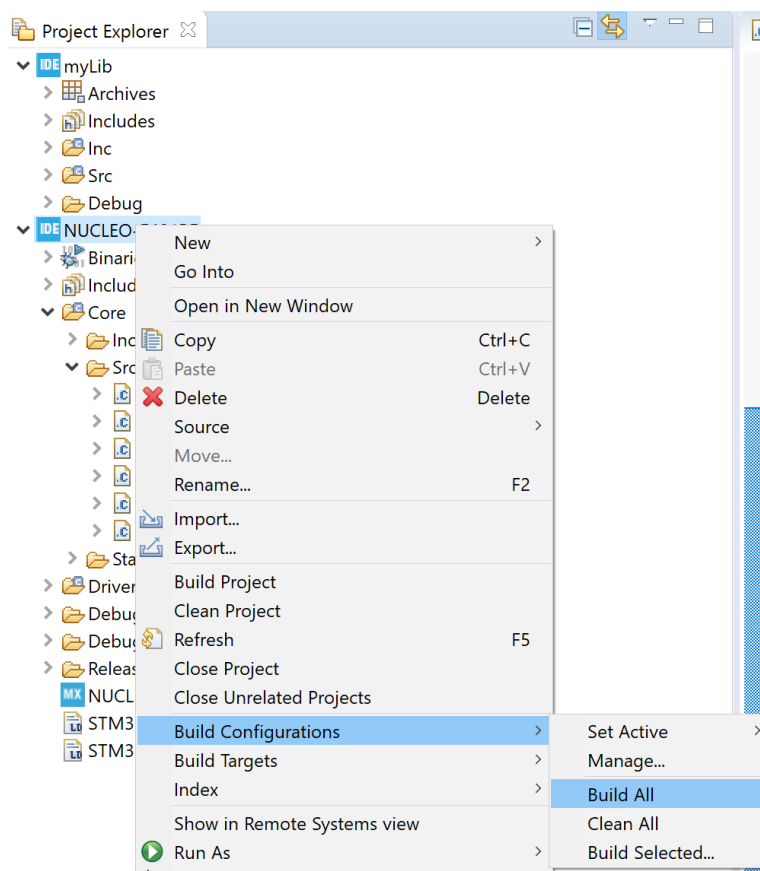
Figure 79. Project build all



2.4.2 Build all build configurations

要生成一个项目的所有生成配置，右键单击该项目并在上下文菜单中选择[Build Configurations]>[Build All]。

Figure 80. Project build-all configurations



2.4.3 Headless build

无头构建旨在用于构建必须集成到脚本控制的构建中的项目，例如用于持续集成过程方法或其他方法的构建服务器上的夜间构建。在这种情况下，STM32CubeIDE GUI永远不会显示，用户不需要与STM32CubeIDE进行任何手动交互。

STM32CubeIDE包含一个运行无头构建的命令文件。例如，当使用Windows®时，它位于C:\ST\STM32CubeIDE_1.7.0\STM32CubeIDE STM32CubeIDE安装文件夹中。bat文件将从命令提示符中运行。

Note: 在运行任何无头构建之前，确保工作空间没有被STM32CubeIDE打开。如果已经使用工作空间运行了STM32CubeIDE，那么无头构建过程就不可能打开并构建项目。

To run headless build in Windows®, use the following procedure:

1. Open a command prompt.
2. Navigate to the STM32CubeIDE installation directory. Open the folder in which the IDE is stored.
For example: `cd C:\ST\STM32CubeIDE_1.7.0\STM32CubeIDE`
3. Enter the following command to build the NUCLEO-F401RE project in the workspace
C:\Users\Name\STM32CubeIDE\workspace_1.7.0:
`$ headless-build.bat -data C:\Users\Name\STM32CubeIDE\workspace_1.7.0 -cleanBuild NUCLEO-F401RE`

To get help on headless build parameters, use headless build with option `-help`. Figure 81 shows the result of command `$ headless-build.bat -help`.

Figure 81. Headless build

```

C:\ST>cd STM32CubeIDE_1.7.0.21alpha1

C:\ST\STM32CubeIDE_1.7.0.21alpha1>headless-build.bat -help
Usage: PROGRAM -data <workspace> -application org.eclipse.cdt.managedbuilder.core.headlessbuild [ OPTIONS ]

-data      {/path/to/workspace}
-import    {[uri:/]/path/to/project}
-importAll {[uri:/]/path/to/projectTreeURI} Import all projects under URI
-build     {project_name_reg_ex{/config_reg_ex} | all}
-cleanBuild {project_name_reg_ex{/config_reg_ex} | all}
-markerType Marker types to fail build on {all | cdt | marker_id}
-no-indexer Disable indexer
-printErrorMarkers Print all error markers
-I         {include_path} additional include_path to add to tools
-include   {include_file} additional include_file to pass to tools
-D         {preproc_define} additional preprocessor defines to pass to the tools
-E         {var=value} replace/add value to environment variable when running all tools
-Ea        {var=value} append value to environment variable when running all tools
-Ep        {var=value} prepend value to environment variable when running all tools
-Er        {var} remove/unset the given environment variable
-T         {toolid} {optionid=value} replace a tool option value in each configuration build
-Ta        {toolid} {optionid=value} append to a tool option value in each configuration build
-Tp        {toolid} {optionid=value} prepend to a tool option value in each configuration build
-Tr        {toolid} {optionid=value} remove a tool option value in each configuration build
Tool option values are parsed as a string, comma separated list of strings or a boolean based on the options type
  
```

2.4.4 Temporary assembly file and preprocessed C code

Save the temporary assembly file by adding the `-save-temps` flag to the compiler:

1. In the menu, select **[Project]>[Properties]**
2. Select **[C/C++ build]>[Settings]**
3. Open the *Tool Settings* tab
4. Add `-save-temps` in the **[C Compiler]>[Miscellaneous]** settings
5. Rebuild the project

汇编程序文件位于生成输出目录中，名称为filename.s。

文件filename.i包含预处理的C代码也会生成。它显示了在预处理器之后但在编译之前的代码。建议检查此文件的内容，以防定义出现问题。

2.4.5 Build logging

要启用或禁用项目生成日志记录，在project Explorer视图中右键单击项目并选择**[Properties]**。然后选择**[C/C++ Build]>[Logging]**。还指定了日志文件的位置和名称。

要为工作空间中的所有项目启用全局生成日志，请选择**[Window], [Preferences]**，并打开**[C/C++, Build, Logging]>[Enable global build logging]**。

2.4.6 Parallel build and build behaviour

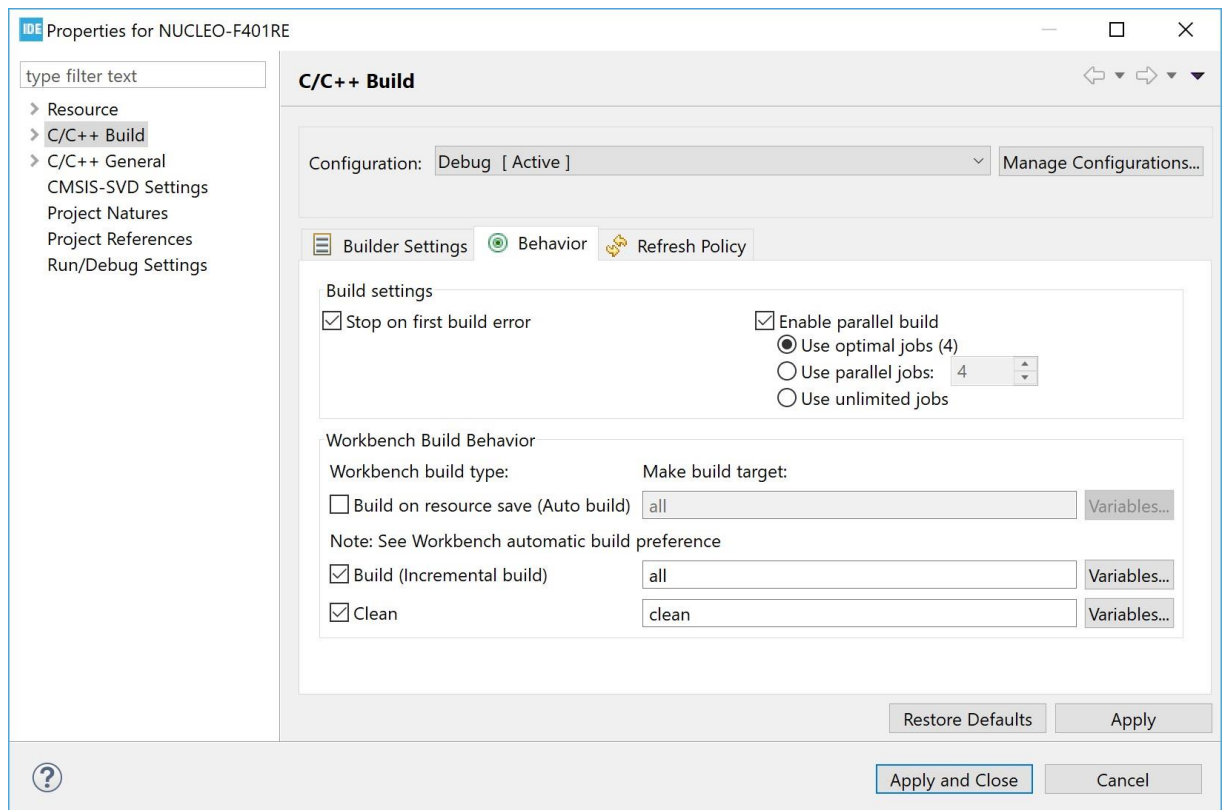
并行构建发生在同时使用多个线程编译和构建代码的情况下。大多数情况下，它可以显著减少构建时间。使用的最佳线程数通常等于计算机的CPU核数。并行构建可以启用和禁用。

To configure parallel build:

1. Right-click on the project in the *Project Explorer* view
2. Select menu **[Project]>[Properties]**
3. Select **[C/C++ Build]** in the *Properties* panel
4. Open the *Behavior* tab and configure **[Enable parallel build]**

The *Behavior* tab also contains build settings on how to behave on errors, build on resource save, incremental build, and clean.

Figure 82. Parallel build



2.4.7 Post-build with makefile targets

It is possible to add advanced post-build scripts by using makefile targets. To do this:

1. Create a new file
2. Name it `makefile.targets`
3. Place it in the root directory of the project

The content of the file must be similar to the example presented below. The example just copies the `elf` generated file to a new file and uses macros `BUILD_ARTIFACT`, `BUILD_ARTIFACT_PREFIX`, `BUILD_ARTIFACT_NAME`, and `BUILD_ARTIFACT_EXTENSION`, which are generated into the makefile by STM32CubeIDE from v1.5.0.

```
secure_target := \
    $(BUILD_ARTIFACT_PREFIX)$(BUILD_ARTIFACT_NAME)-secure.$(BUILD_ARTIFACT_EXTENSION)
main-build: $(secure_target)

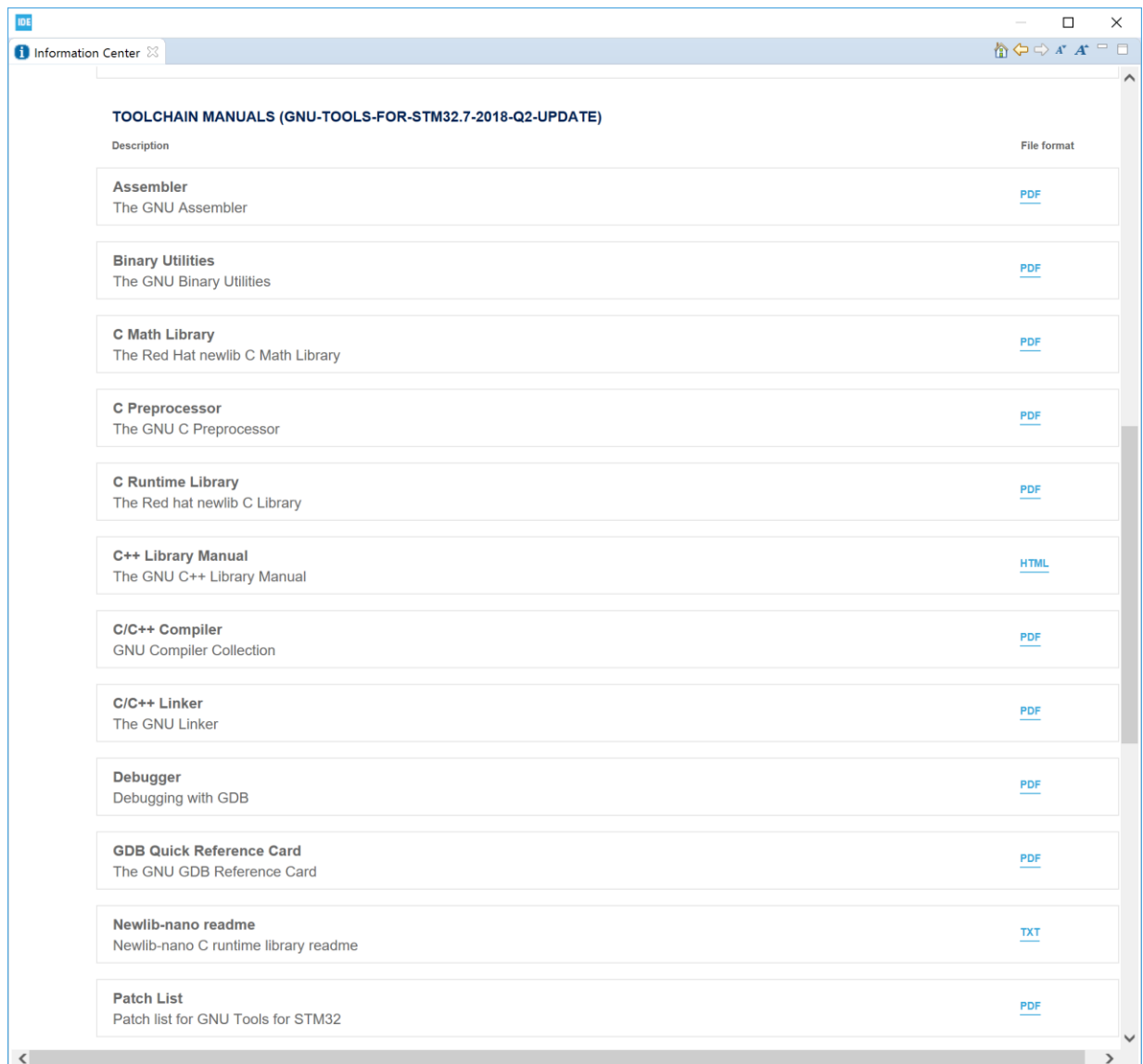
$(secure_target): $(BUILD_ARTIFACT)
    # Do what you want here... simple copy file for demo
    cp "$<" "$@"
```

Note: *make* requires that tabs are used instead of spaces.

2.5 Linking the project

本节包含链接器和链接器脚本文件的基本信息。有关连接器的详细信息可以在GNU连接器手册 ([EXT-05])中找到，该手册可以从信息中心访问。单击[Information Center] 工具栏按钮，打开信息中心视图。使用[C/C++ Linker The GNU Linker PDF]链接打开链接器文档。

Figure 83. Linker documentation



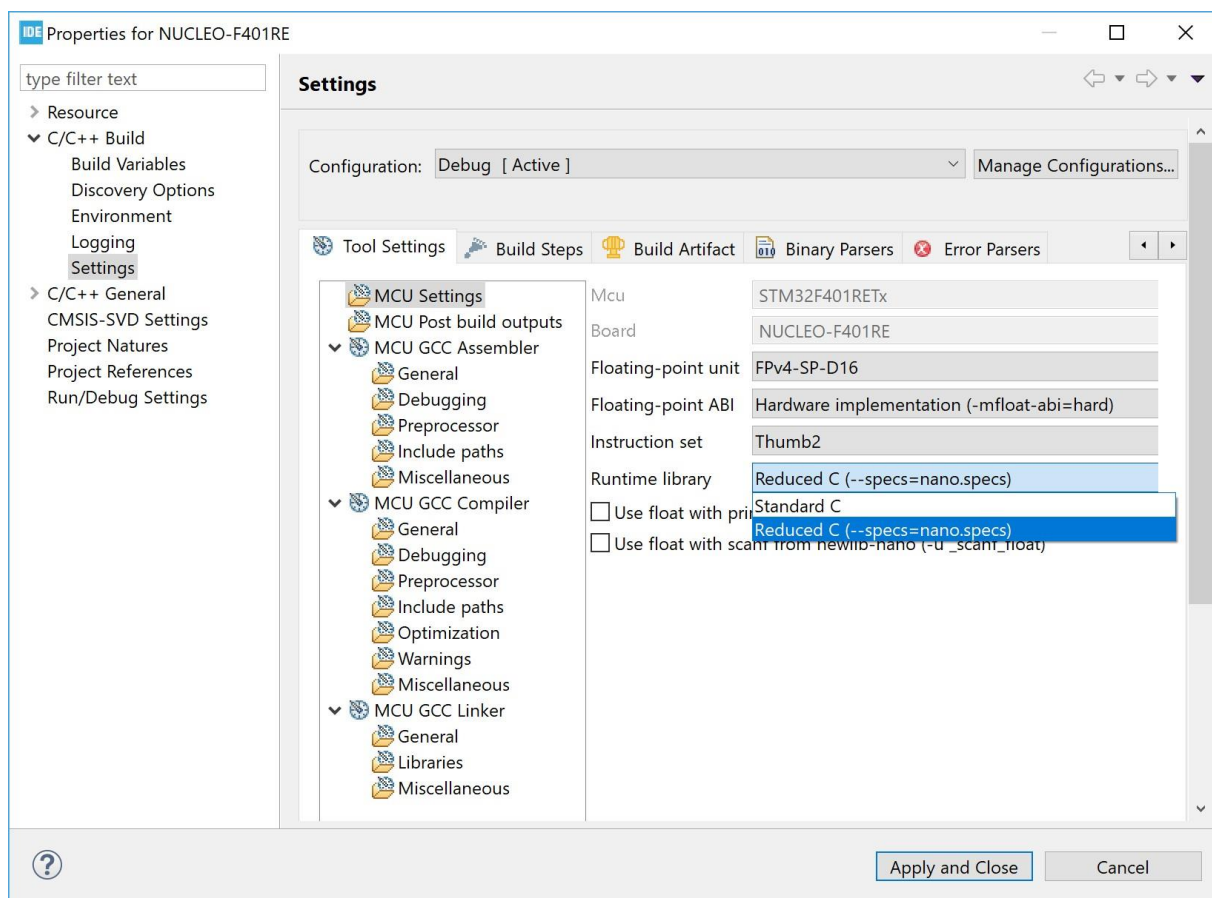
2.5.1 Run time library

STM32CubeIDE中包含的工具链包含两个基于newlib预构建的运行时C库。一个是标准的C newlib库，另一个是简化的C newlib-nano。使用newlib-nano实现更小的代码大小。有关newlib-nano和标准newlib之间区别的信息，请参阅newlib-nano自述文件([ST-09])，可从信息中心访问。

选择要在项目中使用的所需运行时库：

1. Right-click on the project in the *Project Explorer* view
2. Select menu **[Project]>[Properties]**
3. Select **[C/C++ Build]>[Settings]** in the *Properties* panel
4. Open the *Tool Settings* tab, select **[MCU Settings]** and configure the **[Runtime library]** setting

Figure 84. Linker run time library



当使用newlib-nano而浮点数必须由scanf/printf处理时，需要其他选项。原因是newlib-nano和newlib处理浮点数的方式不同。在newlib-nano中，格式化的浮点数输入和输出被实现为弱符号。因此，如果使用-u选项将%f与scanf/printf一起使用，则必须显式指定该符号：

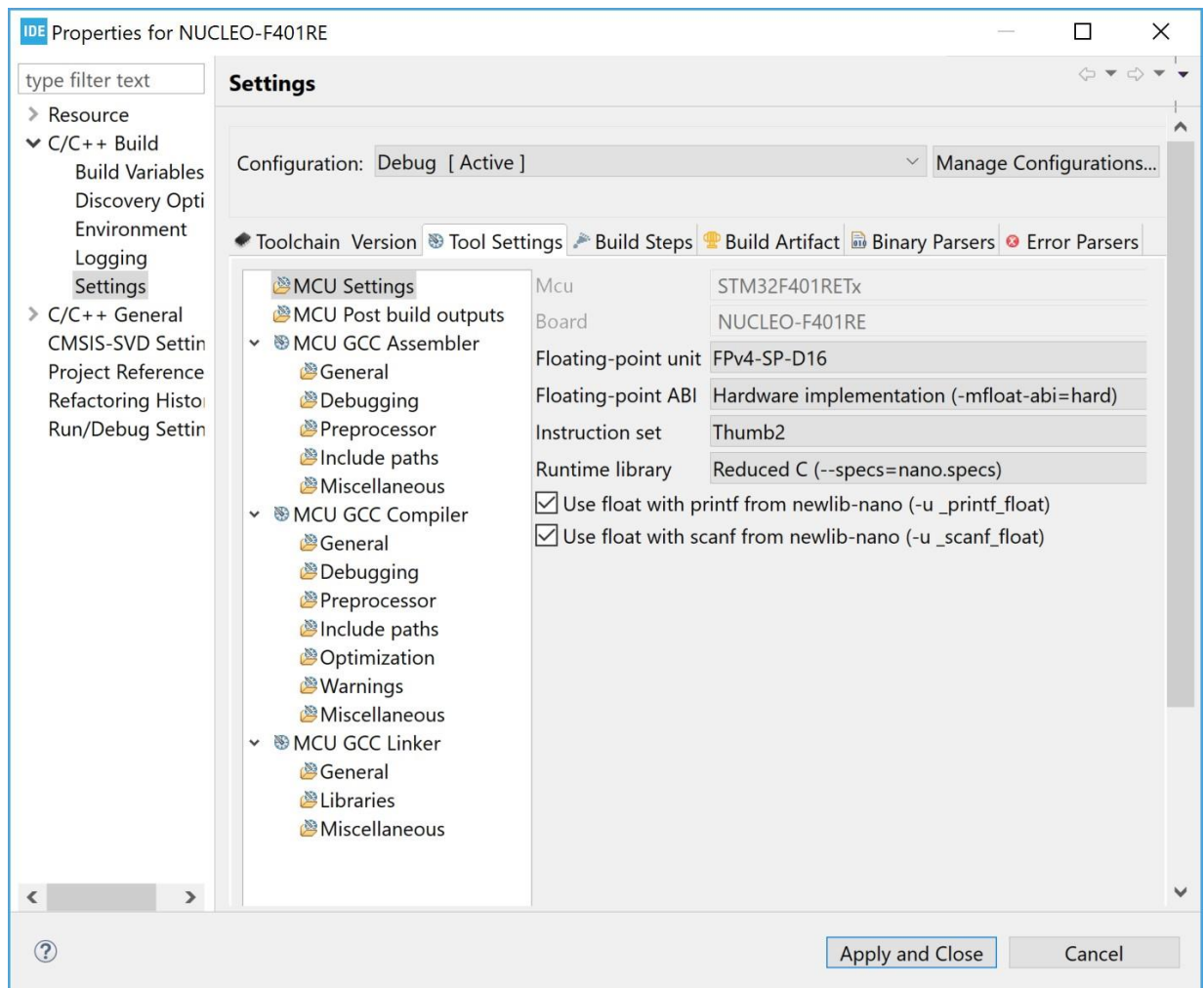
- -u _scanf_float
- -u _printf_float

例如，要使用printf启用输出浮动，命令行如下所示：

```
$ arm-none-eabi-gcc --specs=nano.specs -u _printf_float $(OTHER_LINK_OPTIONS)
```

可以使用工具设置选项卡中[MCU Settings]中的[Use float ...]复选框启用该选项。

Figure 85. Linker newlib-nano library and floating-point numbers

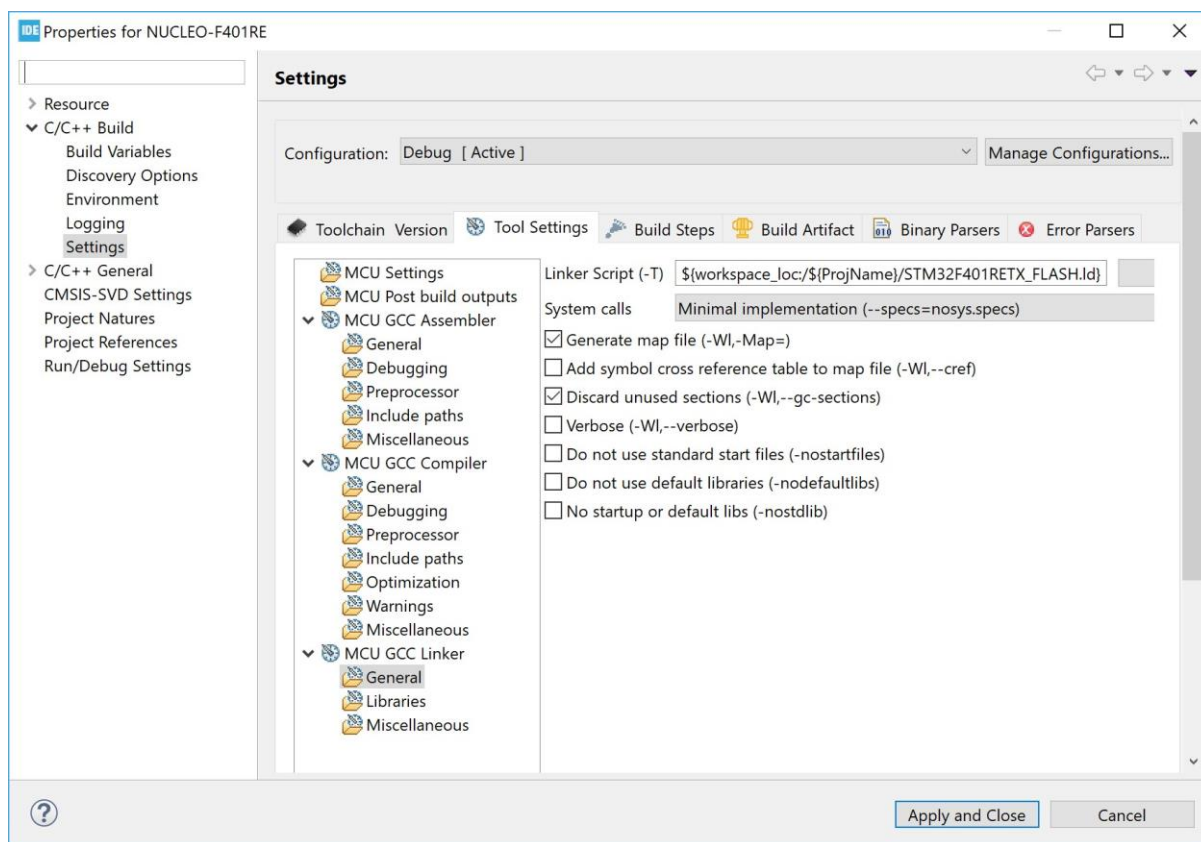


2.5.2 Discard unused sections 丢弃未使用的部分

链接器优化是链接器从输出二进制中删除未使用的代码和数据段(死代码)的过程。运行时和中间件库通常包含许多不是所有应用程序都使用的函数，因此除非从输出二进制文件中删除，否则浪费宝贵的内存。

当使用项目向导创建新项目时，默认配置是链接器丢弃未使用的部分。要检查或更改未使用区段的设置，请随时打开项目的生成设置：

1. Right-click the project in the *Project Explorer* view and select **[Properties]**
2. In the dialog, select **[C/C++ Build]>[Settings]**
3. Select the *Tool Settings* tab in the panel
4. Select **[MCU GCC Linker]>[General]**
5. Configure **[Discard unused sections (-Wl,--gc-sections)]** according to the project requirements
6. Rebuild the project

Figure 86. Linker discard unused sections


2.5.3 Page size allocation for malloc 为malloc分配页面大小

当STM32工具链的GNU Tools与标准C newlib库一起使用时，可以更改malloc的页面大小设置。newlib默认页面大小为4096字节。如果在用户项目中实现了sysconf()函数，则该用户函数将由 _malloc_r()调用。

下面的例子展示了如何实现页面大小为128字节的sysconf()函数。如果应用程序需要使用比默认的4096字节更小的页面大小，则添加类似的函数。

```
/**
*****
** File      : sysconf.c
*****
**/

/* Includes */
#include <errno.h>
#include <unistd.h>

long sysconf(int name)
{
    if (name == _SC_PAGESIZE)
    {
        return 128;
    }
    else
    {
        errno = EINVAL;
        return -1;
    }
}
```

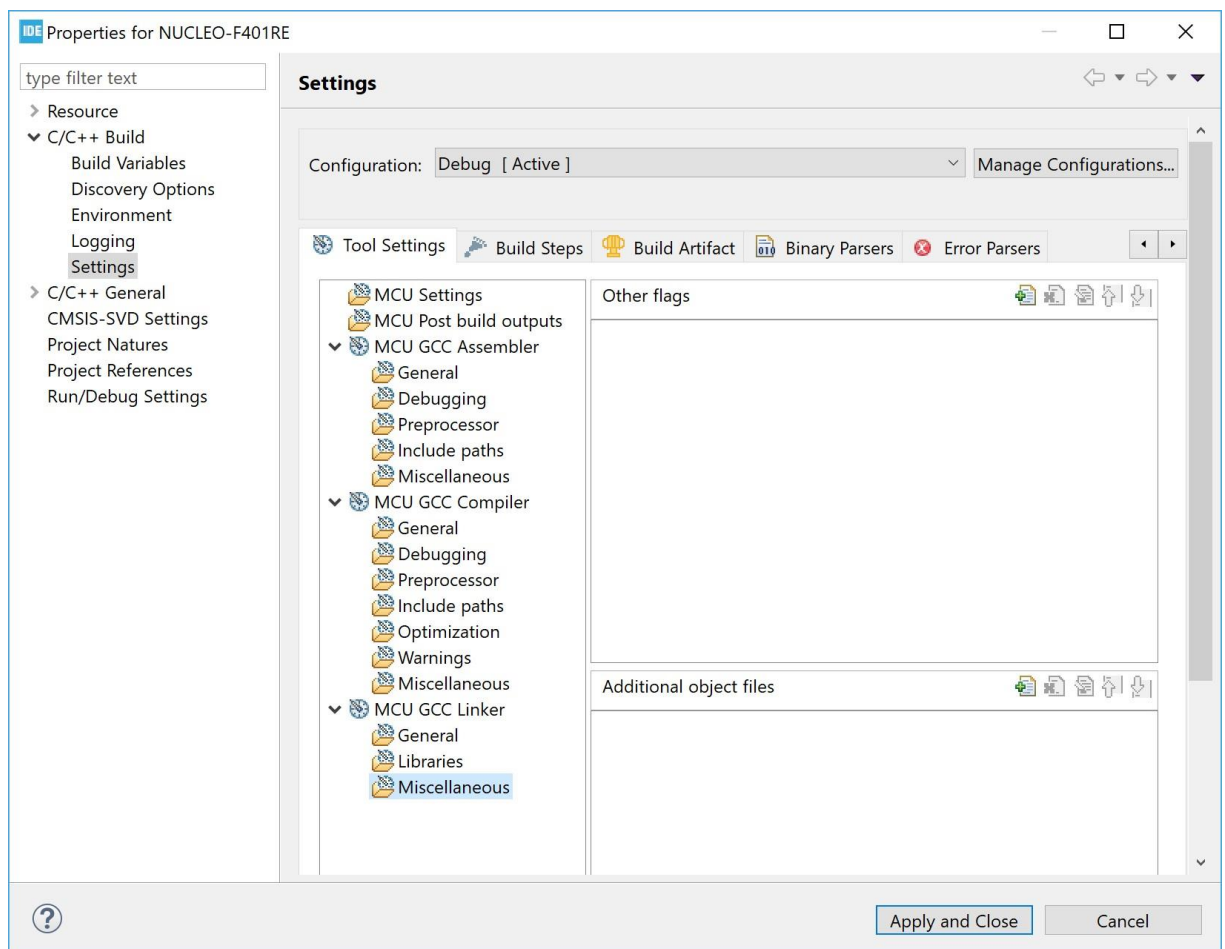
Note: 如果使用“GNU ARM Embedded”工具链，它不会调用应用程序中实现的任何sysconf()函数，而总是使用newlib中的默认sysconf()函数。此外，如果“GNU Tools for STM32”工具链与简化的C newlib-nano库一起使用，则不会调用sysconf()。

2.5.4 Include additional object files

STM32CubeIDE使得包含必须链接到项目的附加目标文件变得很容易。它们可以是来自其他项目的文件，没有源代码的预编译库，或者用其他编译器创建的目标文件。

1. Right-click the project in the *Project Explorer* view and select **[Properties]**
2. In the dialog, select **[C/C++ Build]>[Settings]**
3. Select the *Tool Settings* tab in the panel
4. Select **[MCU GCC Linker]>[Miscellaneous]**
5. Use the **[Add...]** icon to add additional object files in several possible ways:
 - Enter the filenames in the *Add file path* dialog
 - Use the **[Workspace...]** or **[File system...]** buttons to locate the files

Figure 87. Linker include additional object files



2.5.5 Treat linker warnings and errors

链接器通常是沉默的警告。如果项目中缺少包含普通Reset_Handler函数的启动代码，就会看到这种无声警告的一个例子。GNU链接器在正常静默模式下创建一个elf文件，并且只在控制台窗口中报告关于缺少Reset_Handler的警告输出。

Example of warning message:

```
arm-none-eabi-gcc -o "NUCLEO-F401RE.elf" @"objects.list" -mcpu=cortex-m4 -
T"C:\Users\username\STM32CubeIDE\workspace_um\NUCLEO-F401RE\STM32F401RETX_FLASH.ld"
--specs=nosys.specs -Wl,-Map="NUCLEO-F401RE.map" -Wl,--gc-sections -static -
mfpv=fpv4-sp-d16 -mfloat-abi=hard -mthumb -Wl,--start-group -lc -lm -Wl,--end-group
c:\st\stm32cubeide_1.1.0.19w37\stm32cubeide\plugins\com.st.stm32cube.ide.mcu.extern
altools.gnu-tools-for-stm32.7-2018-q2-update.win32_1.0.0.201904181610\tools\arm-
none-eabi\bin\ld.exe: warning: cannot find entry symbol Reset_Handler; defaulting
to 0000000008000000
Finished building target: NUCLEO-F401RE.elf
```

在这种情况下，将创建一个新的elf文件，但是，如果没有检测到警告，则它将无法调试项目，因为程序不包含Reset_Handler函数。通过添加——fatal-warnings选项，可以配置链接器将警告视为错误。

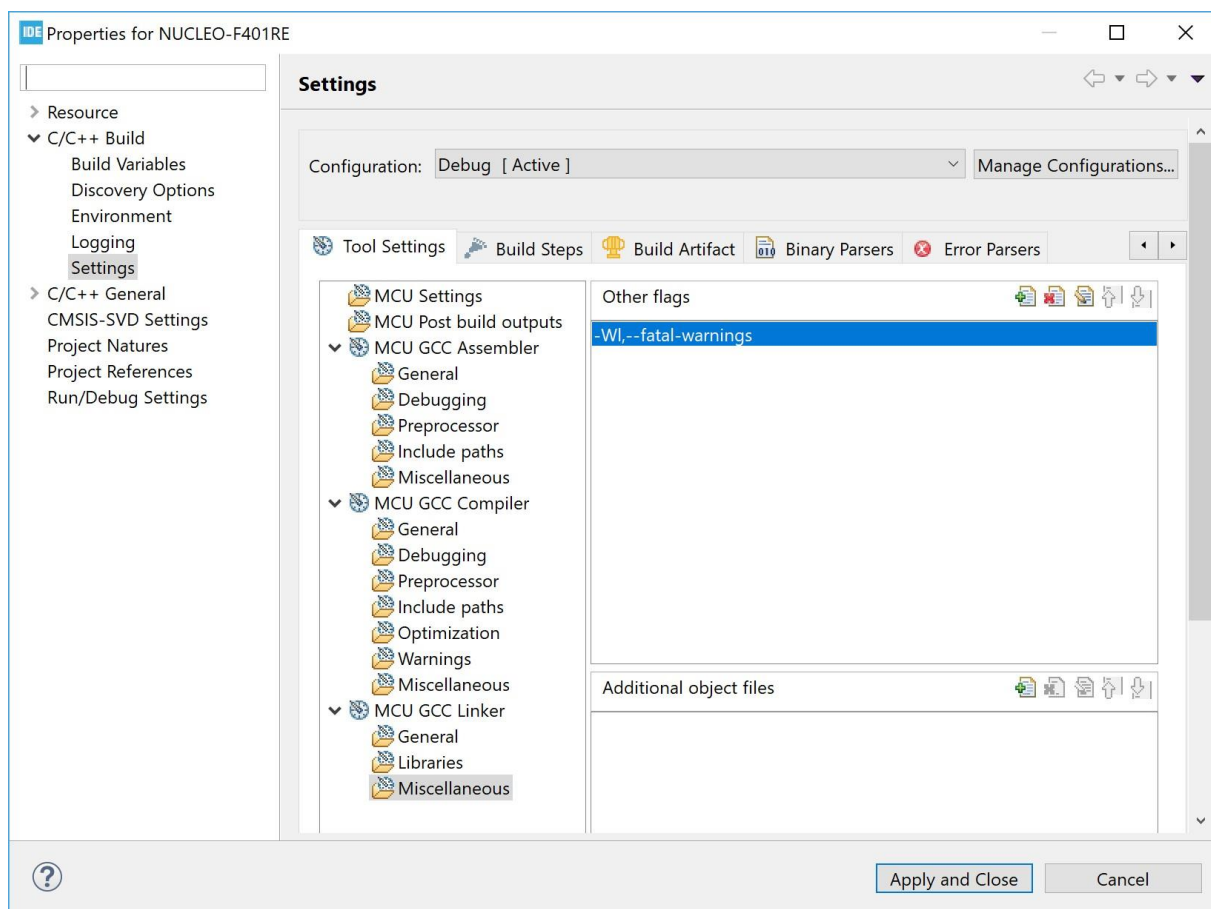
当使用--fatal-warnings选项时，链接器不会生成elf文件，而是在控制台日志中显示一个错误:

```
c:\st\stm32cubeide_1.1.0.19w37\stm32cubeide\plugins\com.st.stm32cube.ide.mcu.extern
altools.gnu-tools-for-stm32.7-2018-q2-update.win32_1.0.0.201904181610\tools\arm-
none-eabi\bin\ld.exe: warning: cannot find entry symbol Reset_Handler; defaulting
to 0000000008000000
collect2.exe: error: ld returned 1 exit status
make: *** [makefile:40: NUCLEO-F401RE.elf] Error 1
"make -j4 all" terminated with exit code 2. Build might be incomplete.
11:26:30 Build Failed. 1 errors, 6 warnings. (took 7s.193ms)
```

To use the -Wl,--fatal-warnings option:

1. Right-click the project in the *Project Explorer* view and select **[Properties]**
2. In the dialog, select **[C/C++ Build]>[Settings]**
3. Select the *Tool Settings* tab in the panel
4. Select **[MCU GCC Linker]>[Miscellaneous]**
5. Add -Wl,--fatal-warnings to the **[Other flags]** field.

Figure 88. Linker fatal warnings



2.5.6 Linker script

链接器脚本文件(.ld)定义了要包含的文件以及内容最终在内存中的位置。链接器脚本文件的一些重要部分将在下一节中描述。有关连接器的详细信息，请阅读C/ c++连接器GNU连接器手册([ST-05])。本手册可在信息中心的文档部分找到。特别考虑第3.6和3.7节。

链接器脚本指定内存区域和堆栈、堆、bss、数据、rodata、文本和程序条目的位置。堆栈和堆的大小可以通过编辑链接器脚本文件中的_Min_Stack_Size和_Min_Heap_Size值来配置。但是，这些值仅被链接器用于验证堆栈和堆是否适合内存。在运行程序时，堆栈或堆可能需要更多内存，如果数据被覆盖，可能会导致意想不到的结果。

表3给出了一个带有512-Kbyte闪存和96-Kbyte SRAM的STM32F4设备的典型程序和内存布局示例。该设备基于Cortex®- M核心，32位地址空间(0x0000 0000到0xFFFF FFFF)。

Table 3. Memory map layout

Example: STM32F4 96-Kbyte SRAM 512-Kbyte Flash memory	Usage	Files Linker script .ld, or .h and .c files	Comment
0xFFFF FFFF 0xE000 0000	Cortex-M4 internal peripherals.	.h and .c files.	SysTick, NVIC, ITM, debug and others.
0xDFFF FFFF 0x6000 0000	External memory FMC (Flexible memory controller).	Must be added in linker script, and .h and .c files. ⁽¹⁾	NOR Flash memory, NAND Flash memory, SPI Flash memory, PSRAM, SDRAM and others.
0x5FFF FFFF 0x4000 0000	STM32 Peripherals.	.h and .c files.	GPIO, ADC, timers, USB, USART and others.
0x2001 8000	96-Kbyte SRAM Stack	Linker script _estack _Min_Stack_Size	The stack contains local data ⁽²⁾
	Heap	_Min_Heap_Size _user_heap_stack	⁽³⁾ Heap used by malloc ⁽⁴⁾ Data
		.bss	Static global data (.bss and .data) .bss == Uninitialized data Cleared to zero by the startup code.
0x2000 0000	Data	.data	.data == Initialized data Copied from Flash memory to SRAM by the startup code.
0x0808 0000	512-Kbyte Flash memory Data	Linker script .data .rodata	Initialized data to copy to SRAM. Read-only data placed in Flash memory.
	Program	ENTRY Reset_Handler ⁽⁵⁾ .text	.text == Program, such as main() in main.c, SystemInit() in system_stm32f4xx.c, Reset_Handler in startup_stm32*.s, g_pfnVectors in startup_stm32*.s, Vector table in startup_stm32*.s.
0x0800 0000	Interrupt vector table	.isr_vector ⁽⁶⁾	
Color legend			
Cortex®-M internal peripherals and STM32 peripherals.			
External memory. Normally the linker script, header files, and C files must be updated to use external memories.			
Flash memory and SRAM where program, data, heap, and stack are located. Usually, when creating a project with STM32CubeIDE, these Flash memory and RAM regions are accessible and usable without any updates of the linker script or other files. The linker script file defines how to place code, data, heap, and stack in memory.			

1. If external memory is used, the memory must be added into the linker script file. See in chapter [Section 2.5.7.1](#) how to add a new memory region.
2. The stack grows downwards and may go into the heap.
3. When running the program, the stack or heap may require more memory, which might lead to unexpected results if data is overwritten.
4. The heap grows upwards and may go into the stack.
5. The linker script file contains the entry point definition of the program. Normally, ENTRY(Reset_Handler).
6. The interrupt vector table contains the reset value of the stack pointer, the start addresses of the program (Reset_Handler), exception handlers and interrupt handlers. Normally the Reset_Handler code and vector table (g_pfnVectors) are available in file <startup_stm32xxx.s>.

请参阅下面由STM32CubeIDE为具有512-Kbyte闪存和96-Kbyte SRAM的STM32F4设备生成的默认链接器脚本。

代码摘录的开头显示了链接器脚本头、条目、堆栈、堆和内存定义。

```
/**
*****
* @file      LinkerScript.ld
* @author    Auto-generated by STM32CubeIDE
* Abstract   : Linker script for NUCLEO-F401RE Board embedding STM32F401RETx Device from
stm32f4 series
*
*              512Kbytes FLASH
*              96Kbytes RAM
*
*      Set heap size, stack size and stack location according
*      to application requirements.
*
*      Set memory bank area and size if external memory is used
*****
* @attention
*
* <h2><center>&copy; Copyright (c) 2020 STMicroelectronics.
* All rights reserved.</center></h2>
*
* This software component is licensed by ST under BSD 3-Clause license,
* the "License"; You may not use this file except in compliance with the
* License. You may obtain a copy of the License at:
*              opensource.org/licenses/BSD-3-Clause
*
*****
*/

/* Entry Point */
ENTRY(Reset_Handler)

/* Highest address of the user mode stack */
_estack = ORIGIN(RAM) + LENGTH(RAM); /* end of "RAM" Ram type memory */

_Min_Heap_Size = 0x200; /* required amount of heap */
_Min_Stack_Size = 0x400; /* required amount of stack */

/* Memories definition */
MEMORY
{
  RAM      (xrw)      : ORIGIN = 0x20000000, LENGTH = 96K
  FLASH    (rx)       : ORIGIN = 0x80000000, LENGTH = 512K
}
```

The continuation of the code excerpt shows the definition of sections.

```

/* Sections */
SECTIONS
{
/* The startup code into "FLASH" Rom type memory */
.isr_vector :
{
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
} >FLASH

/* The program code and other data into "FLASH" Rom type memory */
.text :
{
    . = ALIGN(4);
    *(.text)           /* .text sections (code) */
    *(.text*)          /* .text* sections (code) */
    *(.glue_7)         /* glue arm to thumb code */
    *(.glue_7t)        /* glue thumb to arm code */
    *(.eh_frame)

    KEEP (*(.init))
    KEEP (*(.fini))

    . = ALIGN(4);
    _etext = .;        /* define a global symbols at end of code */
} >FLASH

/* Constant data into "FLASH" Rom type memory */
.rodata :
{
    . = ALIGN(4);
    *(.rodata)         /* .rodata sections (constants, strings, etc.) */
    *(.rodata*)        /* .rodata* sections (constants, strings, etc.) */
    . = ALIGN(4);
} >FLASH

.ARM.extab : {
    . = ALIGN(4);
    *(.ARM.extab* .gnu.linkonce.armextab.*)
    . = ALIGN(4);
} >FLASH

.ARM : {
    . = ALIGN(4);
    __exidx_start = .;
    *(.ARM.exidx*)
    __exidx_end = .;
    . = ALIGN(4);
} >FLASH

.preinit_array :
{
    . = ALIGN(4);
    PROVIDE_HIDDEN (__preinit_array_start = .);
    KEEP (*(.preinit_array*))
    PROVIDE_HIDDEN (__preinit_array_end = .);
    . = ALIGN(4);
} >FLASH

.init_array :
{
    . = ALIGN(4);
    PROVIDE_HIDDEN (__init_array_start = .);
    KEEP (*(SORT(.init_array.*)))
    KEEP (*(.init_array*))
    PROVIDE_HIDDEN (__init_array_end = .);
    . = ALIGN(4);
} >FLASH

```

```

.fini_array :
{
  . = ALIGN(4);
  PROVIDE_HIDDEN (__fini_array_start = .);
  KEEP (*(SORT(.fini_array.*)))
  KEEP (*(.(fini_array*)))
  PROVIDE_HIDDEN (__fini_array_end = .);
  . = ALIGN(4);
} >FLASH

/* Used by the startup to initialize data */
_sidata = LOADADDR(.data);

/* Initialized data sections into "RAM" Ram type memory */
.data :
{
  . = ALIGN(4);
  _sdata = .;          /* create a global symbol at data start */
  *(.data)              /* .data sections */
  *(.data*)             /* .data* sections */
  *(.RamFunc)           /* .RamFunc sections */
  *(.RamFunc*)          /* .RamFunc* sections */

  . = ALIGN(4);
  _edata = .;          /* define a global symbol at data end */
} >RAM AT> FLASH

/* Uninitialized data section into "RAM" Ram type memory */
. = ALIGN(4);
.bss :
{
  /* This is used by the startup in order to initialize the .bss section */
  _sbss = .;           /* define a global symbol at bss start */
  __bss_start__ = _sbss;
  *(.bss)
  *(.bss*)
  *(COMMON)

  . = ALIGN(4);
  _ebss = .;           /* define a global symbol at bss end */
  __bss_end__ = _ebss;
} >RAM

/* User_heap_stack section, used to check that there is enough "RAM" Ram type memory left
*/
._user_heap_stack :
{
  . = ALIGN(8);
  PROVIDE ( end = . );
  PROVIDE ( _end = . );
  . = . + _Min_Heap_Size;
  . = . + _Min_Stack_Size;
  . = ALIGN(8);
} >RAM

/* Remove information from the compiler libraries */
/DISCARD/ :
{
  libc.a ( * )
  libm.a ( * )
  libgcc.a ( * )
}

.ARM.attributes 0 : { *(.ARM.attributes) }
}

```

2.5.6.1 *The ENTRY command defines the start of the program* **ENTRY命令定义了程序的开始**

程序中要执行的第一条指令是用ENTRY命令定义的。

Example:

```
/* Entry Point */
ENTRY(Reset_Handler)
```

GDB使用ENTRY信息，以便在加载程序时将程序计数器(PC)设置为ENTRY地址的值。在本例中，当加载后向GDB提供step或continue命令时，程序从Reset_Handler开始执行。

Note: 如果GDB脚本在加载命令之后包含监视器重置命令，则可以重写程序的开始。然后代码从reset开始运行。

2.5.6.2 *Stack location*

堆栈位置通常由启动文件使用_estack符号使用。启动代码通常使用链接器脚本中给出的地址初始化堆栈指针。对于基于Cortex®-M的设备，堆栈地址也设置在中断向量表的第一个地址。

Example:

```
/* Highest address of the user mode stack */
_estack = ORIGIN(RAM) + LENGTH(RAM); /* end of "RAM" Ram type memory */
```

2.5.6.3 *Define heap and stack minimum sizes*

通常在链接器脚本中定义系统使用的堆和堆栈的最小大小。

Example:

```
_Min_Heap_Size = 0x200; /* required amount of heap */
_Min_Stack_Size = 0x400; /* required amount of stack */
```

这里定义的值通常稍后在链接器脚本中使用，以使链接器能够测试堆和堆栈是否适合内存。如果没有足够的可用内存，链接器就会发出一个错误。

2.5.6.4 *Specify memory regions*

内存区域由名称ORIGIN和LENGTH指定。使用一个属性列表来指定特定内存区域的使用情况也是很常见的，例如(rx)，其中“r”代表只读部分，“x”代表可执行部分。不需要指定任何属性。

Example:

```
/* Memories definition */
MEMORY
{
  RAM      (xrw)      : ORIGIN = 0x20000000,   LENGTH = 96K
  FLASH    (rx)       : ORIGIN = 0x80000000,   LENGTH = 512K
}
```

2.5.6.5 Specify output sections (.text and .rodata)

输出部分定义了'.text'、'.data'或其他数据位于存储器中。下面的例子告诉链接器把所有的部分，如.text，.rodata和其他在Flash内存区域。如果程序中存在一些混合代码，GCC将使用示例中提到的粘合部分。例如，如果某些Arm®代码调用thumb代码，则使用胶水代码，反之亦然。

Example:

```
/* Sections */
SECTIONS
{
  /* The startup code into "FLASH" Rom type memory */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
  } >FLASH

  /* The program code and other data into "FLASH" Rom type memory */
  .text :
  {
    . = ALIGN(4);
    *(.text)           /* .text sections (code) */
    *(.text*)          /* .text* sections (code) */
    *(.glue_7)         /* glue arm to thumb code */
    *(.glue_7t)        /* glue thumb to arm code */
    *(.eh_frame)

    KEEP (*(.init))
    KEEP (*(.fini))

    . = ALIGN(4);
    _etext = .;         /* define a global symbols at end of code */
  } >FLASH
```

2.5.6.6 Specify initialized data (.data)

初始化的数据值需要额外的处理，因为初始化值必须放在闪存中，启动代码必须能够用正确的值初始化RAM变量。下面的例子创建了符号_sidata、_sdata和_edata。在程序启动时，启动代码可以使用这些符号将值从闪存复制到RAM。

Example:

```
/* Used by the startup to initialize data */
_sidata = LOADADDR(.data);

/* Initialized data sections into "RAM" Ram type memory */
.data :
{
  . = ALIGN(4);
  _sdata = .;          /* create a global symbol at data start */
  *(.data)             /* .data sections */
  *(.data*)            /* .data* sections */
  *(.RamFunc)          /* .RamFunc sections */
  *(.RamFunc*)         /* .RamFunc* sections */

  . = ALIGN(4);
  _edata = .;          /* define a global symbol at data end */
} >RAM AT> FLASH
```


2.5.6.7 Specify uninitialized data (.bss)

启动代码必须将未初始化的数据值重置为0:链接器脚本文件必须标识这些变量的位置。下面的例子创建符号`_sbss`和`_ebss`。然后，启动代码可以使用这些符号将未初始化变量的值设置为0。

Example:

```
/* Uninitialized data section into "RAM" Ram type memory */
. = ALIGN(4);
.bss :
{
    /* This is used by the startup in order to initialize the .bss section */
    _sbss = .;          /* define a global symbol at bss start */
    __bss_start__ = _sbss;
    *(.bss)
    *(.bss*)
    *(COMMON)

    . = ALIGN(4);
    _ebss = .;          /* define a global symbol at bss end */
    __bss_end__ = _ebss;
} >RAM
```

2.5.6.8 Check if user heap and stack fit in the RAM

代码的一部分通常专门用于链接器检查所需的堆和堆栈是否与所有其他数据一起放入RAM。

Example:

```
/* User_heap_stack section, used to check that there is enough "RAM" Ram type memory left */
.
.user_heap_stack :
{
    . = ALIGN(8);
    PROVIDE ( end = . );
    PROVIDE ( _end = . );
    . = . + _Min_Heap_Size;
    . = . + _Min_Stack_Size;
    . = ALIGN(8);
} >RAM
```

Note:

栈被放置在RAM的顶部，并在数据之后进行堆，两者之间有一个间隙。见表3。内存映射布局。

2.5.6.9 Linker map and list files

在构建用STM32CubeIDE生成的项目时，将在调试或发布构建输出文件夹中创建映射和列表文件。这些文件包含程序中代码和数据最终位置的详细信息。

Build Analyzer视图可用于详细分析程序的大小和位置。在第8节构建分析器中阅读更多信息。

2.5.7 Modify the linker script

本节介绍需要编辑链接器脚本的常见用例。编辑和管理脚本可以更精确地放置代码和数据。

2.5.7.1 Place code in a new memory region

许多设备都有多个内存区域。可以使用链接器脚本将代码特定地放置在不同的区域。下面的示例展示了如何更新链接器脚本，以支持将代码放置在名为IP_CODE的新内存区域中。

Example:

```
Original MEMORY AREA

/* Memories definition */
MEMORY
{
  RAM      (xrw)      : ORIGIN = 0x20000000,    LENGTH = 96K
  FLASH    (rx)       : ORIGIN = 0x80000000,    LENGTH = 512K
}

Add IP_CODE into MEMORY AREA

/* Memories definition */
MEMORY
{
  RAM      (xrw)      : ORIGIN = 0x20000000,    LENGTH = 96K
  FLASH    (rx)       : ORIGIN = 0x80000000,    LENGTH = 256K
  IP_CODE  (rx)       : ORIGIN = 0x80400000,    LENGTH = 256K
}
```

将下面的代码放在脚本中稍低的位置，在链接器脚本文件的.data和.bss部分之间:

Example:

```
.ip_code :
{
  *(.IP_Code*);
} > IP_CODE
```

这告诉链接器将所有命名为.IP_CODE的节放入IP_CODE内存区域，该区域指定从目标内存地址0x80400000开始。

在C代码中，通过添加告诉编译器哪些函数必须到此部分
__attribute__((section(".IP_Code"))) before the function declaration.

Example:

```
__attribute__((section(".IP_Code"))) int myIP_read()
{
  // Add code here...
  return 1;
}
```

myIP_read()函数现在被链接器放在IP_CODE内存区域中。

2.5.7.2 Place code in RAM

要将代码放入RAM中，需要对链接器脚本和启动代码进行一些修改。下面的示例描述了当内部RAM被分割成几个部分，代码被放置并在其中一个内部RAM部分中执行时应用的更改。

在链接器脚本的memory{}区域中定义一个新的内存区域：

```
Original MEMORY AREA

/* Memories definition */
MEMORY
{
  RAM      (xrw)      : ORIGIN = 0x20000000,    LENGTH = 96K
  FLASH    (rx)       : ORIGIN = 0x80000000,    LENGTH = 512K
}

Split RAM into memory areas RAM1, RAM_CODE, RAM

/* Memories definition */
MEMORY
{
  RAM1      (xrw)      : ORIGIN = 0x20000000,    LENGTH = 16K
  RAM_CODE  (xrw)      : ORIGIN = 0x20004000,    LENGTH = 16K
  RAM       (xrw)      : ORIGIN = 0x20008000,    LENGTH = 64K
  FLASH     (rx)       : ORIGIN = 0x80000000,    LENGTH = 512K
}
```

为链接器脚本中的代码定义输出部分。这必须与属于闪存的负载内存地址(LMA)和RAM中的虚拟内存地址(VMA)一起放置：

```
/* load code used by the startup code to initialize the ram code */
_siram_code = LOADADDR(.RAM_CODE);
.RAM_CODE :
{
  . = ALIGN(4);
  _sram_code = .; /* create a global symbol at ram_code start */
  *(.RAM_Code)    /* .RAM_Code sections */
  *(.RAM_Code*)   /* .RAM_Code* sections */
  . = ALIGN(4);
  _eram_code = .; /* define a global symbol at ram_code end */
} >RAM_CODE AT> FLASH
```

必须初始化RAM代码区，并将代码从闪存复制到RAM代码区。启动代码可以访问位置信息符号_siram_code、_sram_code和_eram_code。在启动文件中添加RAM_CODE的加载地址符号：

```
/* Load address for RAM_CODE */
.word _siram_code;
.word _sram_code;
.word _eram_code;
```

在启动代码中添加一段代码，将RAM代码从Flash memory (LMA)复制到RAM (VMA):

```
Reset_Handler:
    ldr    sp, =_estack          /* set stack pointer */

/* Copy the ram code from flash to RAM */
    movs  r1, #0
    b     LoopRamCodeInit

RamCodeInit:
    ldr  r3, =_sram_code
    ldr  r3, [r3, r1]
    str  r3, [r0, r1]
    adds r1, r1, #4

LoopRamCodeInit:
    ldr  r0, =_sram_code
    ldr  r3, =_eram_code
    adds r2, r0, r1
    cmp  r2, r3
    bcc  RamCodeInit

/* Copy the data segment initializers from flash to SRAM */
    movs  r1, #0
    b     LoopCopyDataInit

CopyDataInit:
```

在C代码中，通过在函数声明前添加attribute ((section(".RAM_Code")))来指示编译器哪些函数必须进入该section:

```
__attribute__((section(".RAM_Code"))) int myRAM_read()
{
    // Add code here...
    return 2;
}
```

有关如何使用STM32CubeIDE从CCM SRAM执行应用程序代码的信息，请参阅[ST-12]。它包含了如何设置链接器脚本和启动代码以从RAM执行函数或中断处理程序的示例。[ST-12]第4章中的例子可以作为如何在RAM中添加其他RAM区域和设置代码段的灵感。

2.5.7.3 Place variables at specific addresses

可以将变量放在内存中的特定地址。要实现这一点，必须修改链接器脚本。本节给出的示例将处理产品VERSION_NUMBER、CRC_NUMBER和BUILD_ID的常量变量放在内存中。

第一步是在链接器脚本中创建一个新的内存区域：

```
Original MEMORY AREA

/* Memories definition */
MEMORY
{
  RAM      (xrw)      : ORIGIN = 0x20000000,    LENGTH = 96K
  FLASH    (rx)       : ORIGIN = 0x80000000,    LENGTH = 512K
}

Add a new 2K FLASH_V memory region at end of flash
/* Memories definition */
MEMORY
{
  RAM      (xrw)      : ORIGIN = 0x20000000,    LENGTH = 96K
  FLASH    (rx)       : ORIGIN = 0x80000000,    LENGTH = 512K-2K
  FLASH_V  (rx)       : ORIGIN = 0x807F800,     LENGTH = 2K
}
```

此时，必须添加内存部分：

```
Place the following a bit further down in the script, between the .data { ... } and the .bss
{ ... } section

.flash_v :
{
  *(.flash_v*);
} > FLASH_V
```

这指示链接器将所有名为flash_v*的节放置到flash_v内存区域的flash_v输出节中，该区域指定从目标内存地址0x807 F800开始。

一个section几乎可以被称为任何东西，除了一些预定义的名称，如“.data”。

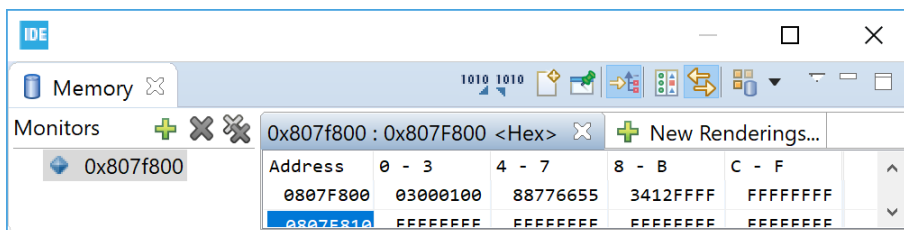
现在，必须定位到FLASH_V内存中的变量必须在C文件中定义属性：

```
__attribute__((section(".flash_v.VERSION"))) const uint32_t VERSION_NUMBER=0x00010003;
__attribute__((section(".flash_v.CRC"))) const uint32_t CRC_NUMBER=0x55667788;
__attribute__((section(".flash_v.BUILD_ID"))) const uint16_t BUILD_ID=0x1234;
```

当调试这个例子并检查内存时，可以观察到：

- Address 0x807 f800 contains VERSION_NUMBER
- Address 0x807 f804 contains CRC_NUMBER
- Address 0x807 f808 contains BUILD_ID

Figure 89. Linker memory output



如果Flash内存中插入的数据顺序很重要，则映射链接器脚本中变量的顺序。这使得可以在任何文件中定义变量。链接器根据文件的链接方式独立地按定义的顺序输出变量。因此，如果CRC_NUMBER在链接器构建文件后以某种方式计算，CRC_NUMBER可以通过另一个工具插入到Flash内存文件中：

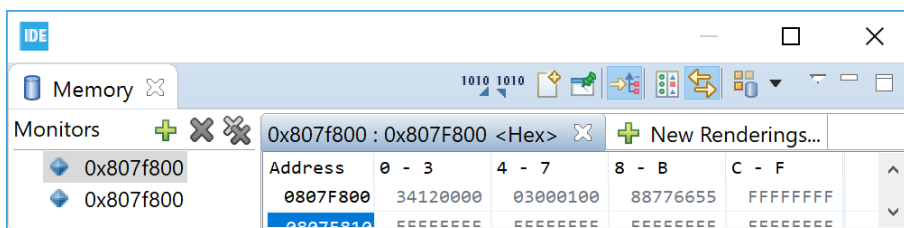
Decide the order in the linker script by adding the specially named sections in order BUILD_ID, VERSION_NUMBER, CRC_NUMBER, and others(*) .

```
.flash_v :
{
  *(.flash_v.BUILD_ID*);
  *(.flash_v.VERSION*);
  *(.flash_v.CRC*);
  *(.flash_v*);
} > FLASH_V
```

When debugging this example and examining the memory, it can be observed that:

- Address 0x807 f800 contains BUILD_ID
- Address 0x807 f804 contains VERSION_NUMBER
- Address 0x807 f808 contains CRC_NUMBER

Figure 90. Linker memory output specified order



2.5.7.4 Linking in a block of binary data

可以将二进制数据块链接到链接的文件中。下面的例子描述了如何包含一个readme.txt文件。

Example:

```
File: readme.txt
Revision: Version 2
Product news: This release ...
```

在项目中包含它的一种方法是在C文件中引用它，使用incbin指令和section上的allocatable ("a")选项:

```
asm(".section .binary_data,\"a\";\"
\".incbin \"../readme.txt\";\"
);
```

然后，新的节binary_data被添加到链接器脚本中，并指示该节必须放入Flash内存中。KEEP()关键字可以用来包围输入节，这样即使没有调用，链接器垃圾收集器也不会消除该节:

```
.binary_data :
{
  _binary_data_start = .;
  KEEP(*(.binary_data));
  _binary_data_end = .;
} > FLASH
```

然后可以从C代码中访问该块:

```
extern int _binary_data_start;
int main(void)
{
  /* USER CODE BEGIN 1 */
  int *bin_area = &_amp;_binary_data_start;
}
```

在调试项目时，可以在Memory视图中观察到二进制数据(在本例中是自述文件)。

Figure 91. Linker memory displaying file readme

Address	0 - 3	4 - 7	8 - B	C - F
08002610	9D220008	89010008	95010008	46696C65
08002620	3A207265	61646D65	2E747874	200D0A52
08002630	65766973	696F6E3A	20566572	73696F6E
08002640	2032200D	0A50726F	64756374	206E6577
08002650	733A2054	68697320	72656C65	61736520
08002660	2E2E2E0D	0A030000	000C0000	000024F4
08002670	00100000	00010000	00180000	20000000
08002680	00040300	206C0300	20040300	20000000
08002690	00000000	00000000	00000000	00000000

2.5.7.5 Locate uninitialized data in memory (NOLOAD)

有时需要在Flash或其他非易失性内存中放置变量，这些变量在启动时不能初始化。在这种情况下，可以在链接器脚本(FLASH_D)中创建一个特定的MEMORY AREA，并在使用该区域的部分中使用NOLOAD指令。

Example:

```
The MEMORY AREA can be defined like this

/* Memories definition */
MEMORY
{
  RAM      (xrw)      : ORIGIN = 0x20000000,    LENGTH = 96K
  FLASH    (rx)       : ORIGIN = 0x80000000,    LENGTH = 512K-4K
  FLASH_D  (rx)       : ORIGIN = 0x807F000,     LENGTH = 2K
  FLASH_V  (rx)       : ORIGIN = 0x807F800,     LENGTH = 2K
}
```

使用NOLOAD指令为FLASH_D添加一个section。这可以使用链接器脚本中下面的代码来完成:

```
Place the following a bit further down in the script

.flash_d (NOLOAD) :
{
  * (.flash_d*);
} > FLASH_D
```

最后，在声明必须位于FLASH_D内存中的变量时，可以通过添加section属性在程序中的某个地方使用数据。

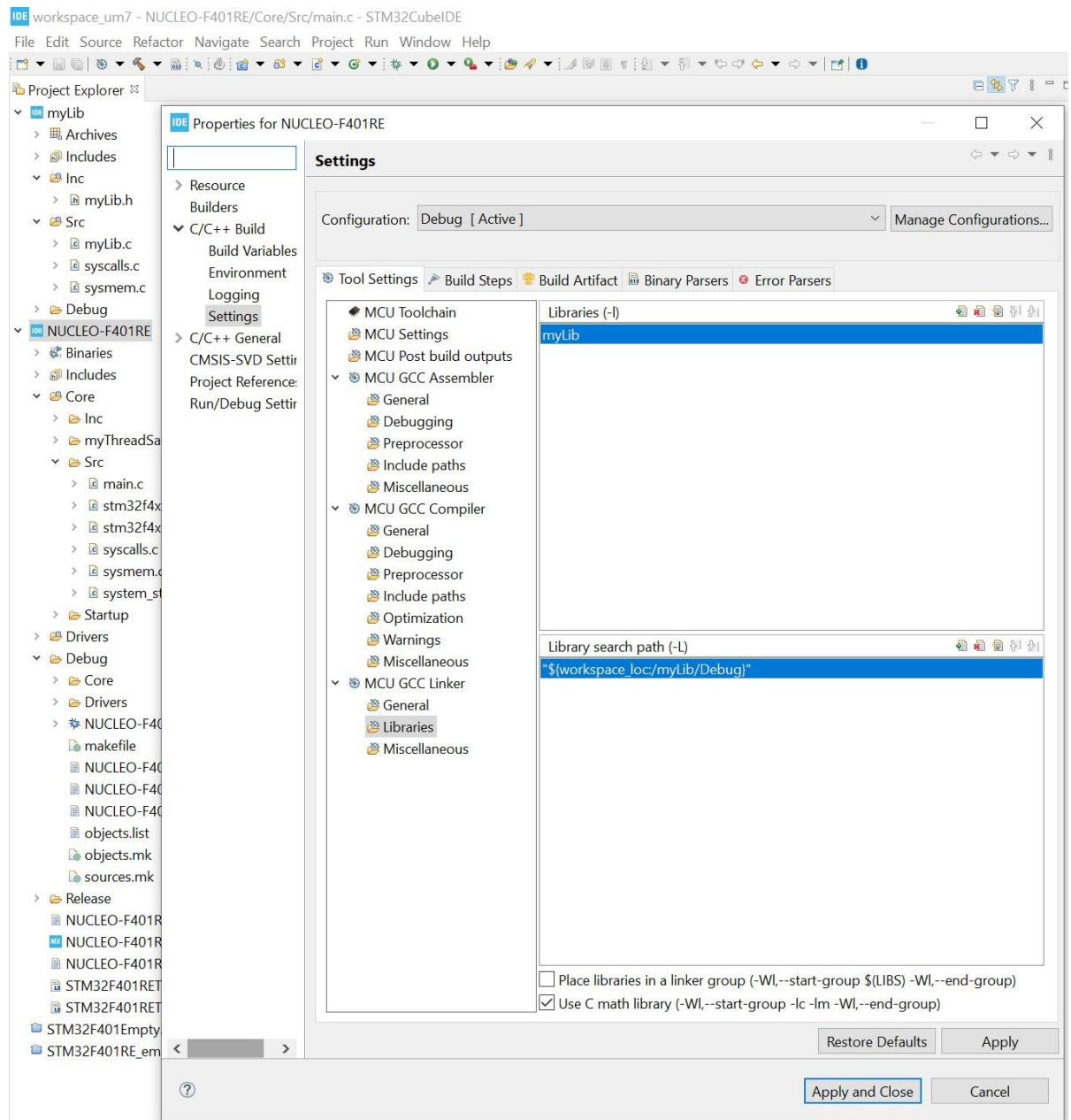
```
__attribute__((section(".flash_d"))) uint32_t Distance;
__attribute__((section(".flash_d"))) uint32_t Seconds;
```

2.5.8 Include libraries

将一个库包含到项目中:

1. Right-click the project where the library must be included in the *Project Explorer* view and select **[Properties]**
2. In the dialog, select **[C/C++ Build]>[Settings]**
3. Select the *Tool Settings* tab in the panel
4. Select **[C Linker]>[Libraries]**
5. Add the library name to the **[Libraries]** field.
 Make sure the library name is added and not the path. According to the GCC convention, the library name is its filename without the *"lib"* prefix and *".a"* extension.
 Example: for a library file named *libmyLib.a*, add the library name *myLib*.
 If by any chance the library name do not comply with the GCC convention, the full library name can be entered, preceded by a colon ":".
 Example: for a library file named *STemWin524b_CM4_GCC.a*, add the library name *:STemWin524b_CM4_GCC.a*.
6. In the **[Library Paths]** list, set the library location path. Do not include the name of the library in the path.
 Example: `${workspace_loc:/myLib/Debug}` is the path to the archive file of the library project *myLib* residing in the same workspace as the application project.
7. Enable **[Place libraries in a linker group (-Wl,--start-group \$(LIBS) -Wl,--end-group)]** if libraries need to be linked several times to resolve circular dependencies.

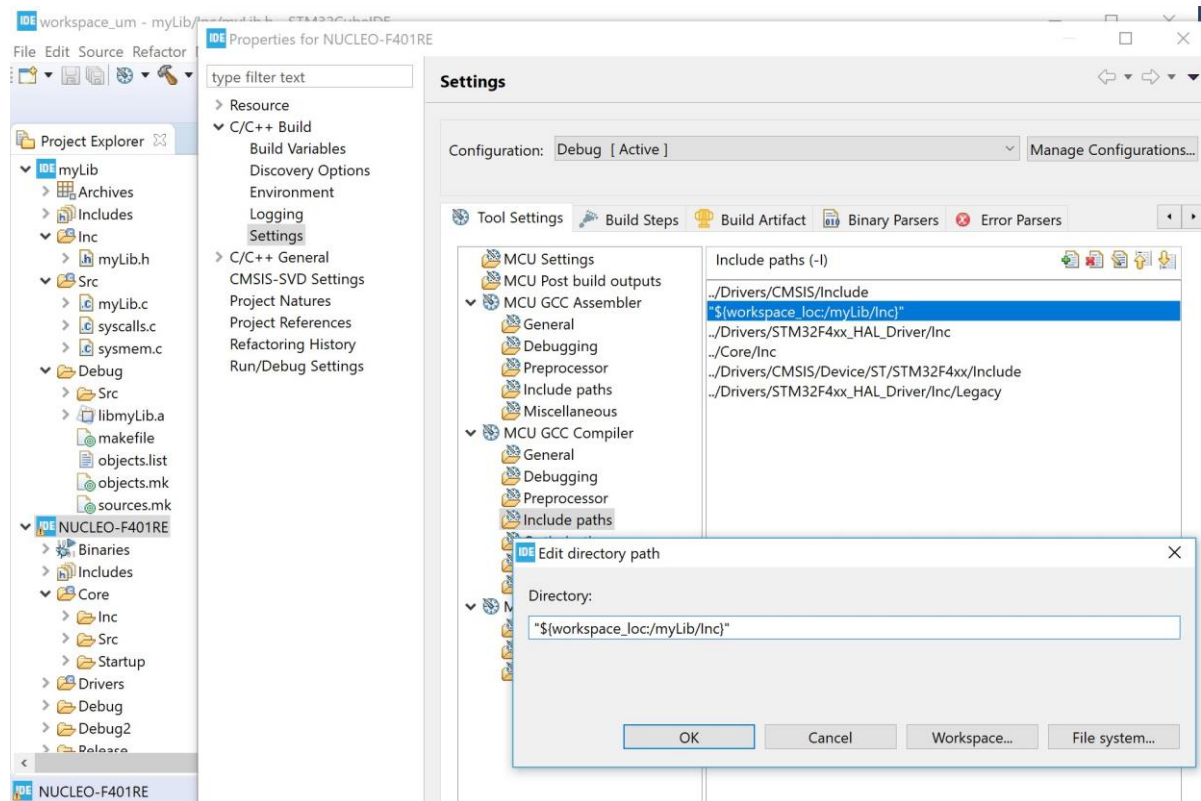
Figure 92. Include a library



头文件的源文件夹也必须添加到[Include paths]字段:

1. Select **[MCU GCC Compiler]>[Include paths]**
2. press the **[Add...]** button and add the paths to the source folders for the header files in the library

Figure 93. Add library header files to the include paths



Note: 由include路径添加的库被认为是静态库，因为它们是由外部方提供的。头文件不会重新扫描，因为外部头文件的内容必须没有更改。如果必须将外部库视为普通的源文件夹，则这些文件夹也必须作为源文件夹添加到项目中。

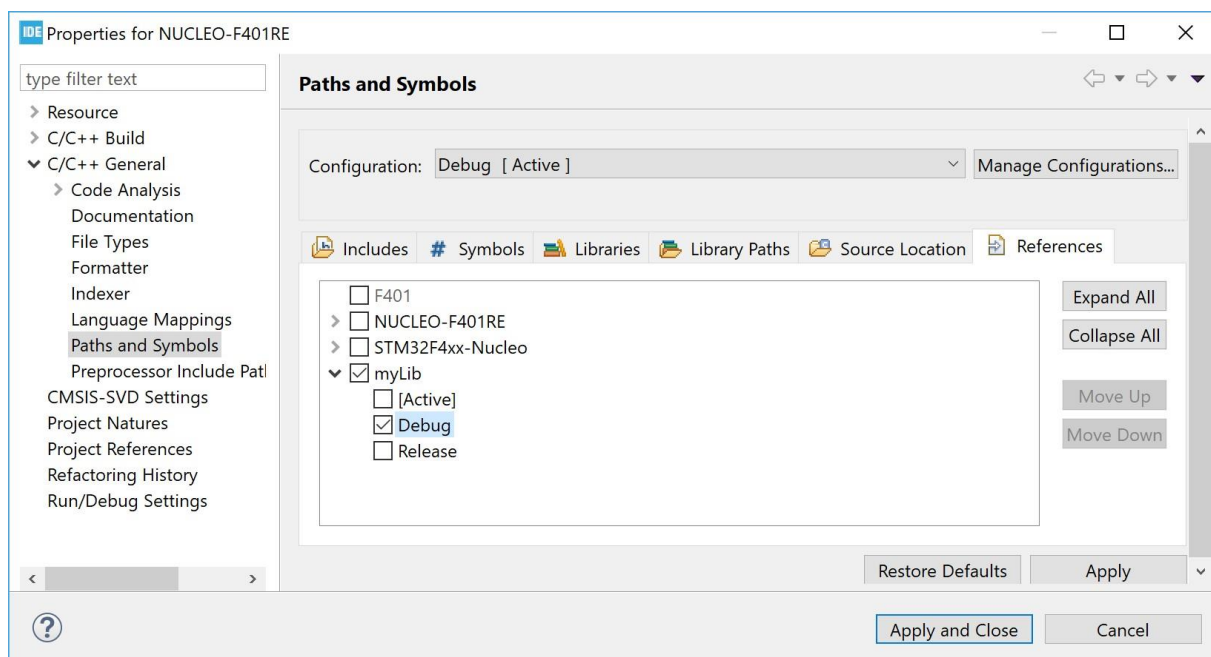
如果一个项目引用了另一个项目、库或普通项目，请参阅2.5.9节引用项目以获得更多信息。

2.5.9 Referring to projects

当一个项目使用来自另一个项目的代码时，两个项目必须相互引用。一个项目引用另一个项目的特定版本：

1. Select instead **[Project]>[Properties]**
2. Select **[C/C++ General]>[Paths and Symbols]**
3. Open the *References* tab
4. select the **[Configuration]** that the current project is referring to

Figure 94. Set project references



Note: 当使用多个项目作为参考时，使用[上移]和[下移]按钮来设置优先级。

正确设置项目引用有很多好处：

- 所涉及的项目没有进行不必要的重建。
- 索引器能够从库中找到函数并打开它们。要使用这种可能性，请按下Ctrl键，然后在编辑器中单击用于在编辑器中打开库源文件的库函数。
- 可以为库中的函数创建调用层次结构。要找到调用层次结构，请标记函数名并按Ctrl+Alt+H在call Hierarchyview中显示调用层次结构。

如果将库项目添加为引用，则将设置该库的“路径和符号”属性页中的所有正确设置。依赖于此属性页的工具设置也会进行调整。

这是添加本地开发的库的推荐方法。有关添加库的更多信息，请参阅第2.5.8节包含库。

另一种让项目相互引用的方法如下：

1. Select [Project]>[Properties]
2. Select [Project References]
3. Select and mark the project for reference

然而，使用这种方法，不可能引用不同的构建配置，库也不会自动设置。

2.6 I/O redirection

C运行时库包含许多函数，包括一些处理I/O的函数。与I/O相关的运行时函数包括printf()、fopen()、fclose()等。通常的做法是将这些函数的I/O重定向到实际的嵌入式平台。例如，printf()输出可以重定向到LCD显示器或串行电缆，而像fopen()和fclose()这样的文件操作可以重定向到Flash内存文件系统中间件。

2.6.1 **printf() redirection**

有几种方法可以执行printf()重定向，例如使用UART或SWV/ITM。另一个解决方案是SEGGER提供的实时传输技术(RTT)。

这三种技术比较如下：

- UART输出可能是最常用的方法，其中嵌入式系统的输出通过RS-232发送到终端。它需要一些CPU开销和中等带宽。
- Instrumentation Trace Macrocell (ITM)输出是高效的，但要求设备支持带有串行连线查看器 (SWV)的Arm®CoreSight™ 调试器技术。这通常是基于Cortex®- M3、Cortex®- M4、Cortex®- M7和Cortex®- M33的设备的情况。但是，SWV信号必须是可用的，并且必须连接到单板上。它需要低CPU开销，但带宽有限。ITM输出将在第4节使用串行连线查看器跟踪 (SWV)进行调试中解释。
- SEGGER在其网站上描述了RTT解决方案。RTT是一个快速的解决方案，但需要SEGGER J-LINK调试探头。

要使用UART或ITM输出启用I/O重定向，必须将syscalls.c文件包含并内置于项目中。当使用printf()时，它调用_write()函数，该函数在syscalls.c中实现。

在创建新的STM32CubeIDE项目时，通常会创建syscalls.c文件并将其包含在项目中。必须修改该文件中的_write()函数，通过修改对io_putchar()的调用来启用printf()重定向。修改_write()的方式取决于硬件和库实现。

下面的示例展示了如何更新syscalls.c，以便使用STM32F4系列设备将printf输出重定向到ITM。这是通过添加一些头文件来访问ITM_SendChar()并调用ITM_SendChar()来实现的。

```
Original _write() function

__attribute__((weak)) int _write(int file, char *ptr, int len)
{
    int DataIdx;

    for (DataIdx = 0; DataIdx < len; DataIdx++)
    {
        __io_putchar(*ptr++);
    }
    return len;
}

Modified with added header files calling ITM_SendChar(*ptr++);

#include "stm32f4xx.h"
#include "core_cm4.h"

__attribute__((weak)) int _write(int file, char *ptr, int len)
{
    int DataIdx;

    for (DataIdx = 0; DataIdx < len; DataIdx++)
    {
        //__io_putchar(*ptr++);
        ITM_SendChar(*ptr++);
    }
    return len;
}
```

可以注意到，syscalls.c中的_write函数包含一个弱属性。这意味着_write函数可以在项目使用的任何C文件中实现。

例如，新的_write()函数可以直接添加到main.c中。在这种情况下，省略弱属性，如下例所示。

```
int _write(int file, char *ptr, int len)
{
    int DataIdx;

    for (DataIdx = 0; DataIdx < len; DataIdx++)
    {
        //__io_putchar(*ptr++);
        ITM_SendChar(*ptr++);
    }
    return len;
}
```

2.7 Thread-safe wizard for empty projects and CDT™ projects

STM32CubeIDE包含一个线程安全的向导，用于生成支持使用资源的文件，这些资源可以由应用程序代码和中断更新，也可以在使用实时操作系统时更新。

Note: 线程安全向导只能用于STM32CubeIDE空项目。对于由STM32CubeMX管理的项目，必须使用STM32CubeMX对话框进行线程安全的实现配置。

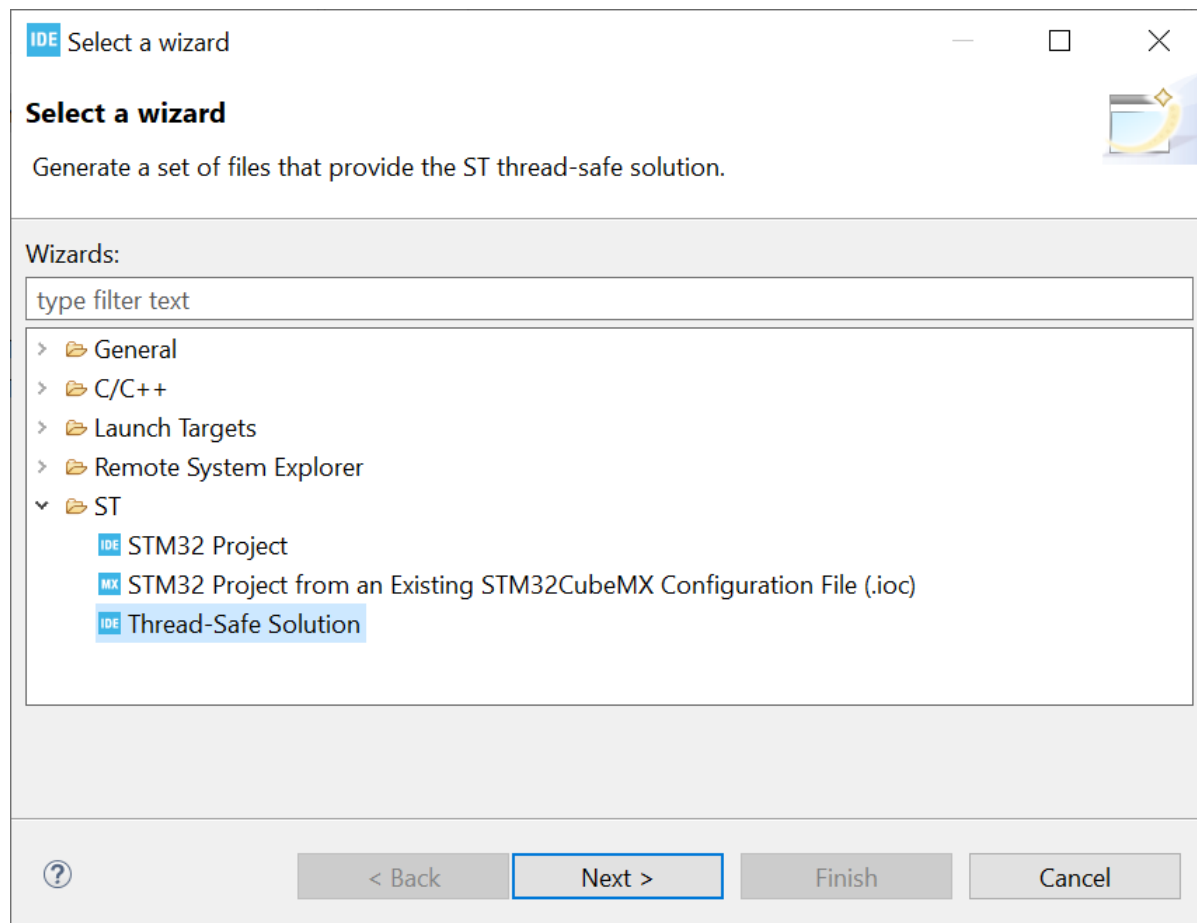
线程安全向导创建三个文件，并将STM32_THREAD_SAFE_STRATEGY定义添加到项目中。这些文件是：

- newlib_lock_glue.c
- stm32_lock_user.h
- stm32_lock.h

首先，在下面的示例中，在空项目中创建了一个myThreadSafe文件夹。在线程安全解决方案向导中选择此文件夹，以便在此文件夹中生成文件。

打开菜单[File]>[New]>[Other...], 得到如图95所示的向导选择窗口。

Figure 95. Select a wizard



在[ST]节点中选择[Thread-Safe Solution]并按[Next >]打开线程安全解决方案向导。

Figure 96. Thread-Safe Solution wizard

IDE Generate Thread-Safe Solution

Thread-Safe Solution

✖ Folder 'STM32F401RE_empty' is not in a source folder.

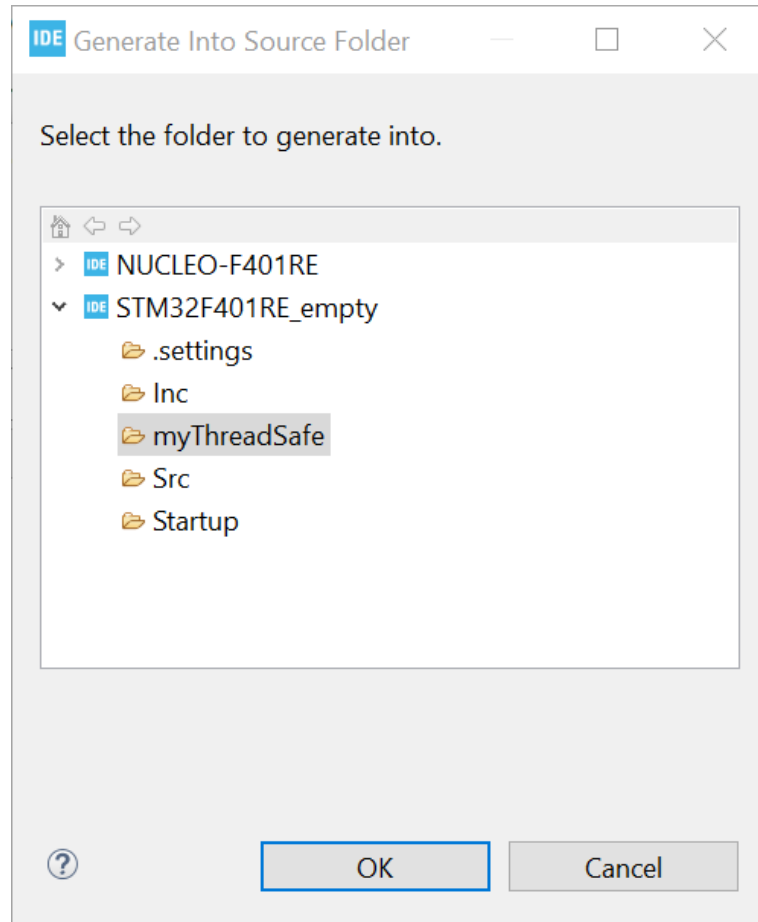
Source folder: STM32F401RE_empty Browse...

Strategy: ▾

? < Back Next > Finish Cancel

按[Browse]打开“生成到源文件夹”对话框。

Figure 97. Thread-safe source folder location



选择要生成文件的源文件夹并按[OK]。

向导建议在五种不同的线程安全策略中进行选择：

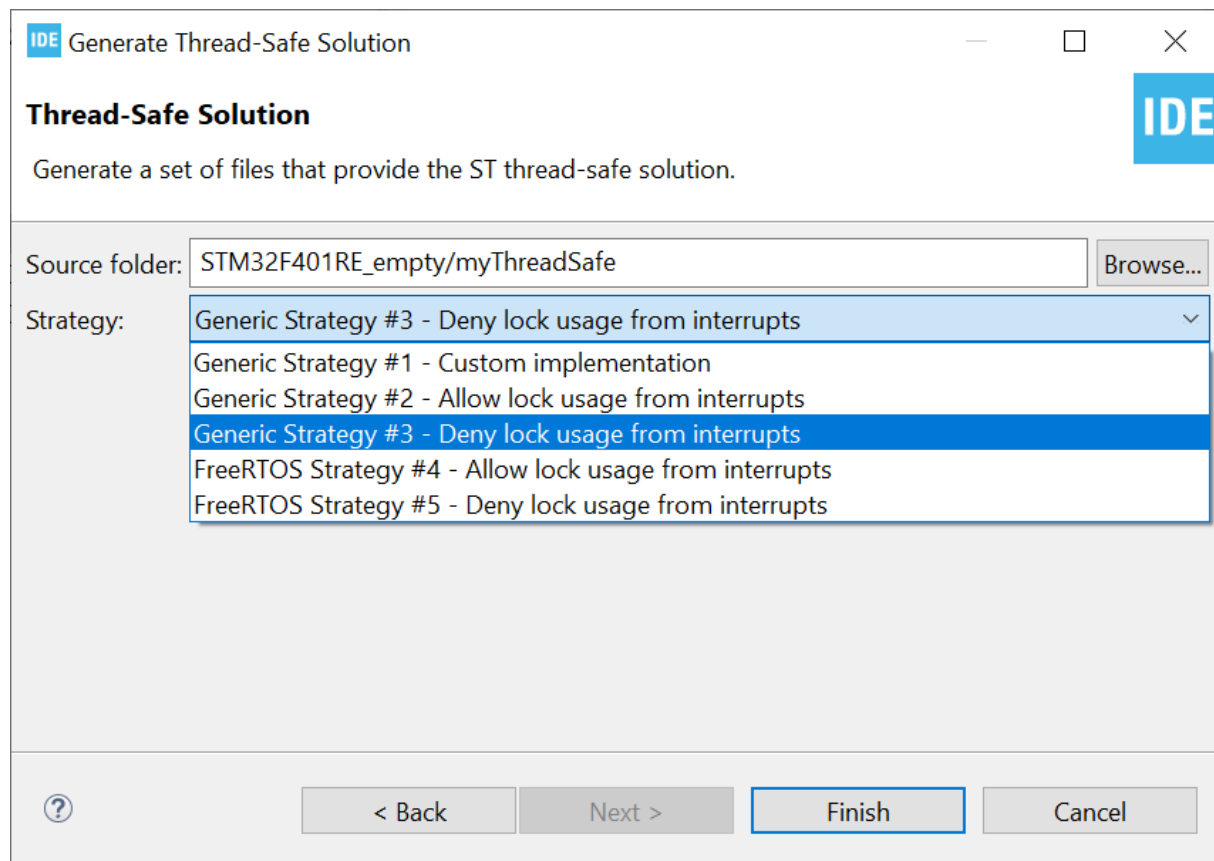
1. User-defined thread-safe implementation. 用户定义的线程安全实现
2. Allow lock usage from interrupts. 允许从中断中使用锁
3. Deny lock usage from interrupts. 拒绝从中断中使用锁
4. Allow lock usage from interrupts. Implemented using FreeRTOS™ locks. 允许从中断中使用锁。使用 FreeRTOS™ 锁实现。
5. Deny lock usage from interrupts. Implemented using FreeRTOS™ locks. 拒绝从中断中使用锁。使用 FreeRTOS™ 锁实现。

stm32_lock.h文件解释了不同的策略。

```
* 1. User defined thread-safe implementation.
*   User defined solution for handling thread-safety.
*   NOTE: The stubs in stm32_lock_user.h needs to be implemented to gain
*   thread-safety.
*
* 2. Allow lock usage from interrupts.
*   This implementation will ensure thread-safety by disabling all interrupts
*   during e.g. calls to malloc.
*   NOTE: Disabling all interrupts creates interrupt latency which
*   might not be desired for this application!
*
* 3. Deny lock usage from interrupts.
*   This implementation assumes single thread of execution.
*   Thread-safety dependent functions will enter an infinity loop
*   if used in interrupt context.
*
* 4. Allow lock usage from interrupts. Implemented using FreeRTOS locks.
*   This implementation will ensure thread-safety by entering RTOS ISR capable
*   critical sections during e.g. calls to malloc.
*   By default this implementation supports 2 levels of recursive locking.
*   Adding additional levels requires 4 bytes per lock per level of RAM.
*   NOTE: Interrupts with high priority are not disabled. This implies
*   that the lock is not thread-safe from high priority interrupts!
*
* 5. Deny lock usage from interrupts. Implemented using FreeRTOS locks.
*   This implementation will ensure thread-safety by suspending all tasks
*   during e.g. calls to malloc.
*   NOTE: Thread-safety dependent functions will enter an infinity loop
*   if used in interrupt context.
```

选择一个策略，如图98

Figure 98. Thread-safe strategy selection

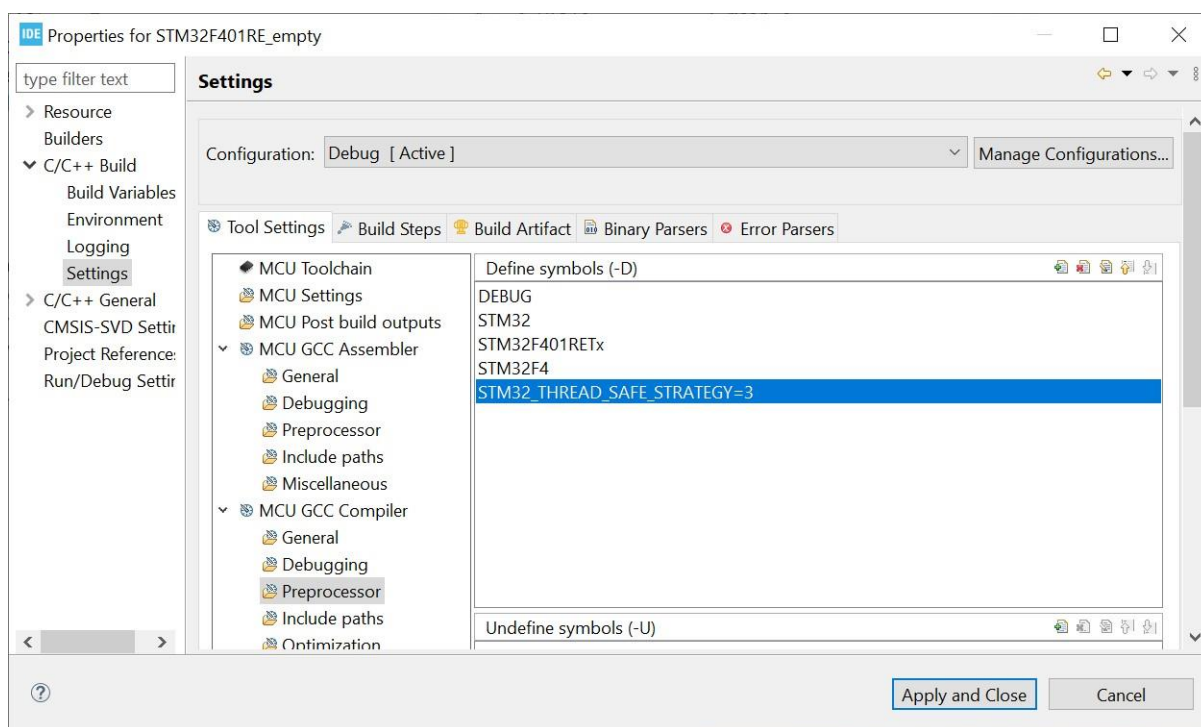


选择一个策略并按[Finish]将文件生成到选定的源文件夹中。

Note: 生成的文件是相同的，包含相同的信息，独立于所选的策略。

向导将一个新定义STM32_THREAD_SAFE_STRATEGY=3添加到项目中，供预处理器在构建项目时使用。定义值根据向导中选择的策略设置。可以通过打开项目属性并查看Tool Settings选项卡来观察定义。

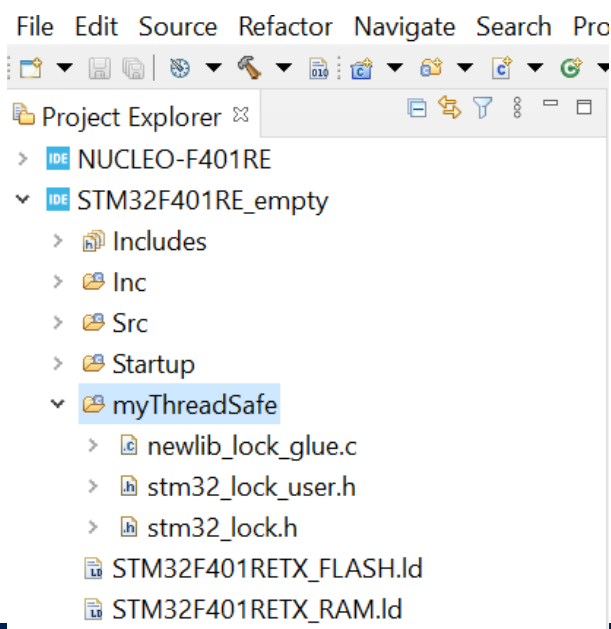
Figure 99. Thread-safe properties



生成的文件显示在Project Explorer中。

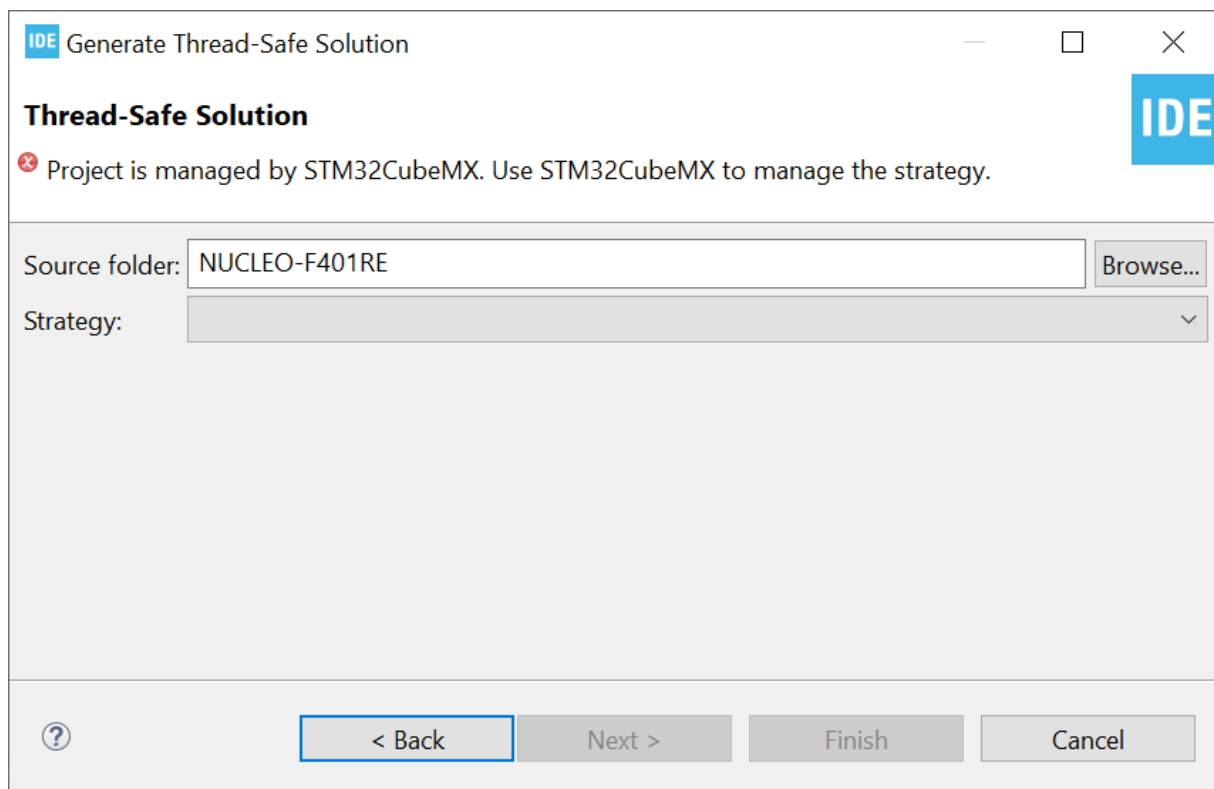
Note: 生成的文件要求项目中包含一些CMSIS头文件。如果缺少这些文件，则必须手动复制并添加到项目中

Figure 100. Thread-safe files



如果在项目由STM32CubeMX管理时启动向导，则会显示一个错误，说明必须使用STM32CubeMX来管理线程安全策略。

Figure 101. Thread-safe error dialog



2.8 Position-independent code

对于在系统中没有定义最终地址位置的应用程序中工作的用户，此部分很有用。这发生在使用引导加载程序时：系统设计人员必须能够定义应用程序的最终位置。在这种情况下，可以使用位置无关代码(PIC)。fpie编译器选项允许编译器/链接器生成与位置无关的可执行文件。

使用选项-fPIE编译会生成与位置无关的可执行文件，这样如果应用程序链接到地址0x800 0000，但放置在地址0x800 1000，它仍然会运行。

但是，本节中的信息并不完整。它描述的解决方案在使用初始化为零的全局数据(.bss)时有效，但在使用初始化的数据时无效，并且有其他几个限制。其中一个限制是不能使用STM32工具链中包含的运行时库，因为这些库在构建时没有使用-fPIE选项进行优化。与其在系统中使用与位置无关的代码，不如考虑其他解决方案。

Alternate solution example:

如果一个系统被设计成在Flash内存的不同插槽中有一个引导加载程序和一个应用程序的多个版本，那么为应用程序设置多个构建配置可能会更容易。每个构建配置都基于自己的链接器脚本文件。在这种情况下，不需要使用与位置无关的代码，因为可以使用运行时库。每个构建配置将应用程序链接到Flash内存中的唯一插槽，每个插槽生成一个elf文件。当将应用程序的新版本下载到插槽时，必须使用正确的elf文件。引导加载程序可以设计为验证elf文件中的地址，如果其中包含超出插槽的地址，则生成错误。应用程序可以将中断向量表复制到RAM中，并根据应用程序存储的插

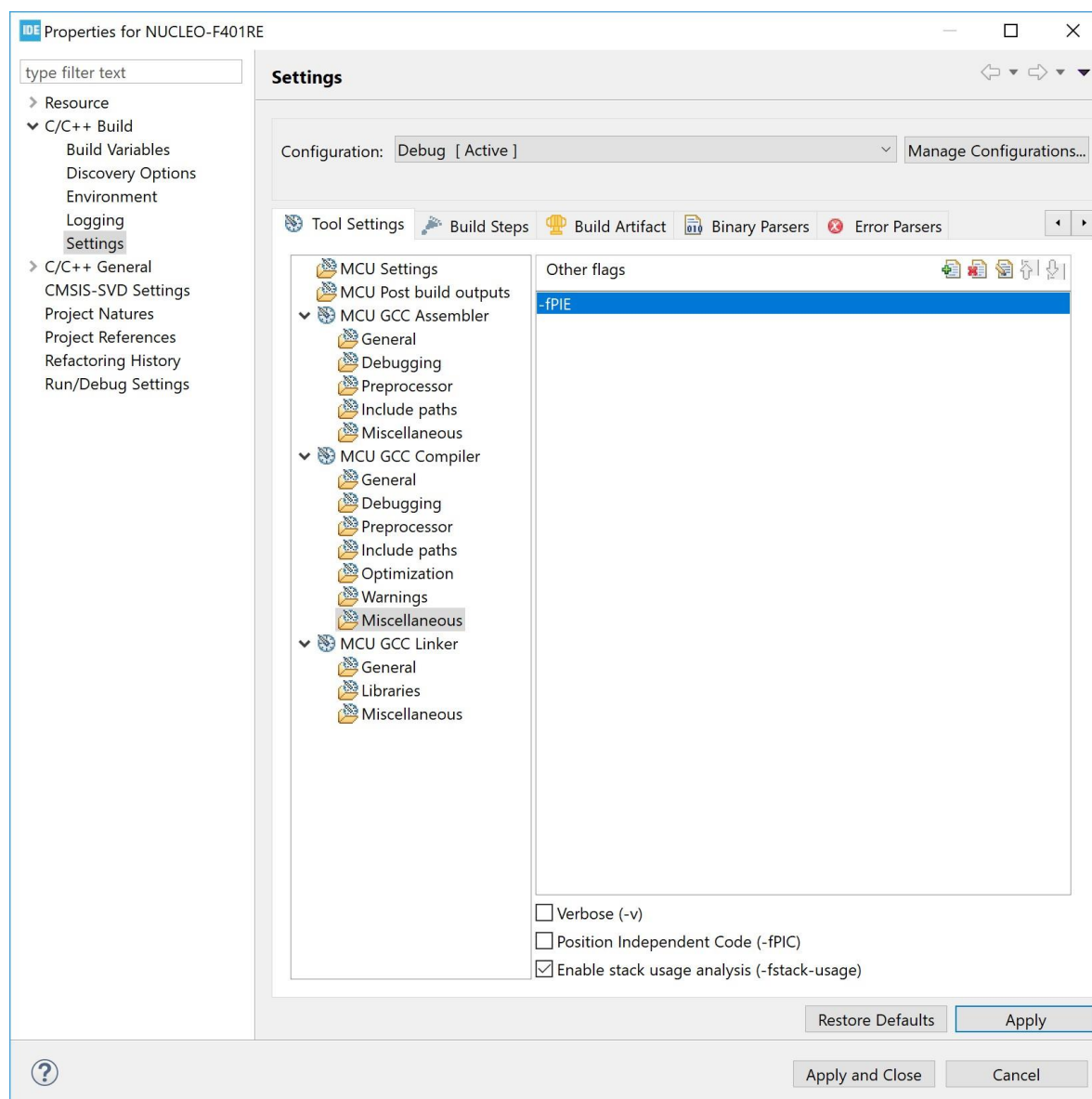
槽更新向量副本。

2.8.1 Adding the `-fPIE` option

To add the `-fPIE` option into the tool settings:

1. Right-click the project in the *Project Explorer* view and select **[Properties]**
2. In the dialog, select **[C/C++ Build]>[Settings]**
3. Select the *Tool Settings* tab in the panel
4. Select **[MCU GCC Compiler]>[Miscellaneous]**
5. Add `-fPIE` to the **[Other flags]** field.

Figure 102. Position independent code, `-fPIE`



2.8.2 Run time library

编译C运行时库时不使用-fPIE选项。因此，在生成与位置无关的可执行文件时，必须避免对库的任何调用。启动代码通常调用libc_init_array。这个调用必须被移除，如下面的例子所示：

```
/* Call static constructors */
/* bl __libc_init_array */
```

2.8.3 Stack pointer configuration

确保堆栈指针设置正确。堆栈指针必须在启动文件中的Reset_Handler中设置，如下例所示。不能假定堆栈指针是通过从vector表中读取它的重置来设置的。

```
Reset_Handler:
    ldr    sp, =_estack          /* set stack pointer */
```

2.8.4 Interrupt vector table

如果程序加载到偏移地址，则必须更新向量表中的向量。如果程序需要向表中的每个向量添加偏移量，它可以将中断向量表复制到RAM中，并将偏移量添加到这个向量表中。

向量基寄存器也必须更改，以便它指向新定位的向量表，如下例所示：

```
/* Set Vector Base Address */
SCB->VTOR=RAM_VectorTable;
```

2.8.5 Global offset table

全局偏移表(GOT)是在构建和使用-fPIE选项时通常存储在数据部分中的地址表。被执行的程序使用它在运行时查找编译时未知的全局变量的地址。如果不需要更改全局变量的位置，则可以将变量放置在与链接程序时相同的位置。然后将GOT表放在Flash内存区域的.text部分中。

下面的例子展示了如何使用.got*部分更新链接器脚本。在本例中，还添加了GOT_START和GOT_END符号，以便工具能够知道GOT的位置和大小。

```
/* The program code and other data into "ROM" Rom type memory */
.text :
{
    . = ALIGN(4);
    *(.text)          /* .text sections (code) */
    *(.text*)         /* .text* sections (code) */
    GOT_START = .;
    *(.got*)
    GOT_END = .;
    *(.glue_7)        /* glue arm to thumb code */
    *(.glue_7t)       /* glue thumb to arm code */
    *(.eh_frame)

    KEEP (*(init))
    KEEP (*(fini))

    . = ALIGN(4);
    _etext = .;        /* define a global symbols at end of code */
} >ROM
```

2.8.6 Interrupt vector table and symbols

当调试带有偏移量的代码时，必须同时指定加载偏移量和新符号地址。要提供的符号地址是.text节地址。链接器脚本可以通过将.isr_vector定义为定位到.text中来更新。这避免了查找.text位置的问题。

```
Remove the following

.isr_vector :
{
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
} >FLASH

Add KEEP(*(.isr_vector)) instead to first location of .text
/* The program code and other data into "FLASH" Rom type memory */
.text :
{
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    *(.text)              /* .text sections (code) */
    *(.text*)             /* .text* sections (code) */
    GOT_START = .;
    *(.got*)
    GOT_END = .;
    *(.glue_7)            /* glue arm to thumb code */
    *(.glue_7t)           /* glue thumb to arm code */
    *(.eh_frame)

    KEEP (*(.init))
    KEEP (*(.fini))

    . = ALIGN(4);
    _etext = .;           /* define a global symbols at end of code */
} >FLASH
```

2.8.7 Debugging position-independent code

当调试位于偏移量的位置无关代码时，必须设置下载偏移量和新符号地址。

Figure 103. Debugging position independent code

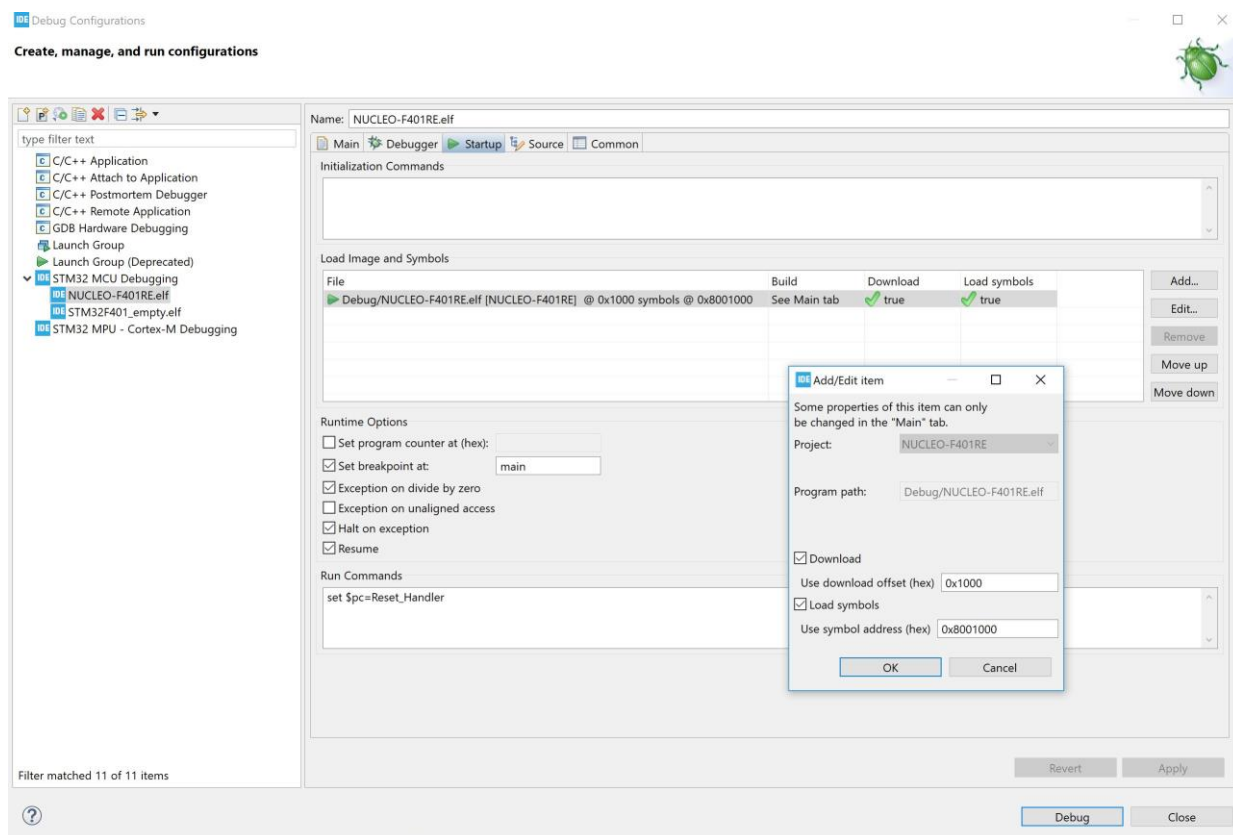


图103演示了一个示例，其中下载偏移量为0x1000，符号地址为0x800 1000。在这种情况下，可以将符号地址设置为0x800 1000，因为.isr_vector被添加到.text部分，如第2.8.6节中断向量表和符号所述。

如果.isr_vector位于.text之外的另一个节中，则必须使用.text节的起始地址并添加偏移量。例如，如果映射文件指出.text从0x0000 0000 0800 0194开始，那么在这种情况下，符号地址必须设置为0x800 1194。

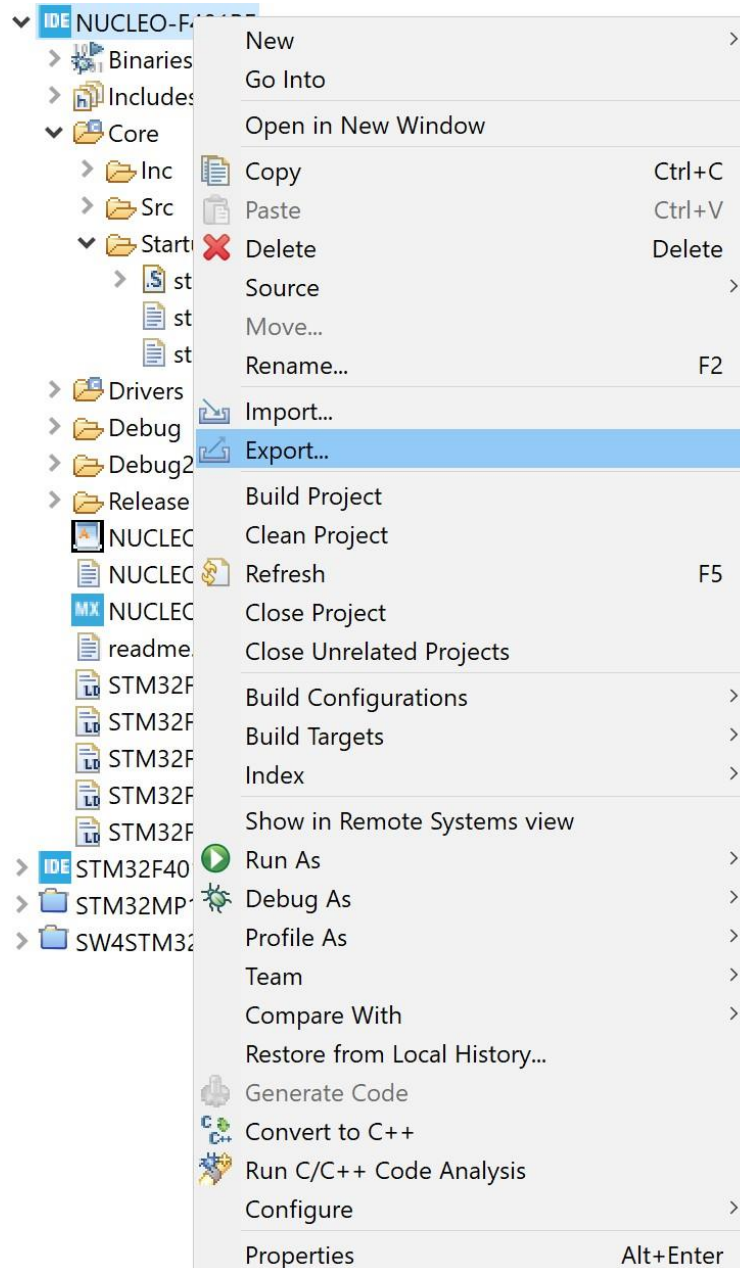
图103显示断点设置在main，程序计数器(\$pc)设置为Reset_Handler符号到[Run Commands]中。这个符号包含Reset_Handler的正确地址，因为gdb使用基符号地址0x800 1000。如果在此调试配置期间没有设置\$pc，则必须禁用[Resume]复选框以使程序在加载后停止。在这种情况下，必须在启动程序之前在Registers视图中手动设置程序计数器。

2.9 Exporting projects

一个项目可以用许多不同的方式导出。本节展示如何将项目导出为压缩zip文件。

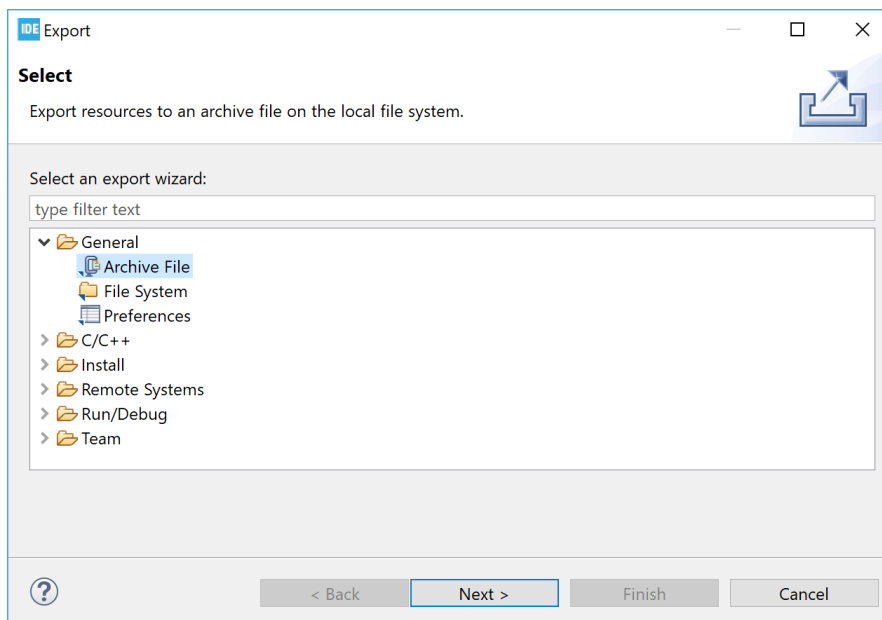
在project Explorer视图中右键单击项目并选择[Export...]

Figure 104. Export project



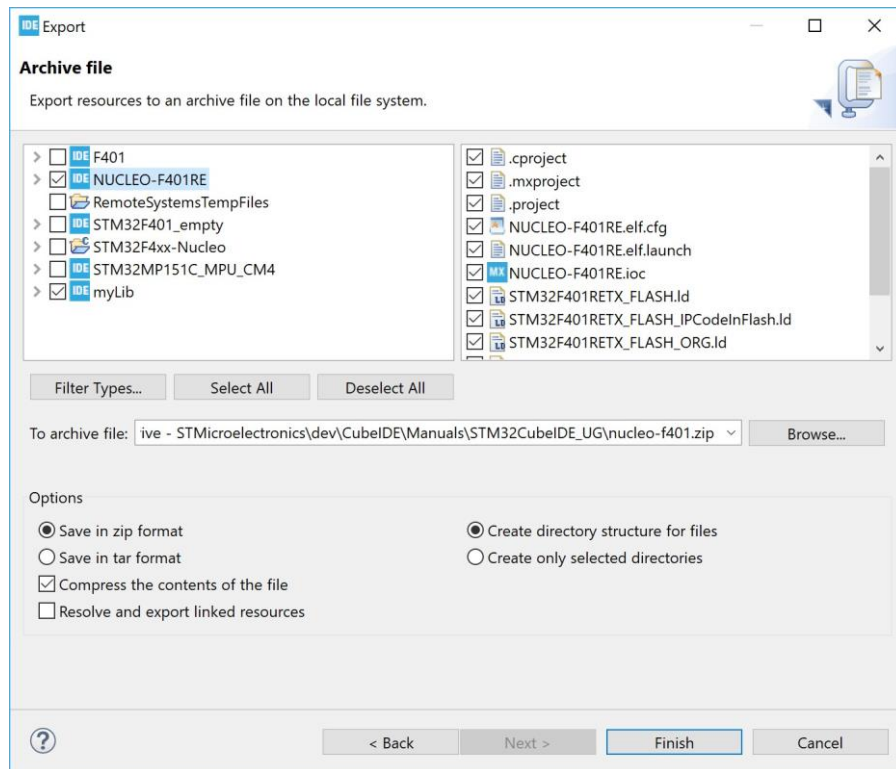
导出对话框打开。选择[General]>[Archive File]，按[Next >]。

Figure 105. Export dialog



“导出”对话框更新。选择要导出的项目。可以从导出中排除一些项目文件。在图106的示例中，包括了所有项目和库文件。必须在[To archive file]字段中输入文件名，可能使用[Browse...]浏览文件的文件夹位置。在本例中，默认选项值保持不变。按[Finish]导出项目并创建zip文件。

Figure 106. Export archive



2.10 Importing existing projects

本节描述将现有项目导入STM32CubeIDE工作区的不同方法。标准Eclipse®导入器能够导入Eclipse®项目。这用于导入用STM32CubeIDE创建的项目。项目导入器也得到扩展，以支持STM32项目和Atollic®TrueSTUDIO®项目的ac6 System Workbench的导入。这些项目在导入阶段转换为STM32CubeIDE项目。

可以导入和调试由其他IDE或工具链开发的现有elf文件。关于如何做到这一点的更多信息，请参见第3.8节。

2.10.1 Importing an STM32CubeIDE project

可以通过许多不同的方式导入项目。本节展示如何导入已导出为压缩zip文件的项目。

- One way to open the *Import* dialog is to use the menu [File]>[Import...]
- Another way is to right-click the *Project Explorer* view and select [Import...]

Figure 107. Import project

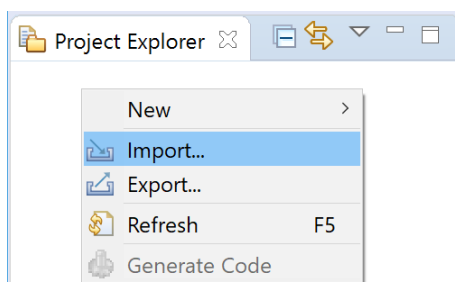


Figure 108. Import dialog

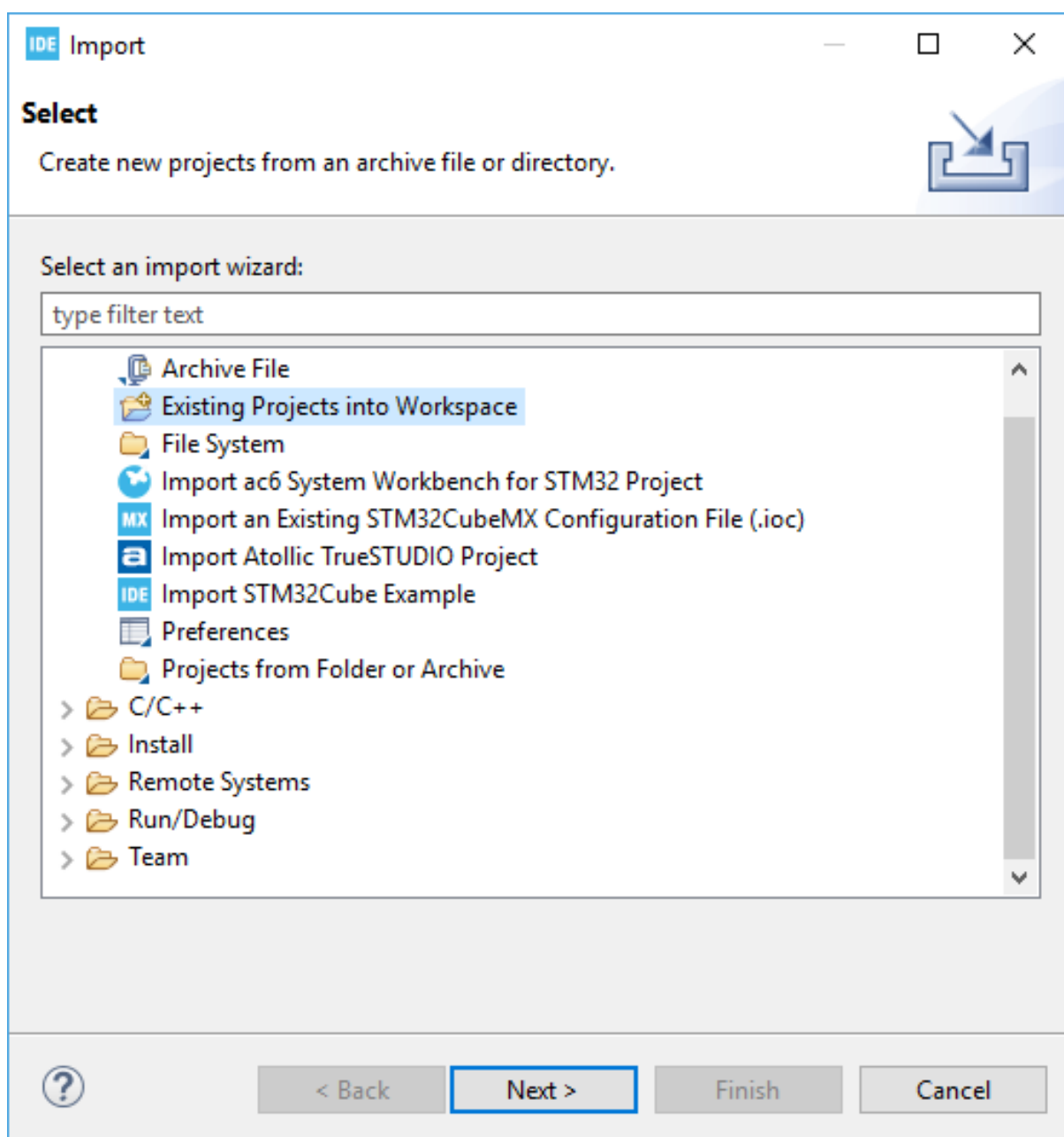
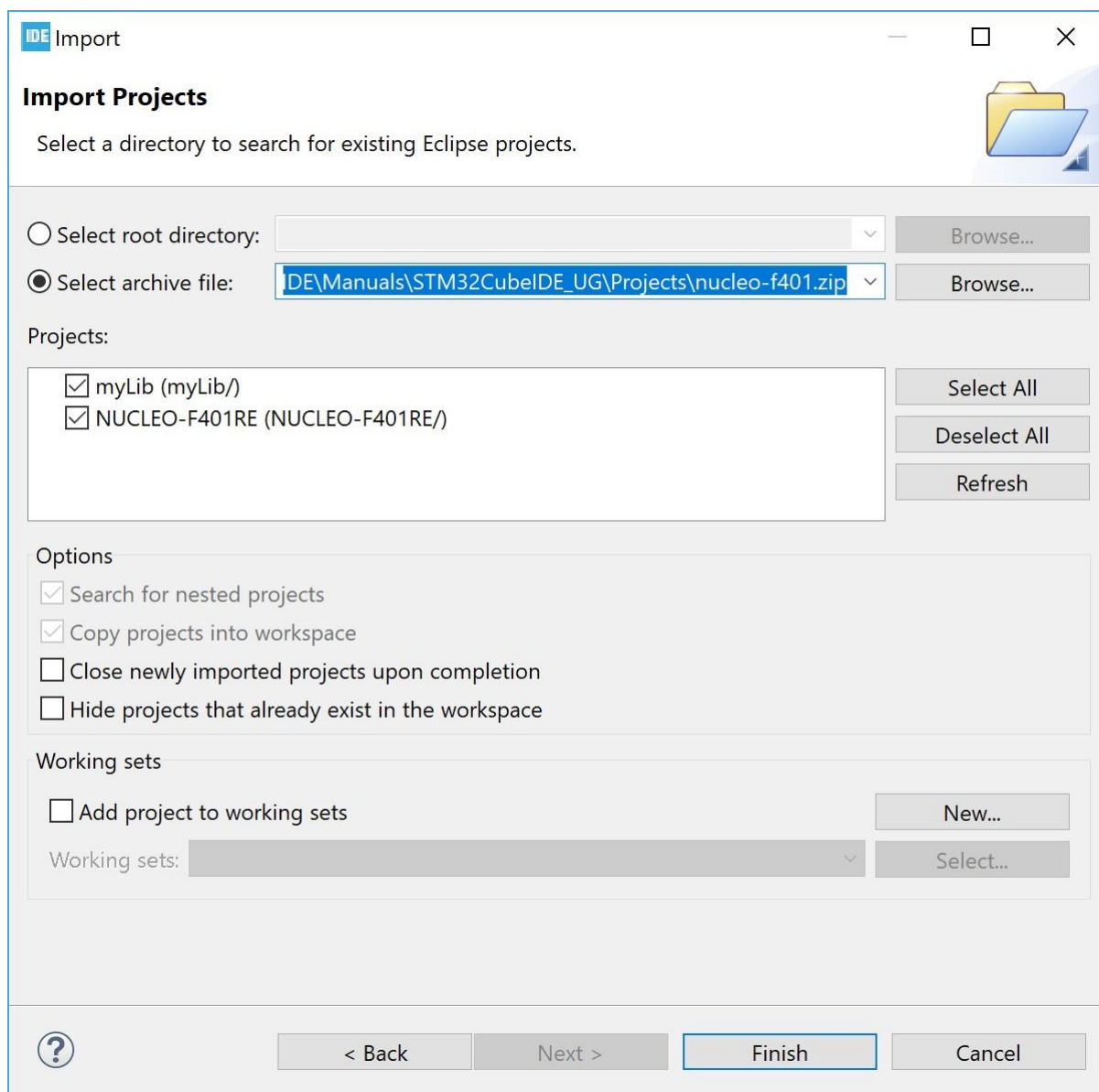


Figure 109. Import projects



2.10.2 Importing System Workbench and TrueSTUDIO® projects

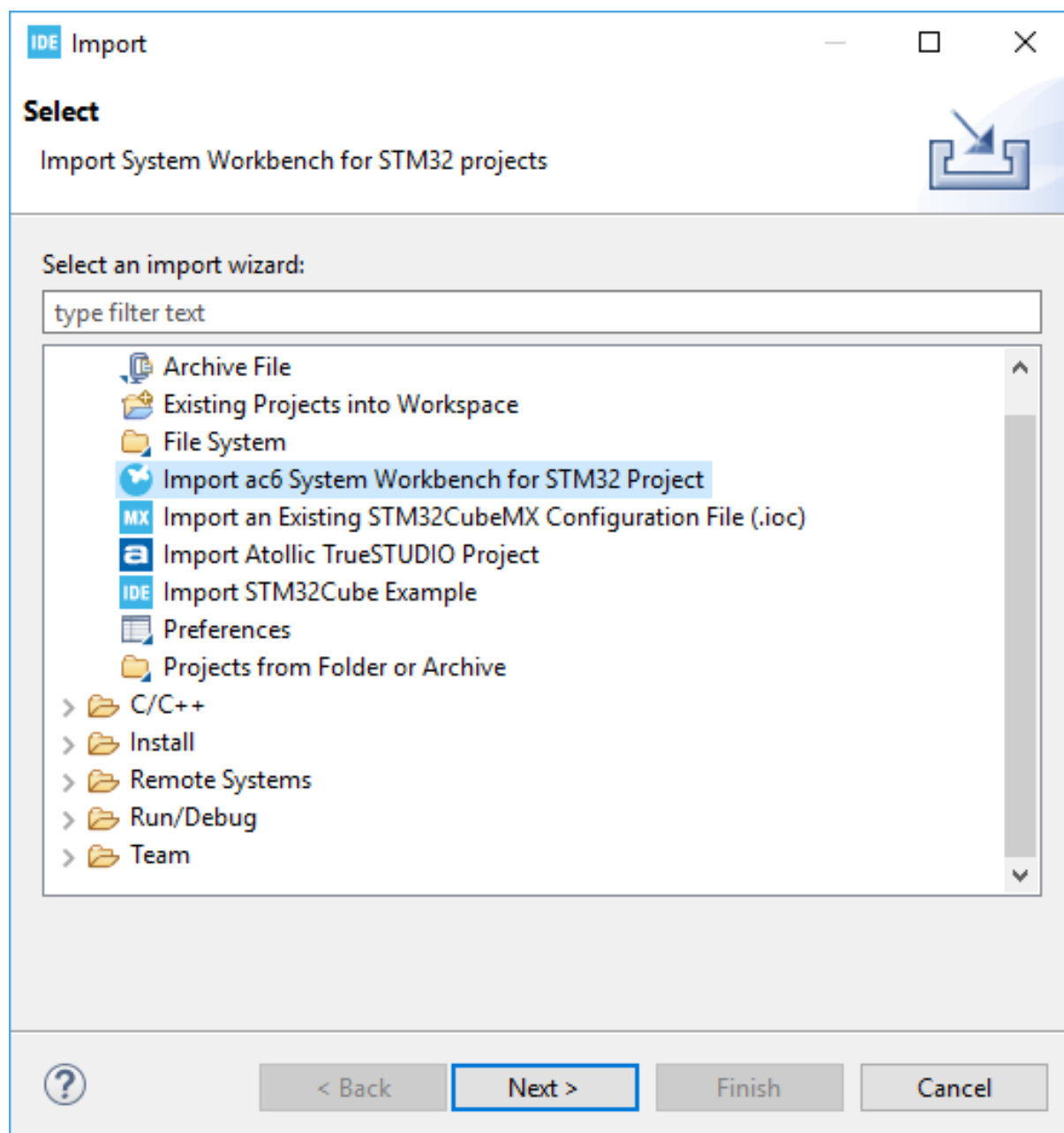
要将STM32项目的ac6系统工作台或Atollic®TrueSTUDIO®项目导入STM32CubeIDE，建议在项目副本上工作：

1. 创建项目的副本，可以是项目文件夹的副本，也可以是zip文件中的项目导出
2. 使用复制的项目导入到STM32CubeIDE

导入复制的项目的方法是通过菜单[File]>[import...]或右键单击project Explorer视图打开import对话框。

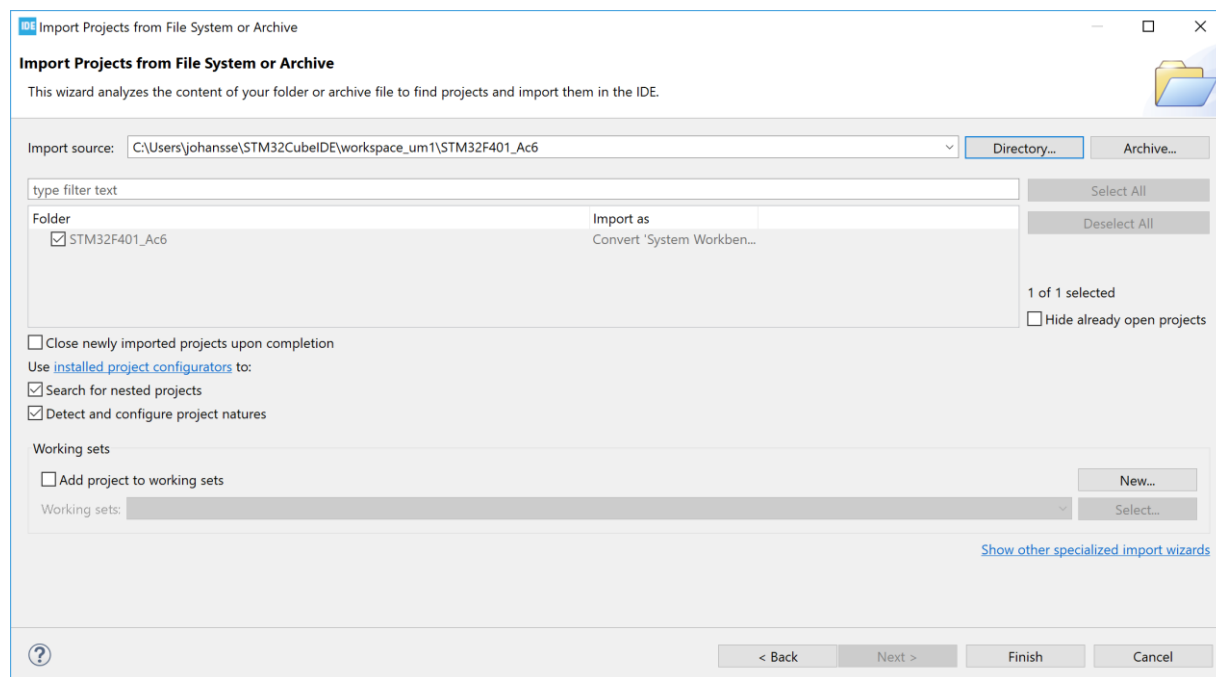
根据用于创建项目的原始工具选择[Import ac6 System Workbench for STM32 project]或[Import Atollic TrueSTUDIO project]，然后按[Next >]。

Figure 110. Import System Workbench projects (1 of 3)



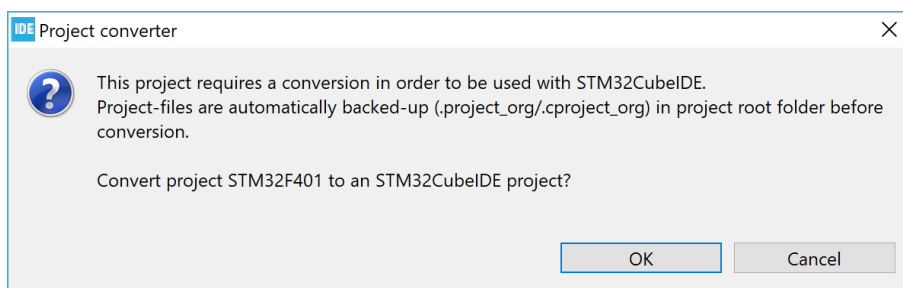
在本例中，ac6项目被复制到STM32CubeIDE工作空间中，因此使用[Directory...]按钮，并选择项目STM32F401_Ac6。导入向导检测到这是一个System Workbench项目。

Figure 111. Import System Workbench projects (2 of 3)



按[Finish]打开Project converter对话框。

Figure 112. Import System Workbench projects (3 of 3)



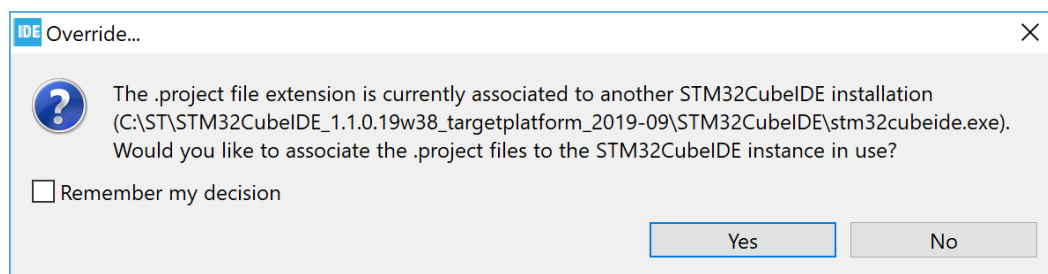
按[OK]将项目转换为STM32CubeIDE项目。

有两个迁移指南解释如何从ac6 System Workbench for STM32 ([ST-06])和Atollic®TrueSTUDIO®迁移到STM32CubeIDE ([ST-05])。这些指南可以从信息中心的技术文档页面打开。

2.10.3 Importing using project files association

当STM32CubeIDE启动时，一个弹出窗口询问.cproject和.project文件是否必须与程序关联。

Figure 113. Import using project files association



如果选择了关联，则在个人计算机文件浏览器中双击.project文件，触发STM32CubeIDE将项目导入到当前工作区。项目转换器调查项目，如果为STM32CubeIDE制作，则直接导入该项目。如果项目来自其他工具，项目转换器将尝试识别它是否是已知的项目格式，在这种情况下，将项目转换为STM32CubeIDE项目，如2.10.2节导入System Workbench和TrueSTUDIO项目所述。

2.10.4 Prevent “GCC not found in path” error

当导入旧项目时，Problems视图中的错误可能会显示“Program”gcc“not found in PATH”。此错误是由项目使用已弃用的发现方法设置引起的。该错误可以通过更新窗口首选项和项目属性设置来删除。

1. Open [Window]>[Preferences]. In the *Preferences* dialog, select [C/C++]>[Property Pages Settings] and enable checkbox [Display “Discovery Options” page].
2. Open [Project Properties]>[C/C++ Build]>[Discovery Options] and disable checkbox [Automate discovery of paths and symbols].

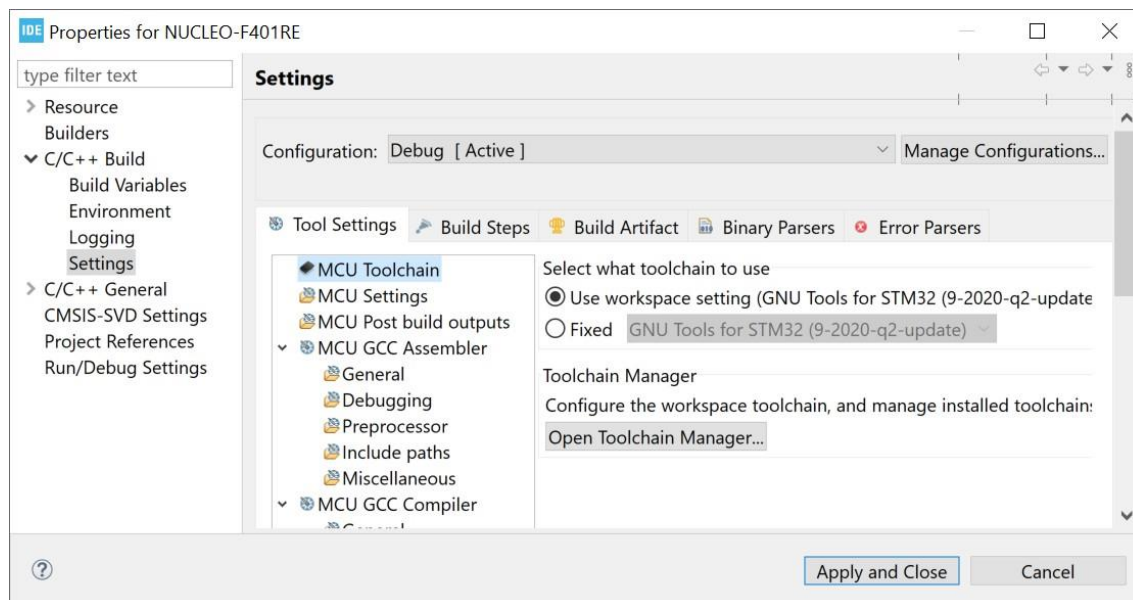
2.11 Toolchain Manager

工具链管理器用于在构建项目时安装工具链、卸载工具链和选择默认的工作区工具链。

从项目属性中的工具设置选项卡打开工具链管理器:

1. Select the **[MCU Toolchain]** node

Figure 114. Open Toolchain Manager

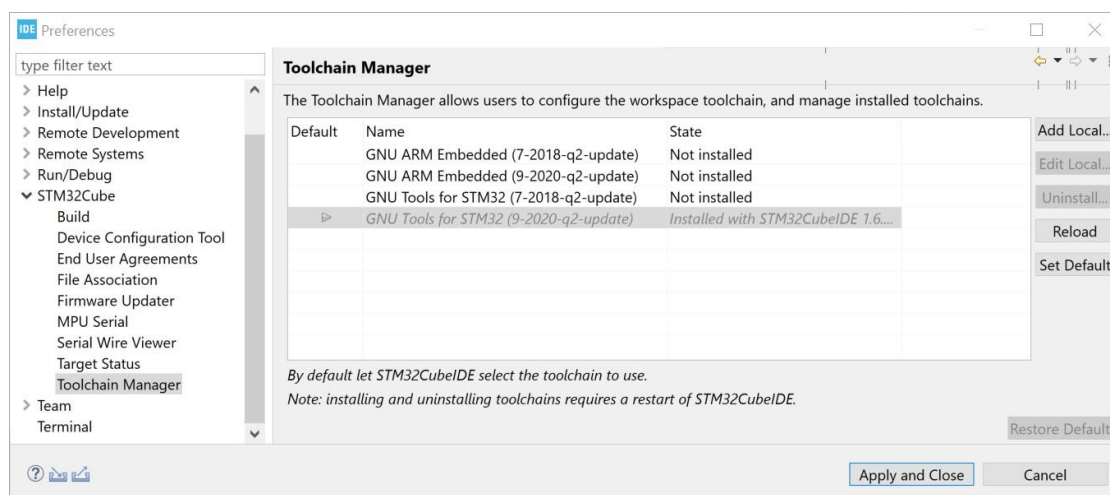


2. Click on **[Open Toolchain Manager...]**

工具链管理器也可以从**[Window]>[Preferences]**菜单打开:

1. Select **[STM32Cube]>[Toolchain Manager]**

Figure 115. Toolchain Manager



工具链管理器中的列如表4所示。

Table 4. Toolchain Manager column details

Name	Description
Default	A green/grey arrow symbol indicates the default workspace toolchain. The arrow color is: - green when the toolchain is manually set as default by the user - grey when the toolchain is selected as default by STM32CubeIDE logic
Name	The name of the toolchain.
State	The state of the toolchain. Toolchains available for download from STMicroelectronics online repository are listed as <i>"installed"</i> or <i>"not installed"</i> . Local toolchains added by the user are listed as <i>"local"</i> .

工具链管理器中的按钮如表5所示。

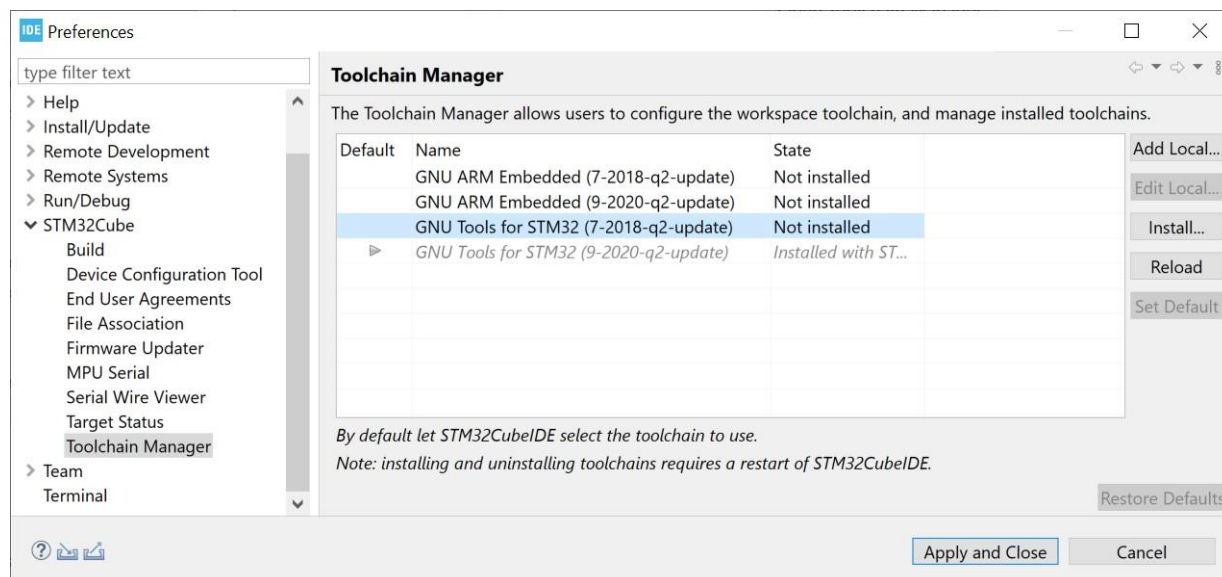
Table 5. Toolchain Manager button information

Name	Description
Add Local...	Add reference to local toolchain.
Edit Local...	Edit reference to local toolchain.
Install... Uninstall... Remove...	The button text depends on the type of the selected toolchain. It is used to: - Install / Uninstall the selected toolchain provided by the repository - Remove the selected local toolchain
Reload	Reload the toolchain list from the repository.
Set Default	Set selected toolchain to be used by default.
Restore Defaults	Restore and use the default toolchain.
Apply and Close	Apply selection and close dialog.
Cancel	Cancel dialog.

2.11.1 Install new toolchain

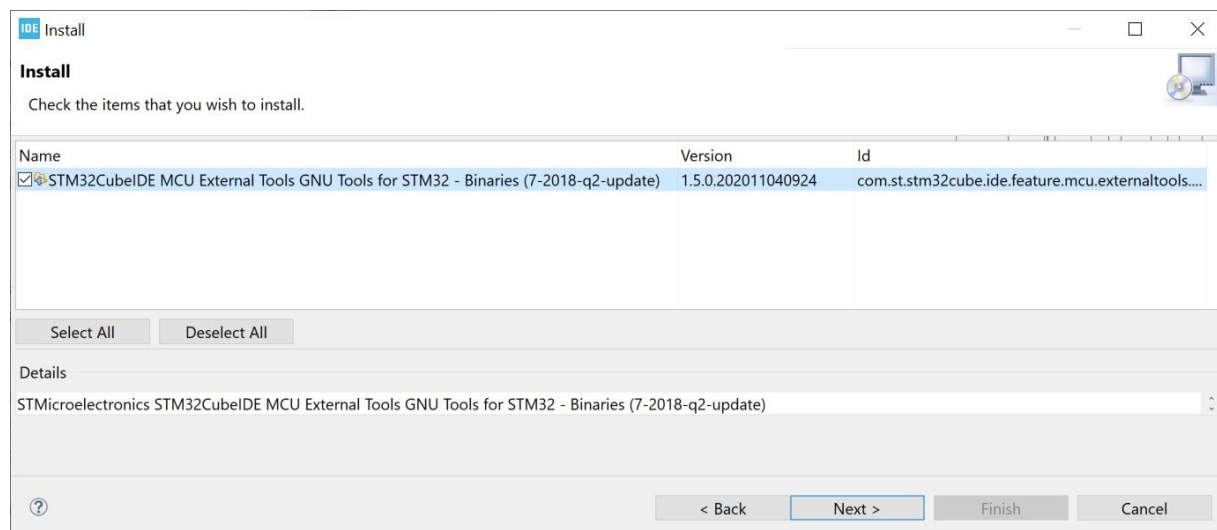
打开工具链管理器来安装一个新的工具链。

Figure 116. Install toolchain



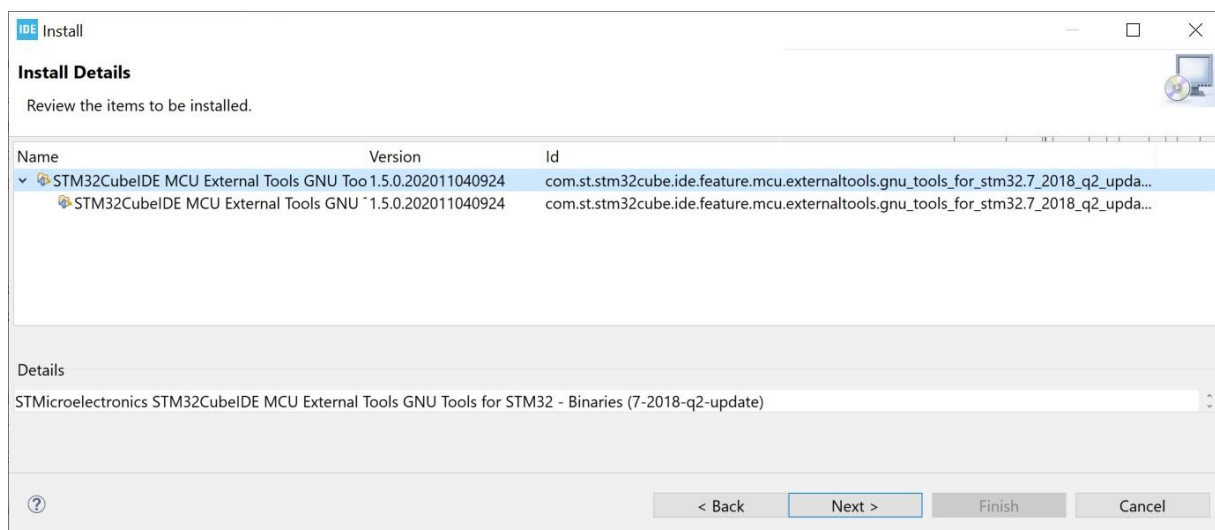
选择要安装的工具链并单击[Install...]. Install对话框打开并显示要安装的项目。

Figure 117. Check items to install



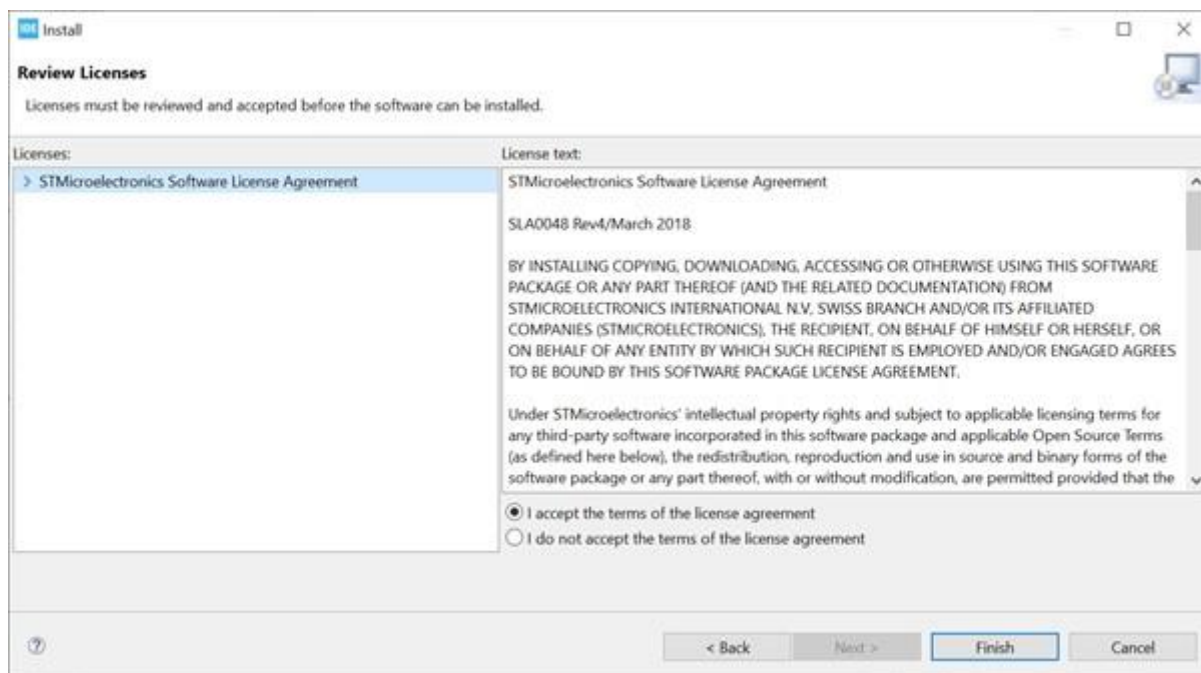
检查要安装的项目并单击[Next].

Figure 118. Review items to install



检查项目并单击[Next].

Figure 119. Review and accept licenses



查看许可，选择[I accept the terms of the license agreements]，然后单击[Finish]。

此时，软件安装开始。STM32CubeIDE窗口底部显示的进度条显示了安装完成率。请等待安装完成。

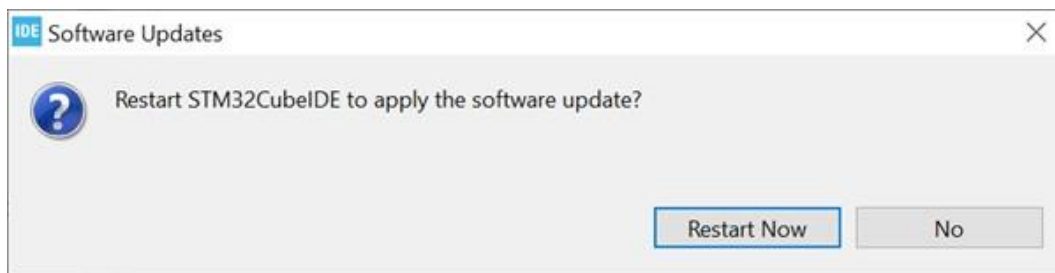
在安装完成之前可能会出现以下警告。

Figure 120. Security warning



在这种情况下，要完成安装，请单击[Install anyway].一段时间后，显示如下对话框。

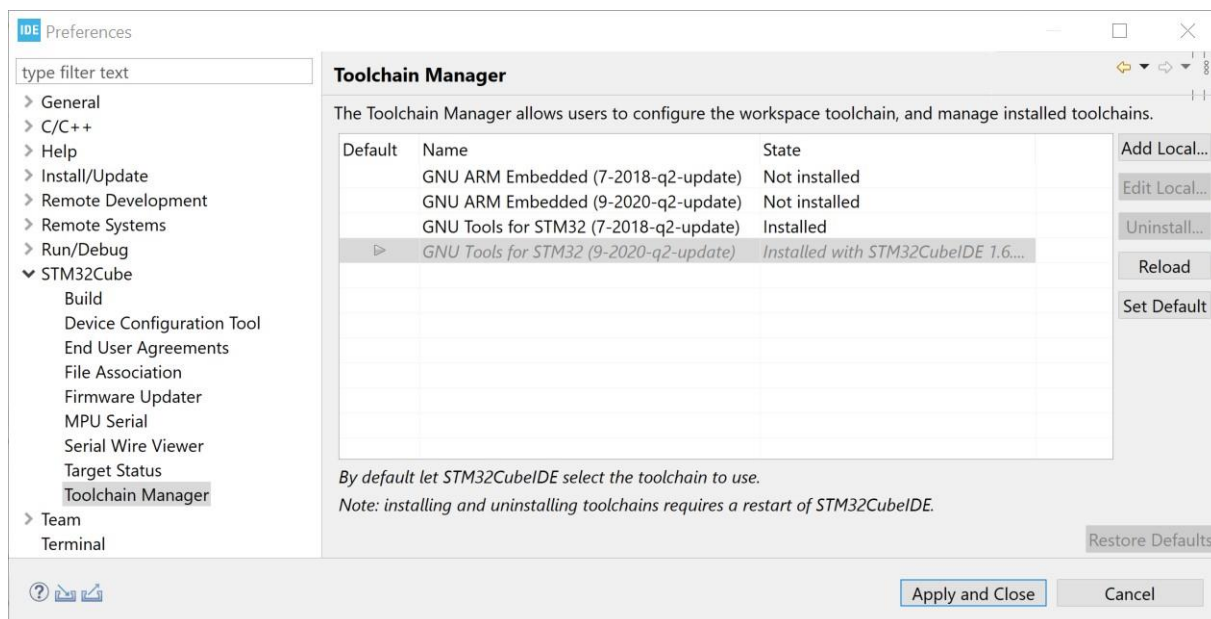
Figure 121. Restart to apply software update



单击[Restart Now]，可以在STM32CubeIDE中使用已安装的工具链。STM32CubeIDE重新启动，可以使用新的工具链。

打开工具链管理器以验证安装。

Figure 122. Toolchain installed



在本例中，图122显示了为STM32安装的两个版本的GNU Tools。

2.11.2 Manage default toolchain

工具链管理器在“默认”列中用箭头突出显示默认工作区工具链。

Figure 123. Default toolchain

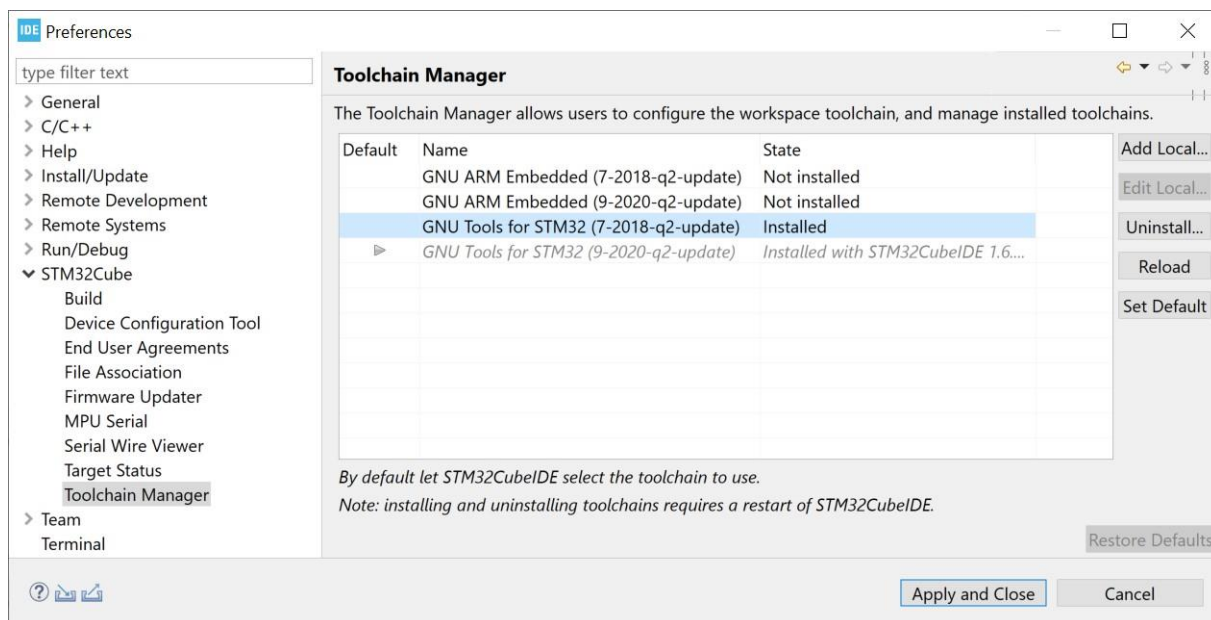
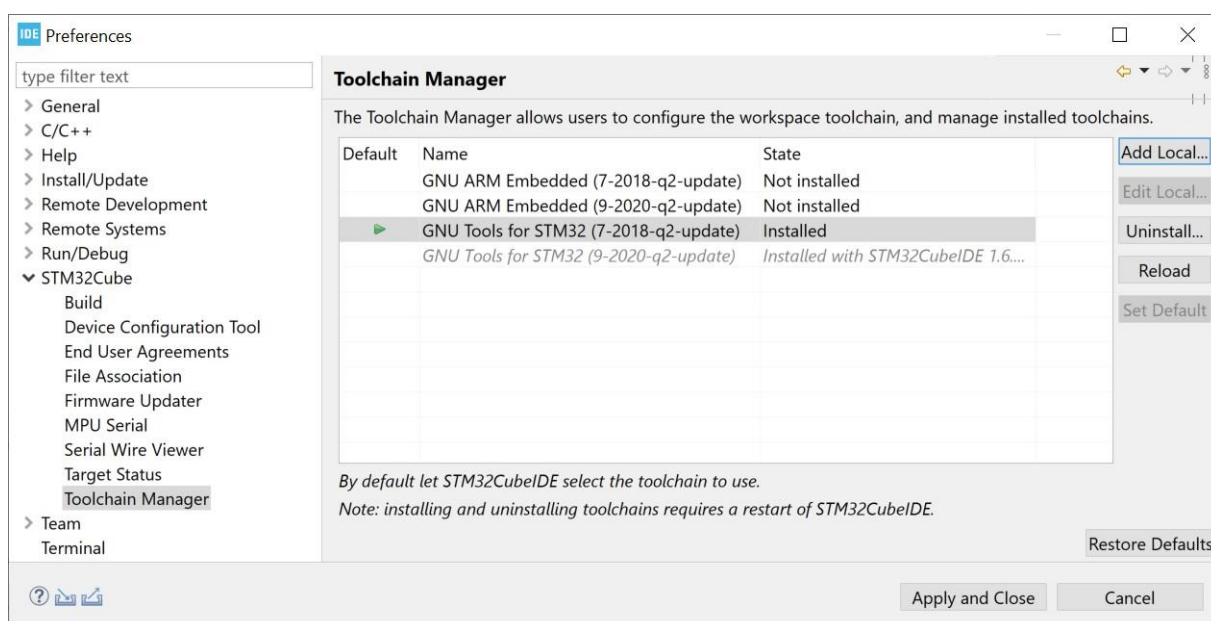


图123显示了STM32版本9-2020-q2-update的GNU Tools是默认的工作区工具链。GNU Tools for STM32 version 7-2018-q2-update行以蓝色标记，这表明该工具链已被选中。可以用鼠标选择表中的任何一行。

单击[Set default]:在工具链管理器的default列中，选中的工具链将被用作默认工作区工具链，并以箭头符号高亮显示。

Figure 124. Default toolchain updated

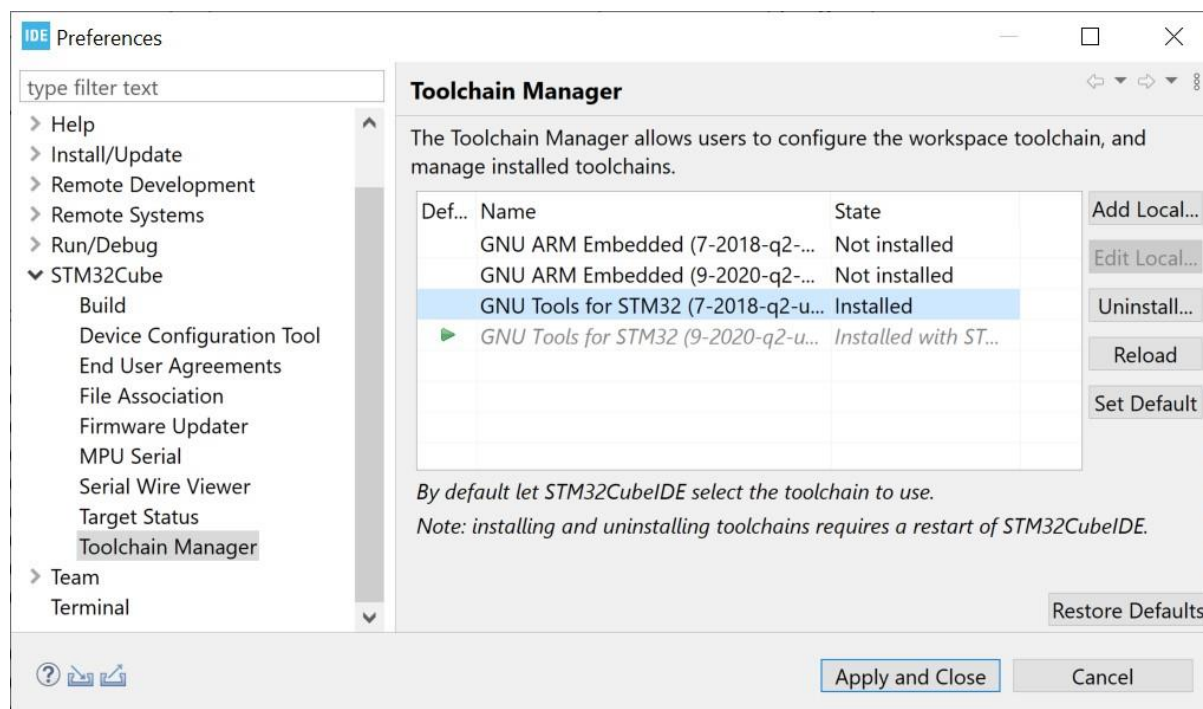


单击[Apply and Close]以应用设置并更新被设置为默认工作区工具链的工具链。

2.11.3 Uninstall toolchain

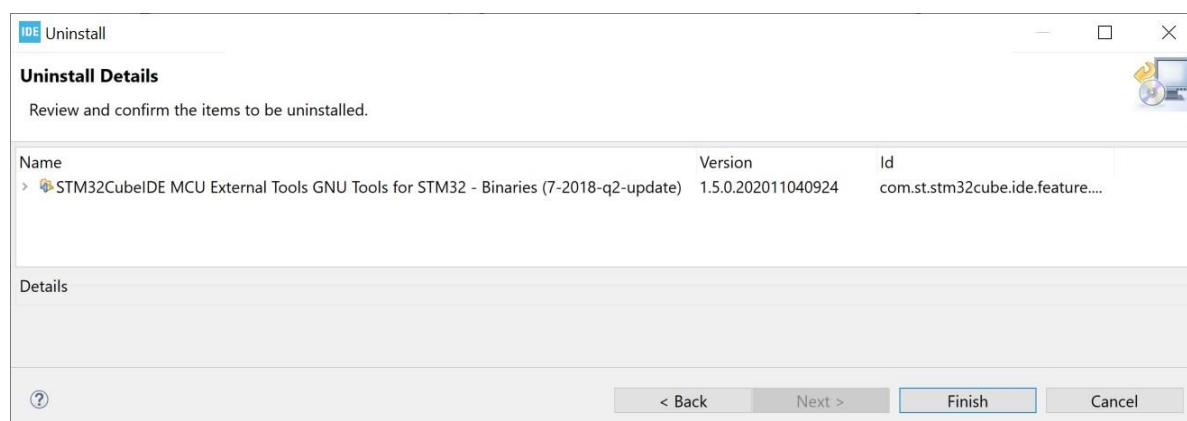
不可能卸载STM32工具链的GNU Tools，默认情况下是在STM32CubeIDE中安装的。任何其他已安装的工具链都可以卸载。

Figure 125. Uninstall toolchain



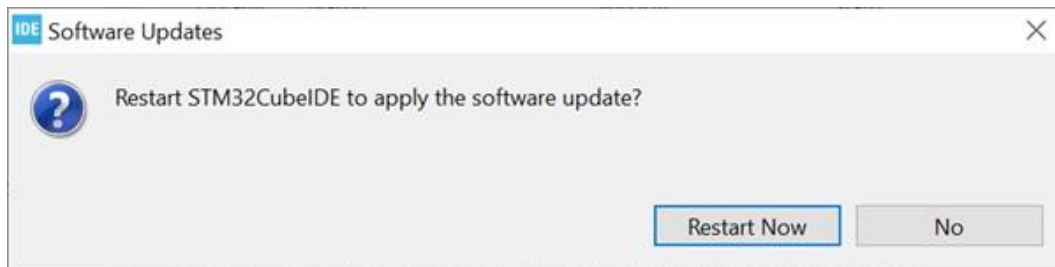
要卸载工具链，请在工具链管理器中选择它，然后单击[Uninstall...].这将要打开卸载对话框。

Figure 126. Uninstall details



单击[Finish]开始卸载软件。弹出“软件更新”对话框。

Figure 127. Software updates

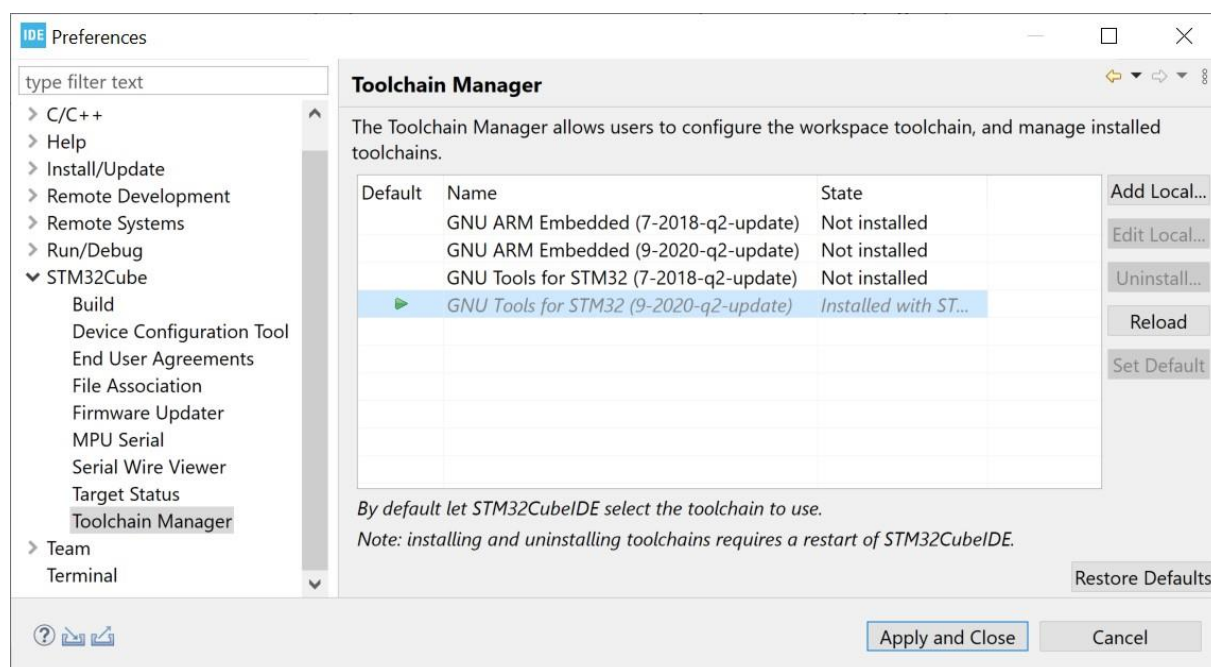


Click on [Restart Now] to apply the software update.

The product is restarted.

Open the *Toolchain Manager* to verify the installation.

Figure 128. Toolchain uninstalled



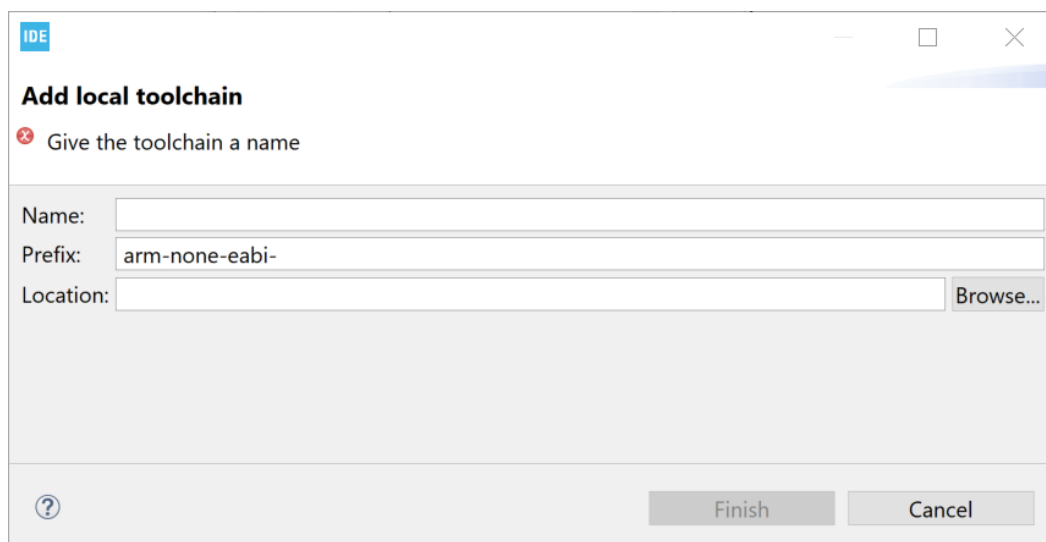
In this case, Figure 128 shows that only one version of *GNU Tools for STM32* is installed.

2.11.4 Using local toolchain

可以添加和使用已经安装的本地GNU ARM工具链。要添加本地工具链，请遵循以下步骤：

1. Open *Toolchain Manager* and press the **[Add Local...]** button.

Figure 129. Add local toolchain



Add local toolchain

Give the toolchain a name

Name:

Prefix:

Location: Browse...

? Finish Cancel

2. Add a name and specify location.

Figure 130. Specify local toolchain location

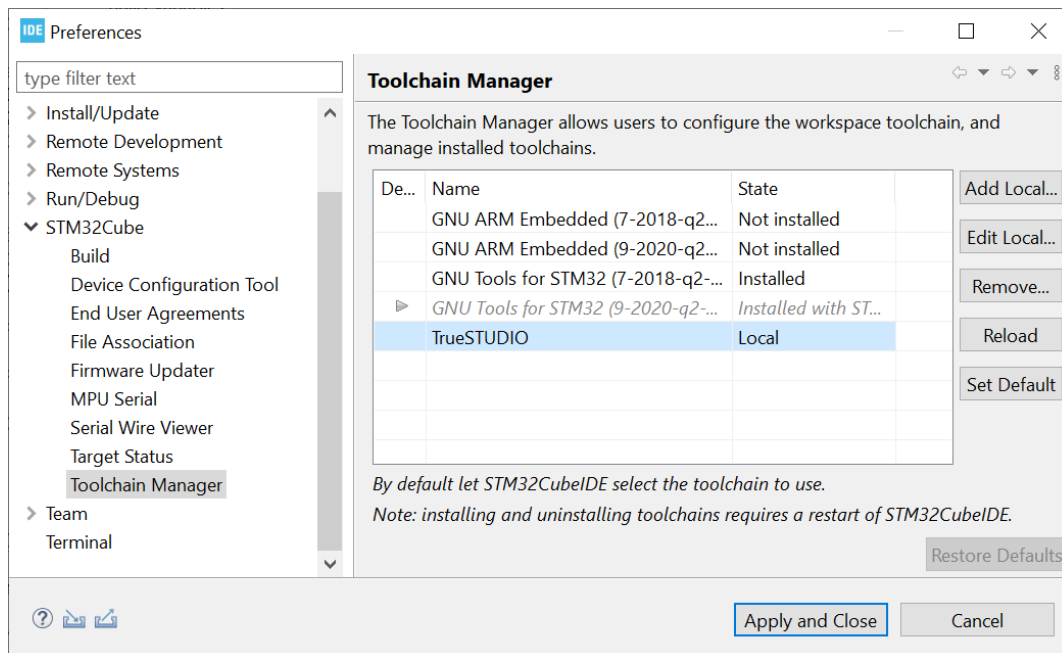
As seen in Figure 130, some naming problems can occur. In this case, the problem results from a wrong prefix that prevents the toolchain application validation.

Update the toolchain prefix. The prefix must end with a dash (-).

Figure 131. Specify local toolchain prefix

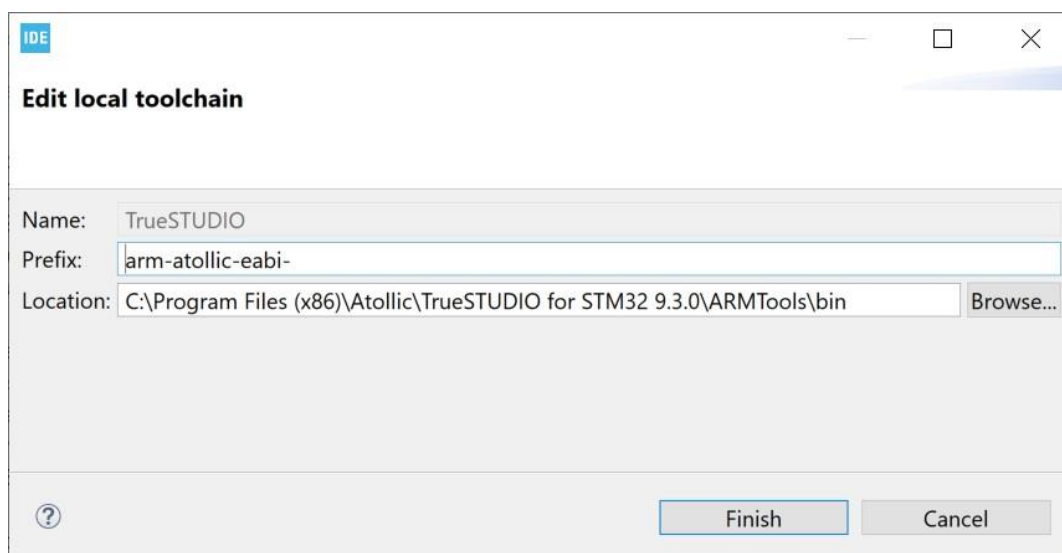
3. Press **[Finish]**.

Figure 132. Local toolchain added



4. Use the **[Edit Local...]** button to edit local toolchain. The *Edit local toolchain* dialog opens, and it is possible to update *Prefix* and *Location*.

Figure 133. Edit local toolchain

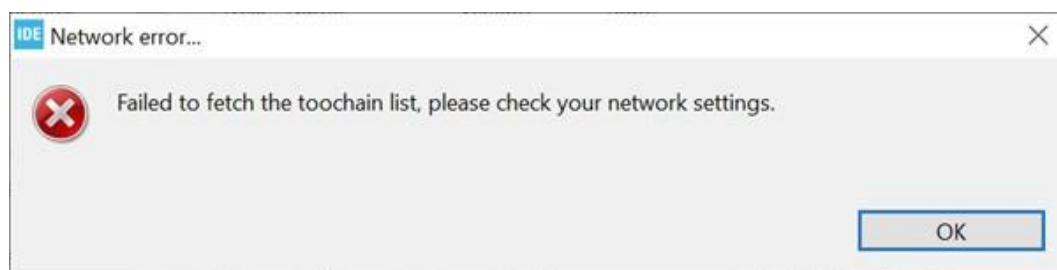


5. Update *Prefix* or *Location* and press **[Finish]** to update local toolchain settings.

2.11.5 Network error

In case of problem to access the update site, the *Network error...* dialog is displayed.

Figure 134. Toolchain network error



Check the network settings. Information on how to configure network proxy settings are described in [Section 1.5.3 Preferences - Network proxy settings](#).

3 Debugging

3.1 Introduction to debugging

STM32CubeIDE包含一个基于GDB命令行调试器的强大图形化调试器。它还为ST-LINK和SEGGER J-Link JTAG探测器捆绑了GDB服务器。

GDB服务器是将PC上的GDB连接到目标系统的程序。STM32CubeIDE调试会话可以自动启动本地GDB服务器或连接到远程GDB服务器。

远程GDB服务器可以运行在同一台PC上，也可以运行在可以通过网络访问并指定主机名或IP地址和端口号的PC上。当连接到远程GDB服务器时，在STM32CubeIDE中启动调试会话之前，必须首先启动此GDB服务器。

当选择“本地自动启动调试”时，STM32CubeIDE在调试过程中会根据需要自动启动和停止GDB服务器，实现GDB服务器的无缝集成。

Note:

在构建必须调试的项目时，建议使用编译器优化级别-O0。使用优化级别-Og进行调试可能有效，但由于编译器代码优化，更高级别的优化很难进行调试。

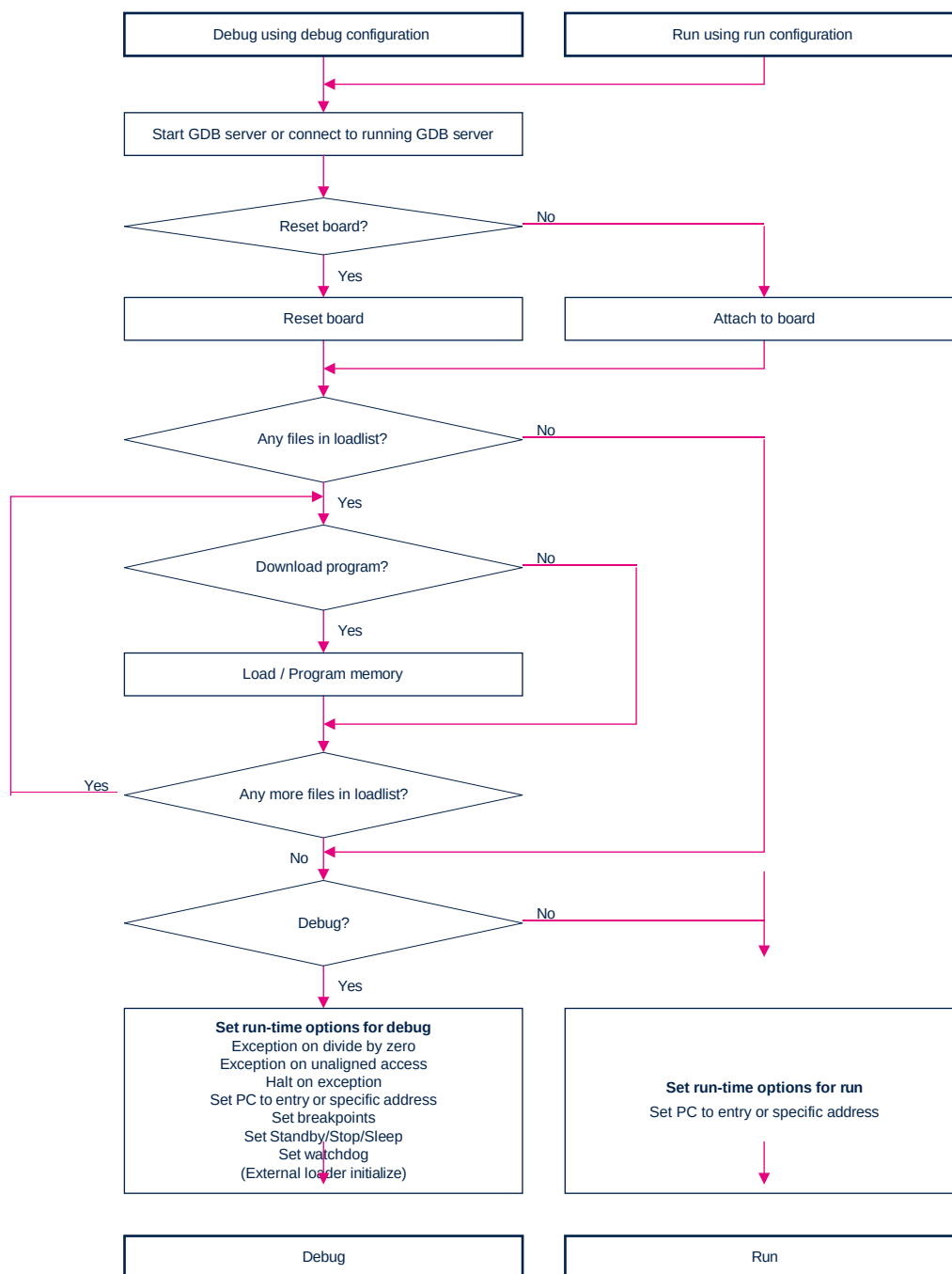
也可以只使用GDB服务器将应用程序下载到目标系统并运行它，而不启动调试会话。这是通过创建运行配置来实现的，这将在本章后面进行描述(请参阅3.7节运行配置)。

STM32CubeIDE可用于调试使用另一个IDE或工具链开发的现有elf文件，通过导入STM32 Cortex®- M可执行文件来导入elf文件。导入STM32 Cortex-M可执行文件。

3.1.1 General debug and run launch flow

调试配置用于调试STM32程序。运行配置用于将一个新程序闪入STM32并启动它。图135中的流程图显示了启动GDB服务器、重置设备、加载程序、设置运行时选项、异常、程序计数器、断点、待机/停止/睡眠、看门狗和启动调试会话时外部加载器初始化的顺序。它还显示调试和运行会话之间的差异。

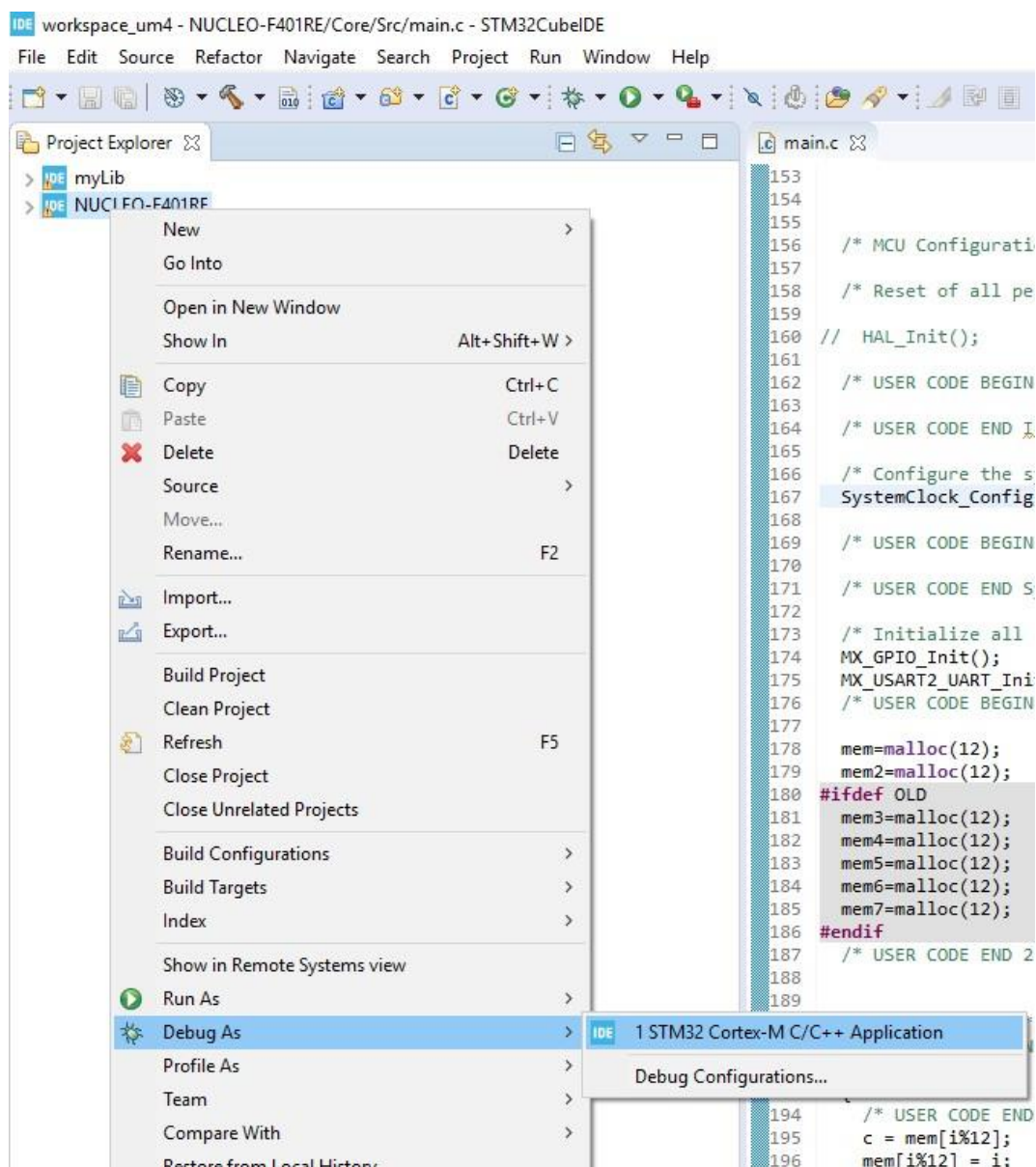
Figure 135. General debug and run launch flowchart



3.2 Debug configurations

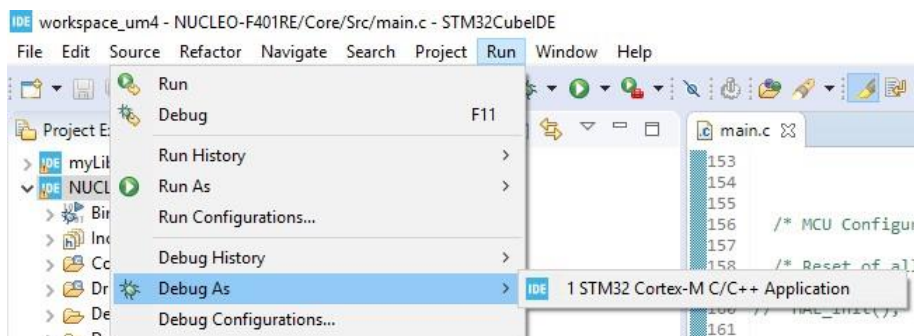
在启动调试会话之前，需要项目的调试配置。要为项目创建第一个调试配置，请在项目资源管理器视图中右键单击项目名称，并选择[Debug As]>[STM32 Cortex-M C/C++ Application]。

Figure 136. Debug as STM32 MCU



另一种创建新的调试配置的方法是在项目资源管理器视图中选择项目名称，并使用菜单[Run]>[Debug As]>[STM32 Cortex-M C/C++ Application]。

Figure 137. Debug as STM32 MCU menu



创建新的调试配置的第三种方法是在project Explorer视图中选择项目名称并按[F11]。

这三种不同的方式打开Debug Configuration对话框。

3.2.1 Debug configuration

The *Debug Configuration* dialog contains the following tabs:

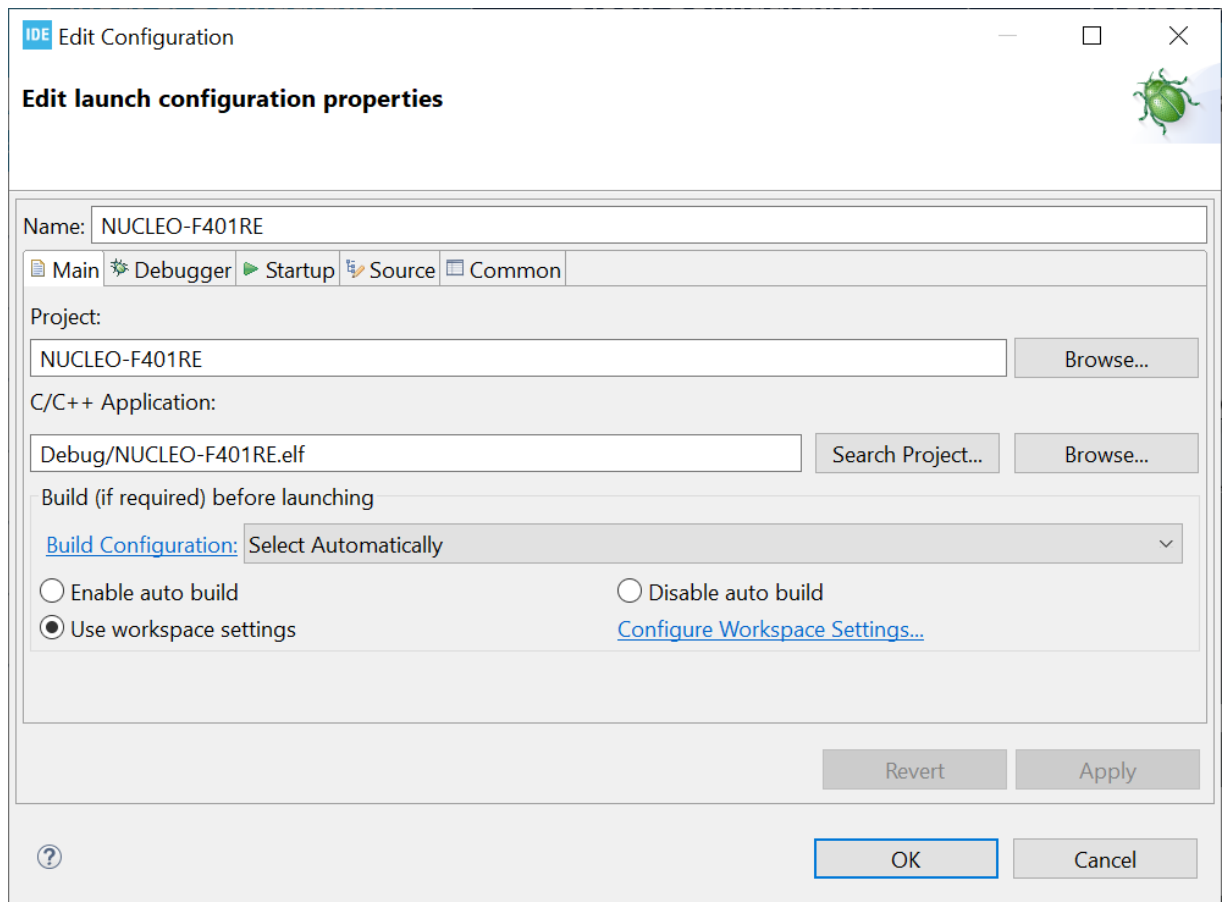
- *Main*
- *Debugger*
- *Startup*
- *Source*
- *Common*

The *Debugger* and *Startup* tabs must be updated when creating a new debug configuration while the others do not require update.

3.2.2 Main tab

Main选项卡包含要调试的C/ c++应用程序的配置。通常，当使用本章前面描述的序列创建调试配置时，不需要在Main选项卡中做任何更改。确保选择了正确的elf文件和项目。

Figure 138. Debug configuration main tab



Note: 可以在Main选项卡中定义是否必须在启动调试会话之前进行构建。

3.2.3 Debugger tab

调试器选项卡配置如何启动GDB服务器并连接到它。它还定义了如果选择[Autostart local GDB server]，则必须使用哪个GDB服务器。

Figure 139. Debug configuration debugger tab

Edit Configuration

Edit launch configuration properties

Name: NUCLEO-F401RE

Main **Debugger** **Startup** **Source** **Common**

GDB Connection Settings

☒ Autostart local GDB server Host name or IP address: localhost

☐ Connect to remote GDB server Port number: 61234

Debug probe: ST-LINK (ST-LINK GDB server) ▼

GDB Server Command Line Options

Show Command Line

Interface

☒ SWD ☐ JTAG

☐ ST-LINK S/N: [dropdown] Scan

Frequency (kHz): Auto ▼

Access port: 0 - Cortex-M4 ▼

Reset behaviour

Type: Connect under reset ▼

Device settings

Debug in low power modes: Enable ▼

Suspend watchdog counters while halted: No configuration ▼

Serial Wire Viewer (SWV)

☐ Enable

Core Clock (MHz): 16.0

☐ Limit SWO clock

Maximum SWO clock (kHz): auto detect

Port number: 61235

RTOS Kernel Awareness

☐ Enable RTOS Proxy

Driver settings

Driver: ThreadX ▼

Port: cortex_m0 ▼

Port number: 60000

Misc

☒ Verify flash download

☒ Enable live expressions

☐ Log to file: C:\Users\johansse\STM32CubeIDE\workspace_um8\NUCLEO-F401RE\ Browse...

☐ External Loader: [dropdown] Scan ☐ Initialize

☐ Shared ST-LINK

☐ Max halt timeout(s): 2

Revert Apply

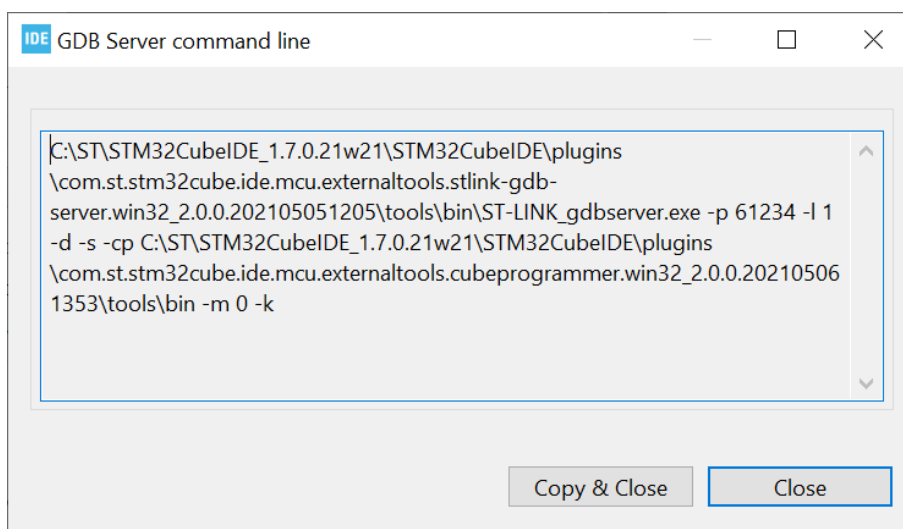
OK Cancel

编辑字段[Port number]包含在字段[Debug probe]中选择的GDB服务器使用的默认值。当选择[Connect to remote GDB server]时，需要设置[Host name or IP address]字段。

字段[**Debug probe**]选择用于调试的探针和GDB服务器。当使用ST-LINK调试探针时，可以使用ST-LINK GDB服务器或OpenOCD。使用SEGGER J-LINK探针时，请使用SEGGER J-LINK GDB服务器。

按下[**Show Command Line**]按钮打开GDB Server命令行对话框。对话框显示如何根据当前[GDB Server Command Line options]设置启动GDB服务器。

Figure 140. GDB server command line dialog



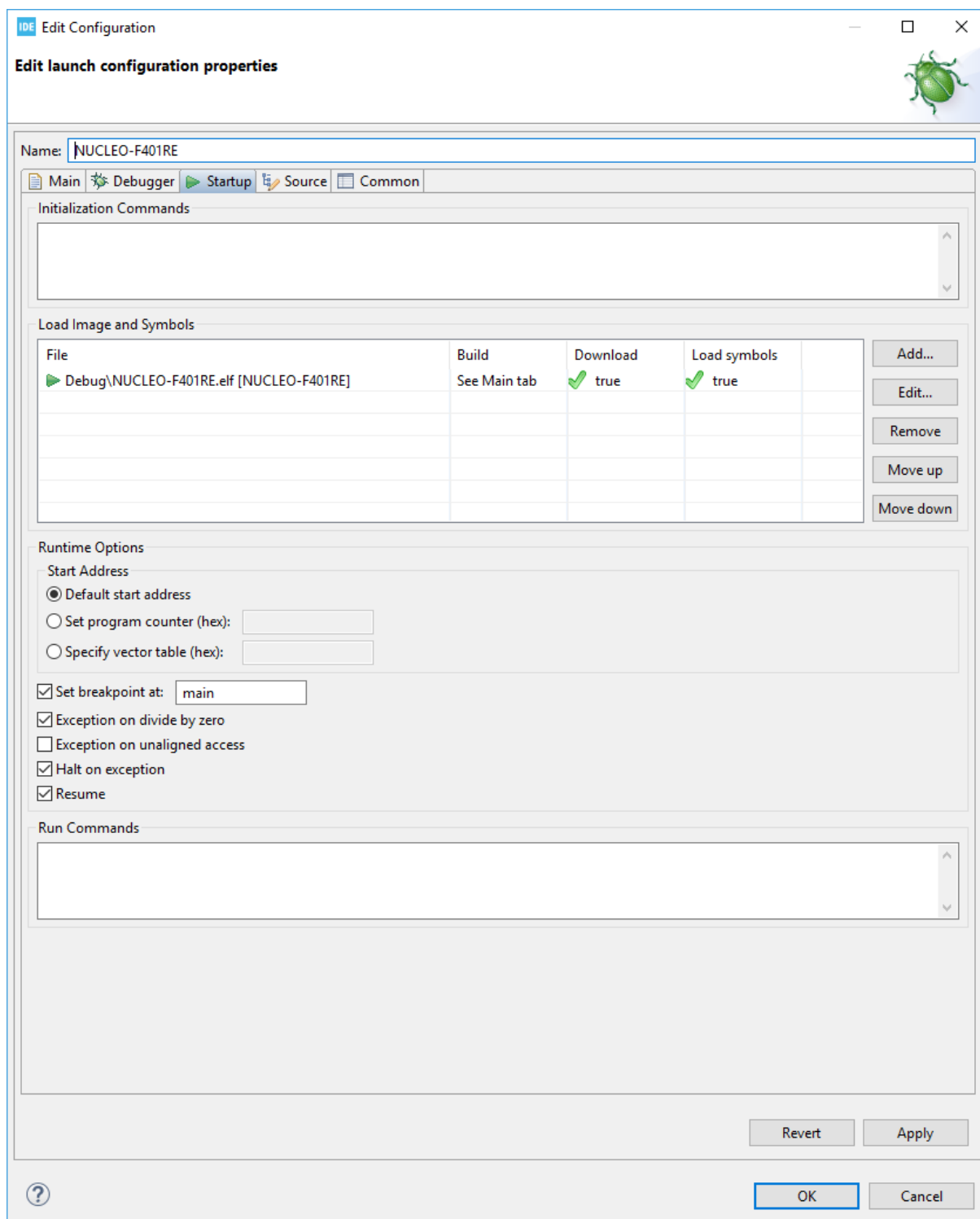
使用[**Copy & Close**]按钮将当前命令行设置复制到剪贴板，例如通过粘贴命令在命令行窗口中手动启动GDB服务器。

[**GDB Server Command Line options**]选项会随着所选[**Debug probe**]的功能而更新。关于这些设置的详细信息，请参见第3.4节使用不同的GDB服务器进行调试及其子节。

3.2.4 Startup tab

Startup选项卡配置如何启动调试会话。

Figure 141. Debug configuration startup tab



如果在发送加载命令之前需要向GDB服务器发送一些命令，则可以使用任何类型的GDB或GDB服务器监控命令更新[Initialization Commands]编辑字段。例如，当使用ST-LINK GDB服务器时，如果在加载前需要擦除flash内存，可以在这里输入一个监视器flash mass_erase命令。

[Load Image and Symbols]列表框必须包含要调试的文件。此列表与以下命令按钮相关联:

- [Add...]: Add new lines for files for download and/or load symbols
- [Edit...]: Edit the selected line
- [Remove]: Remove the selected line from the list
- [Move up]: Move the selected line upwards
- [Move down]: Move selected line downwards

[Runtime Options]部分包含设置开始地址和断点的复选框，以及启用异常处理和恢复。

The start address can be selected as:

- [Default start address]: \$pc is set to the start address found in the last loaded `elf` file
- [Set program counter (hex)]: \$pc is set to the hex value specified in the edit field
- [Specify vector table (hex)]: \$pc is updated with the value found in memory using specified address + offset of 4. This is similar to how \$pc is set by a reset using vector table in a Cortex®-M device

[Set breakpoint at:]复选框默认启用，编辑字段显示main。这意味着，在默认情况下，在程序调试时将断点设置为main。

三个异常复选框，[Exception on divide by zero]，[Exception on unaligned access]和[Halt on exception]，用于在调试应用程序时更容易发现问题。

- 默认启用[Exception on divide by zero]，以便在调试时更容易捕获除零错误
- [Exception on unaligned access]可以启用，以获得异常，如果有任何非对齐访问
- 在默认情况下启用[Halt on exception]，以便在调试过程中发生异常错误时程序执行停止。如果发生异常，可以使用Fault Analyzer视图来查找问题的位置

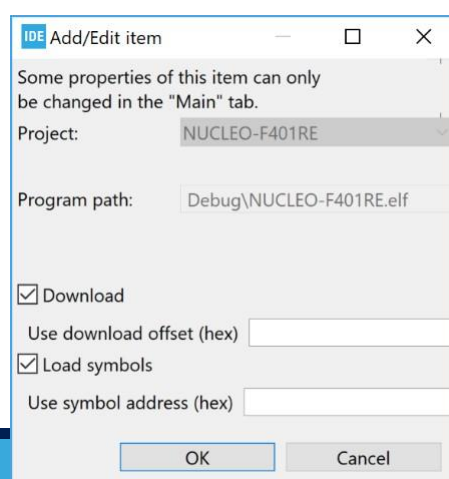
Note:

The application software needs to enable **Exception on divide by zero** and **Exception on unaligned access** if they must be issued also when running the application and not only during debugging. The CMSIS Cortex®-M header files contain defines to update the SCB Configuration Control Register. For instance, `core_cm4.h` contains the SCB->CCR register, and `SCB_CCR_DIV_0_TRP` and `SCB_CCR_UNALIGN_TRP` defines.

当[Resume]选择被启用时，在加载后向GDB发出继续命令以启动程序。通常，在这种情况下，如果在main处设置了断点，则程序在main处中断。否则，当[Resume]选择被禁用时，程序将停留在链接器脚本中指定的ENTRY位置，通常是Reset_Handler函数。在这种情况下，可能需要一个步骤来在编辑器中显示Reset_Handler函数。

当选中列表框中的一行并按下[Edit...]时，将出现以下对话框，用于选择是否必须下载文件以及是否必须加载符号。

Figure 142. Add/Edit item

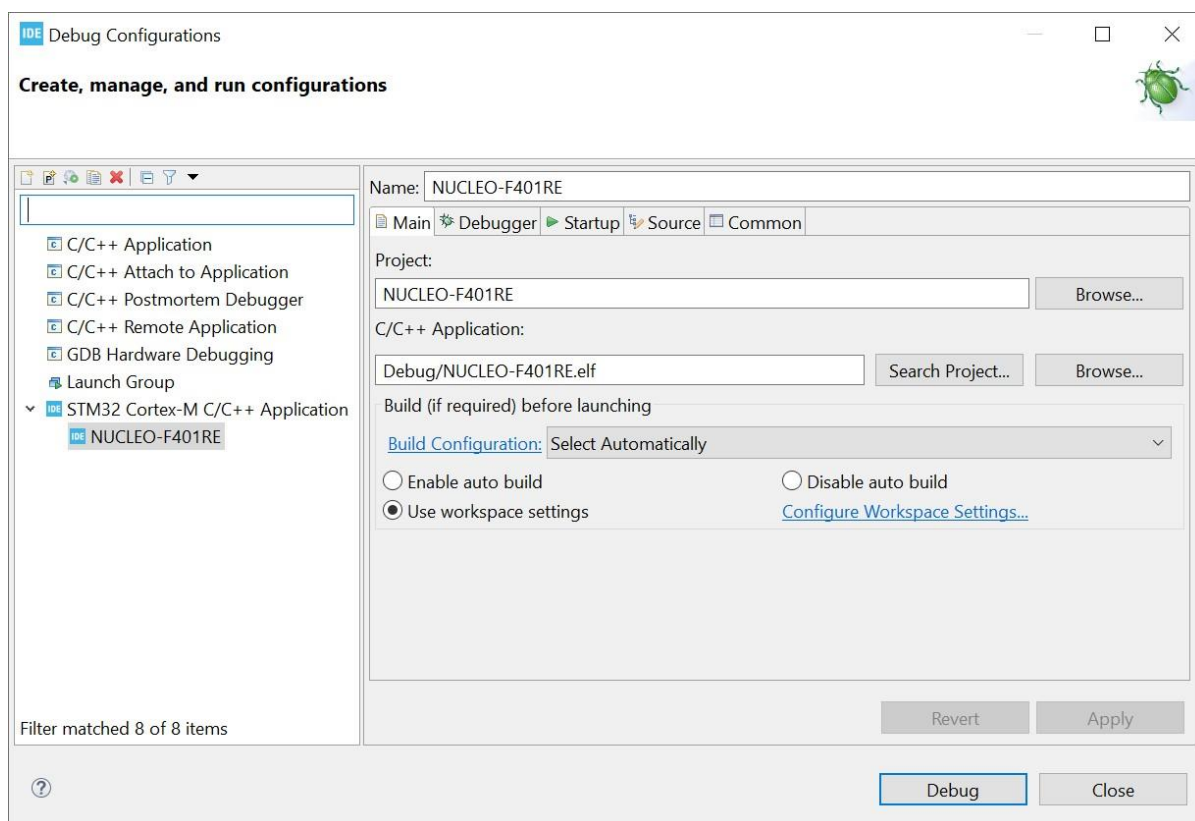


3.3 Manage debug configurations

每个项目都可以有几个调试配置。创建现有调试配置的副本并对其进行一些更改是很容易的。例如，一种配置可能包含新程序的闪存加载，而另一种配置不加载任何程序。

当从菜单[Run]>[Debug Configurations...]打开调试配置时，会打开调试配置对话框。该对话框在左侧包含一个带有工具栏的导航窗口，在右侧包含3.2节调试配置中描述的选项卡和字段的调试配置。

Figure 143. Manage debug configurations



右窗格顶部的[Name]字段可以使用反映配置的调试配置的名称进行编辑。然后，当按[Apply]时，该名称出现在左侧[STM32 Cortex-M C/C++ Application]节点下的导航窗口中。

导航窗口左侧的工具栏包含用于管理配置的图标，例如复制或删除选定的配置。

Figure 144. Manage debug configurations toolbar



These icons are used for the following purpose, from left to right:

- Create new launch configuration
- New launch of configuration prototype
- Export launch configuration
- Duplicate currently selected launch configuration
- Delete selected launch configuration(s)
- Collapse all expanded launch configurations
- Filter launch configurations

3.4 Debug using different GDB servers

STM32CubeIDE includes the following GDB servers:

- ST-LINK GDB server
- OpenOCD GDB server
- SEGGER J-Link GDB server

All three GDB servers support normal debug, live expressions and SWV.

所有GDB服务器还支持使用RTOS代理对Microsoft®Azure®RTOS ThreadX和FreeRTOS™操作系统进行RTOS内核感知调试。RTOS代理包含在STM32CubeIDE中。

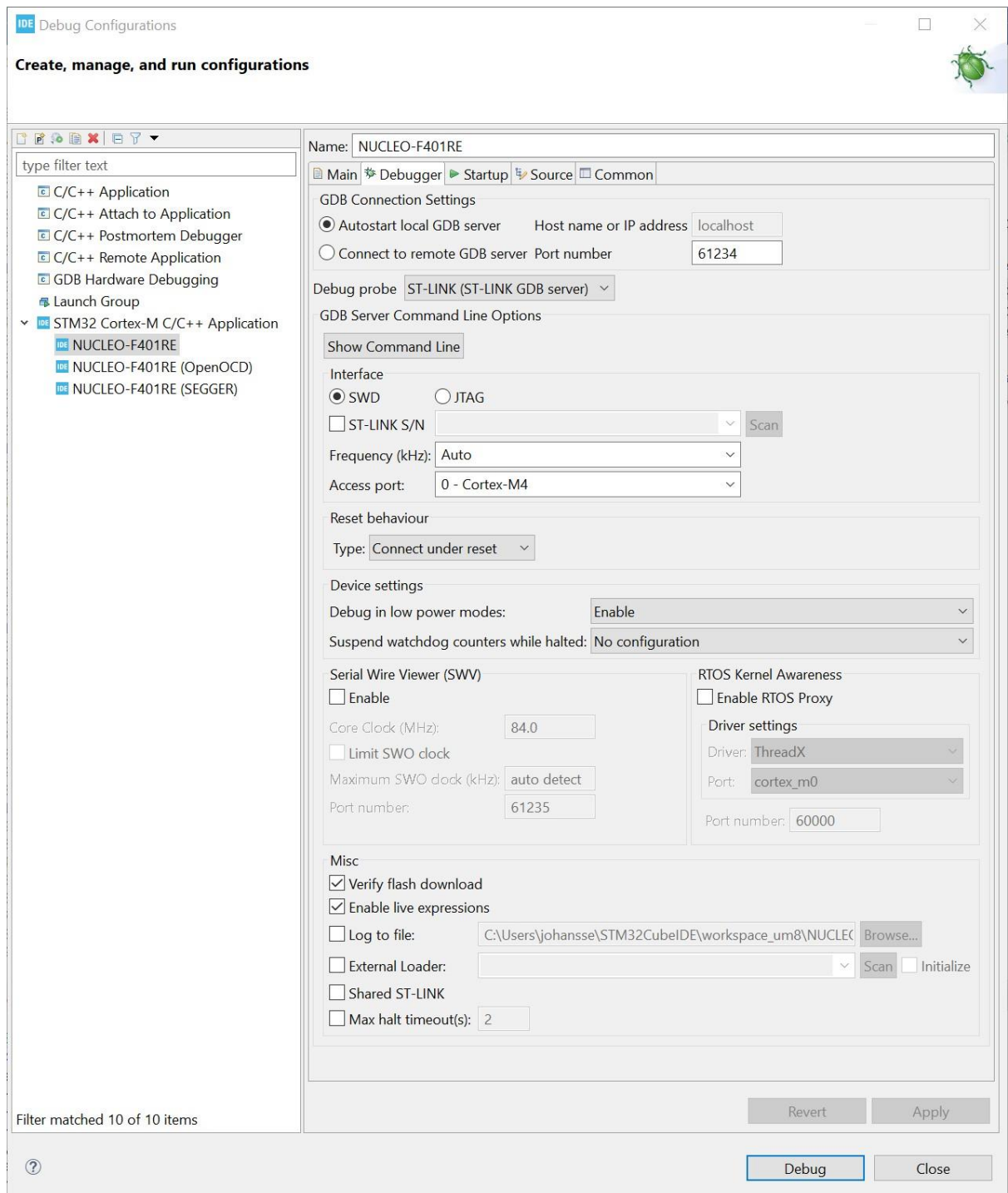
启动这些GDB服务器时使用不同的命令行选项。因此，调试配置对话框中的调试器选项卡根据所选GDB服务器显示不同的设置。本节描述每个服务器的单独设置。

3.4.1 Debug using the ST-LINK GDB server

通常，当使用ST-LINK GDB服务器进行调试时，不需要更新调试器选项卡中的[GDB Server

Command Line Options]。在某些情况下，必须更新默认配置，例如，如果打算使用SWV或如果多个STM32板连接到PC。

Figure 145. ST-LINK GDB server debugger tab



在[Interface]中选择t [SWD] or [JTAG]来定义ST-LINK探针必须与微控制器连接的方式。SWD接口通常是首选。如果要使用SWV，则必须选择该选项。

当使能[ST-LINK S/N]时，必须在edit/list字段中输入要使用的ST-LINK探针的序列号。[Scan]按钮可以扫描和列出所有检测到的连接到PC的ST-LINK设备。扫描完成后，这些ST-LINK设备的S/N会显示在列表框中，用户可以在列表框中选择ST-LINK设备。当使能[Use specific ST-LINK S/N]时，启动ST-LINK GDB服务器，只与指定S/N的ST-LINK连接。

[Frequency (kHz)]选项定义了ST-LINK和STM32设备之间的通信速度。选择[Auto]时，使用ST-LINK提供的最大速度。在硬件限制的情况下降低频率。

[Access port]选项仅在调试多核STM32设备时使用。在这种情况下，ST-LINK连接到设备上，并且必须通知ST-LINK GDB服务器要调试的核心。

[Reset behaviour]包含[Type] and [Halt all cores]的选择。[Halt all cores]选项只对多核设备可见。

The [Type] can be set as follows:

- [Connect under reset] (default): ST-LINK复位线激活，复位时ST-LINK以SWD或JTAG方式连接。然后复位线被停用。
- [Software system reset]: 系统复位是由写入寄存器的软件激活的。这将重置核心和外围设备，并且可以重置整个系统，因为目标的复位引脚是由自己断言的。
- [Hardware reset]: ST-LINK复位线激活和去激活(复位线上有脉冲)，ST-LINK以SWD或JTAG方式连接。
- [Core reset]: 核心重置是通过在寄存器中写入软件来激活的(在Cortex® - M0、Cortex® - M0+ 和Cortex® - M33核心上不可能实现)。这只重置核心，不重置外设或复位引脚。
- [None]: 用于附加到一个运行的目标，其中程序已经下载到设备中。启动选项卡中不能有任何文件程序命令。

Note:

如果调试配置包括Flash编程，则所选择的重置行为将被覆盖，在这种情况下，ST-LINK GDB服务器使用STM32CubeProgrammer (STM32CubeProg)命令行程序STM32_Programmer_CLI对Flash内存进行编程。此程序总是由ST-LINK GDB服务器以mode=UR reset=hwRst启动，以便在加载新程序时完成设备重置，忽略[None]选项的选择。这确保了设备编程是正确的。

[Halt all cores]只能在调试多核设备时使用。[Halt all cores]选项对于单核设备不可见。

[Device settings]包含[Debug in low power modes]和[Suspend watchdog counters while halted]的选择。这些可以定义为:

- [No configuration]
- [Enable]
- [Disable]

只有选中[SWD]接口时，才能使用[Serial Wire Viewer (SWV)]选项。启用[SWV]时，需要配置[Clock Settings]。

[Core Clock]必须设置为设备速率。有关SWV配置的更多信息，请参见第4.2.1节SWV调试配置。

[RTOS Kernel Awareness]选项用于在ThreadX和FreeRTOS™操作系统中启用RTOS内核感知调试。当启用rtos内核感知调试并启动调试会话时，debug视图将列出所有线程。通过在Debug视图中选择一个线程，编辑器中将显示线程执行的当前行。关于rtos内核感知调试的更多信息，请参见第6.3节。

The [Misc] selections contains:

- [Verify flash download]
- [Enable live expressions] (为了能够在调试期间使用Live Expressions视图，必须在启动期间启用Live表达式机制。默认启用。)
- [Log to file] (在调试问题时启用。它以较高的日志级别启动ST-LINK GDB服务器，并将日志保存到一个文件中。)

- **[External Loader]** (如果必须加载到非内部STM32闪存，则启用)。一个[扫描]按钮可以访问STM32CubeProgrammer外部Flash加载器文件。

当 **[External Loader]** 被启用时，还有一个 **[Initialize]** 选择。当它被启用时，STM32CubeProgrammer中的Init()函数在重置后被调用。它可以用来配置外部内存访问的设备。通常，初始化必须由调试的应用程序完成。

- **[Shared ST-LINK]** (如果在调试会话期间其他程序必须能够连接到同一个ST-LINK，则必须启用Shared ST-LINK。)具体请参见3.6.2共享ST-LINK。

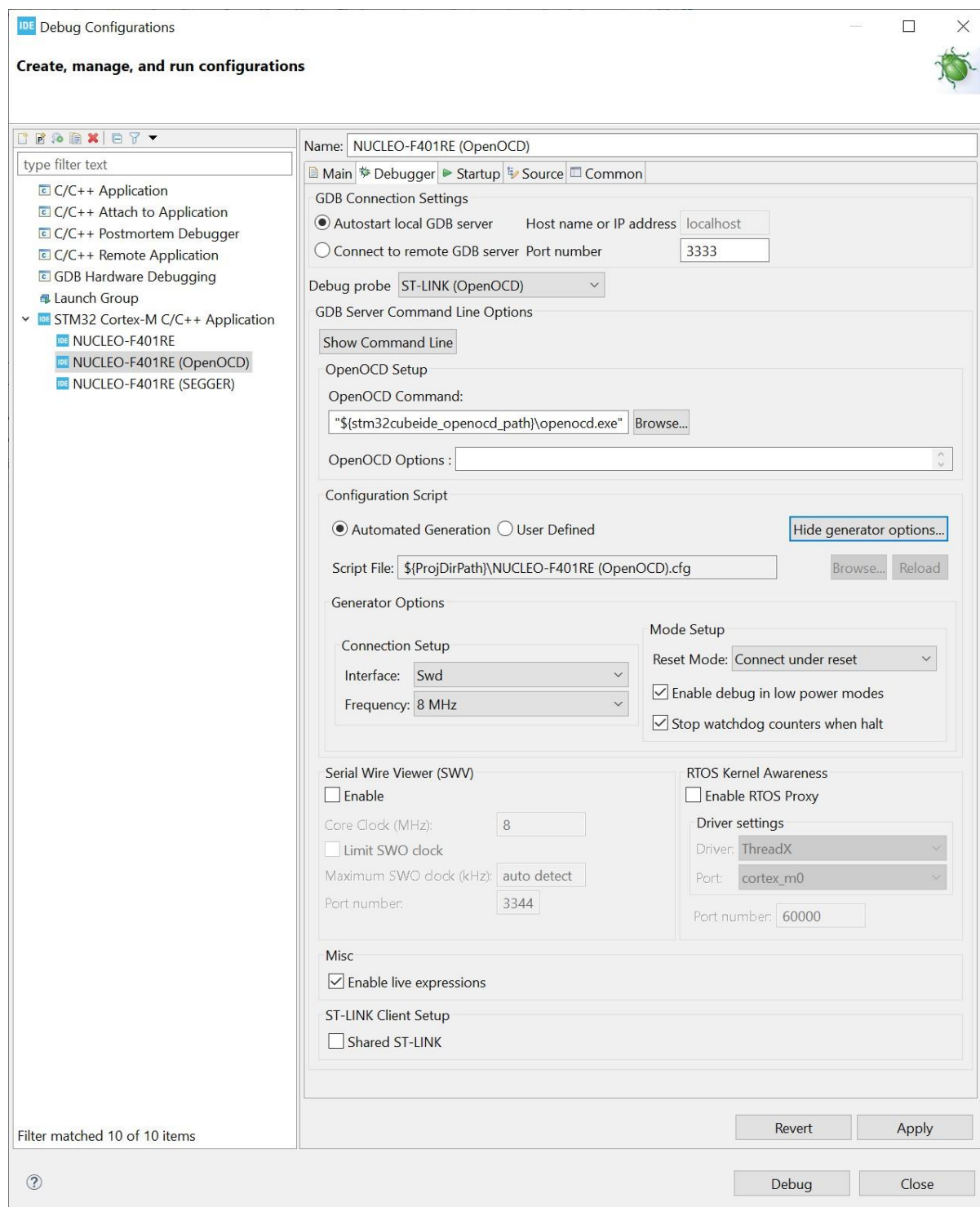
ST-LINK GDB服务器的详细描述可在ST-LINK GDB服务器手册([ST-07])中获得，该手册可从信息中心获得。

Note: *STM32_Programmer_CLI is used by the ST-LINK GDB server to program the STM32 or external Flash memory. In this case, such external Flash memory programming is automatically done using the external loader.*

3.4.2 Debug using OpenOCD and ST-LINK

当使用OpenOCD时，调试器选项卡中的[GDB Server Command Line Options]包含一个生成器选项切换字段，它在[Show generator options...] and [Hide generator options...]之间交替。当该字段被设置为[Hide generator options...]时，对话框将显示额外的[GDB Server Command Line Options]，如图146所示。

Figure 146. OpenOCD debugger tab



[OpenOCD Command]字段包含调试时使用的OpenOCD .exe文件。[Browse]按钮可用于选择另一个版本的OpenOCD。

[OpenOCD Options]编辑字段可用于在启动OpenOCD时添加额外的命令行参数。

[**Configuration Script**]选项可以是[**Automated Generation**] or [**User Defined**]。当选择[**Automated Generation**]时，将根据调试器选项卡中的选择自动创建openocd.cfg文件。当选择[**User Defined**]时，文件必须在[**Script File**]编辑字段中指定。

[**Interface**]选项[**Swd**] or [**Jtag**]选择ST-LINK探针必须与微控制器连接的方式。[**Swd**]通常是首选。

“频率选择”用于配置ST-LINK与STM32设备之间的通信速率。[**Reset Mode**]选项包括:

- [**Connect under reset**] (default): ST-LINK复位线激活，复位时ST-LINK以SWD或JTAG方式连接。然后复位线被停用。
- [**Hardware reset**]: ST-LINK复位线激活和去激活(复位线上有脉冲)，ST-LINK以SWD或JTAG方式连接。
- [**Software system reset**]: 系统复位是由写入寄存器的软件激活的。这是在重置核心和外围设备，并且可以重置整个系统，因为目标的复位引脚是由自己断言的。
- [**Core reset**]: 核心重置是通过在寄存器中写入软件来激活的(在Cortex®- M0、Cortex®- M0+和Cortex®- M33核心上不可能实现)。这只是重置核心，而不是外围设备或复位引脚。
- [**None**]: 用于附加到一个运行的目标，其中程序已经下载到设备中。启动选项卡中不能有任何文件程序命令。

[**Enable debug in low power modes**]在低功耗模式下也可以使用STM32设备进行调试。

[**Stop watchdog counters when halt**] 当调试会话停止STM32设备时，停止看门狗。否则可能触发看门狗中断。

只有选中[**SWD**]接口时，才能使用[**Serial Wire Viewer (SWV)**]选项。启用[**SWV**]时，需要配置[时钟设置]。[**Core Clock**]必须设置为设备速率。有关SWV配置的更多信息，请参见第4.2.1节SWV调试配置。

[**RTOS Kernel Awareness**]选项用于在ThreadX和FreeRTOS™操作系统中启用RTOS内核感知调试。当启用rtos内核感知调试并启动调试会话时，debug视图将列出所有线程。通过在Debug视图中选择一个线程，编辑器中将显示线程执行的当前行。关于rtos内核感知调试的更多信息，请参见第6.3节。

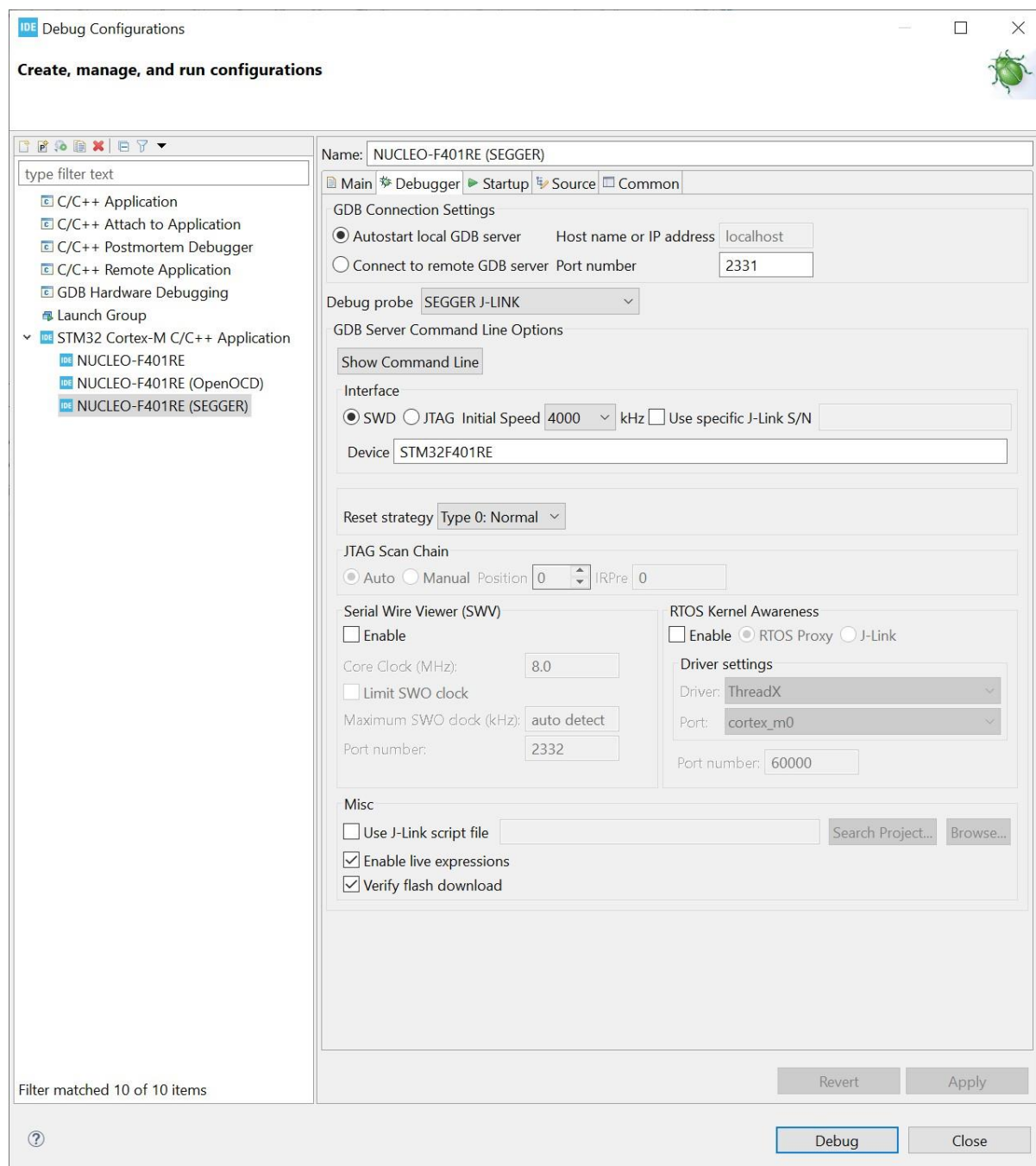
如果要在调试期间使用live expressions视图，必须启用[**Enable live expressions**]。

如果其他程序在调试会话期间必须连接到同一个ST-LINK，则必须启用[**Shared ST-LINK**]。具体请参见3.6.2共享ST-LINK。

3.4.3 Debug using SEGGER J-Link

当在调试器选项卡中选择[**SEGGER J-LINK**]时，[**GDB服务器命令行选项**]对应于SEGGER J-LINK GDB服务器。

Figure 147. SEGGER debugger tab



[Interface]选项[SWD]或[JTAG]选择SEGGER J-Link探针必须与微控制器连接的方式。[SWD]接口通常是首选;如果使用SWV，这是必需的。

[Initial Speed]项配置SEGGER J-Link与STM32设备之间使用的通信速度。

当使能[Use specific J-Link S/N]时，在edit/list字段中输入调试时使用的J-Link的S/N。当使能[Use specific J-Link S/N]时，SEGGER J-Link GDB服务器将启动，只与指定S/N的J-Link连接。

如果包含条目，则使用设备编辑字段。如果使用STM32CubeIDE中使用的默认设备名启动SEGGER J-Link GDB服务器存在问题，则可以使用该字段。在这种情况下，在编辑字段中输入SEGGER GDB服务器使用的设备名称。

The **[Reset strategy]** selection contains:

- **[Type 0: Normal]** - Default.
- **[None]** -用于附加到正在运行的目标。在这种情况下，程序必须已经下载到设备中。启动选项卡中不能有任何文件程序命令。

[JTAG Scan Chain]选项只能在**[JTAG]**接口被选中时使用。

只有选中**[SWD]**接口时，才能使用**[Serial Wire Viewer (SWV)]**选项。启用**[SWV]**时，需要配置**[Clock Settings]**。**[Core Clock]**必须设置为设备速率。有关SWV配置的更多信息，请参见第4.2.1节SWV调试配置。

[RTOS内核感知]选项用于在ThreadX和FreeRTOS™操作系统中启用RTOS内核感知调试。当启用rtos内核感知调试并启动调试会话时，debug视图将列出所有线程。通过在Debug视图选择一个线程，编辑器中将显示线程执行的当前行。关于rtos内核感知调试的更多信息，请参见第6.3节。

The **[Misc]** selections contains:

- **[Use J-Link script file]**
- **[Enable live expressions]**
为了能够在调试期间使用Live Expressions视图，必须在启动期间启用Live表达式机制。
- **[Verify flash download]**
- **[Select RTOS variant]**如果**[线程感知RTOS支持]**与**[FreeRTOS]**和**[embOS]**一起使用，则可以使用列表框。

当使用**[Thread-aware RTOS support]**时，更新启动选项卡:禁用**[Resume]** and **[in Run Commands]**，添加线程2并继续。这将强制在发送continue命令之前切换线程上下文。

Note: SEGGER J-Link GDB服务器的详细描述可以在SEGGER J-Link手册中找到，该手册可以从“信息中心”访问。

3.5 Start and stop debugging

当使用首选的JTAG探测为项目创建调试配置时，就可以进行调试了。在下面的章节中，将使用ST-LINK GDB服务器。但是，STM32项目的调试方式与ST-LINK GDB服务器、OpenOCD或SEGGER J-Link之间的选择是完全独立的。

Perform the following steps to prepare for debug:

1. 确定单板是否支持JTAG调试、SWD调试或两者都支持。swd模式调试通常是首选。
2. 在ITAC接口和目标板之间连接ITAC电缆。当使用意法半导体STM32 Nucleo和Discovery板时，ST-LINK通常集成在板上。此外，大多数意法半导体STM32评估板包含嵌入式ST-LINK。
3. 连接PC与JTAG探针之间的USB线缆。
4. 确保目标板有适当的电源连接。一旦执行了上述步骤，就可以启动调试会话。

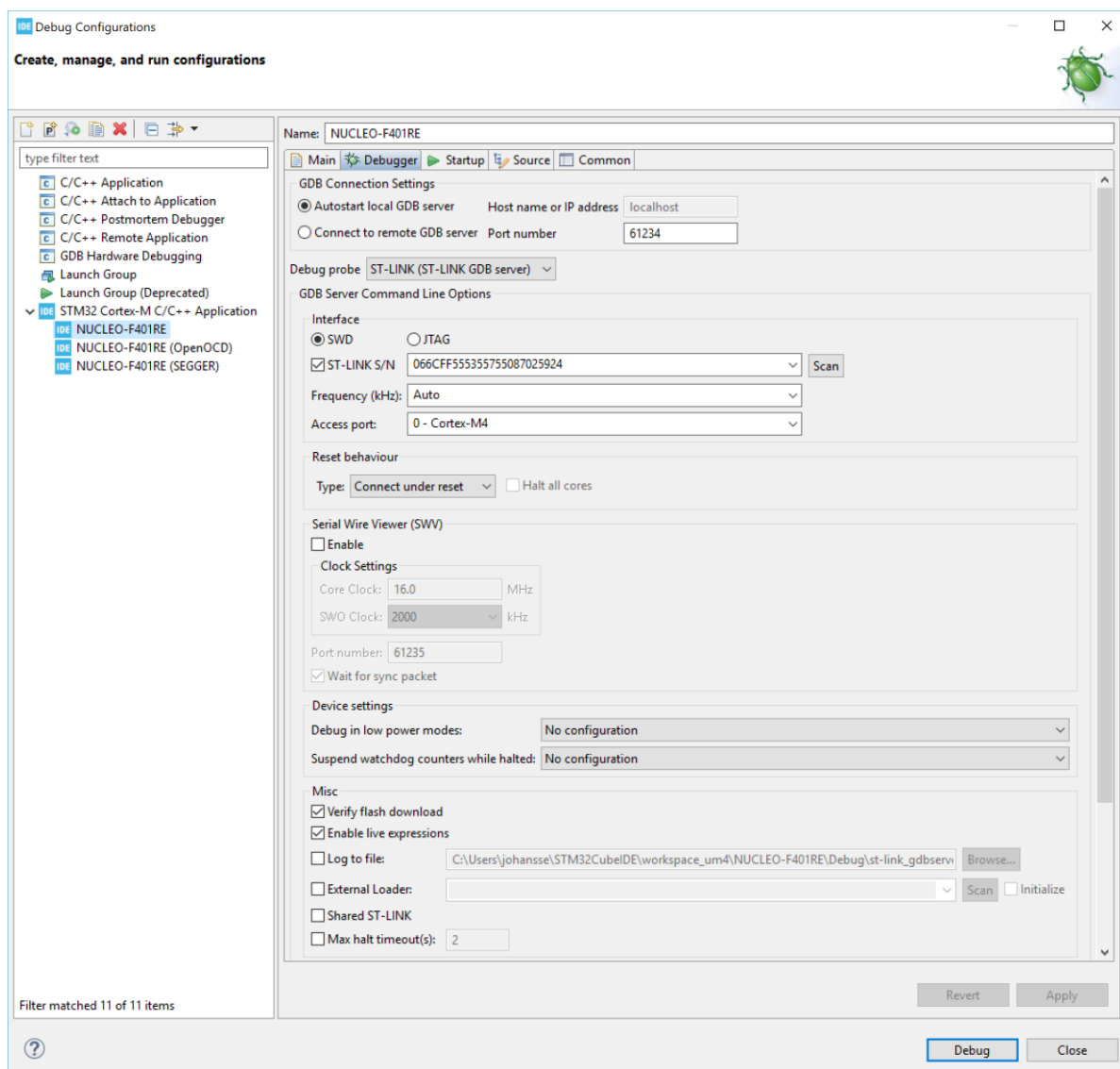
3.5.1 Start debugging

在project Explorer视图中右键单击项目名称，打开Debug Configurations对话框，并选择[Debug As]>[Debug Configurations...]。

也可以使用菜单[Run]>[Debug Configurations...]打开对话框，这将打开调试配置对话框

Note: 可以在“项目管理器”视图选择项目，并按[F11]在调试会话关闭后重新启动调试会话。

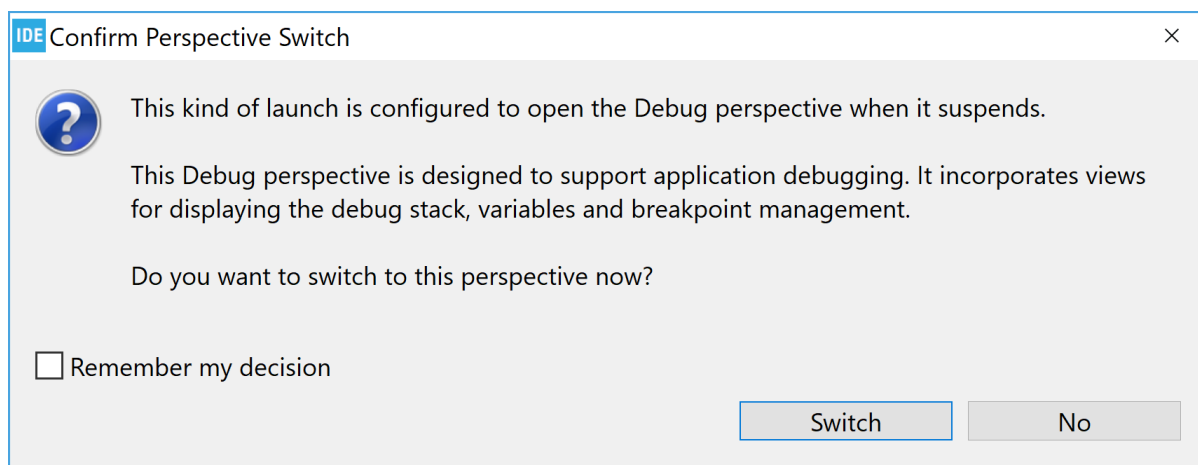
Figure 148. Debug configurations



在左侧窗格中选择要使用的调试配置。如果所有调试配置都已完成，按[Debug]按钮启动调试会话。如果进行文件更新，则构建项目，但构建取决于调试配置。

STM32CubeIDE启动调试器并打开以下对话框。

Figure 149. Confirm perspective switch

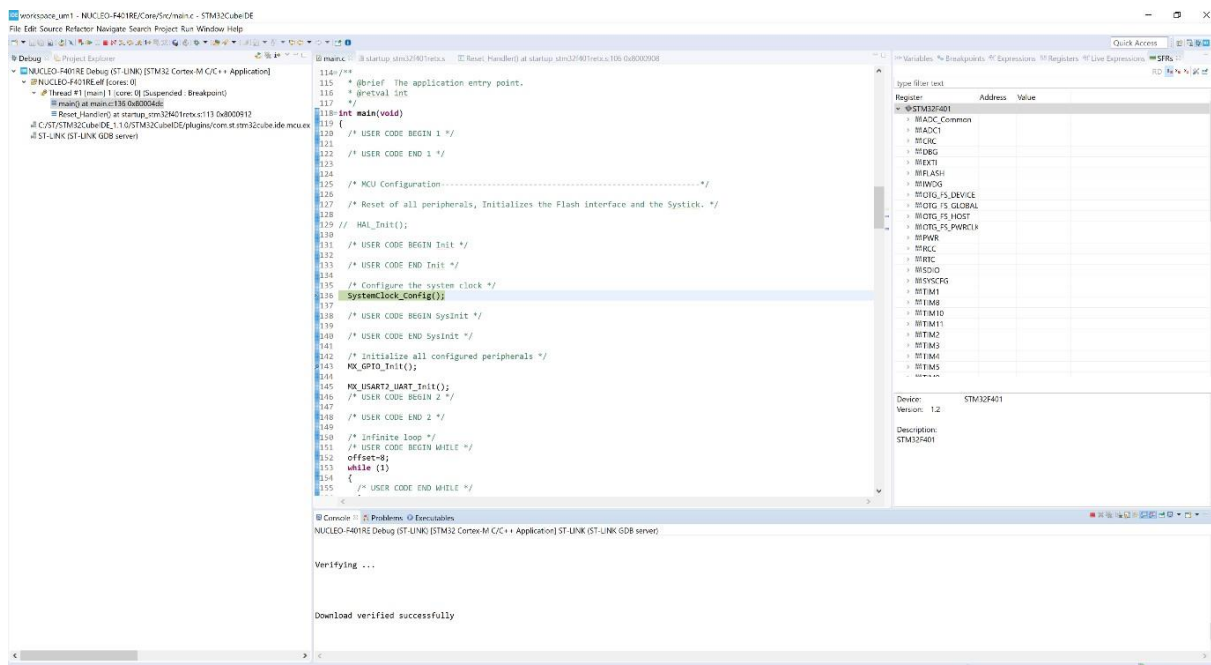


建议启用[Remember my decision]，按[Switch]。它打开Debug透视图，该透视图提供了许多适合调试的视图和窗口。

3.5.2 Debug perspective and views

Debug透视图包含调试过程中经常使用的菜单、工具栏和视图。

Figure 150. Debug perspective



- Variables视图，它在程序未运行时自动显示局部变量及其当前值。
- Breakpoints视图，它显示当前的断点。可以在列表中禁用或启用断点。断点视图还包含一个工具栏，例如，可以通过单击[Skip All Breakpoints]图标来删除断点和跳过断点。
- 表达式视图，用于添加和查看表达式。表达式可以是单个全局变量、结构或计算某些变量的表达式。这些值只在程序停止时更新。可以在Editor中选择一个全局变量，并将其拖到表达式视图中，而不是输入变量名。
- Registers视图，显示调试过的设备当前值。这些值只在程序停止时更新。
- Live Expressions视图，它显示在程序执行期间定期采样和更新的表达式值。该视图允许创建自动求值的数学表达式，例如(Index*4+Offset)。动态表达式视图更新要求在调试配置中启用动态表达式。详细信息请参见3.6.1活动表达式视图。
- SFRs视图，显示调试设备中的特殊函数寄存器。详细信息请参阅第5节特殊功能寄存器(SFRs)。
- 控制台视图，它显示不同的控制台输出。默认情况下，显示来自GDB服务器日志的控制台输出。可以通过按控制台视图右侧的[Display Selected console]图标来更改控制台日志。

其他视图在调试时也很有用，其中：

- 调试器控制台视图，如果需要手动输入GDB命令，可以使用该视图。打开调试器控制台视图最简单的方法是使用[Quick Access]字段并在该字段中输入调试器。它列出了包含调试器控制台视图的选项。选择它以打开视图。可以在调试器控制台视图中输入GDB。

例如，要显示来自地址0x8000000的16个字的内存，输入GDB命令x /16 0x8000000

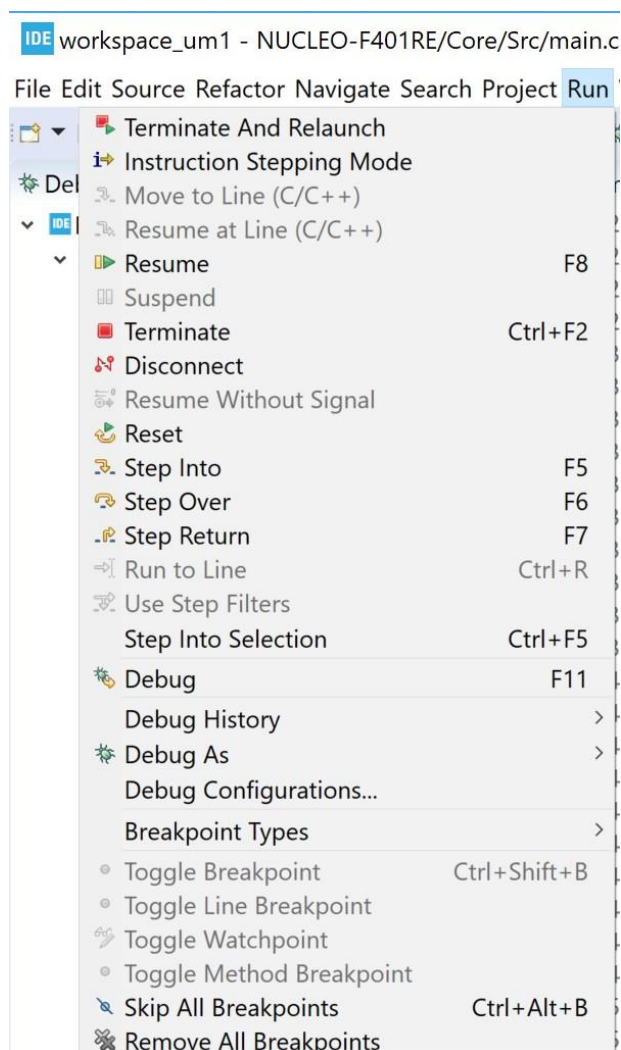
```
x /16 0x8000000
0x8000000: 0x20018000 0x080008b1 0x080007e9 0x080007f7
0x8000010: 0x080007fd 0x08000803 0x08000809 0x00000000
0x8000020: 0x00000000 0x00000000 0x00000000 0x0800080f
0x8000030: 0x0800081d 0x00000000 0x0800082b 0x08000839
```

- Memory和Memory Browser视图，可用于显示和更新内存数据。
- 反汇编视图，用于查看和步进程序集代码。
- SWV视图。有关详细信息，请参阅第4节使用串行连线查看器跟踪(SWV)进行调试。
- Fault Analyzer视图。详细信息请参见第7节故障分析器。

3.5.3 Main controls for debugging

Debug透视图中的[Run]菜单包含许多执行控制函数。

Figure 151. [Run] menu



或者，Debug透视图工具栏具有以下主要调试控件图标。

Figure 152. Debug toolbar



这些图标用于以下目的，从左到右:

- Reset the device and restart the debug session
- Skip all breakpoints (**Ctrl+Alt+B**)
- Terminate and relaunch
- Resume (**F8**)
- Suspend
- Terminate (**Ctrl+F2**)
- Disconnect
- Step into (**F5**)
- Step over (**F6**)
- Step return (**F7**)
- Instruction stepping mode (assembler stepping)

按[**Terminate and relaunch**]终止当前调试会话，如果源代码被修改，则构建一个新程序，并重新启动调试会话。

当按[**Instruction stepping mode**]时，将打开拆解视图，进一步步进使用汇编程序指令步进级别。再次按[**Instruction stepping mode**]切换回C/ C++级步进。

3.5.4 Run, start and stop a program

使用工具栏图标如下所示来运行、步进或停止程序:

- Run the program with the [**Resume**] toolbar icon ([**F8**])
- Step into a function with the [**Step into**] toolbar icon ([**F5**])
- Step over a function with the [**Step over**] toolbar icon ([**F6**])
- Step until return from a function with the [**Step return**] toolbar icon ([**F7**])
- Abort running program with the [**Suspend**] toolbar icon

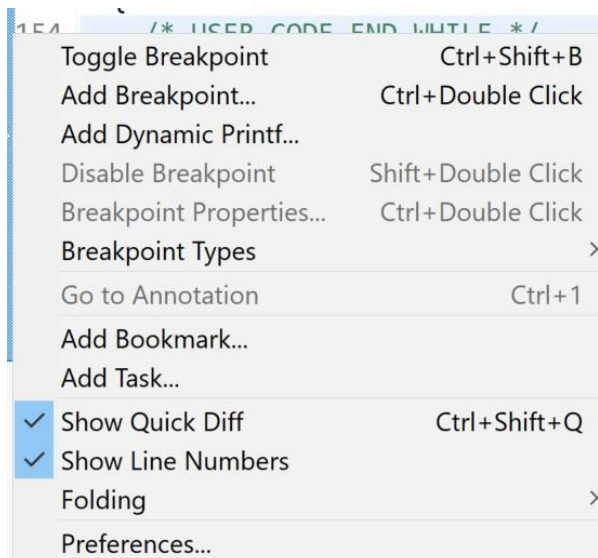
3.5.5 Set breakpoints

在调试会话期间设置断点并让代码执行直到到达断点是很常见的。

3.5.5.1 Standard breakpoint

在源代码行的标准代码断点可以通过在编辑器左侧空白处双击或在C/C++源代码编辑器左侧空白处右击来轻松插入。在后一种情况下，建议使用上下文菜单。

Figure 153. Debug breakpoint



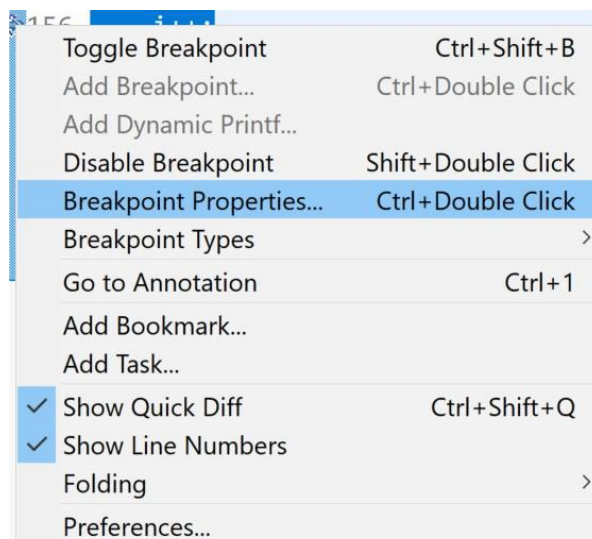
选择[Toggle Breakpoint]菜单命令在相应的源代码行上设置或删除断点。

3.5.5.2 Conditional breakpoint

在源代码行上设置标准断点时，程序每次到达该行时就会中断。如果这不是理想的行为，则可以在断点上设置一个条件，以规定程序是否应该在该断点上实际中断。

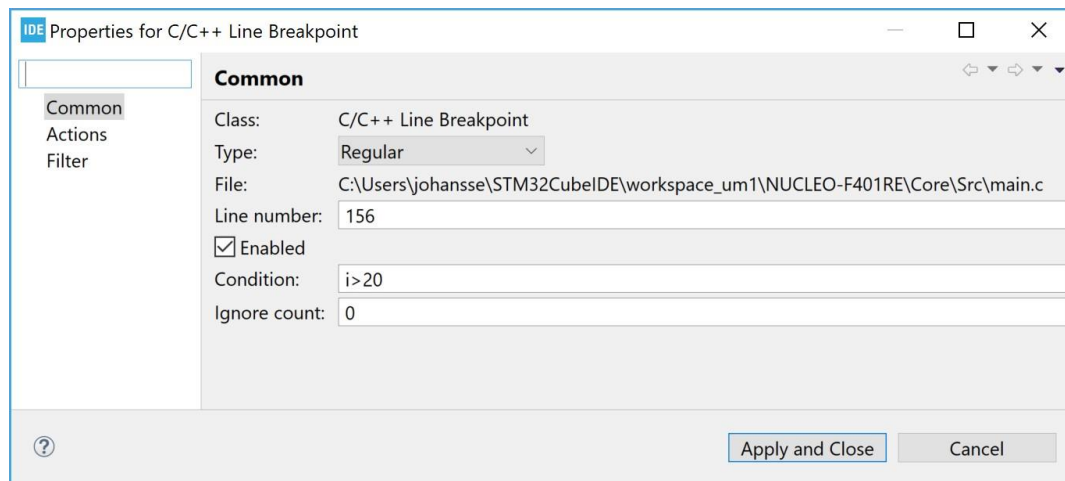
在设置了断点的行中，右键单击编辑器左侧可见的断点图标来更新断点属性。[断点属性]也可以从断点视图中打开。

Figure 154. Breakpoint properties



选择[Breakpoint Properties...]. 打开以下窗口。在下面的示例中，输入*i>20*作为条件。

Figure 155. Conditional breakpoint



设置了上面的条件后，程序在每次执行行时中断，然后GDB测试条件并在变量*i*不大于20时重新启动执行。GDB需要一段时间来评估情况。

这些条件是用c风格编写的。因此，可以编写*i%2==0* "这样的表达式来设置更复杂的条件。

3.5.6 Attach to running target

可以通过JTAG/SWD将STM32CubeIDE和调试器连接到嵌入式目标，而无需执行重置。当试图解决很少出现的问题时，这种方法非常有用。通过学习如何使用Fault Analyzer视图(请参阅第7节 Fault Analyzer)，可以进一步简化在CPU崩溃时查找问题的根本原因。

在尝试这种方法之前，请考虑在错误的状态下停止应用程序是否会潜在地损害硬件(例如在电机控制器应用程序的情况下)。这是因为当GDB连接到目标时，CPU将被暂停。这种行为无法修改。

更新调试配置并附加到运行目标需要以下三到四个步骤：

1. 修改调试配置以附加到运行目标
2. 将调试探测连接到嵌入的目标
3. 使用修改后的调试配置启动调试会话
4. 可选地，使用故障分析工具分析CPU故障状况(请参阅故障分析工具)

Step 1: Modify the debug configuration

STM32CubeIDE中默认生成的调试配置包含重置设备和下载新程序的设置，并在main上设置断点。当连接到一个正在运行的系统时，这是没有任何用处的，这个系统可能已经崩溃，也可能没有。

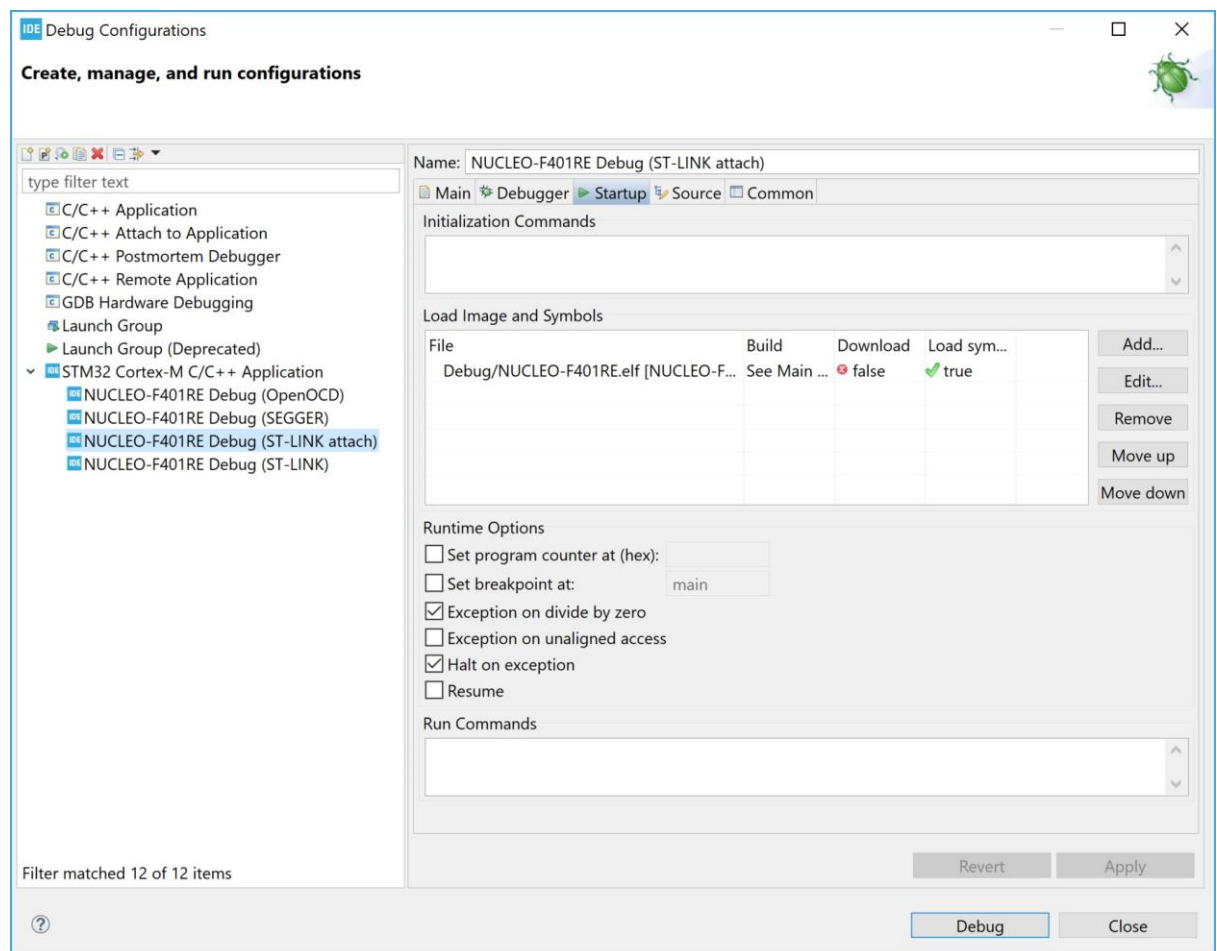
要创建修改后的调试配置，请执行以下步骤：

1. Open the *Debug Configurations* dialog.
2. In the left frame of the *Debug Configurations* dialog, select the debug configuration associated to the project to debug and make a copy of this by right-clicking it and selecting **[Duplicate]**.
3. Give the duplicate debug configuration a name.
4. Update the *Debugger* tab in *Debug Configurations*:
 - When using ST-LINK GDB server and OpenOCD, select **[None]** as **[Reset behaviour]**.
 - When using SEGGER J-Link GDB server, select **[None]** as **[Reset strategy]**.

5. Change needed/recommended in the *Startup* tab of *Debug Configurations* for both ST-LINK GDB server and SEGGER J-Link GDB server:
 - Disable file **[Download]** in **[Load Image and Symbols]**.
 - Disable **[Set program counter at (hex)]**.
 - Disable **[Set breakpoint at]**.
 - **[Exception on divide by zero]** and **[Exception on unaligned access]** can be disabled or enabled.
 - Disable **[Resume]**.

If the **[Resume]** is enabled, the debugger stops the target during connection and, after a short period of time, sends a `continue` command.

Figure 156. Startup tab attach



Step 2: Connect ST-LINK or SEGGER J-Link to the embedded target

首先将ST-LINK或SEGGER J-link连接到计算机。然后将其连接到嵌入式目标。没有复位。

Step 3: Start a debug session using the modified debug configuration

Important:

不要使用错误的调试配置启动调试会话，这可能会重编程并重置目标。使用 **[Run]>[Debug Configurations...]**，在左侧选择修改后的调试配置，单击 **[Debug]**。这是在完全控制调试配置的情况下启动调试会话并防止可能的重置的最安全方法。

调试器现在连接到嵌入的目标，该目标将自动停止。此时，可以在应用程序中研究不同的状态寄存器和变量。如果CPU崩溃了，可以使用Fault Analyzer来更好地了解根本原因。

3.5.7 Restart or terminate debugging

本节介绍重新启动和停止调试会话的各种方法。

3.5.7.1 Restart

在调试过程中，有时需要重新启动程序以更仔细地检查调试过程中观察到的问题。请使用工具栏按钮 **[Reset the chip and restart debug session]** 或菜单命令 **[Run]>[Restart]** 重新启动程序。这将重置设备，并在调试配置中启用 **[Resume]** 时启动程序。

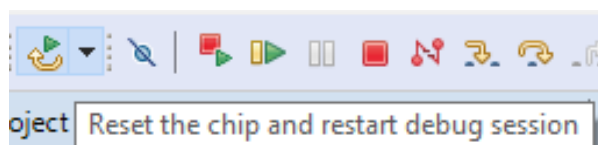
Note:

为了使重启工作，中断向量必须配置并与硬件复位一起使用。这通常是位于闪存中的STM32程序的情况。但是，如果程序位于其他地方，例如在RAM中，则可能需要进行一些手动处理，以使程序从预期的Reset_Handler开始。

3.5.7.2 Restart configurations

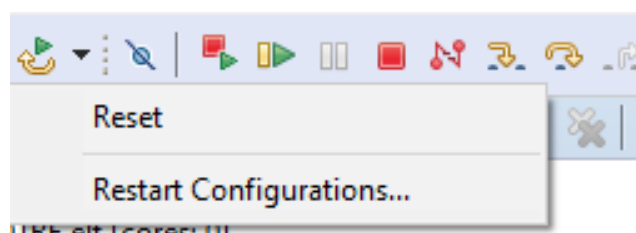
可以创建重新启动配置，定义必须如何执行调试会话的重置和重新启动。单击 **[Reset the chip and restart debug session]** 工具栏图标右侧的箭头。

Figure 157. Reset the chip toolbar



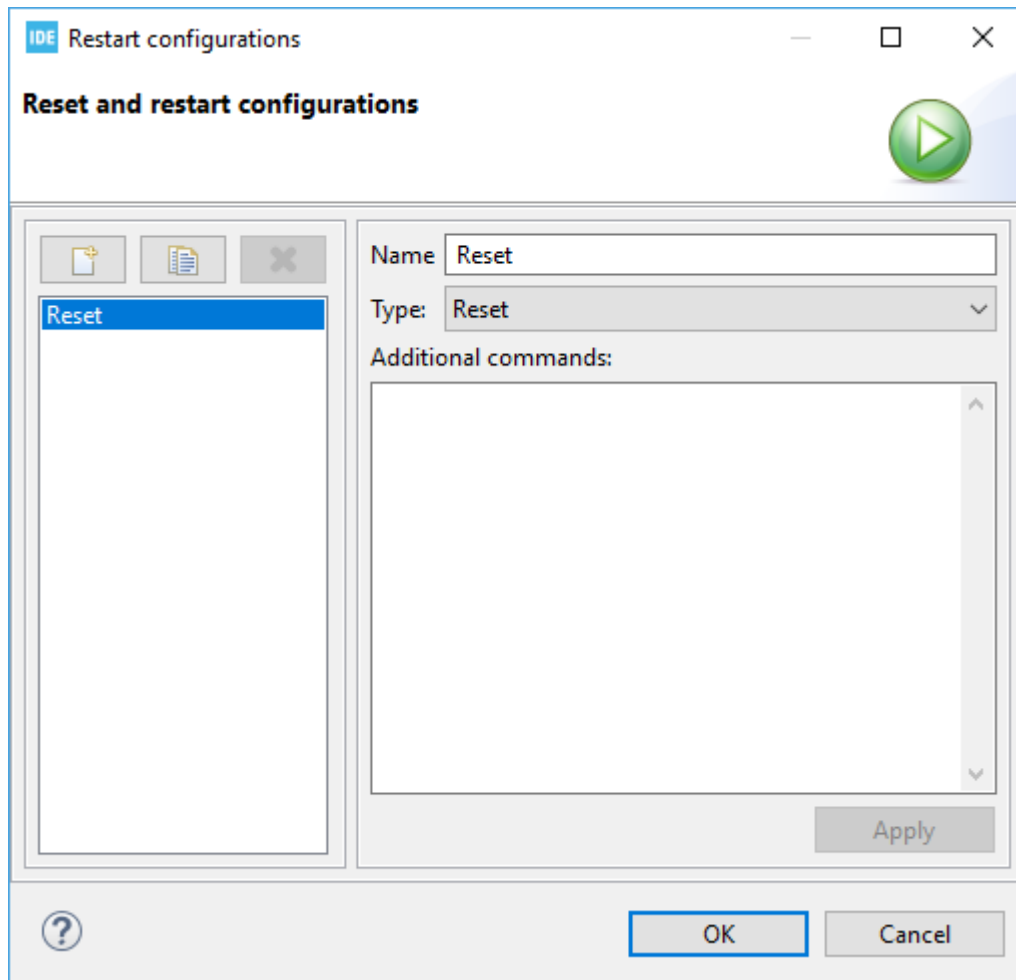
这将展开菜单 **[Restart Configurations...]** 选项。

Figure 158. Restart configurations selection



当选择[Restart Configurations...]时，将打开重启配置对话框。

Figure 159. Restart configurations dialog



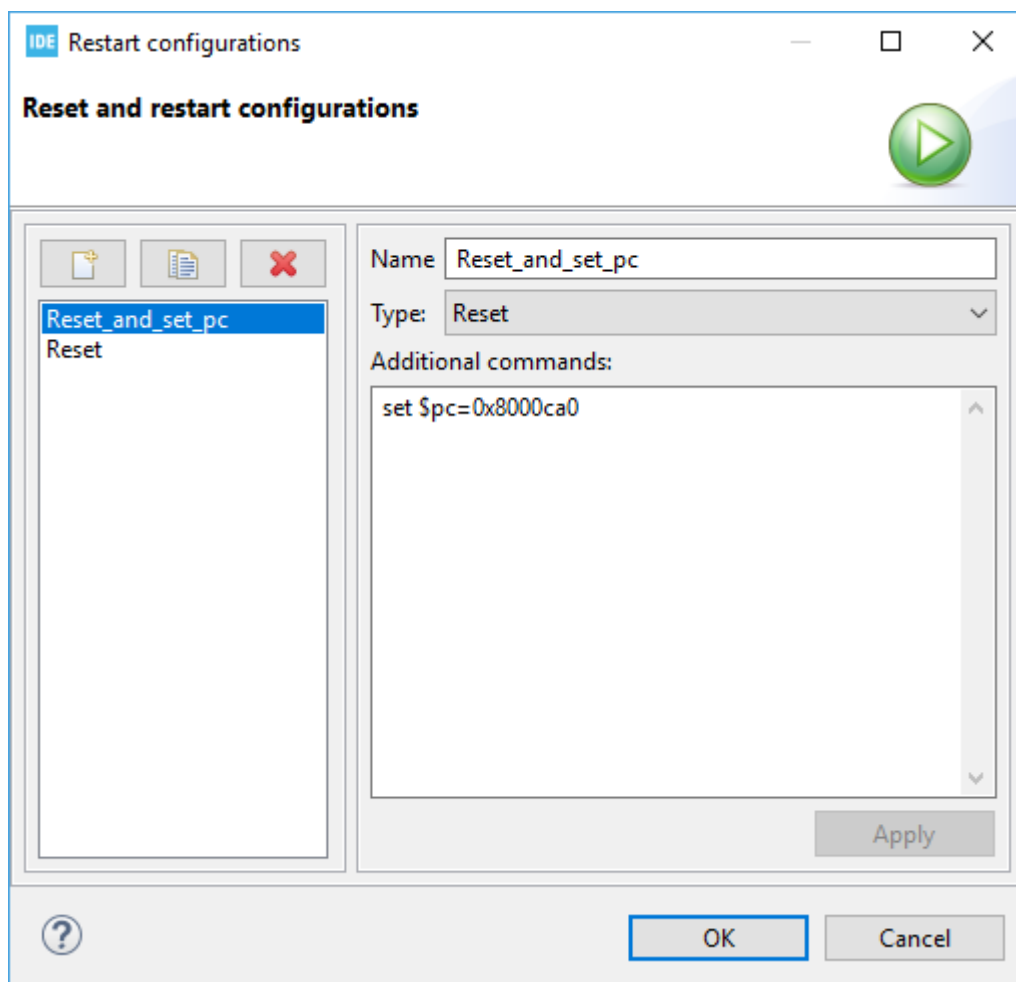
对话框包含一个左右面板

- 左侧窗格用于选择和创建新的重启配置，复制现有的重启配置，以及删除所选的重启配置。缺省重启配置不能删除。
- 右侧窗格用于设置[Name]，并选择所选配置的重置[Type]。还可以添加用于重置的其他命令。

Press **[Apply]** to save a setting.

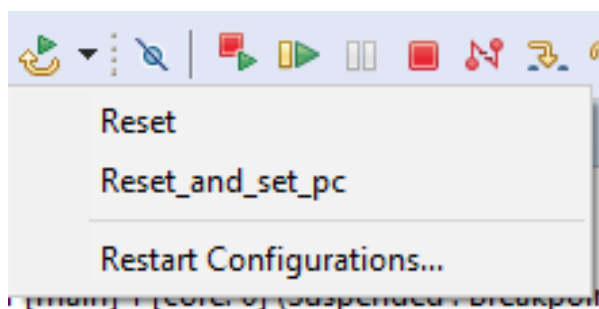
图160显示了一个设置，其中创建了一个新的重启配置，其中包含一个额外的命令，将pc设置为0x8000ca0。

Figure 160. Restart configurations dialog with additional command



定义了几个重置配置后，它们将按使用顺序出现在工具栏下拉菜单中。选择需要重置的设备。

Figure 161. Select restart configuration



3.5.7.3 Terminate

停止调试会话的最常用方法是单击[Terminate]工具栏按钮。也可以使用[Run]>[Terminate]菜单停止调试会话。当调试会话停止时，STM32CubeIDE自动切换到C/ c++透视图。

3.5.7.4 Terminate and relaunch

如果在调试期间对源代码进行了更改，则使用[终止并重新启动]工具栏按钮。菜单命令[**Terminate And Relaunch**]也可用于此目的。这将停止调试会话，重新构建程序，并重新启动加载新程序的调试会话

3.6 Debug features

3.6.1 Live Expressions view

STM32CubeIDE中的Live Expressions视图与Expression视图非常相似，不同之处在于所有表达式都是在调试执行期间实时采样的。采样速度由被采样的表达式数量决定。采样表达式数量的增加导致采样速率变慢。该视图显示了许多不同类型的全局变量。该视图还允许用户创建自动求值的数学表达式，例如($i * 4 + \text{offset}$)。

Figure 162. Live Expressions

Expression	Type	Value
(x)= i	uint32_t	2
(x)= offset	uint32_t	8
(x)= i*4+offset	unsigned int	16
Add new expression		

视图可以解析复杂的数据类型，并像c语言结构一样显示复杂的数据类型。同一时间只能使用一种数字格式。若要更改此格式，请使用下拉菜单。

Figure 163. Live expressions number format

Expression	Type	Value
(x)= i	uint32_t	2
(x)= offset	uint32_t	8
(x)= i*4+offset	unsigned int	16
Add new expression		

当程序运行时，变量值可以在Live Expressions视图中动态更改。选择变量并更改其值。它要求在表达式中只使用单个变量名，并且不涉及计算。

Note: 为了能够在调试期间使用Live Expressions视图，必须在启动期间启用Live表达式机制。

3.6.2 Shared ST-LINK

在ST-LINK GDB服务器和OpenOCD的调试配置中的调试器选项卡中，有一个选项启用共享ST-LINK。当启用共享ST-LINK时，与ST-LINK的通信将通过ST-LINK服务器进行。当共享ST-LINK被启用时，ST-LINK服务器使得多个程序可以访问同一个ST-LINK。

STM32CubeProgrammer (STM32CubeProg)也包含共享ST-LINK的配置。这意味着当在

STM32CubeIDE的调试配置中启用共享ST-LINK时，可以调试程序并让STM32CubeProgrammer同时访问和读取设备闪存和RAM。

启用共享模式会导致STM32CubeIDE启动ST-LINK服务器(如果它还没有运行)，并使用默认端口7184监听TCP连接。这个默认端口不能从STM32CubeIDE编辑。

3.6.3 Debug multiple boards

使用两个ST-LINK或SEGGER J-Link探头同时调试多个板是可能的。两个探针连接到两个不同的微控制器上，通过不同的USB端口连接到一台PC上。在本节中，让我们假设使用了两个不同的板/微控制器:HW_A和HW_B。

可以运行一个STM32CubeIDE实例，其中包含HW_A的一个项目和HW_B的一个项目。默认端口为：

- 61234 for ST-LINK GDB server
- 3333 for OpenOCD
- 2331 SEGGER J-Link

这在调试配置对话框中的调试器选项卡中显示。如果其中一个项目要使用另一个端口，则必须更改端口号，例如端口61244。

调试配置可以使用GDB连接选择[自动启动本地GDB服务器]。注意，当调试多个板时，两个或多个调试探头连接到PC;必须为每个调试配置选择正确的序列号。

当为两个项目配置了调试配置，以便每个板都与特定探测相关联时，就应该首先单独测试和调试每个板。当确认这是工作的时候，可以同时开始两个目标的调试，如下所示：

1. Start to debug HW_A.
2. The perspective switches automatically to the *Debug* perspective in STM32CubeIDE when a debug session for HW_A is started.
3. Switch to the C/C++ perspective.
4. Select the project for HW_B and start debugging it. The *Debug* perspective opens again.
5. There are two application stacks/nodes in the *Debug* view, one for each project. When changing the selected node in the *Debug* view, the related editor, variable view and others are updated to present information associated to the selected project.

也可以手动启动GDB服务器:在调试配置中选择[Connect to remote GDB server]。在这种情况下，确保启动GDB服务器时使用了定义要使用的各个端口和序列号的参数，并且在每个项目的Debug Configurations对话框中使用了相应的端口号。

下面是SEGGER J-Link GDB服务器连接到SEGGER J-Link的示例，端口=2341,S/N=123456789:

```
>JLinkGDBServerCL.exe -port 2341 -if SWD -select usb=123456789
```

关于手动启动GDB服务器时使用的命令行参数的信息，在从信息中心获得的GDB服务器手册中提供。

3.6.4 STM32H7 multicore debugging

关于如何在STM32CubeIDE中使用STM32H7多核设备的信息可在[ST-09]中获得。

3.6.5 STM32MP1 debugging

关于如何在STM32CubeIDE中使用STM32MP1设备的信息可在[ST-08]中获得。

建议用户在[www.st.com/en/microcontrollers-microprocessors/ STM32MP1 - Series](http://www.st.com/en/microcontrollers-microprocessors/stm32mp1-series)上随时更新STM32MP1系列的文档演进。

3.6.6 STM32L5 debugging

关于如何在STM32CubeIDE中与TrustZone®一起使用STM32L5设备的信息可在[ST-10]中获得。

Note: TrustZone是Arm Limited(或其子公司)在美国和其他地方的注册商标。

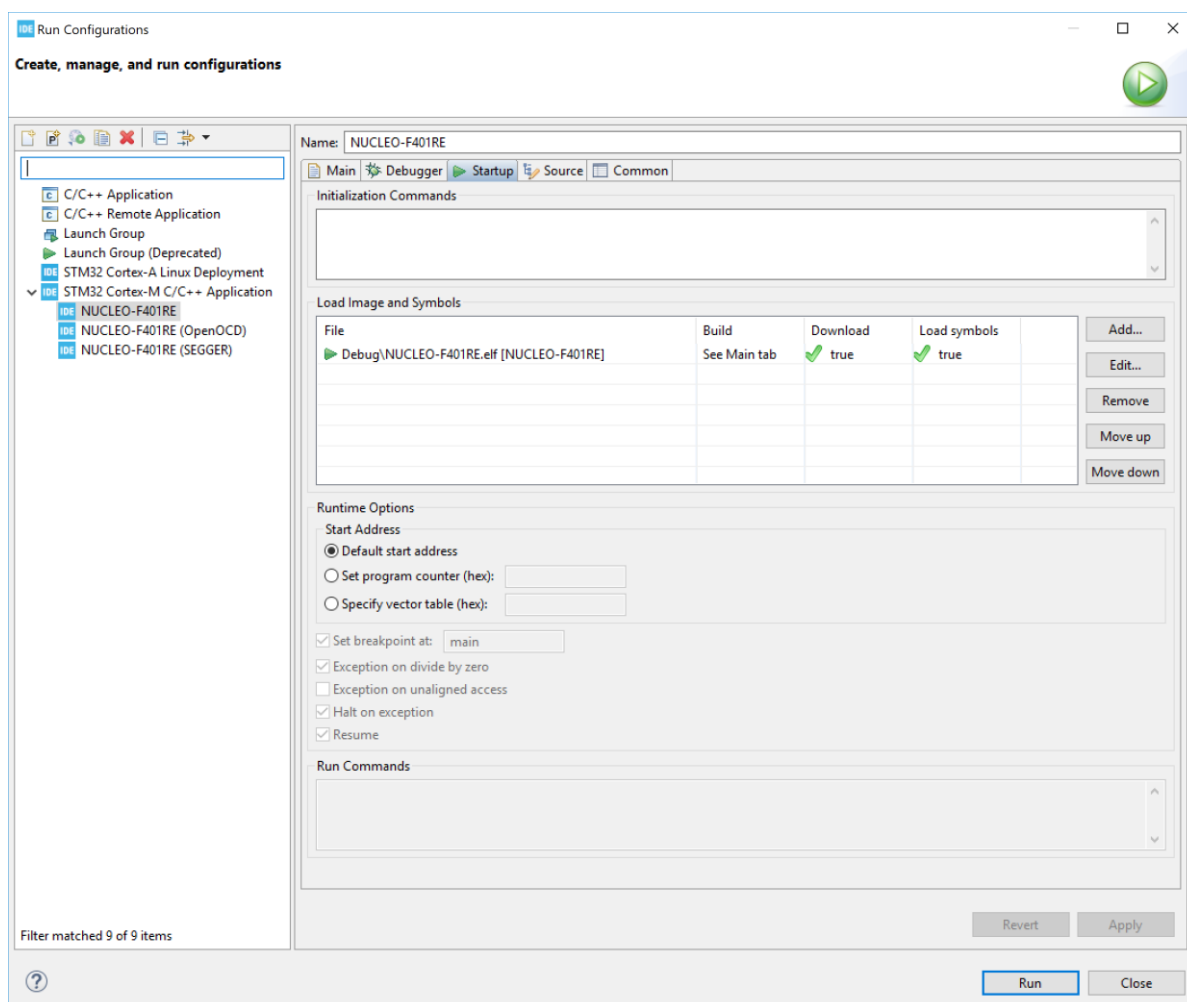
3.7 Run configurations

可以创建运行配置来下载应用程序并重置目标，而无需启动完整的调试会话。“运行配置”对话框类似于“调试配置”对话框，但是“启动”选项卡下部禁用的小部件不会执行。当运行一个运行配置时，指定的程序被闪烁，但是，在程序计数器设置之后，程序在目标中开始执行，STM32CubeIDE中的“运行”会话被关闭。

要为项目创建运行配置，在项目资源管理器视图中右键单击项目名称，并选择[Run As]>[STM32 Cortex-M C/C++ Application]

另一种创建运行配置的方法是在项目资源管理器视图中选择项目名称，并使用菜单[Run]>[Run As]>[STM32 Cortex-M C/C++ Application]。

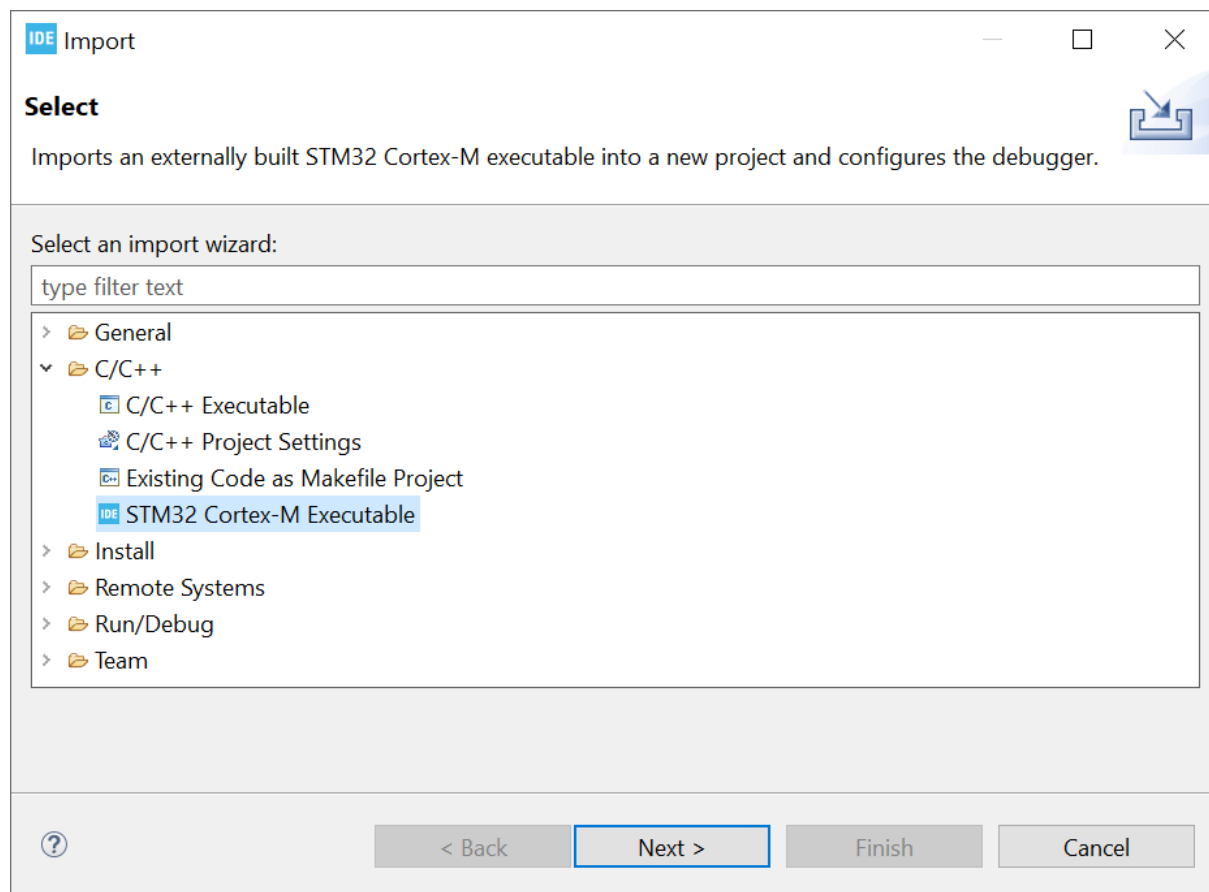
Figure 164. Run configurations startup tab



3.8 Import STM32 Cortex®-M executable

Use menu [File]>[Import...] to open the *Import* dialog.

Figure 165. Cortex®-M executable import dialog



Select **[STM32 Cortex-M Executable]** and press **[Next >]**.

Figure 166. STM32 Cortex®-M executable dialog

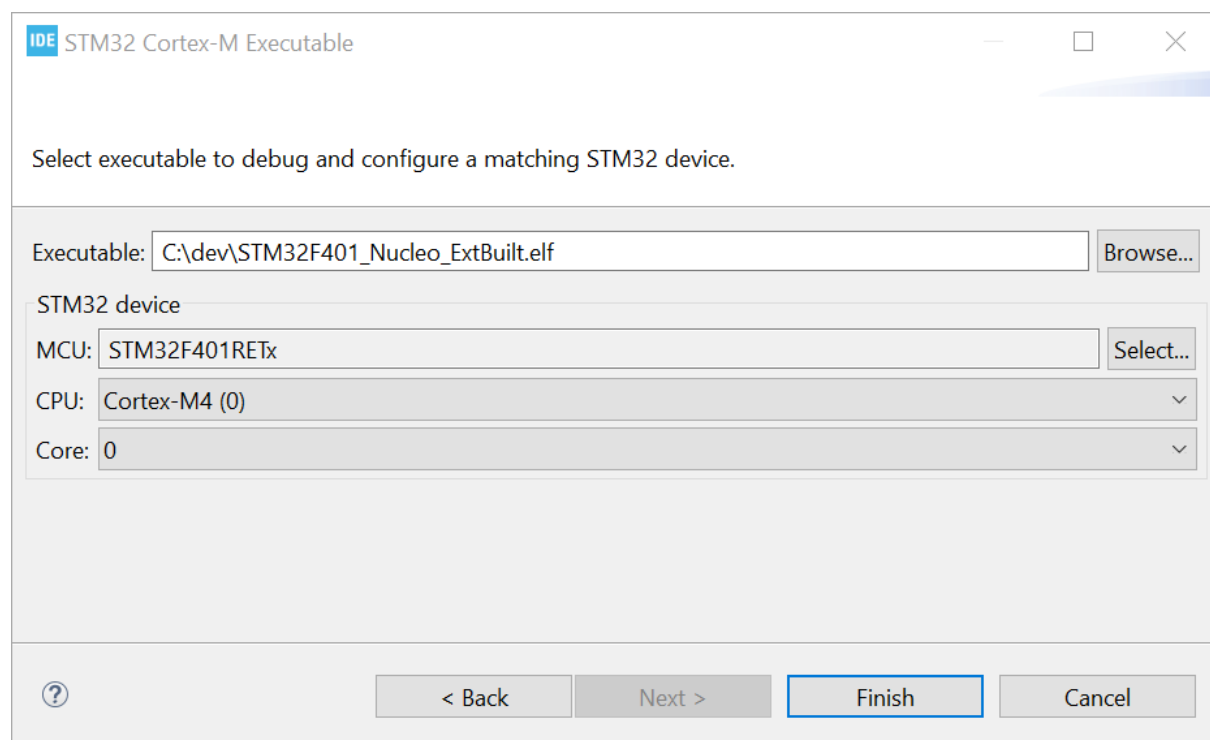
Use the **[Browse...]** button and select the `elf` file to import. When the `elf` file is selected, the STM32 device must be selected manually so that STM32CubeIDE can be used for debugging. Press **[Select...]** to open the *MCU/MPU Selector* dialog.

Figure 167. STM32 Cortex®-M executable MCU/MPU selection

Select the microcontroller or microprocessor to be used. The search field can be used to find the device. Press [OK] once the device is selected.

As a result, the CPU and core are presented in the dialog.

Figure 168. STM32 Cortex®-M CPU and core



IDE STM32 Cortex-M Executable

Select executable to debug and configure a matching STM32 device.

Executable: C:\dev\STM32F401_Nucleo_ExtBuilt.elf Browse...

STM32 device

MCU: STM32F401RETx Select...

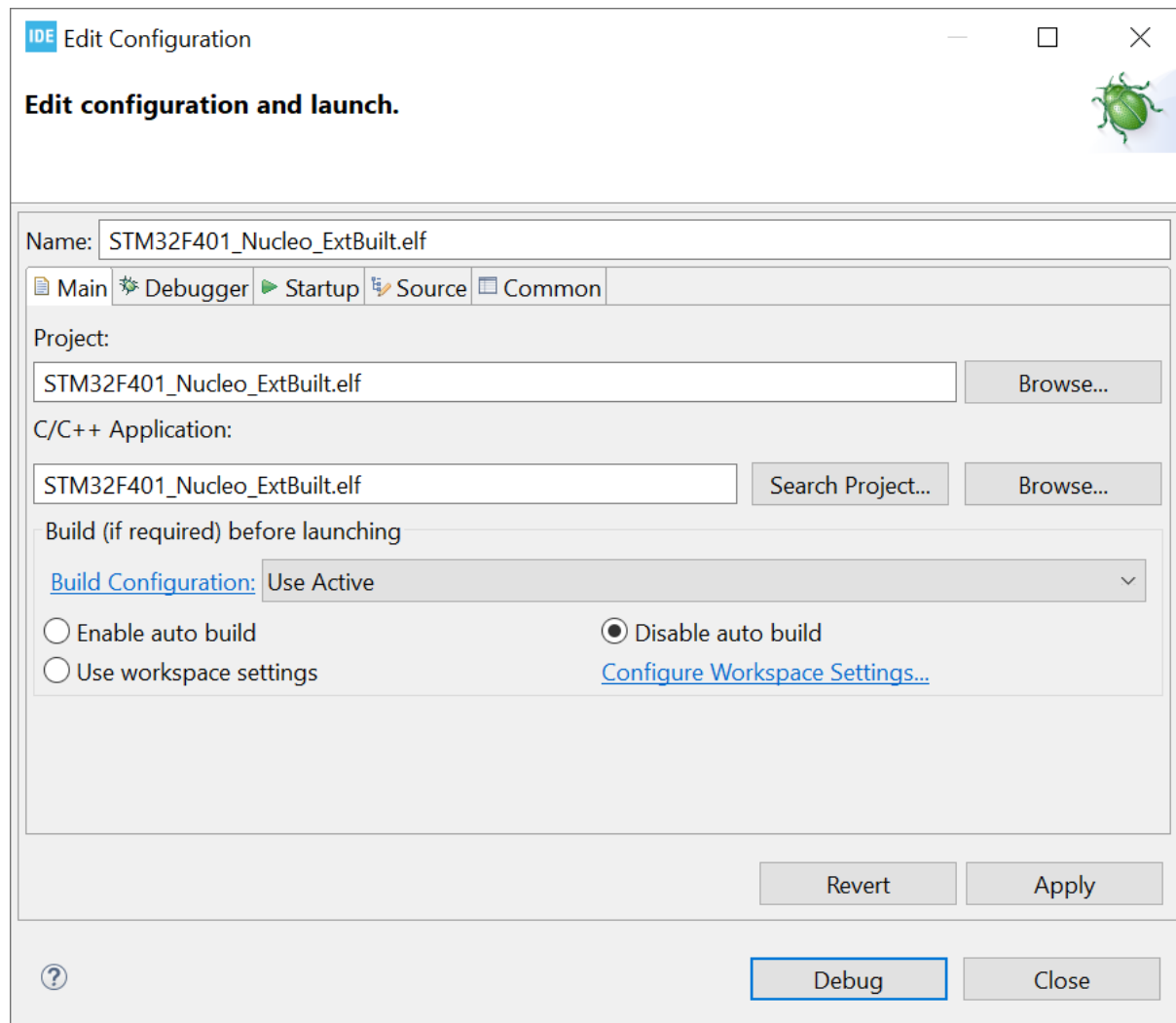
CPU: Cortex-M4 (0) ▾

Core: 0 ▾

? < Back Next > Finish Cancel

Press **[Finish]** and the debug configuration dialog automatically opens.

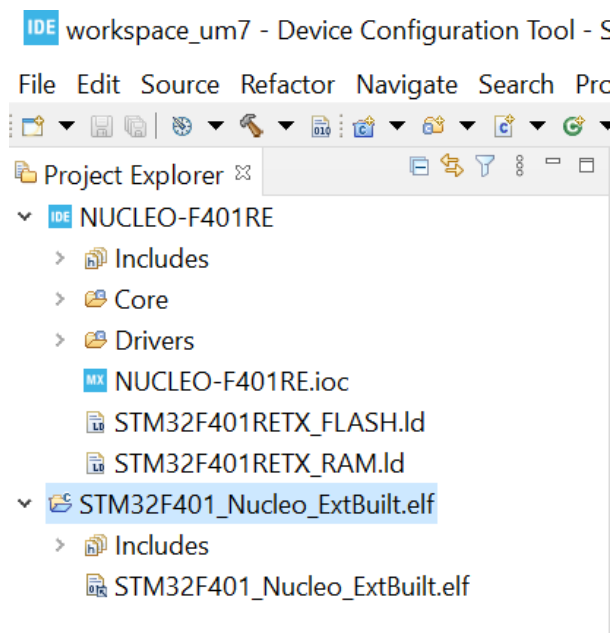
Figure 169. Cortex®-M debug configuration for imported project



The debug configuration can then be set up in similar way as with any other STM32CubeIDE project. Once the configuration is completed, press **[Debug]** to start a debug session.

The imported project is displayed in the *Project Explorer* view.

Figure 170. Project explorer view with imported project



4 Debug with Serial Wire Viewer tracing (SWV)

4.1 Introduction to SWV and ITM

本节提供有关如何在STM32CubeIDE中使用串行连线查看器跟踪(SWV)的信息。

STM32中的系统分析和实时跟踪需要多种交互技术:串行连线查看器(SWV)、串行连线调试(SWD)、仪器跟踪宏单元(ITM)和串行连线输出(SWO)。这些技术是Arm®CoreSight™ 调试器技术的一部分。它们解释如下。

串行连线调试(SWD)是一个类似于JTAG的调试端口。它提供了相同的调试功能(运行、在断点上停止、单步),但使用了更少的引脚。它用2引脚接口(一个时钟引脚和一个双向数据引脚)取代了JTAG连接器。单独的SWD端口不允许实时跟踪。

串行线输出(SWO)引脚可以与SWD组合使用。处理器使用它来发出实时跟踪数据,从而用第三个引脚扩展两个SWD引脚。两个SWD引脚和SWO引脚的组合可以在兼容的Arm®处理器中实现串行线查看器(SWV)实时跟踪。

注意,SWO只是一个引脚,很容易设置产生比SWO能够发送的更多数据的配置。

串行线查看器(SWV)是一种使用串行线调试(SWD)端口和串行线输出(SWO)引脚的实时跟踪技术。Serial Wire Viewer提供高级的系统分析和实时跟踪,而不需要暂停处理器来提取调试信息。

串行连线查看器(SWD)提供以下类型的目标信息:

- Event notification on data reading and writing 数据读写的事件通知
- Event notification on exception entry and exit 异常进入和退出的事件通知
- Event counters 事件计数器
- Timestamp and CPU cycle information, which can be used for program statistical profiling 时间戳和CPU周期信息,可用于程序统计分析

Instrumentation Trace Macrocell (ITM)允许应用程序将任意数据写入SWO引脚,这些数据可以在调试器中进行解释和可视化。例如,ITM可用于将printf()输出重定向到调试器中的SWV控制台视图。标准是将端口0用于此目的。

ITM端口有32个通道。将不同类型的数据写入不同的ITM通道允许调试器以不同的方式解释或可视化不同通道上的数据。

将一个字节写入ITM端口只需要一个写入周期,因此几乎不占用应用程序逻辑的执行时间。

基于SWV和ITM跟踪数据,STM32CubeIDE可以提供具有特殊SWV视图的高级调试器功能。

Note: . Arm®在Cortex®- M0或Cortex®- M0+内核中不包括SWV/ITM。因此,基于这些核心的STM32器件,如STM32L053微控制器,不支持SWV/ITM。

4.2 SWV debugging

要调试和使用STM32CubeIDE中的串行连线查看器(Serial Wire Viewer, SWV), JTAG探针和GDB服务器必须支持SWV。该板还必须支持SWD,并且SWO引脚需要可用并连接到JTAG探针。

以下部分描述了创建调试配置、SWV设置配置的过程，以及如何在调试会话中使用SWV跟踪

4.2.1 SWV debug configuration

Step 1: Open the *Debug Configurations* dialog

使用实例菜单[Run]>[Debug Configurations...]并选择要更新的STM32 Cortex®-M调试配置。

Step 2: Select the SWD interface

Select the [SWD] interface in the *Debug Configurations* dialog.

Step 3: Enable SWV

Enable [SWV] in the *Debug Configurations* dialog.

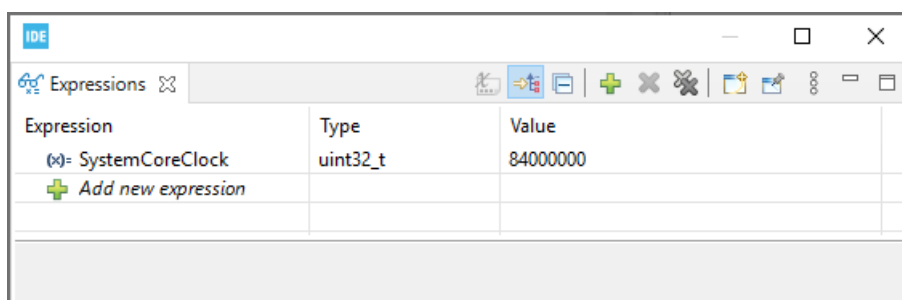
Step 4: Enter the core clock frequency

在调试配置对话框中输入[Core Clock]频率。这必须与要执行的应用程序设置的值相对应。

通常，当使用从STM32固件示例导入的项目或使用STM32CubeMX创建的项目时，核心时钟设置存储在SystemCoreClock变量中。检查核心时钟值的一种方法是启动一个调试会话，并将SystemCoreClock变量添加到Expressions视图。在读取该值之前，请确保应用程序已经配置了系统核心时钟。

如果SystemCoreClock没有更新，则更改程序并添加对函数SystemCoreClockUpdate()的调用。重新构建程序，重新启动调试并再次检查SystemCoreClock值。

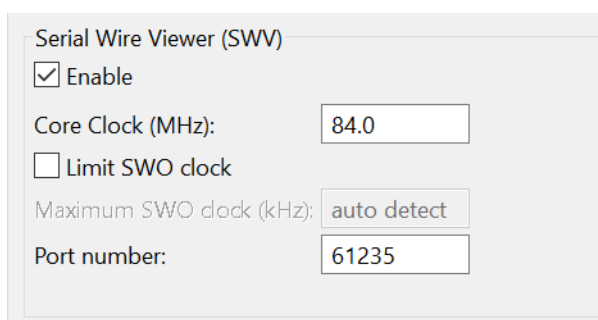
Figure 171. SWV core clock



Step 5: Enter the SWO clock frequency

调试配置对话框中的[Serial Wire Viewer (SWV)]选项只能在选中[SWD]接口时使用。启用[SWV]时，需要配置[Clock Settings]。[Core Clock]必须设置为设备速率。SWO时钟自动设置为最高可能的速度，这取决于所使用的调试探头和核心时钟。但是，如果已调试的硬件不允许太多的SWO时钟速度，则可以启用[Limit SWO clock]并输入以kHz为单位的最大SWO时钟速度。SWV[Port number]必须设置为用于SWV数据通信的端口号。SWV端口不能设置为GDB连接[Port number]。

Figure 172. SWV debug configuration



Step 6: Save the configuration

Press [Apply] in the *Debug Configurations* dialog to save the configuration.

Step 7: Start a debug session



Press **[Debug]** to start a debug session. Make sure that the probe and board are connected.

Step 8: Possibly suspend the target

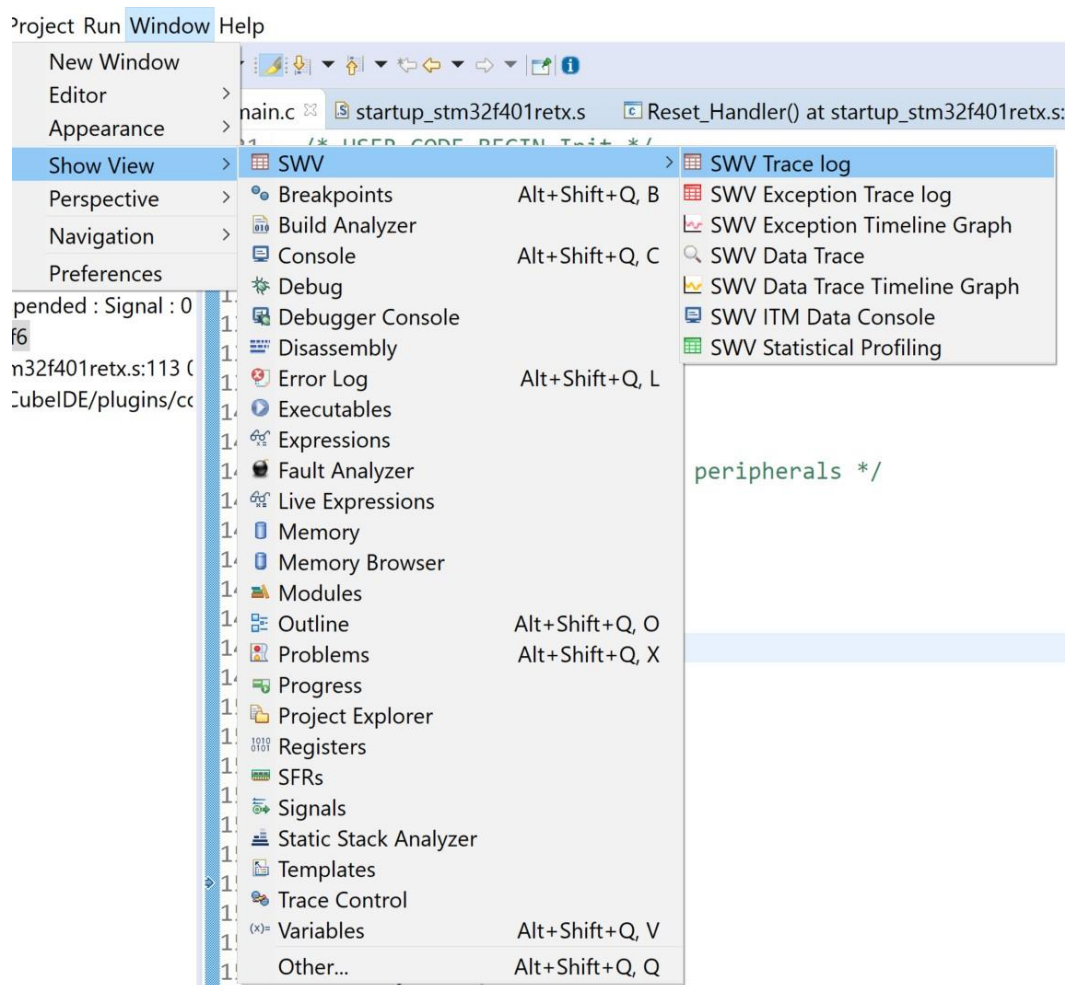
[挂起]目标，如果它没有停止在一个断点。

Step 9: Open a SWV view

打开SWV视图之一。对于第一次使用SWV的用户，建议打开SWV Trace日志视图，因为它可以很好地概述传入的SWV包以及跟踪的工作情况。

选择[Window]>[Show View]>[SWV]>[SWV Trace log] menu命令，打开SWV跟踪日志视图。

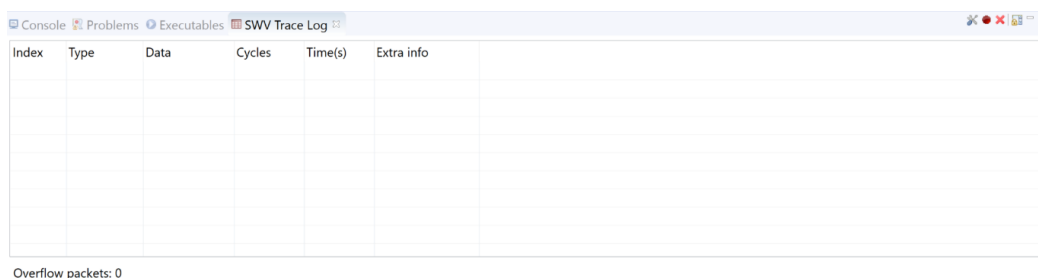
Figure 173. SWV show view



Step 10: View the trace log

The SWV Trace log view is now visible.

Figure 174. SWV Trace log view



Overflow packets: 0

4.2.2 SWV settings configuration

Step 1: Open the *Serial Wire Viewer* settings

在SWV跟踪日志视图中单击[Configure Trace]工具栏按钮，打开Serial Wire Viewer设置对话框。

Figure 175. SWV [Configure Trace] toolbar button

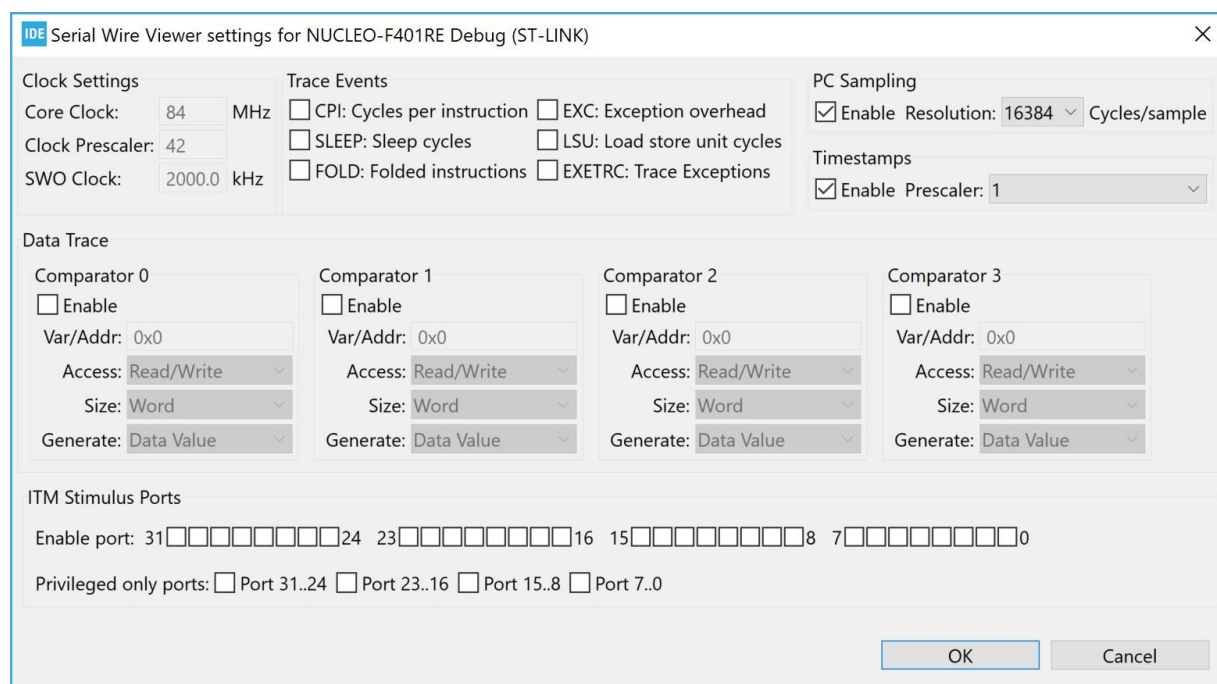


Note: *[Configure Trace]* 工具栏按钮在所有SWV视图中都可用。

Step 2: Configure the trace data

在Serial Wire Viewer设置对话框中配置要跟踪的数据。本例启用了[PC Sampling]和[Timestamps]。

Figure 176. SWV settings dialog



SWV设置对话框有以下配置:

- **[Clock Settings]:** 这些字段被禁用，并且仅显示在调试会话的调试配置中使用和配置的值。如果需要更改这些值，请关闭调试会话并打开调试配置以修改它们。
- **[Trace Events]:** The following events can be traced.
 - **[CPI]:** 每条指令循环。对于指令使用的第一个周期以外的每一个周期，内部计数器都增加1。计数器(DWT CPI计数)最多可以计数到256，然后设置为0。每次发生这种情况，就会发送一个这样的包。这是处理器性能的一个方面，用于计算每秒指令数。该值越低，性能越好。
 - **[SLEEP]:** 睡眠周期。CPU处于休眠模式的周期数。在DWT睡眠计数寄存器中计数。每当CPU处于256个周期的睡眠模式时，就会发送一个这样的数据包。这用于调试功耗或等待外部设备时。
 - **[FOLD]:** 折叠的指令。计算被折叠(删除)指令数量的计数器。每折叠256条指令(0个周期)将收到一个这样的事件。计数在DWT折叠计数寄存器。
 分支折叠是一种技术，在大多数分支的预测中，分支指令完全从呈现给执行管道的指令流中删除。分支折叠可以显著提高分支的性能，使分支的CPI小于1。
 - **[EXC]:** 异常的开销。DWT异常计数寄存器跟踪异常开销中花费的CPU周期数。这包括堆栈操作和返回，但不包括处理异常代码所花费的时间。当计时器溢出时，将发送其中一个事件。用于计算程序的实际异常处理成本。
 - **[LSU]:** 加载存储单元周期。DWT LSU计数寄存器计算处理器在第一个周期之后处理LSU操作的总周期数。当计时器溢出时，将发送其中一个事件。
 通过这种测量，可以跟踪内存操作所花费的时间。
 - **[EXETRC]:** 跟踪异常。无论何时发生异常，都会发送异常进入、异常退出和异常返回事件。可以在SWV异常跟踪日志视图中监视这些事件。从这个视图，可以跳转到该异常的异常处理程序代码。
- **[PC Sampling]:** 启用此功能将开始以某个周期间隔对程序计数器进行采样。由于SWO引脚带宽有限，不建议采样太快。使用**[Resolution]**(周期/采样设置)进行实验，以便能够经常采样。抽样的结果被用于SWV统计分析视图。
- **[Timestamps]:** 必须启用，以便了解事件发生的时间。**[Prescaler]**应该只作为减少溢出数据包的最后努力来更改。
- **[Data Trace]:** 它可以跟踪多达四个不同的C变量符号，或内存的固定数字区域。为此，启用一个比较器，并输入要跟踪的变量或内存地址的名称。跟踪变量的值可以在数据跟踪和数据跟踪时间轴图视图中显示。
- **[ITM Stimulus Ports]:** 有32个可用的ITM端口可供应用程序使用。例如，CMSIS函数ITM_SendChar可用于将字符发送到端口0(参见第4.3.5节SWV ITM数据控制台和printf重定向)。在SWV ITM Data Console视图中显示来自ITM端口的报文。

Note: 建议限制跟踪的数据量。大多数STM32微控制器读写数据的速度都快于最大SWO引脚吞吐量。

过多的跟踪数据会导致数据溢出、丢包和可能损坏的数据。为了获得最佳性能，只跟踪手头任务所需的数据。

运行SWV时溢出表明SWV被配置为跟踪超过SWO引脚能够处理的数据。在这种情况下，减少跟踪的数据量。

启用[**Timestamps**]来使用STM32CubeIDE中的任何时间轴视图。缺省值[Prescaler]为1。保持该值，除非发生与SWV包溢出相关的问题。

下面提供了三个示例来说明SWV跟踪配置:

- **Example 1:**要跟踪全局变量的值，启用[Comparator]并输入要跟踪的变量名或内存地址。
跟踪变量的值同时显示在数据跟踪和数据跟踪时间轴图视图中。
- **Example 2:**要分析程序执行情况，启用[PC采样]。在开始时，建议为[Cycles/sample]设置一个较高的值。
然后在SWV统计概要视图中显示PC抽样的结果。
- **Example 3:**要跟踪程序执行过程中发生的异常，启用[Trace Event EXETRC: Trace Exceptions]。
然后，在SWV Exception Trace Log视图中显示有关异常的信息。

Step 3: Save the SWV configuration

单击[OK]按钮保存SWV配置。该配置与其他调试配置一起保存，在更改之前保持有效。

4.2.3 SWV tracing

Step 1: Start SWV trace recoding

在SWV视图之一中按下[Start/Stop Trace]工具栏按钮，将SWV设置发送到目标板并启动SWV跟踪重新编码。此工具栏按钮可在所有SWV视图中使用。在正确配置SWV之前，单板不会发送任何SWV报文。如果目标板上的配置寄存器被重置，则必须重新发送SWV配置。实际跟踪直到目标开始执行才开始。

Figure 177. SWV [Start/Stop Trace] toolbar button



Note: 在目标运行时不能配置跟踪。在尝试向单板发送新配置之前，暂停调试。每个新的或更新的配置都必须发送到单板才能生效。按下 [Start/Stop Trace]按钮，将配置信息发送到单板。

Step 2: Start the target

按Debug透视图顶部的[Resume]工具栏按钮来启动目标。

Step 3: SWV Trace Log view

在“SWV Trace Log”视图下显示SWV报文。

Figure 178. SWV Trace Log PC sampling

IDE

SWV Trace Log

Index	Type	Data	Cycles	Time(s)	Extra info
10362	PC Sample	0x8000508	169777034	2.021155 s	
10363	PC Sample	0x8000516	169793417	2.021350 s	
10364	PC Sample	0x8000528	169809800	2.021545 s	
10365	PC Sample	0x8000500	169826183	2.021740 s	
10366	PC Sample	0x8000510	169842566	2.021935 s	
10367	PC Sample	0x80004f2	169858949	2.022130 s	
10368	PC Sample	0x8000504	169875332	2.022325 s	
10369	PC Sample	0x8000516	169891715	2.022520 s	
10370	PC Sample	0x8000528	169908098	2.022715 s	

Overflow packets: 0

Step 4: Clear collected SWV data

当目标没有运行时，可以通过按[Remove all collected SWV data]工具栏按钮清除收集的SWV数据。此工具栏按钮可在所有SWV视图中使用。

Figure 179. [Remove all collected SWV data] toolbar button

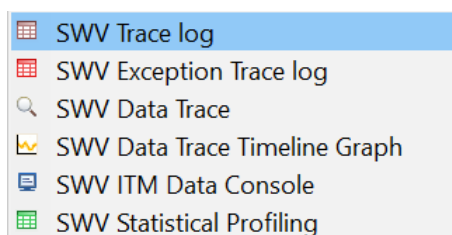


4.3 SWV views

显示SWV跟踪数据的SWV视图如下：

- *SWV Trace Log*: 在电子表格中列出所有传入的SWV数据包。作为跟踪质量的第一个诊断非常有用。
- *SWV Exception Trace Log*: 该视图有两个选项卡，一个类似于SWV Trace Log视图，另一个选项卡显示有关异常事件的统计信息。
- *SWV Data Trace*: 在内存中追踪多达四个不同的符号或区域。
- *SWV Data Trace Timeline Graph*: 显示变量值随时间分布的图形显示。
- *SWV ITM Data Console*: 打印来自目标应用程序的可读文本输出。这通常是通过printf()完成的，输出被重定向到ITM通道0。
- *SWV Statistical Profiling*: 显示基于程序计数器(PC)采样的统计信息。显示在各种函数中所花费的执行时间。

Figure 180. SWV views selectable from the menu



Note: 为了同时跟踪各种事件，可以同时打开多个SWV视图。SWV视图工具栏包含这些常用的控件图标。

Figure 181. SWV views common toolbar



这些图标用于以下目的，从左到右：

- Configure trace
- Start/Stop trace
- Remove all collected SWV data
- Scroll lock
- Minimize

-
- Maximize

SWV图形视图工具栏包含这些额外的控制图标。

Figure 182. SWV graph views extra toolbar



这些图标用于以下目的，从左到右：

- Save graph as image
- Switch between seconds and cycle scale
- Adjust the Y-axis to best fit
- Zoom in
- Zoom out

4.3.1 SWV Trace Log

SWV跟踪日志视图在电子表格中列出所有传入的SWV包。通过选择要复制的行并按Ctrl+C，可以将此视图中的数据以CSV格式复制到其他应用程序。可以使用Ctrl+V命令将复制的数据粘贴到另一个应用程序中。

Figure 183. SWV Trace Log PC sampling and exceptions

Index	Type	Data	Cycles	Time(s)	Extra info
25012	PC Sample	0x80004f6	258481871	3.077165 s	
25013	PC Sample	0x8000508	258498254	3.077360 s	
25014	PC Sample	0x8000518	258514637	3.077555 s	
25015	Exception entry	SYSTICK (EXC 15)	258522309	3.077647 s	
25016	Exception exit	SYSTICK (EXC 15)	258522367	3.077647 s	
25017	Exception return	N/A (EXC 0)	258522374	3.077647 s	
25018	PC Sample	0x80004fc	258531017	3.077750 s	
25019	PC Sample	0x800050e	258547400	3.077945 s	
25020	PC Sample	0x800051e	258563783	3.078140 s	
25021	PC Sample	0x80004fa	258580166	3.078335 s	

Overflow packets: 0

SWV Trace Log视图中的列信息如表6所示。

Table 6. SWV Trace Log columns details

Name	Description
Index	The packet ID. Shared with the other SWV packets.
Type	The type of packet (example PC sample, data PC value (comp 1), exceptions, overflow).
Data	The packet data information.
Cycles	The timestamp of the packet in cycles.
Time(s)	The timestamp of the packet in seconds.
Extra info	Optional extra packet information.

4.3.2 SWV Exception Trace Log

SWV异常跟踪日志视图由两个选项卡组成。

Data tab

第一个选项卡类似于SWV Trace Log视图，但仅限于异常事件。它还提供关于事件类型的附加信息。数据可以复制和粘贴到其他应用程序中。每一行都链接到对应异常处理程序的代码。双击事件，在Editor视图中打开相应的中断处理程序源代码。

Note: 在串行连线查看器设置对话框中启用 */Trace Event EXETRC: Trace Exceptions* 来跟踪程序执行期间的异常。启用 [Timestamps] 记录每个中断包的周期和时间。

Figure 184. SWV Exception Trace Log – Data tab

Index	Type	Name	Peripheral	Function	Cycles	Time(s)	Extra info
17629	Exception exit	SYSTICK (EXC 15)		SysTick_Handler()	58204401	692.909536 ms	
17630	Exception return	N/A (EXC 0)			58205926	692.927690 ms	Timestamp delayed. Packet delayed.
17636	Exception entry	SYSTICK (EXC 15)		SysTick_Handler()	58288335	693.908750 ms	
17637	Exception exit	SYSTICK (EXC 15)		SysTick_Handler()	58288393	693.909440 ms	
17638	Exception return	N/A (EXC 0)			58288400	693.909524 ms	
17644	Exception entry	SYSTICK (EXC 15)		SysTick_Handler()	58372327	694.908655 ms	
17645	Exception exit	SYSTICK (EXC 15)		SysTick_Handler()	58372385	694.909345 ms	
17646	Exception return	N/A (EXC 0)			58372392	694.909429 ms	

Overflow packets: 0

“SWV异常跟踪日志-数据”页签中的列信息如表7所示。

Table 7. SWV Exception Trace Log – Data columns details

Name	Description
Index	The exception packet ID. Shared with the other SWV packets.
Type	Each exception generates three packets: Exception entry, Exception exit and then an Exception return packet.
Name	The name of the exception. Also the exception or interrupt number.
Peripheral	The peripheral for the exception.
Function	The name of the interrupt handler function for this interrupt. Updated when debug is paused. Is cached during the whole debug session. By double clicking the function, the editor will open that function in the source code.
Cycles	The timestamp for the exception in cycles.
Time(s)	The timestamp for the exception in seconds.
Extra info	Optional extra information about that packet.

Statistics tab

第二个选项卡显示有关异常事件的统计信息。在优化代码时，这些信息可能很有价值。包含到编辑器中异常处理程序源代码的超文本链接。

Figure 185. SWV Exception Trace Log – Statistics tab

Exception	Handler	% of	Number of	% of excepti...	% of debug time	Total runtime	Avg runtime	Fastest	Slowest	First	First (s)	Latest	Latest (s)
SYSTICK (EXC 15)	SysTick_Handler()	100.00000%	2172	100.00000%	0.0690%	40309	57	57	58	71567	851.988095 µs	58372327	694.908655 ms
Total for all			2172		0.0690%	40309	18						

Overflow packets: 0

“SWV异常跟踪日志-统计”页签中的列信息如表8所示。

Table 8. SWV Exception Trace Log – Statistics columns details

Name	Description
Exception	The name of the exception provided by the manufacturer. Also the exception or interrupt number.
Handler	The name of the interrupt handler for this interrupt. Updated when debug is paused. Is cached during the whole debug session. By double clicking the handler, the editor will open that function in the source code.
% of	This exception type's share, in percentage, of all exceptions.
Number of	The total number of entry packets received by SWV of this exception type.
% of exception time	How big part of the execution time for all exceptions that this exception type have.
% of debug time	How big part of the total execution time for this debug session that this exception type have. All the timers are restarted when the Empty SWV-Data button is pressed.
Total runtime	The total execution time in cycles for this exception type.
Avg runtime	The average execution time in cycles for this exception type.
Fastest	The execution time in cycles for the fastest exception of this exception type.
Slowest	The execution time in cycles for the slowest exception of this exception type.
First	The first encounter of an entry event for this exception type in cycles.
First(s)	The first encounter of an entry event for this exception type in seconds.
Latest	The latest encounter of an entry event for this exception type in cycles.
Latest(s)	The latest encounter of an entry event for this exception type in seconds.

4.3.3 SWV Data Trace

SWV Data Trace视图最多跟踪内存中的四个不同符号或区域。例如，全局变量可以通过名称引用。数据可以在“读”、“写”和“读/写”三个阶段进行跟踪

在串行连线查看器设置中启用[Data Trace]。在图186中，程序中的两个全局变量pos1和pos2在[Write]访问时被跟踪。

Figure 186. SWV Data Trace configuration

Serial Wire Viewer settings for NUCLEO-F401RE Debug (ST-LINK)

Clock Settings
 Core Clock: 84 MHz
 Clock Prescaler: 42
 SWO Clock: 2000.0 kHz

Trace Events
☐ CPI: Cycles per instruction ☐ EXC: Exception overhead
☐ SLEEP: Sleep cycles ☐ LSU: Load store unit cycles
☐ FOLD: Folded instructions ☐ EXETRC: Trace Exceptions

PC Sampling
☒ Enable Resolution: 16384 Cycles/sample

Timestamps
☒ Enable Prescaler: 1

Data Trace

Comparator	Enable	Var/Addr	Access	Size	Generate
Comparator 0	☒	pos1	Write	Word	Data Value + PC
Comparator 1	☒	pos2	Read	Word	Data Value + PC
Comparator 2	☐	0x0	Read/Write	Word	Data Value
Comparator 3	☐	0x0	Read/Write	Word	Data Value

ITM Stimulus Ports
 Enable port: 31 ☐ 24 ☐ 23 ☐ 16 ☐ 15 ☐ 8 ☐ 7 ☐
 Privileged only ports: ☐ Port 31..24 ☐ Port 23..16 ☐ Port 15..8 ☐ Port 7..0

OK Cancel

当在调试器中运行程序并启用SWV跟踪时，当在[Watch]列表中选择[Comparator 0]和pos1数据时，SWV数据跟踪视图将显示此信息。

Figure 187. SWV Data Trace

SWV Data Trace

Watch

Comp	Name	Value
0	pos1	10
1	pos2	0

History (pos1)

Access	Value	PC	Cycles	Time
WRITE	8	0x8000578	642414276	7.647789 s
WRITE	1	0x8000578	645655051	7.686370 s
WRITE	2	0x8000578	649164268	7.728146 s
WRITE	3	0x8000578	652673485	7.769922 s
WRITE	4	0x8000578	656182631	7.811698 s
WRITE	5	0x8000578	659691850	7.853474 s
WRITE	6	0x8000578	663004479	7.892910 s
WRITE	7	0x8000578	666513696	7.934687 s
WRITE	9	0x8000578	673532061	8.018239 s
WRITE	10	0x8000578	677041280	8.060015 s

SWV Data Trace中的列信息如表9所示。

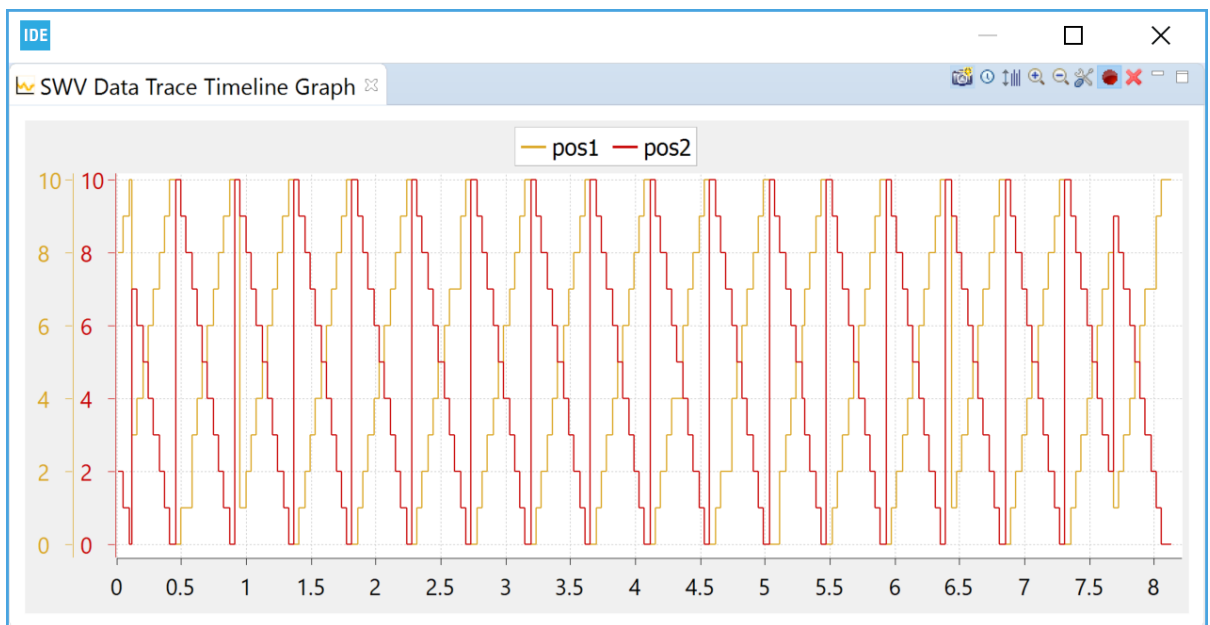
Table 9. SWV Data Trace columns details

Name	Description
Access	Read or Write access type.
Value	The value of data read or written.
PC	The PC location where read or write access occurs.
Cycles	The timestamp for the packet in cycles.
Time(s)	The timestamp for the packet in seconds.

4.3.4 SWV Data Trace Timeline Graph

SWV数据跟踪时间轴图视图包含一个图形显示，显示变量值随时间的分布。它应用于SWV Data Trace中的变量或内存区域。当使用显示全局变量pos1和pos2上下计数的时间轴图时，将显示以下内容。

Figure 188. SWV Data Trace Timeline Graph



SWV数据跟踪时间轴图具有以下特性：

- 通过单击相机工具栏按钮，可以将图形另存为JPEG图像文件。
- 该图默认以秒为单位显示时间，但可以通过单击时钟工具栏按钮更改为周期。
- y轴可以通过单击y轴工具栏按钮调整为最适合。
- 通过单击工具栏上的[+]和[-]按钮来放大和缩小。
- 在调试运行时缩放范围是有限的。当调试被暂停时，缩放细节是可用的。

4.3.5 SWV ITM Data Console and printf redirection

SWV ITM数据控制台打印来自目标应用程序的可读文本输出。通常，这是通过printf()完成的，输

出被重定向到ITM通道0。其他ITM通道可以获得它们自己的控制台视图。要使用SWV ITM Data Console视图，首先在Serial Wire Viewer设置对话框中启用一个或多个32个ITM端口。

Figure 189. SWV settings

在SWV ITM Data Console视图中显示来自ITM端口的报文。应用程序可以使用CMSIS函数 `ITM_SendChar()` 将字符发送到端口0，并且可以将 `printf()` 函数重定向到使用 `ITM_SendChar()` 函数。下面描述了如何在ITM上设置 `printf` 重定向：

1. Configure first file `syscalls.c`. Usually, the `syscalls.c` file is located in the same source folder as `main.c`.
If no `syscalls.c` file is available in the project, it can be copied from another STM32CubeIDE project. One way to get the file is to create a new STM32 empty project for the device. In the `Src` folder, this project contains a `syscall.c` file. Copy this file to a source folder in the project where it is needed.
2. Inside the `syscalls.c` file, replace the `_write()` function with code calling `ITM_SendChar()` instead of `__io_putchar()`

```
int _write(int file, char *ptr, int len)
{
    int DataIdx;

    for (DataIdx = 0; DataIdx < len; DataIdx++)
    {
        //__io_putchar(*ptr++);
        ITM_SendChar(*ptr++);
    }
    return len;
}
```

3. Locate the `core_cmX.h` file, which contains the function `ITM_SendChar()`. The `core_cmX.h` file is included by the Device Peripheral Access Layer header file (for instance `stm32f4xx.h`, which in turn must be included in the `syscalls.c` file).

```
#include "stm32f4xx.h"
```

Use the *Include Browser* view to find the Device Peripheral Access Layer header file. Drop the core file in the *Include Browser* view, and check which files are including the `core_cmX.h` file.

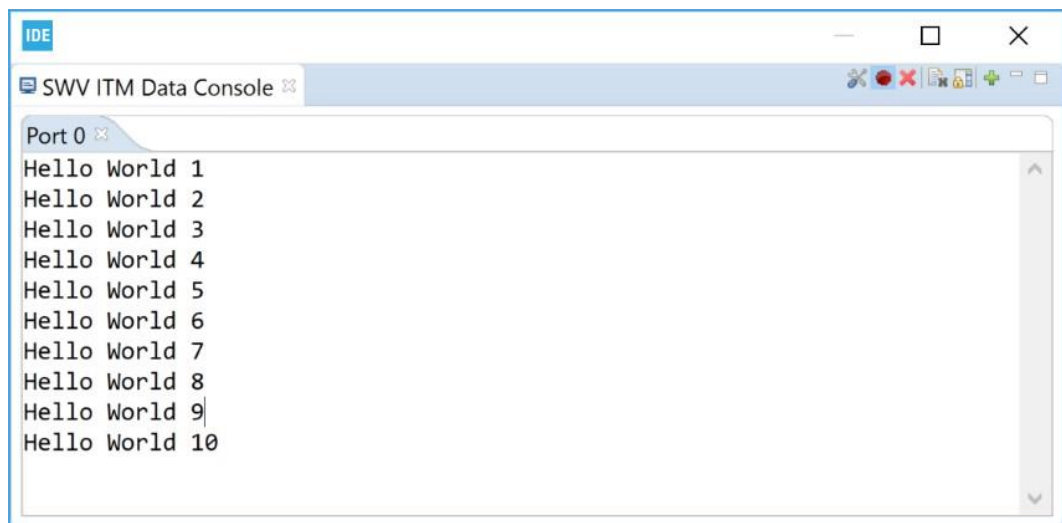
- Test by adding include `stdio.h` and call to `printf()` into the application. Make sure that `printf()` is not called too often.

```
#include <stdio.h>

printf("Hello World %d\n", pos1);
```

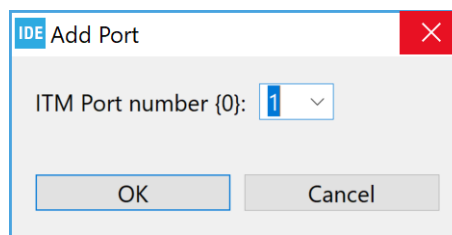
- Start a debug session and enable **[ITM port 0]** in the *SWV ITM Data Console* view.
- Open the *SWV ITM Data Console* view and start tracing using the red **[Start/Stop Trace]** button on the toolbar in this view.
- Start the program. Print commands are logged to the *Port 0* tab in the view.

Figure 190. SWV ITM Data Console



- It is possible to open new port x tabs (x from 1 to 31) by pressing the green **[+]** button on the toolbar. This opens the *Add Port* dialog. In the dialog select the **[ITM Port number]** to be opened to display it as a tab in the *SWV ITM Data Console* view.

Figure 191. SWV ITM port configuration



Note: Study the `ITM_SendChar()` function to learn how to write a function that transmits characters to another ITM port channel.

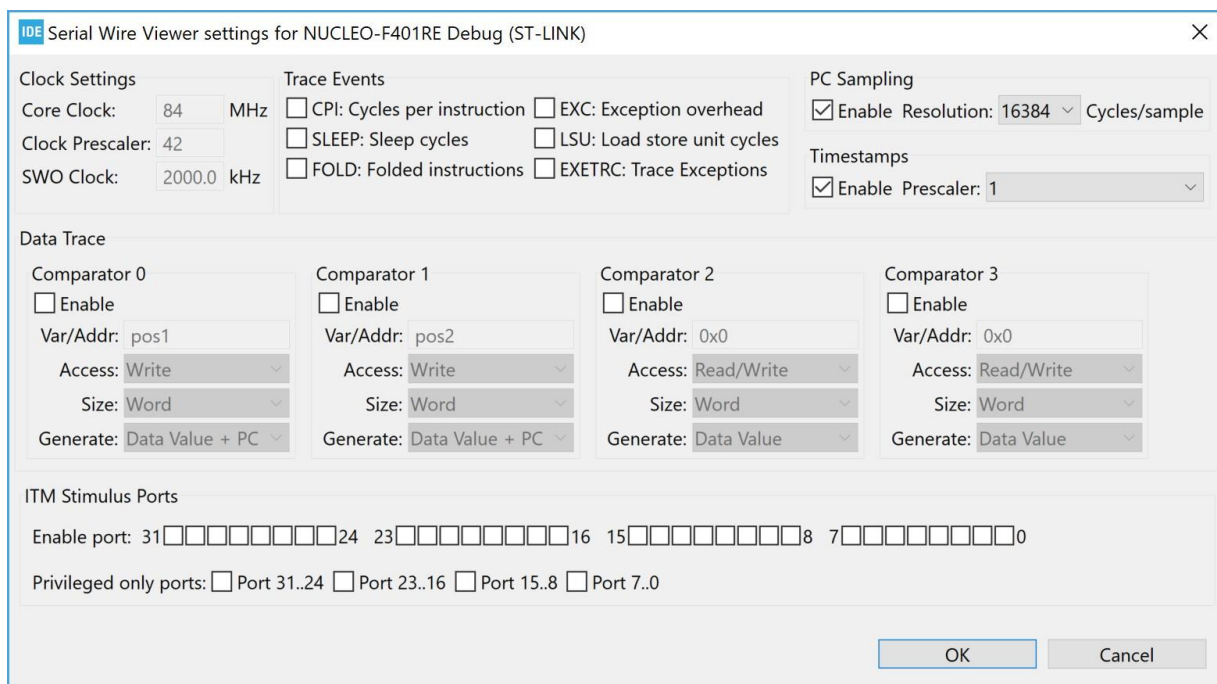
4.3.6 SWV Statistical Profiling

SWV统计分析视图显示基于程序计数器(PC)采样的统计信息。它显示了在各种函数中所花费的执行时间。这在优化代码时很有用。数据可以复制和粘贴到其他应用程序中。在暂停调试时更新视图。

1. 配置SWV发送程序计数器样本，如图192所示。启用[PC Sampling] [PC采样]和[Timestamps] [时间戳]。

根据给定的[Core Clock]周期间隔，SWV将程序计数器值报告给STM32CubeIDE。将[PC Sampling]设置为较高的[Cycle/sample]值，以避免接口溢出。

Figure 192. SWV PC sampling enable



2. 通过选择[Window]>[Show View]>[SWV Statistical Profiling]打开SWV Statistical Profiling视图。视图是空的，因为还没有收集数据。
3. 按红色[Start/Stop Trace]按钮将配置发送到单板。
4. 恢复程序调试。当代码在目标系统中执行时，STM32CubeIDE通过SWV开始收集关于函数使用情况的统计信息。
5. 暂停(Pause)调试。视图显示收集的数据。调试会话的时间越长，收集的统计信息就越多。

Figure 193. SWV Statistical Profiling

Function	% in use	Samples	Start addr...	Size
main()	59.06%	48575	0x80005bd	0x100
readSpeed()	24.82%	20413	0x80004d5	0x46
readTemp()	16.04%	13191	0x800051b	0x2e
HAL_IncTick()	0.07%	56	0x8000b1d	0x34
SysTick_Handler()	0.01%	8	0x80009e5	0xc
writeSpeed()	0.00%	1	0x8000549	0x46

Overflow packets: 0 PC Samples: 82244

Note: 双击SWV Statistical Profiling视图中的函数行，在编辑器中打开包含该函数的文件。
SWV统计分析中的列信息如表10所示。

Table 10. SWV Statistical Profiling columns details

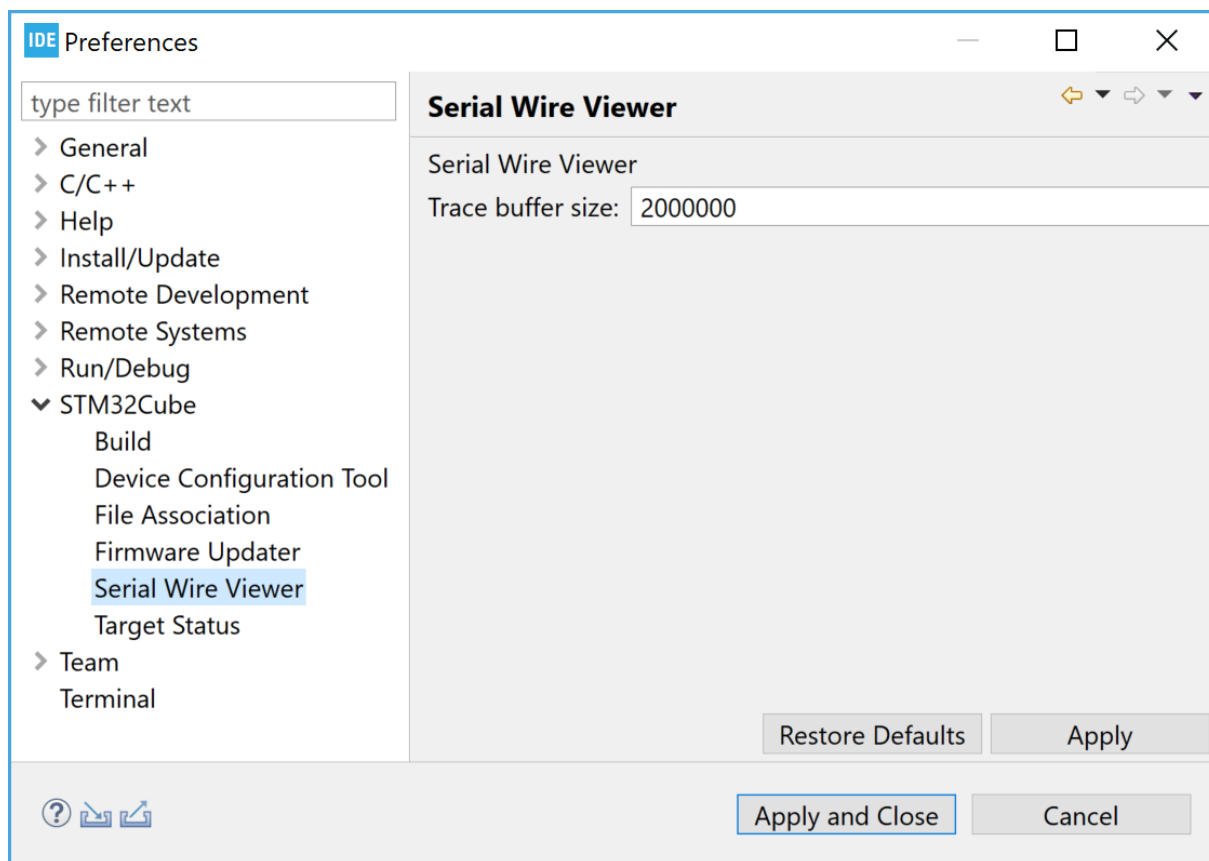
Name	Description
Function	The name of the function which is calculated by comparing address information in SWV packets with the program elf file symbol information.
% in use	The calculated percentage of time the function is used.
Samples	The number of samples received from the function.
Start address	The start address for the function.
Size	The size of the function.

4.4 Change the SWV trace buffer size

传入的SWV包保存在Serial Wire Viewer跟踪缓冲区中，该缓冲区的缺省最大大小为2000000包。为了跟踪更多的数据包，这个数字必须增加。

选择[Windows]>[Preferences]菜单。在Preferences对话框中，选择[STM32Cube]>[Serial Wire Viewer]。如果需要，更新[Trace buffer size]。

Figure 194. SWV Preferences



缓冲区存储在堆中。首先选择[Windows]>[Preferences]菜单显示已分配的堆。在Preferences对话框中，选择[General]。启用[Show heap status]在STM32CubeIDE的右下角显示当前堆和已分配的内存。STM32CubeIDE可以分配的内存量有一个上限。在调试会话期间，可以增加此限制以存储更多信息。

要更新内存限制，请执行以下操作：

1. 导航到STM32CubeIDE安装目录。打开存放IDE的文件夹。
2. 编辑stm32cubeide.ini文件并将-Xmx1024m参数更改为所需的大小(以兆字节为单位)。
3. Save the file and restart STM32CubeIDE.

4.5 Common SWV problems

在尝试首次使用SWV跟踪时，可能会出现以下问题：

- 在当前使用的调试配置中未启用SWV。
SWV Trace没有启动，需要按下某些SWV视图工具栏上的红色Start/Stop Trace按钮来启用SWV并将SWV配置发送到目标板。然后启动程序以接收SWV数据。对于某些SWV视图，需

要再次停止程序以可视化接收到的SWV信息。

- SWO接收过多的数据。减少为跟踪启用的数据量。
- JTAG探测、GDB服务器、目标板或其他部分都不支持SWV。

- 目标[Core Clock]设置不正确。选择正确的[Core Clock]非常重要。

如果目标[Core Clock]的频率未知，有时可以通过在程序循环中设置断点并在命中断点时打开 Expressions 视图来找到它。

点击[Add new expression]，输入SystemCoreClock并按[Enter]。这是一个全局变量，根据CMSIS标准，必须由软件设置为[Core Clock]的正确速度。

在CMSIS标准库中，main()中可以包含一个名为SystemCoreClockUpdate()的函数来设置SystemCoreClock变量。使用Variable视图来跟踪它。

Note: 如果软件在运行时动态地改变CPU时钟速度，这可能会导致SWV停止，因为在执行期间时钟突然变得错误。

要确保接收到所有数据，请执行以下步骤：

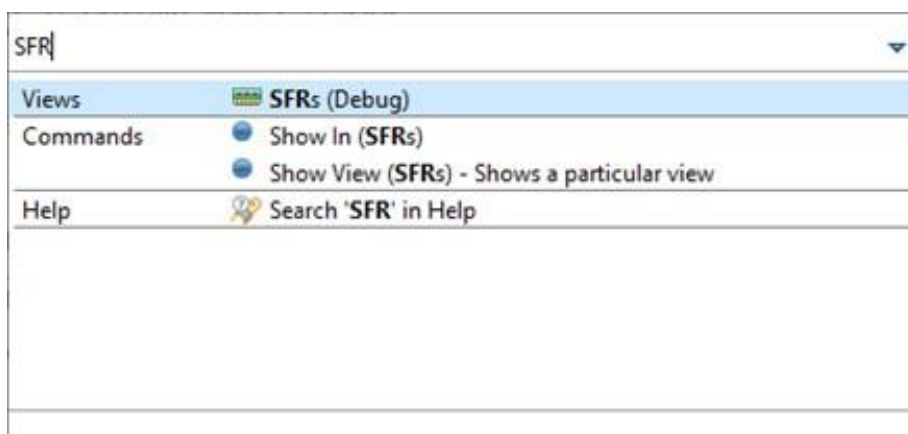
1. 打开SWV配置。禁用除[PC Sampling]和[Timestamps]以外的所有跟踪。将[Resolution]设置为尽可能高的值。
2. Save, and open the *SWV Trace Log* view.
3. Start tracing.
4. 确保在SWV Trace Log视图中都能看到传入的数据包。

5 Special Function Registers (SFRs)

5.1 Introduction to SFRs

特殊功能寄存器(SFRs)可以通过SFRs视图查看、访问和编辑。该视图显示当前项目的信息。如果选择了另一个项目，则其内容将发生变化。要从菜单中打开视图，选择[Window]>[Show View]>[SFRs]菜单命令或使用[Quick Access]字段，搜索“SFR”，并从视图选择它。

Figure 195. Open the SFRs view using the [Quick Access] field



5.2 Using the SFRs view

SFRs视图包含有关项目中使用的STM32设备的外设、寄存器和位字段的信息。在调试项目时，用从目标读取的值填充寄存器和位字段。视图包含两个主要节点，Cortex®- M节点和STM32节点。Cortex®- M节点包括常见的Cortex®- M核心信息，STM32节点包括STM32设备特定的外围设备。

工具栏按钮位于SFRs视图的右上角。

Figure 197. SFRs view toolbar buttons



工具栏中的[RD]按钮用于强制读取所选寄存器。即使寄存器或寄存器中的某些位字段包含在SVD文件中设置的ReadAction属性，它也会导致读取寄存器。

当按下[RD]按钮读取寄存器时，视图中可见的所有其他寄存器也将再次读取，以反映所有寄存器的更新。

必须停止程序才能读取寄存器。

基本格式按钮([X16]，[X10]，[X2])用于改变寄存器显示的基本格式。

[Configure SVD settings]按钮打开当前项目的CMSIS-SVD设置属性面板。[Pin]按钮(“不跟随”选择)可以用来保持当前显示的SVD文件的焦点，即使项目资源管理器视图切换到另一个项目。

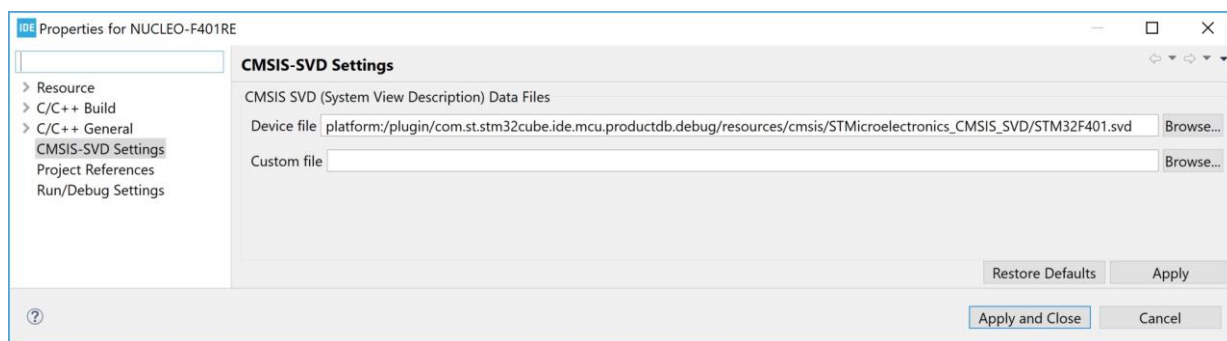
5.3 Updating CMSIS-SVD settings

项目的SFRs视图可以显示该项目的两个CMSIS-SVD(系统视图描述)文件：

- STM32CubeIDE选择的默认文件是项目中所选设备的SVD文件
- 另一个文件可以是用于可视化特定用户硬件配置的自定义SVD文件

要更新设置，请使用SFRs视图中的[Configure SVD settings]工具栏按钮打开CMSIS- SVD settings属性。

Figure 198. SFRs CMSIS-SVD Settings



所有SVD文件必须符合Arm®网站上提供的CMSIS-SVD规范中概述的语法。如果不满足这些要求，SFRs视图可能不会显示任何寄存器信息。

[Device file]字段用于SVD (System View Description)文件。该文件必须描述整个设备。其他视图可能会从该字段指出的SVD文件中获取信息，因此建议仅对包含完整STM32设备描述的SVD文件使用该字段。更新的SVD文件可以从意法半导体(STMicroelectronics)网站(www.st.com)的设备描述部分的HW模型、CAD库和SVD栏目获得。

[Custom file]字段可用于定义与定制硬件相关的特殊函数寄存器，以简化不同寄存器状态的查看。

另一个可能的用例是创建一个SFR收藏夹文件，其中包含[Device file]中的内容子集。例如，这个子集可以由经常检查的寄存器组成。如果指出了[Custom file]，则在SFRs视图中创建一个新的top-node，其中包含与[Custom file]相关的寄存器信息。

用户可以更改两个字段，并且可以同时使用两个字段。

Note:

- 当寄存器和位字段的值列具有写访问权限时，可以在这些值列中写入新的值。
- 当使用ST-LINK GDB服务器时，可以在目标运行时使用SFRs视图。然而，在这种情况下，调试配置中的[Live expression]选项必须启用。
- 当使用OpenOCD或SEGGER J-Link时，不可能在目标正在运行时使用SFRs视图。
- SFRs视图在C/ c++ Editing透视图中也很有用，但是只显示寄存器的名称和地址。

6 RTOS-aware debugging

实时操作系统(RTOS)向设计中添加了不同类型的对象，如线程、信号量和计时器。

STM32CubeIDE包括一组专门的视图来处理Microsoft®Azure®RTOS ThreadX和FreeRTOS™内核对象。

这些视图显示了在逐级执行代码时或在调试会话期间当程序碰到断点时RTOS对象的状态。

Note: FreeRTOS是亚马逊在美国和/或其他国家的商标。

所有其他商标均为其各自所有者的财产。

6.1 Azure® RTOS ThreadX

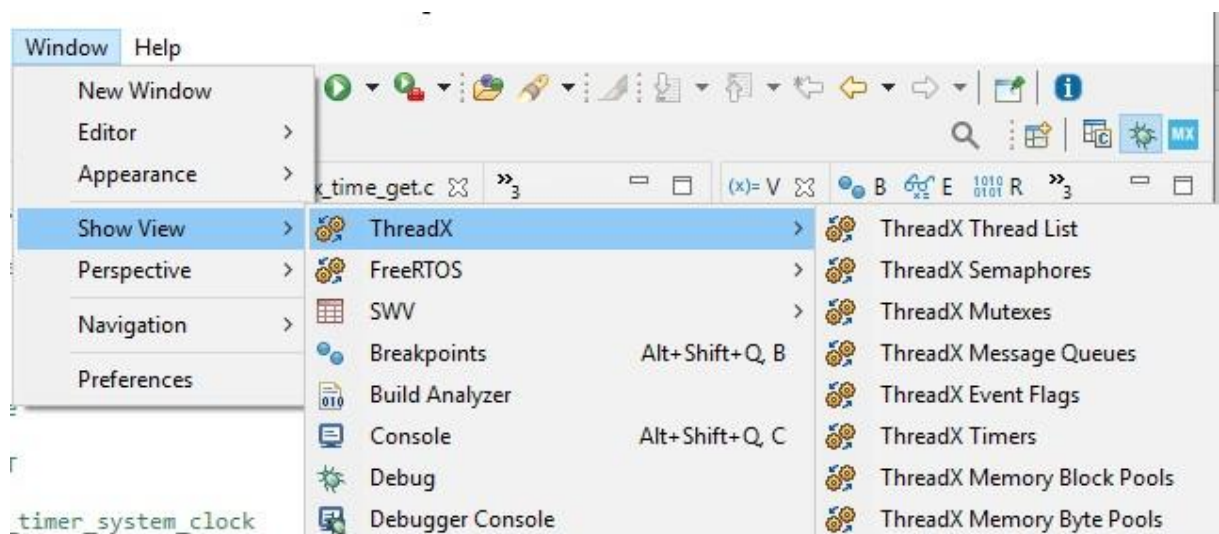
The following views are available for ThreadX:

- *ThreadX Thread List*
- *ThreadX Semaphores*
- *ThreadX Mutexes*
- *ThreadX Message Queues*
- *ThreadX Event Flags*
- *ThreadX Timers*
- *ThreadX Memory Block Pools*
- *ThreadX Memory Byte Pools*

6.1.1 Finding the views

在Debugger透视图，从菜单中打开与threadx相关的视图。选择菜单命令 [Window]>[Show View]>[ThreadX]>[...]或使用[Quick Access]搜索“ThreadX”并从视图选择。

Figure 199. ThreadX views selectable from the menu



6.1.2 ThreadX Thread List view

ThreadX Thread List视图显示有关目标系统中所有可用线程的详细信息。线程列表在每次目标执行挂起时自动更新。

每种类型的线程参数都有一列，每个线程都有一行。如果自上次挂起调试器以来线程的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 200. ThreadX Thread List view (default)

Name	Priority	State	Run Count	Stack Start	Stack End	Stack Size	Stack Ptr	Stack Usage
Main Thread	5	SUSPENDED (MySemaphore_1)	1	0x24001634	0x24001833	512	0x240016cc	Disabled
System Timer Thread	0	SUSPENDED	0	0x24000a18	0x24000e17	1024	0x24000dcc	Disabled
Thread One	10	SUSPENDED (MyMutex_1)	1	0x2400183c	0x24001a3b	512	0x240018d4	Disabled
Thread Two	10	RUNNING	1	0x24001a44	0x24001c43	512	0x24001bfc	Disabled
Idle								

由于性能原因，“Stack Usage”列默认关闭。要启用堆栈分析，请使用ThreadX Thread List视图工具栏中的[Toggle Stack Checking]工具栏按钮(图201中粉红色圈出的部分)。

Figure 201. ThreadX Thread List view (Stack Usage enabled)

Name	Priority	State	Run Count	Stack Start	Stack End	Stack Size	Stack Ptr	Stack Usage
Main Thread	5	SUSPENDED (Event Flag)	2	0x240015a4	0x240017a3	512	0x24001614	512
System Timer Thread	0	SUSPENDED	505	0x24000984	0x24000d83	1024	0x24000c4c	1024
Thread One	10	READY	381	0x240017ac	0x240019ab	512	0x24001824	512
Thread Two	8	RUNNING	126	0x240019b4	0x24001bb3	512	0x24001b0c	512
Idle								

ThreadX Thread List视图中的列信息如表11所示。

Table 11. ThreadX Thread List details

Name	Description
N/A	A green arrow symbol indicates the currently running thread.
Name	The name assigned to the thread.
Priority	The thread priority.
State	The current state of the thread.
Run Count	The threads run counter.
Stack Start	The start address of the stack area.
Stack End	The end address of the stack area.
Stack Size	The size of the stack area (bytes).
Stack Ptr	The address of the stack pointer.
Stack Usage	The maximum thread stack (bytes). By default, ThreadX fills every byte of thread stacks with a 0xEF data pattern during thread creation. See the note below for more information.

Note: 如果所有线程的“堆栈使用情况”列包含的值与“堆栈大小”列相同，则原因可能是在任务创建期间线程堆栈没有被0xEF数据模式填充。如果在构建ThreadX内核时禁用了堆栈数据模式填充，就会发生这种情况。通常，使用<tx_user.h>文件，其中包含TX_DISABLE_STACK_FILLING定义。如下例所示注释此定义，并重新构建项目以解决问题。很高兴知道<tx_user.h>文件还包含TX_ENABLE_STACK_CHECKING定义，如果检测到堆栈损坏，可以启用该定义来进行运行时堆栈检查。更多的信息可以在ThreadX用户指南中找到。

ThreadX头文件中tx_user.h文件的示例，注释TX_ENABLE_STACK_CHECKING定义：

```
/* Determine is stack filling is enabled. By default, ThreadX stack filling is enabled,
   which places an 0xEF pattern in each byte of each thread's stack. This is used by
   debuggers with ThreadX-aw:areness and by the ThreadX run-time stack checking feature. */

/* #define TX_DISABLE_STACK_FILLING */

/* Determine whether or not stack checking is enabled. By default, ThreadX stack checking is
   disabled. When the following is defined, ThreadX thread stack checking is enabled. If
   stack
   checking is enabled (TX_ENABLE_STACK_CHECKING is defined), the TX_DISABLE_STACK_FILLING
   define is negated, thereby forcing the stack fill which is necessary for the stack
   checking
   logic. */

/*#define TX_ENABLE_STACK_CHECKING*/
```

6.1.3 ThreadX Semaphores view

ThreadX Semaphores视图显示有关目标系统中所有可用资源信号量的详细信息。每当目标执行被挂起时，视图都会自动更新。

每种类型的信号量参数各有一列，每个信号量各有一行。如果自上次挂起调试器以来，特定信号量的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 202. ThreadX Semaphores view

Name	Count	Suspended
MySemaphore_1	0	Main Thread
MySemaphore_2	5	
MySemaphore_3	8	

Table 12. ThreadX Semaphores details

Name	Description
Name	The name assigned to the semaphore.
Count	The current semaphore count.
Suspended	The threads currently suspended because of the semaphore state.

6.1.4 ThreadX Mutexes view

ThreadX互斥对象视图显示了目标系统中所有可用互斥对象的详细信息。每当目标执行被挂起时，视图都会自动更新。

每种类型的互斥量参数各有一列，每个互斥量各有一行。如果自上次挂起调试器以来，特定互斥锁的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 203. ThreadX Mutexes view

Name	Owner	Owner Count	Suspended
MyMutex_1	Main Thread	1	Thread One, Thread Two
MyMutex_2		0	
MyMutex_3		0	

Table 13. ThreadX Mutexes details

Name	Description
Name	The name assigned to the mutex.
Owner	The thread that currently owns the mutex.
Owner Count	The mutex owner count (number of get operations performed by the owner thread).
Suspended	The threads currently suspended because of the mutex state.

6.1.5 ThreadX Message Queues view

ThreadX Message Queues视图显示有关目标系统中所有可用消息队列的详细信息。每当目标执行被挂起时，视图都会自动更新。

每种类型的消息队列参数各有一列，每个消息队列各有一行。如果自上次挂起调试器以来，特定消息队列的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 204. ThreadX Message Queues view

Name	Address	Capacity	Used	Free	Message size	Suspended
Message Queue One	0x240002b8	10	0	10	1	
Message Queue Two	0x2400003c	10	0	10	1	

Table 14. ThreadX Message Queues details

Name	Description
Name	The name assigned to the message queue.
Address	The address of the message queue.
Capacity	The maximum number of entries allowed in the queue.
Used	The current number of used entries in the queue.
Free	The current number of free entries in the queue.
Message size	The size (in 32-bit words) of each message entry.
Suspended	The threads currently suspended because of the message queue state.

6.1.6 ThreadX Event Flags view

ThreadX Event Flags视图显示有关目标系统中所有可用事件标志组的详细信息。每当目标执行被挂起时，视图都会自动更新。

每种类型的参数各有一列，每个事件标志组各有一行。如果自上次挂起调试器以来，特定事件标志组的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 205. ThreadX Event Flags view

Name	Flags	Suspended
Event Flag1	0	Main Thread
Event Flag2	0	

Table 15. ThreadX Event Flags details

Name	Description
Name	The name assigned to the event flag group.
Flags	The current value of the event flag group.
Suspended	The threads currently suspended because of the event flag group.

6.1.7 ThreadX Timers view

ThreadX Timers视图显示有关目标系统中所有可用软件计时器的详细信息。每次暂停目标执行时，计时器视图都会自动更新。

每种类型的计时器参数各有一列，每个计时器各有一行。如果自上次挂起调试器以来特定计时器的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 206. ThreadX Timers view

Name	Remaining	Re-init	Function
MyTimer_1	68	100	0x80005d1 <MyTimerFunction1>
MyTimer_2	72	200	0x80005f3 <MyTimerFunction2>
MyTimer_3	276	500	0x8000615 <MyTimerFunction3>

Table 16. ThreadX Timers details

Name	Description
Name	The name assigned to the timer.
Remaining	The remaining number of ticks before the timer expires.
Re-init	The timer re-initialization value (ticks) after expiration. It contains value 0 for one-shot timers.
Function	The address and name of the function that is called when the timer expires.

6.1.8 ThreadX Memory Block Pools view

ThreadX Memory Block Pools视图显示有关目标系统中所有可用内存块池的详细信息。每当目标执行被挂起时，视图都会自动更新。

每种类型的参数都有一列，每个内存块池都有一行。如果自上次挂起调试器以来，特定内存块池的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 207. ThreadX Memory Block Pools view

Name	Address	Used	Free	Total	Block size	Pool size	Suspended
MyBlockPool_1	0x240005ec <P...	0	3	3	28	100	
MyBlockPool_2	0x240004ac <P...	0	4	4	40	200	
MyBlockPool_3	0x2400034c <P...	0	5	5	52	300	

Table 17. ThreadX Memory Block Pools details

Name	Description
Name	The name assigned to the memory block pool.
Address	The starting address of the memory block pool.
Used	The current number of allocated blocks.
Free	The current number of free blocks.
Total	The total number of memory block pools available.
Block size	The size (bytes) of each block.
Pool size	The total pool size (bytes).
Suspended	The threads currently suspended because of the memory block pool state.

6.1.9 ThreadX Memory Byte Pools view

ThreadX Memory Byte Pools视图显示有关目标系统中所有可用内存字节池的详细信息。每当目标执行被挂起时，视图都会自动更新。

每种类型的参数都有一列，每个内存字节池都有一行。如果自上次挂起调试器以来，特定内存字节池的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 208. ThreadX Memory Byte Pools view

Name	Address	Used	Free	Size	Fragments	Suspended
Byte Pool	0x24000f84 "\214\021"	1664	6528	8192	7	

Table 18. ThreadX Memory Byte Pools details

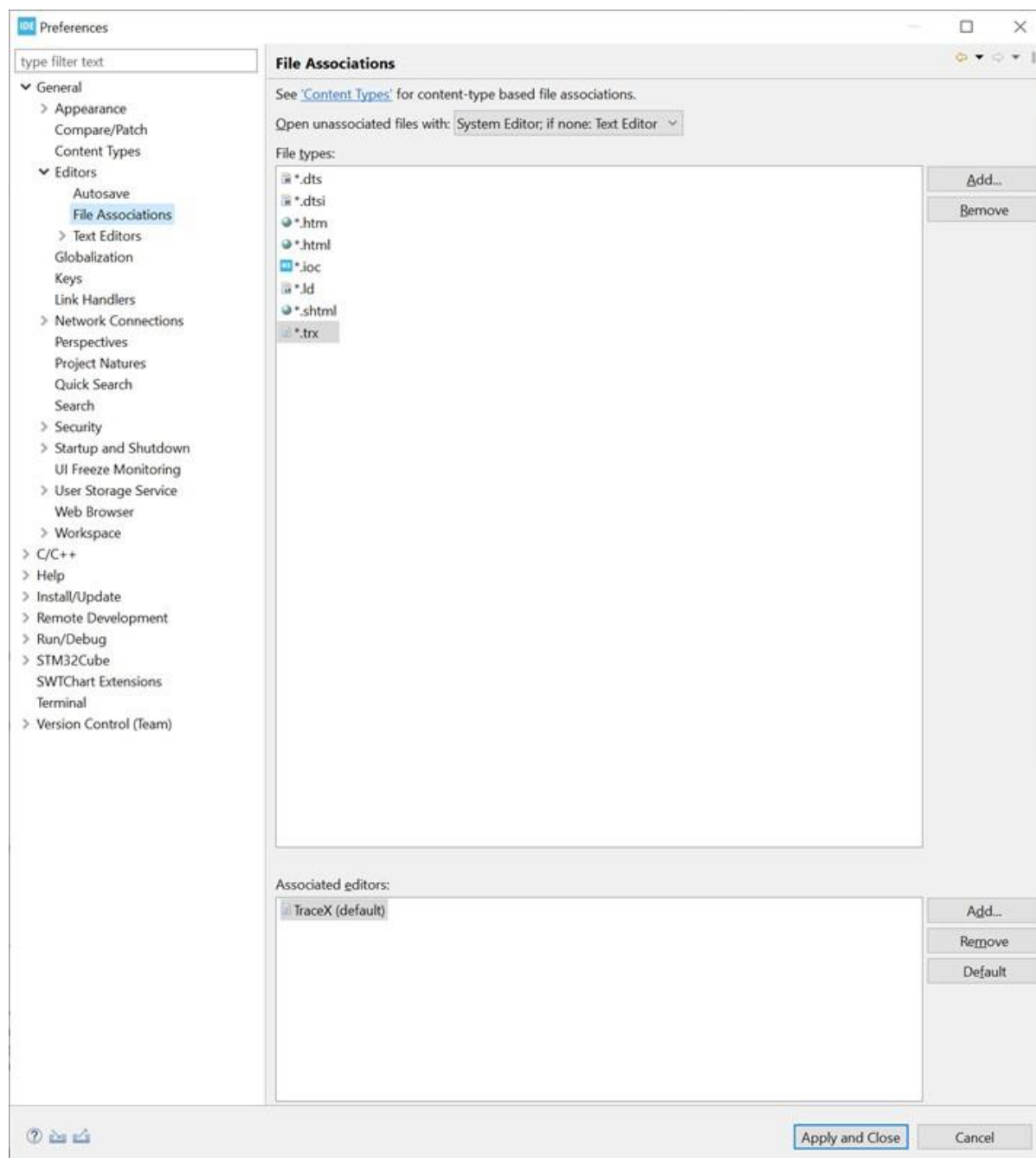
Name	Description
Name	The name assigned to the memory byte pool.
Address	The starting address of the memory byte pool.
Used	The current number of allocated bytes.
Free	The current number of free bytes.
Size	The number of fragments.
Fragments	The size (bytes) of each block.
Suspended	The threads currently suspended because of the memory byte pool state.

6.1.10 Azure® RTOS TraceX tool

Important: Microsoft® Azure® RTOS TraceX 工具(TraceX)仅适用于 Windows®。

要在数据导出时自动打开 TraceX，请选择 **[Windows]>[Preferences]** 菜单，通过 Preferences 窗口将 .trx 文件类型与 TraceX 关联起来，如图 209 所示。

Figure 209. File associations



Azure®RTOS ThreadX内核可以在MCU RAM缓冲区中生成各种系统事件。TraceX应用程序稍后可以“偏离目标”地分析这些事件。这需要将RAM缓冲区导出为合适的文件格式。可以使用 Azure®RTOS ThreadX视图菜单中的[Export trace]按钮导出跟踪数据，如图210和图211所示。

Figure 210. RAM buffer export (1 of 2)

Name	Pri...	State	Run Count	Stack Start	Stack End	Stack Size	Stack Ptr	Stack Usa...
sine wave	8	SLEEP (1)	124	0x24002...	0x24002...	1024	0x24002...	Disabled
System Timer T...	0	SUSPENDED	123	0x24013...	0x24013...	1024	0x24013...	Disabled
thread 0	1	SLEEP (10)	13	0x24000...	0x24000...	1024	0x24000...	Disabled
thread 1	16	READY	1874	0x24000...	0x24000...	1024	0x24000...	Disabled
→ thread 2	16	RUNNING	1876	0x24000...	0x24000...	1024	0x24000...	Disabled
thread 3	8	SUSPENDED (semaph...	62	0x24000...	0x24001...	1024	0x24000f...	Disabled
thread 4	8	SLEEP (2)	62	0x24001...	0x24001...	1024	0x24001...	Disabled
thread 5	4	SUSPENDED (event fl...	13	0x24001...	0x24001...	1024	0x24001...	Disabled
thread 6	8	SUSPENDED (mutex 0)	62	0x24001...	0x24001...	1024	0x24001...	Disabled
thread 7	8	SLEEP (2)	62	0x24001...	0x24002...	1024	0x24001f...	Disabled
Idle								

Figure 211. RAM buffer export (2 of 2)

Name	Pri...	State	Run Count	Stack Start	Stack	ThreadX Semaphores	ThreadX Mutexes	ThreadX Message Queues	ThreadX Event Flags	ThreadX Timers	ThreadX Memory Block Pools	ThreadX Memory Byte Pools
sine wave	8	SLEEP (1)	124	0x24002...	0x240...							
System Timer T...	0	SUSPENDED	123	0x24013...	0x240...							
thread 0	1	SLEEP (10)	13	0x24000...	0x240...							
thread 1	16	READY	1874	0x24000...	0x240...							
→ thread 2	16	RUNNING	1876	0x24000...	0x240...							
thread 3	8	SUSPENDED (semaph...	62	0x24000...	0x24001...	1024	0x24000f...	Disabled				
thread 4	8	SLEEP (2)	62	0x24001...	0x24001...	1024	0x24001...	Disabled				
thread 5	4	SUSPENDED (event fl...	13	0x24001...	0x24001...	1024	0x24001...	Disabled				
thread 6	8	SUSPENDED (mutex 0)	62	0x24001...	0x24001...	1024	0x24001...	Disabled				
thread 7	8	SLEEP (2)	62	0x24001...	0x24002...	1024	0x24001f...	Disabled				

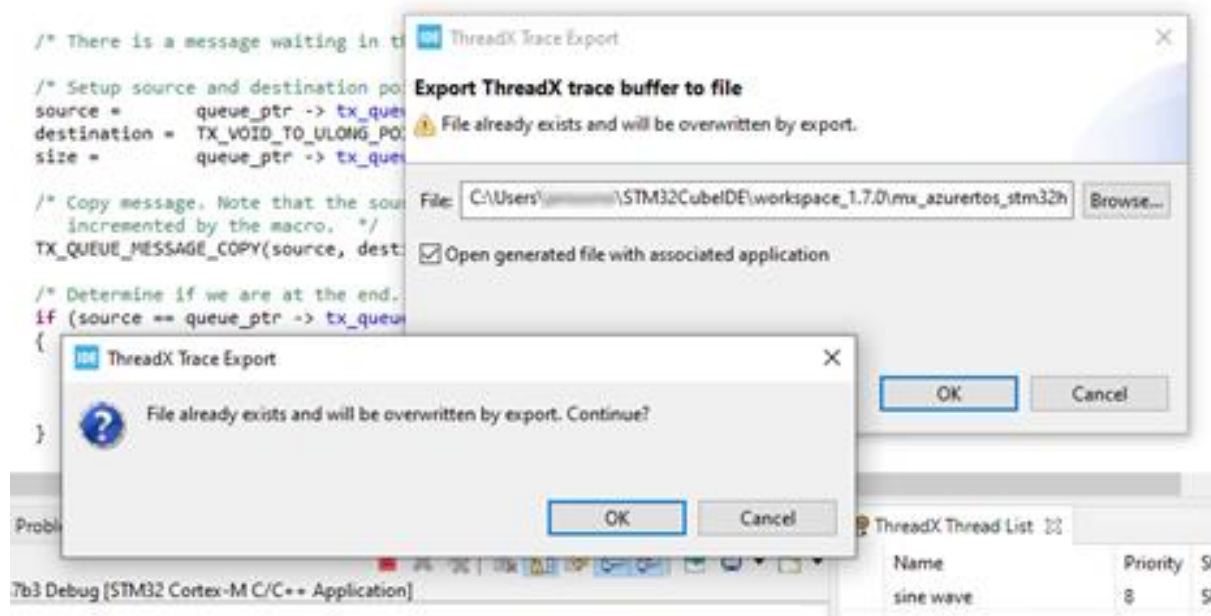
导出跟踪有四个先决条件:

- Azure®RTOS ThreadX内核必须在构建时启用跟踪
 - 嵌入式STM32CubeMX编辑器提供了GUI支持来启用跟踪事件
- 在导出任何数据之前，必须调用tx_trace_enable()函数
- 跟踪导出操作不能在内核API中执行，以避免数据文件损坏
- RAM缓冲区只能在目标停止时读取

On run

导出时，导出目的地将从活动调试上下文(\$PATH_TO_PROJECT/TraceX/\$ launch_config .trx)中预填充。如果以前的跟踪数据已经存在，将提示用户是否覆盖它们，如图212所示。

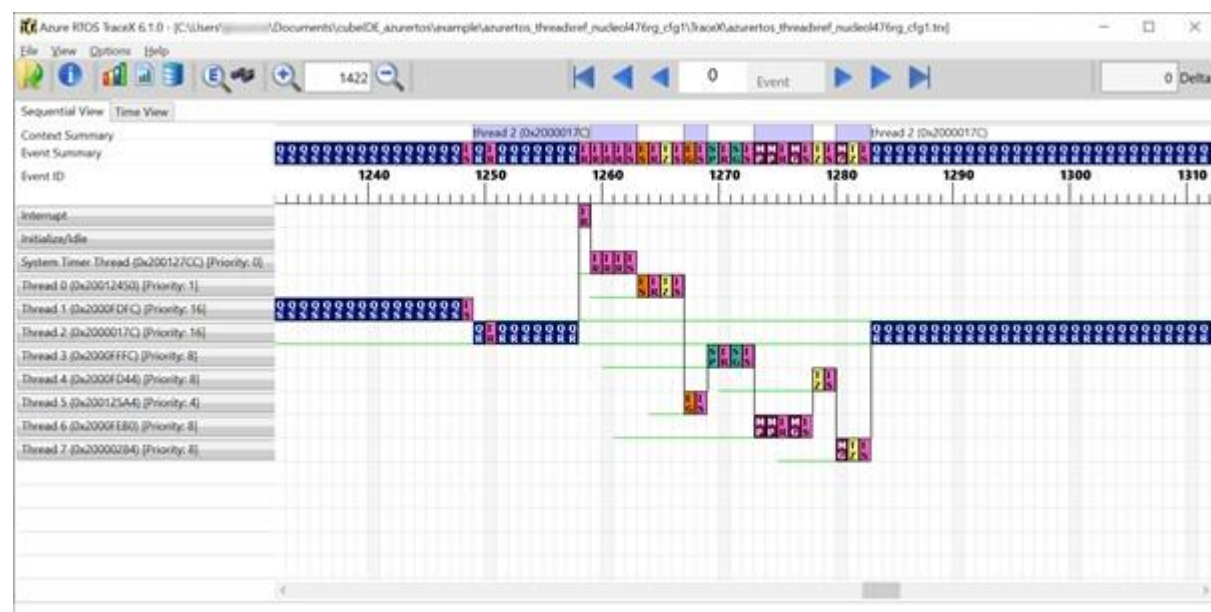
Figure 212. Existing trace overwrite



On export

当将数据导出到TraceX时，STM32CubeIDE从目标读取RAM缓冲区。然后使用相应的数据创建一个*.trx文件，稍后可以使用TraceX工具打开。默认情况下，在项目中创建了一个TraceX目录，其中包含.trx文件。

Figure 213. TraceX analysis



Remember:

- 导出函数只有在TraceX初始化后才能工作(tx_trace_enable())。该工具导出最后N个跟踪事件。
- 如果在内核API中执行导出，则有导出跟踪数据损坏的风险。要避免这种损坏，请确保在目标位于合适的位置时执行导出。例如，在内核API之外设置断点或配置跟踪全回调。

6.2 FreeRTOS™

The following views are available for FreeRTOS™:

- *FreeRTOS Task List*
- *FreeRTOS Timers*
- *FreeRTOS Semaphores*
- *FreeRTOS Queues*

6.2.1 Requirements

为了能够使用关于RTOS状态的详细信息请参见FreeRTOS™相关视图，必须配置FreeRTOS™内核中的某些文件。以下部分描述了一些必需的配置。详细信息请参考FreeRTOS参考手册。

6.2.1.1 Enable trace information

freeRTOSConfig.h中的定义configUSE_TRACE_FACILITY必须启用(设置为1)。它会导致额外的结构成员和函数被包含在构建中，并在FreeRTOS任务列表视图中启用实例堆栈检查，并在FreeRTOS Semaphores视图中列出信号量类型。

Example:

```
freeRTOSConfig.h
#define configUSE_TRACE_FACILITY 1
```

6.2.1.2 Add to registry

应用软件必须调用vQueueAddToRegistry()函数来使FreeRTOS Queues和FreeRTOS Semaphores视图能够显示对象。该函数将一个对象添加到FreeRTOS™Queue注册表中，并接受两个参数，第一个是队列句柄，第二个是队列的描述，在FreeRTOS™相关视图中显示。

Example:

```
vQueueAddToRegistry(mailId, "osMailQueue");
vQueueAddToRegistry(osQueueHandle, "osQueue");
vQueueAddToRegistry(osSemaphoreHandle, "osSemaphore");
```

6.2.1.3 RTOS profiling information

要获得有效的RTOS运行时统计信息，应用程序必须设置一个运行时统计时间库。建议time-base时钟运行速度至少比用于处理RTOS tick中断的时钟频率快10倍。为了启用FreeRTOS™运行时统计数据的收集，文件freeRTOSConfig.h必须包括：

1. Define configGENERATE_RUN_TIME_STATS 1
2. Define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() to call the function that configures a timer to be used for profiling
3. Define portGET_RUN_TIME_COUNTER_VALUE() to call the function that reads the current value from the profiling timer

Example:

```
freeRTOSConfig.h
#define configGENERATE_RUN_TIME_STATS 1
#define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() configureRunTime()
#define portGET_RUN_TIME_COUNTER_VALUE() getRunTimeCounter()
```

Or, if a run time variable is available in the system:

```
freeRTOSConfig.h
#define configGENERATE_RUN_TIME_STATS 1
#define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() (RunTime=0UL)
#define portGET_RUN_TIME_COUNTER_VALUE() RunTime
```

如果FreeRTOS任务列表视图中的“运行时间”列在进行这三个设置后显示为N/A，则如果项目没有使用优化级别-O0构建，则可能会出现这个问题。原因很可能在ulTutoralRunTime的tasks.c声明中找到。

Example:

```
#if ( configGENERATE_RUN_TIME_STATS == 1 )
PRIVILEGED_DATA static uint32_t ulTaskSwitchedInTime = 0UL;
/*< Holds the value of a timer/counter the last time a task was switched in. */
PRIVILEGED_DATA static uint32_t ulTotalRunTime = 0UL;
/*< Holds the total amount of execution time as defined by the run time counter clock. */
#endif
```

Solutions:

- Either declare the variable as volatile:

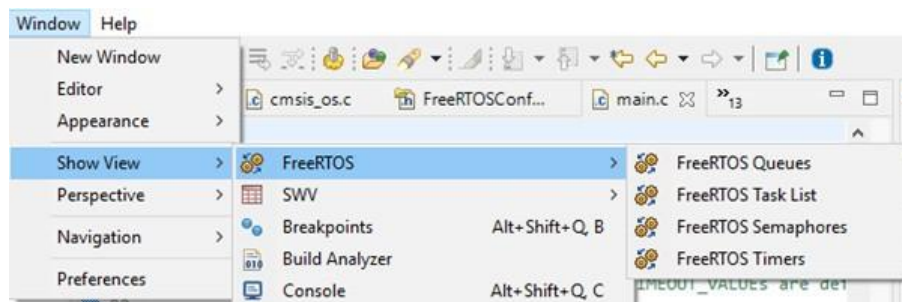
```
PRIVILEGED_DATA volatile static uint32_t ulTotalRunTime = 0UL;
/*< Holds the total amount of execution time as defined by the run time counter clock.
*/
```

- Or simply change the optimization level only for tasks.c by
 1. Right-clicking it in *Project Explorer* view and open *Properties*
 2. Select **[Properties]>[C/C++ Build]>[Settings]>[Tool Settings]>[Optimization]**
 3. Set **[Optimization Level]** to **None (-O0)**

6.2.2 Finding the views

在调试器透视图图中，FreeRTOS™ 相关的视图从菜单中打开。选择菜单命令[Window]>[Show View]>[FreeRTOS]>[...]或使用[Quick Access]，搜索“FreeRTOS”并在视图图中选择。

Figure 214. FreeRTOS™-related views selectable from the menu



6.2.3 FreeRTOS Task List view

FreeRTOS任务列表视图显示目标系统中所有可用任务的详细信息。任务列表在每次目标执行挂起时自动更新。

每种类型的任务参数各有一列，每个任务各有一行。如果自上次暂停调试器以来，任务的任何参数的值发生了更改，则相应的行将以黄色突出显示，如图215中的示例所示。

Figure 215. FreeRTOS Task List (default)

Name	Priority (Base/...	Start of Stack	Top of Stack	State	Event Object	Min Free Stack	Run Time (%)
⇒ IDLE	0/0	0x200001c0	0x20000558 <ucHeap=1308>	RUNNING		Disabled	99%
LEDThread	3/3	0x20000150	0x20000308 <ucHeap=716>	SUSPENDED		Disabled	0%
Time Svc	2/2	0x20000630	0x200009b0 <ucHeap=2420>	BLOCKED	TimeQ	Disabled	1%

由于性能原因，堆叠分析(最小空闲堆栈列)默认关闭。要启用堆栈分析(参见图217)，请使用FreeRTOS任务列表视图工具栏中的Toggle stack Checking工具栏按钮，如图216所示。

Figure 216. FreeRTOS™ Toggle Stack Checking

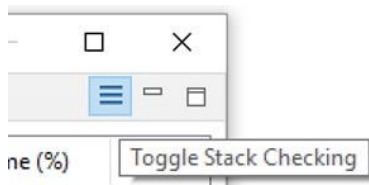


Figure 217. FreeRTOS Task List (Min Free Stack enabled)

Name	Priority (Base/...	Start of Stack	Top of Stack	State	Event Object	Min Free Stack	Run Time (%)
IDLE	0/0	0x200003c0	0x20000960 <ucHeap=1316>	RUNNING		>256	99%
LEDThread	3/3	0x20000150	0x200002b8 <ucHeap=636>	DELAYED		>256	0%
Tmr Svc	2/2	0x20000630	0x200009b0 <ucHeap=2420>	BLOCKED	TmrQ	>256	1%

图217中的FreeRTOS Task List视图包含一个Min Free Stack列。如果项目使用以下定义集构建，列信息将更改为Stack Usage:

```
#define configRECORD_STACK_HIGH_ADDRESS 1
```

在本例中，完整的堆栈使用情况按照已使用/总数(%已使用)的格式显示，如图218所示。

Figure 218. FreeRTOS Task List with ConfigRECORD_STACK_HIGH_ADDRESS enabled

Name	Priority (B...	Start of S...	Top of St...	State	Event Object	Stack Usage	Run Time...
IDLE	0/0	0x20000...	0x20000...	RUNNING		96B / 2052B (4.7%)	N/A
THREAD1	24/24	0x20001...	0x20001...	DELAYED		144B / 512B (28.1%)	N/A
THREAD2	24/24	0x20001...	0x20001...	DELAYED		144B / 512B (28.1%)	N/A
Tmr Svc	2/2	0x20000...	0x20000...	BLOCKED	TmrQ	168B / 1028B (16.3%)	N/A

FreeRTOS Task List视图中的列信息如表19所示。

Table 19. FreeRTOS Task List details

Name	Description
N/A	A green arrow symbol indicates the task currently running.
Name	The name assigned to the task.
Priority (Base/Actual)	The task base priority and actual priority. The base priority is the priority assigned to the task. The actual priority is a temporary priority assigned to the task due to the priority inheritance mechanism.
Start of Stack	The address of the stack region assigned to the task.
Top of Stack	The address of the saved task stack pointer.
State	The current state of the task.
Event Object	The name of the resource that has caused the task to be blocked.
Min Free Stack ⁽¹⁾	The stack “high watermark”. Displays the minimum number of bytes left on the stack for a task. A value of 0 (most likely) indicates that a stack overflow has occurred. <i>Note: This feature must be enabled in the “View” toolbar.</i>
Run Time (%)	The run time statistics provide information on the percentage of time the task has been used. This can be used for profiling the system during development.

1. 当使用configRECORD_STACK_HIGH_ADDRESS = 1构建应用程序时，列名被更改为“Stack Usage”。它以“已使用/总数(%已使用)”的详细格式显示堆栈使用情况。

6.2.4 FreeRTOS Timers view

FreeRTOS Timers视图显示了关于目标系统中所有可用软件计时器的详细信息。每当目标执行被挂起时，视图都会自动更新。每种类型的计时器参数各有一列，每个计时器各有一行。如果计时器的任何参数的值自上次挂起调试器以来发生了更改，则相应的行将以黄色突出显示。

Figure 219. FreeRTOS Timers

Name	Active	Period	Type	Id	Callback
myTimerTEST	True	200	Auto-Reload	0x0	0x8000429 <osTimerCallback>

FreeRTOS Timers视图中的列信息如表20所示。

Table 20. FreeRTOS Timers details

Name	Description
Name	The name assigned to the timer.
Active	The active status information.
Period	The time (in ticks) between timer start and the execution of the callback function.
Type	The type of timer. Auto-reload timers are automatically reactivated after expiration. One-shot timers expire only once.
Id	The timer identifier.
Callback	The address and name of the callback function executed when the timer expires.

- Note:**
1. 如果name字段中没有出现名称，请检查计时器是否创建有名称。调用xTimerCreate()时的第一个参数必须包含计时器名称字符串。
 2. 使用软件定时器时，会自动创建Tmr Svc任务和TmrQ队列。这些对象分别显示在FreeRTOS Task List视图和FreeRTOS Queues视图中。

6.2.5 FreeRTOS Semaphores view

FreeRTOS Semaphores视图显示了目标系统中所有可用同步对象的详细信息，包括：

- Mutexes
- Counting semaphores
- Binary semaphores
- Recursive semaphores

每当目标执行被挂起时，视图都会自动更新。每种类型的信号量参数各有一列，每个信号量各有一行。如果信号量的任何参数的值自上次挂起调试器以来发生了更改，则相应的行将以黄色突出显示。

Figure 220. FreeRTOS Semaphores

Name	Address	Type	Size	Free	# Blocked tasks
osSemaphore	0x20000058	BINARY_SEMAPHORE	1	0	0

- Note:** 如果“Type”提示“N/A”，请确认FreeRTOSconfig.h文件中的“define configUSE_TRACE_FACILITY”已启用。

FreeRTOS Semaphores视图中的列信息如表21所示。

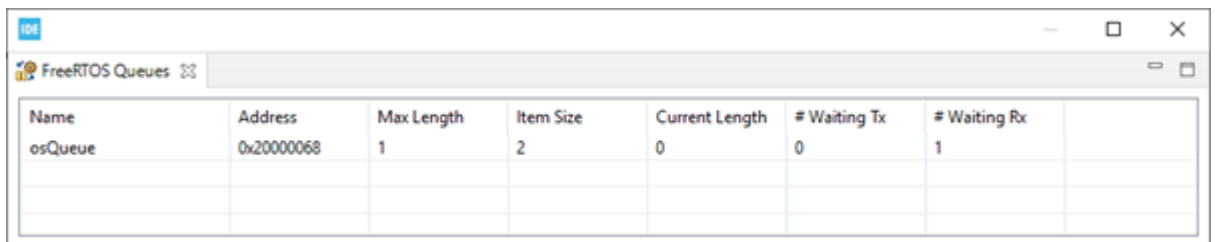
Table 21. FreeRTOS Semaphores details

Name	Description
Name	The name assigned to the semaphore.
Address	The address of the object.
Type	The type of the object.
Size	The maximum number of owning tasks.
Free	The number of free slots currently available.
#Blocked tasks	The number of tasks currently blocked waiting for the object.

6.2.6 FreeRTOS Queues view

FreeRTOS Queues视图显示关于目标系统中所有可用队列的详细信息。每当目标执行被挂起时，视图都会自动更新。每种类型的队列参数各有一列，每个队列各有一行。如果自上次挂起调试器以来队列的任何参数的值发生了更改，则相应的行将以黄色突出显示。

Figure 221. FreeRTOS Queues



Name	Address	Max Length	Item Size	Current Length	# Waiting Tx	# Waiting Rx
osQueue	0x20000068	1	2	0	0	1

FreeRTOS Queues视图中的列信息如表22所示。

Table 22. FreeRTOS Queues details

Name	Description
Name	The name assigned to the queue in the queue registry.
Address	The address of the queue.
Max Length	The maximum number of items that the queue can hold.
Item Size	The size in bytes of each queue item.
Current Length	The number of items currently in the queue.
#Waiting Tx	The number of tasks currently blocked waiting to be sent to the queue.
#Waiting Rx	The number of tasks currently blocked waiting to be received from the queue.

6.3 RTOS-kernel-aware debug

STM32CubeIDE中的RTOS内核感知调试使用RTOS代理支持Microsoft®Azure®RTOS ThreadX和FreeRTOS™操作系统。RTOS代理包含在STM32CubeIDE中，可以与ST-LINK GDB服务器、OpenOCD和SEGGER J-Link GDB服务器一起使用。

当启用rtos内核感知调试并启动调试会话时，debug视图将列出所有线程。通过在Debug视图中

选择一个线程，线程当前上下文可以在视图中显示。例如，变量、寄存器、编辑器视图反映活动堆栈框架。

图222显示了一个调试会话。ThreadX线程列表视图显示消息队列接收者线程正在运行。这也可以在Debug视图中看到。在Debug视图中，选择MsgSenderThreadTwo_Entry函数，编辑器区域显示线程正在睡眠状态中等待500毫秒。

Figure 222. RTOS-kernel-aware debug

The screenshot shows the STM32CubeIDE debug interface. The ThreadX Thread List at the bottom displays the following threads:

Name	Pri.	State	Run Count	Stack Start	Stack End	Stack Size	Stack Ptr	Stack Usage
Message Queue Receiver Thread	10	RUNNING	875	0x2400139c	0x2400159b	512	0x2400141c	Disabled
Message Queue Sender Thread One	5	SLEEP (200)	74	0x2400098c	0x2400118b	512	0x2400105c	Disabled
Message Queue Sender Thread Two	5	SLEEP (404)	30	0x24001194	0x24001393	512	0x24001264	Disabled
System Timer Thread	0	SUSPENDED	874	0x24000570	0x2400096f	1024	0x2400083c	Disabled
Idle								

The main editor window shows the `MsgSenderThreadTwo_Entry` function, which is currently in a sleep state (indicated by the green highlight on the `tx_thread_sleep(500);` line). The function is part of the `tx_thread_sleep` module.

要启用RTOS内核感知调试，调试配置对话框中的调试器选项卡包含启用RTOS代理、驱动程序 (RTOS ThreadX或FreeRTOS™)、端口(Cortex®core)和与代理一起使用的端口号配置的设置。RTOS选项卡还包含一个驱动程序设置选项，以选择驱动程序(“ThreadX”或“FreeRTOS”)和使用的端口。“自动检测”驱动程序设置仍处于实验阶段。

Figure 223. RTOS-kernel-awareness debug configuration

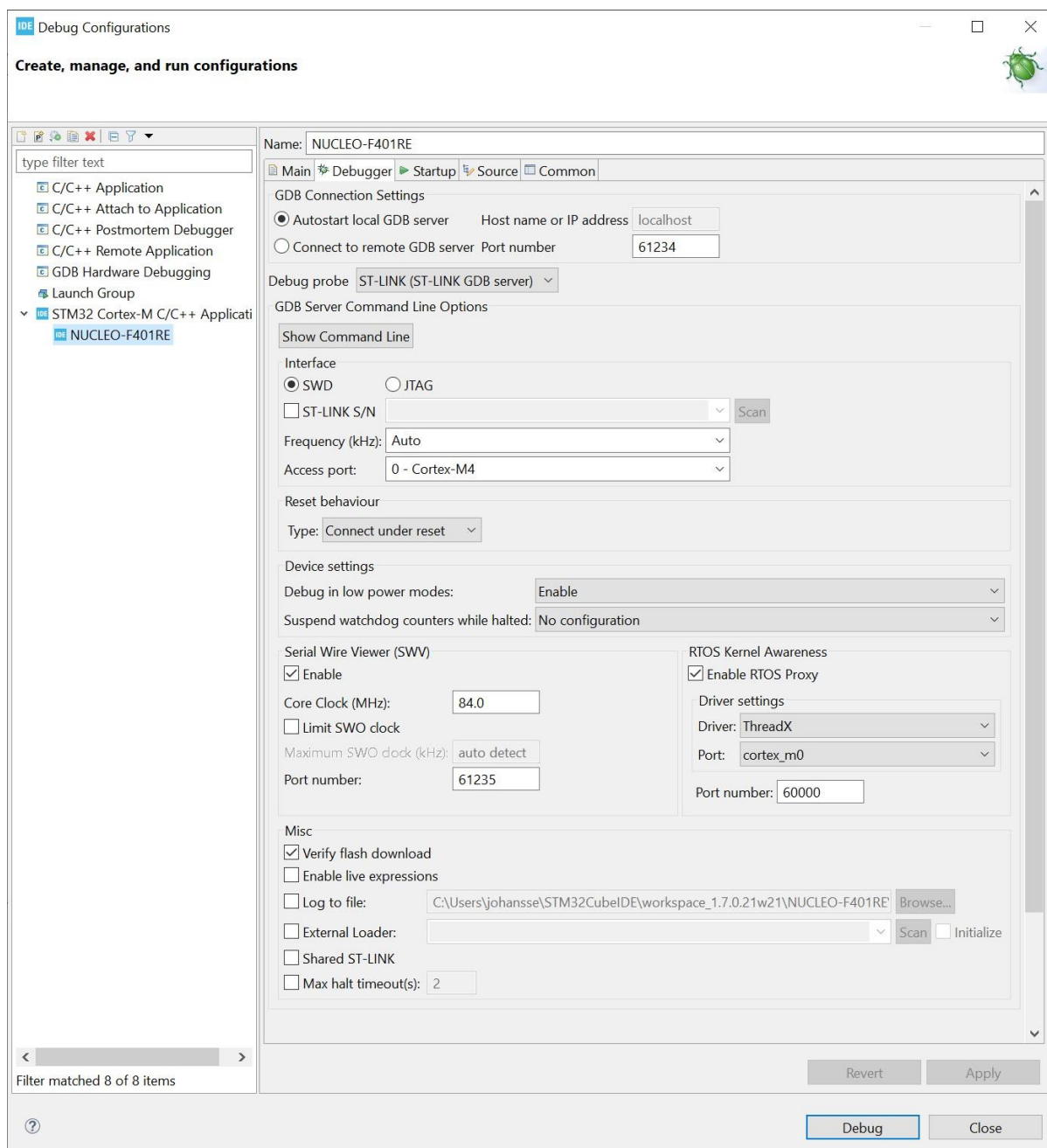
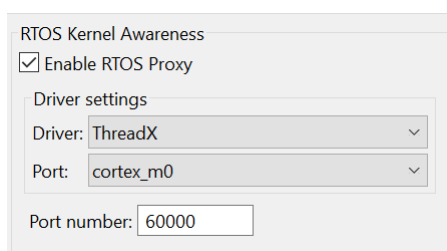


Figure 224. ThreadX-kernel-awareness debug configuration



端口选择中列出了支持的内核。所列项取决于所选的RTOS驱动程序，如图225和图226所示。

Figure 225. ThreadX port configuration

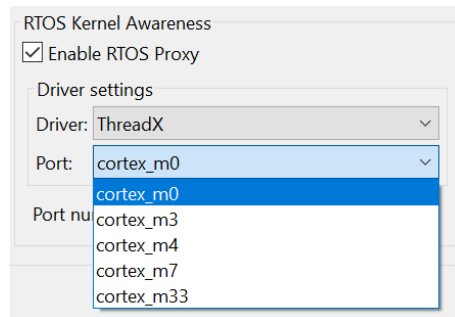
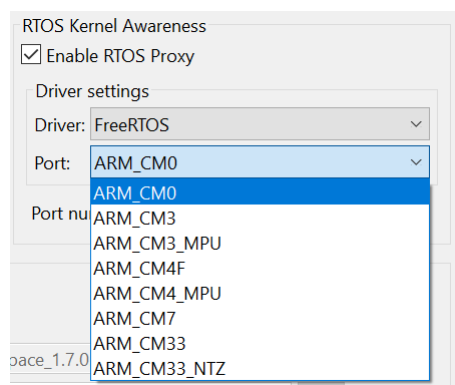


Figure 226. FreeRTOS™ port configuration



Known limitations

- 当与ST-LINK GDB服务器一起使用时，必须禁用活动表达式
- 换出线程的寄存器视图内容与一些寄存器的活动CPU上下文混合在一起(上下文切换器不保存所有寄存器)
- 寄存器视图浮点寄存器没有正确更新

7 Fault Analyzer

7.1 Introduction to the Fault Analyzer

STM32CubeIDE故障分析仪功能解释从Cortex®-M嵌套向量中断控制器(NVIC)提取的信息，以确定导致故障的原因。该信息在Fault Analyzer视图中可视化。它有助于识别和解决当CPU被应用软件驱动进入故障状态时发生的难以发现的系统故障。

这些条件包括：

- Accessing invalid memory locations访问无效内存位置
- Accessing memory locations on misaligned boundaries访问未对齐边界上的内存位置
- Executing undefined instruction执行未定义的指令
- Division by zero

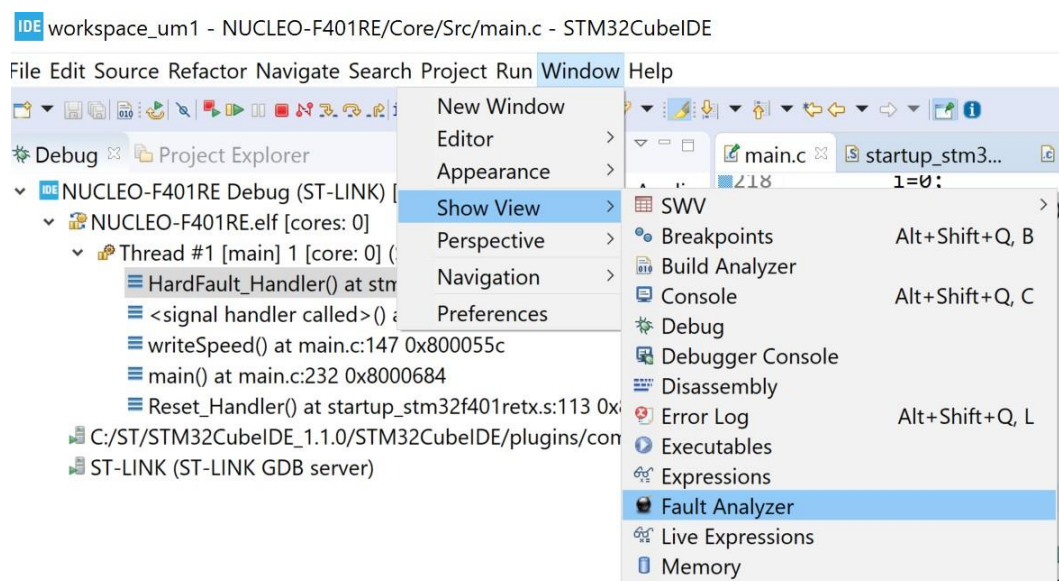
发生错误时，调试器中显示发生错误的代码行。该视图显示错误条件的原因。故障大致分为硬件故障、总线故障、使用故障和内存故障。

- 硬错误和总线错误发生在通过总线(外设寄存器或内存位置)进行无效的访问尝试时
- 使用错误是非法指令或其他程序错误的结果
- 内存故障包括试图访问非法位置或违反内存保护单元(MPU)维护的规则。

为了进一步帮助故障分析，异常堆栈帧可视化选项提供了崩溃时MCU寄存器值的快照。将错误隔离到单个指令可以重建错误指令执行时MCU的状态。

在调试器透视图，Fault Analyzer视图从菜单中打开。选择菜单命令[Window]>[Show View]>[Fault Analyzer]或使用[Quick Access]字段，搜索“Fault Analyzer”并从视图选择它。

Figure 227. Open the Fault Analyzer view



7.2 Using the *Fault Analyzer* view

Fault Analyzer视图有五个主要部分，可以展开和折叠。这些部分包含不同类型的信息，以便更好地理解导致特定故障发生的原因。这些部分是：

- Hard Fault Details
- Bus Fault Details
- Usage Fault Details
- Memory Management Fault Details
- Register Content During Fault Exception

当故障发生时，可以通过按视图中的按钮[Open editor on fault location]和[Open disassembly on fault location]。

图228显示了检测到错误时Fault Analyzer视图的示例。在本例中，该错误是由一个项目在HardFault_Handler()中停止调试器时进行除零操作引起的。

当出现这种情况时，打开Fault Analyzer视图将显示错误的原因。在本例中，它显示[Usage Fault Detected]和[Attempt to perform a division by zero (DIVBYZERO)]。故障异常期间的寄存器内容显示问题发生时的寄存器值。

Figure 228. Fault Analyzer view

IDE

Fault Analyzer

Hard Fault Detected

Hard Fault Details

- Bus, memory management or usage fault (FORCED)
- Failed vector fetch (VECTBL)
- Debug event (DEBUGEVT)

Bus Fault Details

- Instruction access violation (IBUSERR)
- Precise data access violation (PRECISERR)
- Imprecise data access violation (IMPRECISERR)
- Unstacking error (UNSTKERR)
- Stacking error (STKERR)
- Floating point lazy state preservation error (LSPERR)

Bus fault address register (BFAR): 0xe0

Usage Fault Details

- Attempt to execute an undefined instruction (UNDEFINSTR)
- Attempt to switch to invalid state (INVSTATE)
- Attempt to do exception with bad value in EXEC_RETURN number (INVPC)
- Attempt to execute a coprocessor instruction (NOCP)
- Attempt to perform an unaligned access (UNALIGNED)
- Attempt to perform a division by zero (DIVBYZERO)

Memory Management Fault Details

- Instruction access violation (IACCVIOL)
- Data access violation (DACCVIOL)
- Unstacking error (MUNSTKERR)
- Stacking error (MSTKERR)
- Floating point lazy state preservation error (MLSPERR)

Mem manage address register (MMFAR): 0xe0

Register Content During Fault Exception

Name	Value
sp (M...	0x20017fc0
r0	0x0
r1	0x0
r2	0x0
r3	0x8000000
r12	0x0
lr	0x8000685
pc	0x800055c
xpsr	0x21000000

The value of the stack pointer when the fault occurred. Please verify that this value points to a valid stack memory region.

MSP = Main Stack Pointer
PSP = Process Stack Pointer

Fault Analyzer视图包含这些工具栏按钮:

Figure 229. Fault Analyzer toolbar



- 第一个工具栏按钮(左)通过使用堆栈中的PC和LR寄存器中的信息以及调试elf文件中的符号信息，打开故障定位返回地址上的编辑器。
- 第二个工具栏按钮(中间)使用堆栈中的PC和LR寄存器中的信息以及调试elf文件中的符号信息，打开故障定位返回地址的Disassembly视图。
- 第三个工具栏按钮(右)在打开错误位置的Editor或Disassembly视图时选择使用PC或LR寄存器。

图230和图231显示了使用工具栏按钮打开的Editor和Disassembly视图，以查找示例中的故障位置。

Figure 230. Fault analyzer open editor on fault

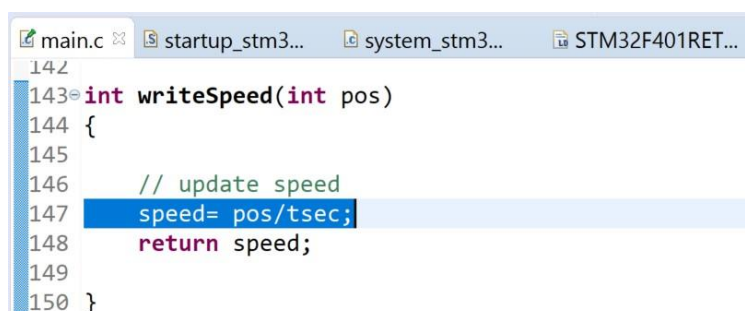
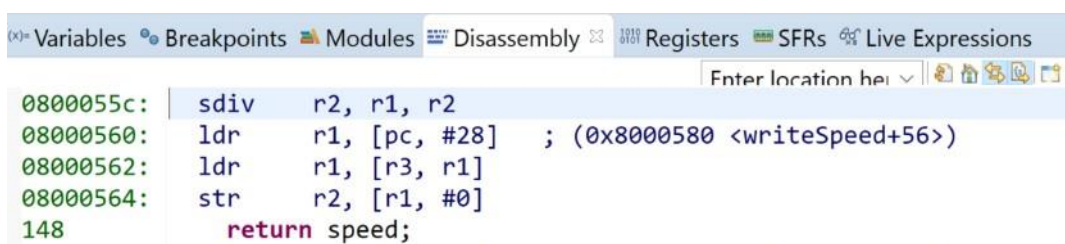


Figure 231. Fault Analyzer open disassembly on fault



Note:

故障分析器可以在所有STM32项目上使用。它不需要特殊的代码和特殊的构建配置。所有数据都是为Cortex®-M寄存器收集的。符号信息从调试elf文件中读取。

8 Build Analyzer

8.1 Introduction to the Build Analyzer

STM32CubeIDE Build Analyzer特性详细解释elf文件中的程序信息，并在视图中显示这些信息。如果在与elf文件相同的文件夹中找到名称相似的地图文件，则也会使用地图文件中的信息，甚至可以显示更多信息。

Build Analyzer视图对于优化或简化程序非常有用。视图包含两个选项卡，Memory Regions和Memory Details选项卡：

- 如果elf文件包含相应的映射文件，则用数据填充Memory Regions选项卡。当映射文件可用时，该选项卡可以被视为内存区域的简要摘要，其中包含有关区域名称、起始地址和大小的信息。大小信息还包括总大小、区域的空闲和使用部分以及使用率。
- Memory Details选项卡包含基于elf文件的详细程序信息。不同的节名显示地址和大小信息。每个部分都可以展开和折叠。当展开一个section时，会列出该section中的函数/数据。每个函数/数据都包含地址和大小信息。

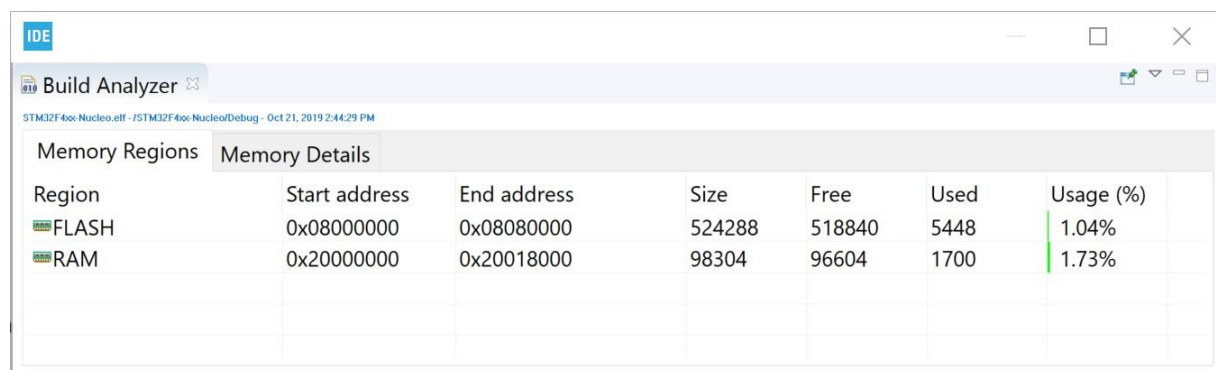
8.2 Using the Build Analyzer

Build Analyzer视图在C/ c++透视图默认是打开的。如果视图是关闭的，它可以从菜单中打开。选择菜单命令[Window]>[Show View]>[Build Analyzer]或使用[Quick Access]字段，搜索“Build Analyzer”并从视图中选择它。

当Build Analyzer视图打开时，在Project Explorer视图中选择一个elf文件。然后，使用来自该文件的信息更新Build Analyzer视图。当选择了elf文件，并且在同一文件夹中找到名称相似的映射文件时，视图也会使用该映射文件中的附加信息。

如果选择了project Explorer视图中的项目节点，Build Analyzer视图也会更新。在这种情况下，构建分析器使用与项目的当前活动构建配置相对应的elf文件。

Figure 232. Build analyzer



Region	Start address	End address	Size	Free	Used	Usage (%)
FLASH	0x08000000	0x08080000	524288	518840	5448	1.04%
RAM	0x20000000	0x20018000	98304	96604	1700	1.73%

8.2.1 Memory Regions tab

Build Analyzer视图中的Memory Regions选项卡显示了基于相应映射文件的信息。如果没有显示任何信息，说明没有找到对应的映射文件。当找到一个映射文件时，将显示区域名称、起始地址、结束地

址、区域总大小、空闲大小、已使用大小和使用信息。

这些区域通常在构建程序时使用的链接器脚本文件(.ld)中定义。如果内存区域的位置或大小必须改变，请更新链接器脚本文件。

Note: 如果elf文件没有对应的映射文件，则Memory Regions选项卡为空。

Figure 233. Memory Regions tab

Region	Start address	End address	Size	Free	Used	Usage (%)
RAM	0x20000000	0x20018000	96 KB	16.23 KB	79.77 KB	83.09%
FLASH	0x08000000	0x08040000	256 KB	236.17 KB	19.83 KB	7.75%
FLASH_ICONS	0x08040000	0x08050000	64 KB	44.47 KB	19.53 KB	30.52%
FLASH_IMAGES	0x08050000	0x08070000	128 KB	10.81 KB	117.19 KB	91.55%
FLASH_SOUND	0x08070000	0x0807f000	60 KB	7.75 KB	52.25 KB	87.08%
FLASH_D	0x0807f000	0x0807f800	2 KB	1.99 KB	8 B	0.39%
FLASH_V	0x0807f800	0x08080000	2 KB	1.99 KB	12 B	0.59%

列信息如表23所示。

Table 23. Memory Regions tab information

Name	Description
Region	Name of memory region (if a corresponding map file is found).
Start address	The start address of the region, defined in the linker script.
End address	End address of the region.
Size	The total size of memory region.
Free	The free size in the memory region.
Used	The used size in the memory region.
Usage %	The percentage of used size relative to the total memory region size. See Table 24 for the bar icon color information.

“使用率(%)”列中有一个与百分比值对应的栏形图标。根据使用内存的百分比，条有不同的颜色。

Table 24. Memory Regions usage color

Usage color	Description
Green	Less than 75% of memory used.
Yellow	75% to 90% of memory used.
Red	More than 90% of memory used.

8.2.2 Memory Details tab

Build Analyzer视图的Memory Details选项卡包含elf文件的信息。可以展开Memory Details选项卡中的每个部分，以便可以看到各个函数和数据。选项卡显示了包含名称、运行地址、加载地址和大小信息的列。

Figure 234. Memory Details tab

Name	Run address (VMA)	Load address (LMA)	Size
RAM	0x20000000		96 KB
> .data	0x20000000	0x08004f49	12 B
> .bss	0x2000000c		78.25 KB
_user_heap_stack	0x2001390c		1.5 KB
FLASH	0x08000000		256 KB
> FLASH_ICONS	0x08040000		64 KB
> FLASH_IMAGES	0x08050000		128 KB
> .flash_images	0x08050000	0x08050000	117.19 KB
image3	0x08064c08	0x08064c08	34.18 KB
image2	0x080561a8	0x080561a8	58.59 KB
image1	0x08050000	0x08050000	24.41 KB
> FLASH_SOUND	0x08070000		60 KB
> .flash_sound	0x08070000	0x08070000	52.25 KB
> FLASH_D	0x0807f000		2 KB
> .flash_d	0x0807f000		8 B
> FLASH_V	0x0807f800		2 KB
> .flash_v	0x0807f800	0x0807f800	12 B

列信息如表25所示。

Table 25. Memory Details tab information

Name	Description
Name	Name of memory region, section, function, and data. A green icon is used to mark functions while the blue icon is used for data variables.
Run Address (VMA)	The Virtual Memory Address contains the address used when the program is running.
Load Address (LMA)	The Load Memory Address is the address used for load, for instance for the initialization values of global variables.
Size	Used size (total size for <i>Memory Regions</i>).

Note: 只有在找到相应的映射文件时，才显示内存区域名。

8.2.2.1 Size information

Memory Details选项卡中的大小信息是根据elf文件中的符号大小计算出来的。如果要调查对应的映射文件，则它可能包含不同的大小值。C文件的大小通常是正确的，但汇编程序文件的值取决于大小信息如何写入汇编程序文件。函数使用的常量必须在section定义中定义。在section的末尾，链接器使用size指令计算函数的大小。

Example: Reset_Handler in startup.s file

这个例子展示了如何在汇编程序启动文件中编写Reset_Handler，以便在elf文件中的Reset_Handler大小信息中包含常量_sidata、_sdata、_edata、_sbss和_ebss。如果这些常量是在Reset_Handler节定义之外定义的，则它们的大小不包括在Reset_Handler的计算大小中。为了将它们包含在Reset_Handler的大小中，这些定义必须放在Reset_Handler部分中，如下面的代码示例所示。

```
.section .text.Reset_Handler
.weak Reset_Handler
.type Reset_Handler, %function

Reset_Handler:
ldr sp, =_estack /* set stack pointer */

/* Copy the data segment initializers from flash to SRAM */
movs r1, #0
b LoopCopyDataInit

CopyDataInit:
ldr r3, =_sidata

/* initialization code data, bss, ... */
...

/* Call the application's entry point */
bl main
bx lr

/* start address for the initialization values defined in linker script */
.word _sidata
.word _sdata
.word _edata
.word _sbss
.word _ebss

.size Reset_Handler, .-Reset_Handler
```

8.2.2.2 Sorting

可以通过单击列名更改Memory Details选项卡列的排序顺序。

Figure 235. Memory Details sorted by size

Name	Run address (VMA)	Load address (LMA)	Size
> FLASH	0x08000000		256 KB
▼ FLASH_IMAGES	0x08050000		128 KB
▼ .flash_images	0x08050000	0x08050000	117.19 KB
image2	0x080561a8	0x080561a8	58.59 KB
image3	0x08064c08	0x08064c08	34.18 KB
image1	0x08050000	0x08050000	24.41 KB
▼ RAM	0x20000000		96 KB
> .bss	0x2000000c		78.25 KB
> .user_heap_stack	0x2001390c		1.5 KB
> .data	0x20000000	0x08004f49	12 B
> FLASH_ICONS	0x08040000		64 KB
> FLASH_SOUND	0x08070000		60 KB
▼ FLASH_D	0x0807f000		2 KB
> .flash_d	0x0807f000		8 B
▼ FLASH_V	0x0807f800		2 KB
> .flash_v	0x0807f800	0x0807f800	12 B

8.2.2.3 Search and filter

可以通过在搜索字段中输入字符串来过滤Memory Detailstab中的信息。图236显示了包含字符串“sound”的名称的搜索示例。

Figure 236. Memory Details search and filter

Name	Run address (VMA)	Load address (LMA)	Size
▼ FLASH_SOUND	0x08070000		60 KB
▼ .flash_sound	0x08070000	0x08070000	52.25 KB
sound1	0x08070000	0x08070000	19.53 KB
sound2	0x08074e20	0x08074e20	19.53 KB
sound4	0x0807afc8	0x0807afc8	8.3 KB
sound3	0x08079c40	0x08079c40	4.88 KB

8.2.2.4 Calculate the sum of sizes

Memory Details选项卡中几行大小的总和可以通过在视图中选择这些行来计算。选择的总和显示在视图中的Name列上方。

Figure 237. Sum of sizes

Name	Run address (VMA)	Load address (LMA)	Size
FLASH	0x08000000		256 KB
FLASH_IMAGES	0x08050000		128 KB
.flash_images	0x08050000	0x08050000	117.19 KB
image2	0x080561a8	0x080561a8	58.59 KB
image3	0x08064c08	0x08064c08	34.18 KB
image1	0x08050000	0x08050000	24.41 KB
RAM	0x20000000		96 KB
FLASH_ICONS	0x08040000		64 KB
FLASH_SOUND	0x08070000		60 KB
.flash_sound	0x08070000	0x08070000	52.25 KB
FLASH_D	0x0807f000		2 KB
FLASH_V	0x0807f800		2 KB

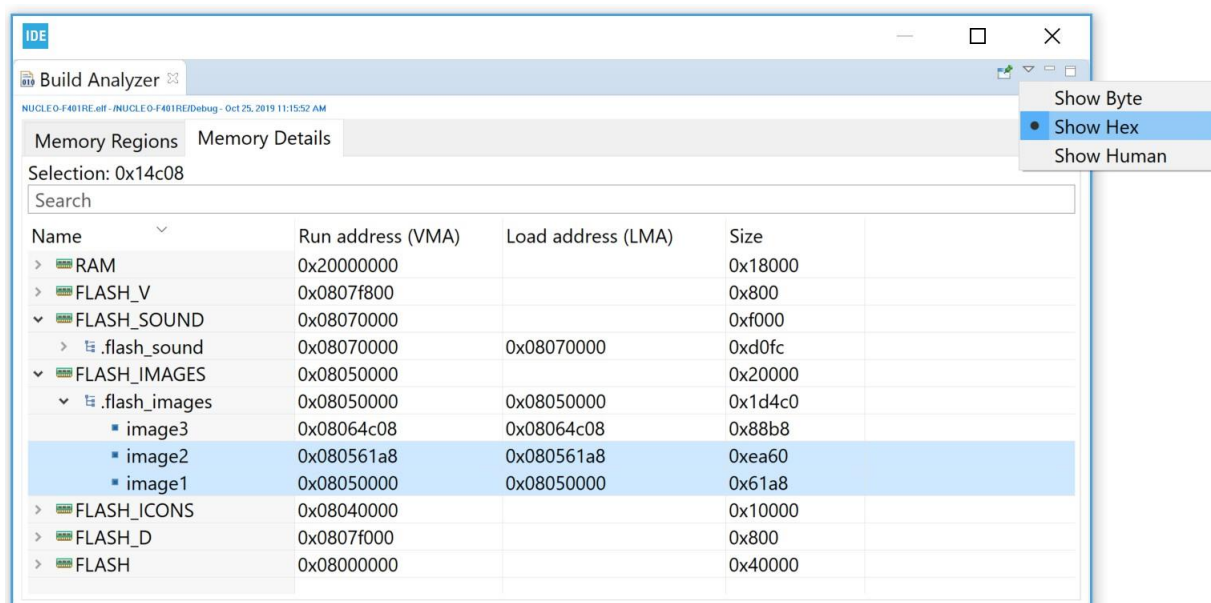
8.2.2.5 Display the size information in byte format

Build Analyzer视图可以根据[Show Byte]、[Show Hex]或[Show Human]选择以不同的格式显示大小信息。Build Analyzer工具栏中的图标用于在这些格式之间切换。首选[Show Byte]或[Show Hex]时复制和粘贴数据到Excel®文档以后的计算。

Figure 238. Show byte count

Name	Run address (VMA)	Load address (LMA)	Size
RAM	0x20000000		98304
FLASH_V	0x0807f800		2048
FLASH_SOUND	0x08070000		61440
.flash_sound	0x08070000	0x08070000	53500
FLASH_IMAGES	0x08050000		131072
.flash_images	0x08050000	0x08050000	120000
image3	0x08064c08	0x08064c08	35000
image2	0x080561a8	0x080561a8	60000
image1	0x08050000	0x08050000	25000
FLASH_ICONS	0x08040000		65536
FLASH_D	0x0807f000		2048
FLASH	0x08000000		262144

Figure 239. Show hex count

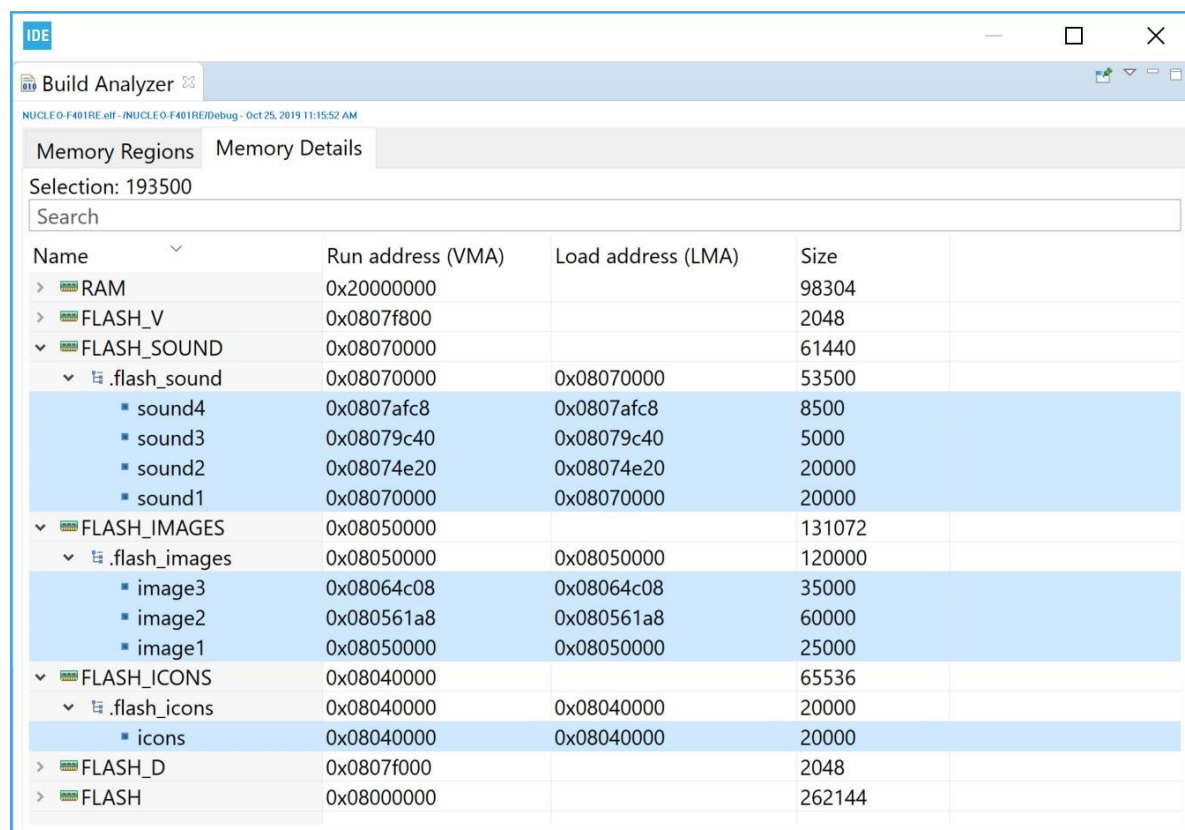


8.2.2.6

Copy and paste

通过选择要复制的行并按Ctrl+C，可以将“内存详细信息”选项卡中的数据以CSV格式复制到其他应用程序。可以使用Ctrl+V命令将复制的数据粘贴到另一个应用程序中。

Figure 240. Copy and paste



用Ctrl+C复制图240中所选的行，得到如下的Ctrl+V结果:

```
"sound4";"0x0807afc8";"0x0807afc8";"8500"  
"sound3";"0x08079c40";"0x08079c40";"5000"  
"sound2";"0x08074e20";"0x08074e20";"20000"  
"sound1";"0x08070000";"0x08070000";"20000"  
"image3";"0x08064c08";"0x08064c08";"35000"  
"image2";"0x080561a8";"0x080561a8";"60000"  
"image1";"0x08050000";"0x08050000";"25000"  
"icons";"0x08040000";"0x08040000";"20000"
```

9 Static Stack Analyzer

9.1 Introduction to the Static Stack Analyzer

STM32CubeIDE静态堆栈分析器基于构建的程序计算堆栈使用情况。它详细分析了gcc生成的.su文件和elf文件，并在视图中显示了结果信息。

视图包含两个选项卡，List和Call Graph选项卡。

List选项卡填充了程序中包含的每个函数的堆栈使用情况。该选项卡为每个功能列出一行，每行由功能、本地成本、类型、位置和信息列组成。

Figure 241. Static Stack Analyzer List tab

Function	Local cost	Type	Location	Info
main	88	STATIC	main.c:79	
TIM_Ti1_SetConfig	16	STATIC	stm32f4xx_hal_tim.c:4540	
TIM_SlaveTimer_SetConfig	12	STATIC	stm32f4xx_hal_tim.c:4983	
TIM_CCxChannelCmd	8	STATIC	stm32f4xx_hal_tim.c:4739	
TIM_Base_SetConfig	0	STATIC	stm32f4xx_hal_tim.c:4481	
SystemInit	0	STATIC	system_stm32f4xx.c:148	
HAL_TIM_TriggerCallback	0	STATIC	stm32f4xx_hal_tim.c:4364	
HAL_TIM_SlaveConfigSync...	16	STATIC	stm32f4xx_hal_tim.c:4143	
HAL_TIM_ReadCapturedVal...	0	STATIC	stm32f4xx_hal_tim.c:4217	
HAL_TIM_PeriodElapsedCal...	0	STATIC	stm32f4xx_hal_tim.c:4304	
HAL_TIM_PWM_PulseFinish...	0	STATIC	stm32f4xx_hal_tim.c:4349	
HAL_TIM_OC_DelayElapsed...	0	STATIC	stm32f4xx_hal_tim.c:4319	
HAL_TIM_IRQHandler	8	STATIC	stm32f4xx_hal_tim.c:2809	
HAL_TIM_IC_Start_IT	8	STATIC	stm32f4xx_hal_tim.c:1672	

Call Graph选项卡包含一个可扩展的列表，其中包含程序中包含的函数。表示函数调用其他函数的行可以展开以查看调用层次结构。

Figure 242. Static Stack Analyzer Call Graph tab

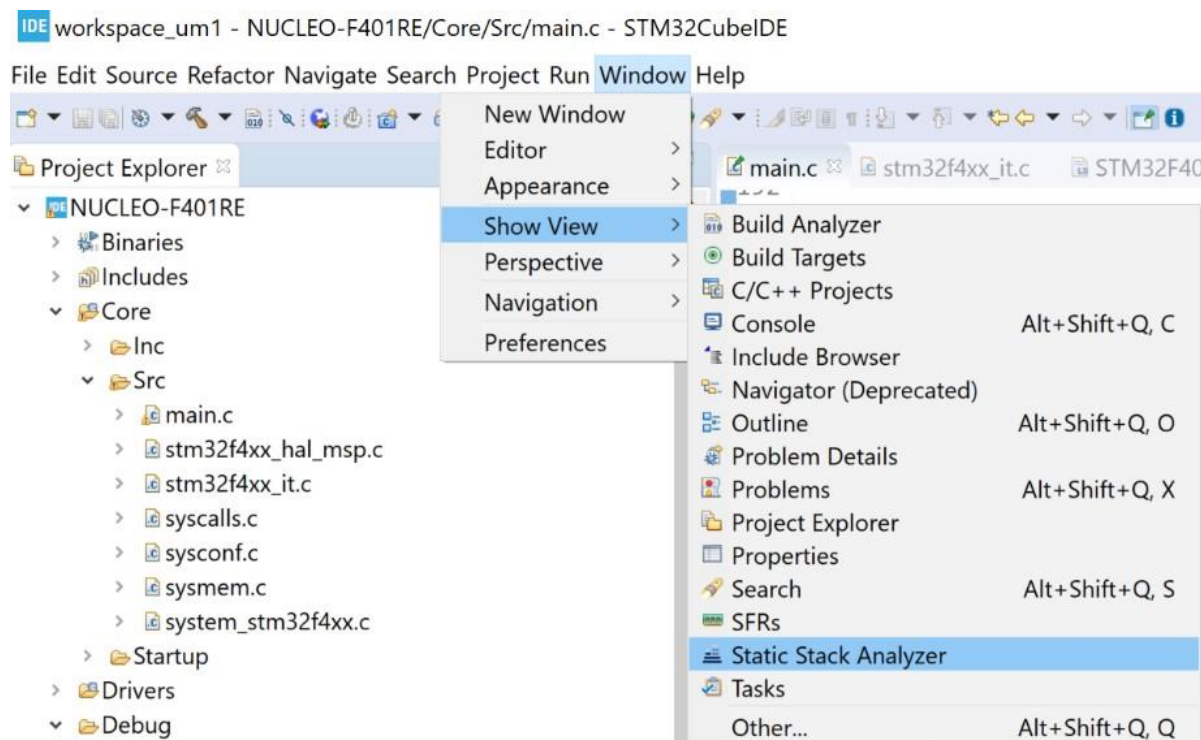
Function	Depth	Max cost	Local cost	Type	Location	Info
main						
UsageFault_Handler	?	?	0	STATIC	stm32f4xx_it.c:116	Max cost uncertain. Recursive
ADC_IRQHandler	?	?	0	STATIC	stm32f4xx_it.c:103	Max cost uncertain. Recursive. No stack usage information available for this ...
BusFault_Handler	?	?	0	STATIC	stm32f4xx_it.c:77	Max cost uncertain. Recursive
HardFault_Handler	?	?	0	STATIC	stm32f4xx_it.c:90	Max cost uncertain. Recursive
MemManage_Handler	?	?	0	STATIC	stm32f4xx_it.c:173	Max cost uncertain. No stack usage information available for this function
Reset_Handler	7	184	0	STATIC	stm32f4xx_it.c:68	Max cost uncertain
TIM4_IRQHandler	3	8	0	STATIC	stm32f4xx_it.c:147	Max cost uncertain
NMI_Handler	0	0	0	STATIC	stm32f4xx_it.c:156	Max cost uncertain. No stack usage information available for this function
PendSV_Handler	0	0	0	STATIC	stm32f4xx_it.c:129	Max cost uncertain. No stack usage information available for this function
frame_dummy	0	0	0	STATIC	stm32f4xx_it.c:138	Max cost uncertain. No stack usage information available for this function
SysTick_Handler	1	0	0	STATIC	stm32f4xx_it.c:138	Max cost uncertain. No stack usage information available for this function
SVC_Handler	0	0	0	STATIC	stm32f4xx_it.c:138	Max cost uncertain. No stack usage information available for this function
DebugMon_Handler	0	0	0	STATIC	stm32f4xx_it.c:138	Max cost uncertain. No stack usage information available for this function
_do_global_ctors_aux	0	0	0	STATIC	stm32f4xx_it.c:138	Max cost uncertain. No stack usage information available for this function
_fini	0	0	0	STATIC	stm32f4xx_it.c:138	Max cost uncertain. No stack usage information available for this function

9.2

Using the Static Stack Analyzer

静态堆栈分析视图在C/ C++透视图默认是打开的。如果视图是关闭的，它可以从菜单中打开。选择菜单命令[Window]>[Show View]>[Static Stack Analyzer]。另一种打开静态堆栈分析视图的方法是在[Quick Access search bar]中键入“静态堆栈分析”，并从视图选择它

Figure 243. Open the Static Stack Analyzer view



在项目资源管理器中选择已构建的项目时，将填充静态堆栈分析器视图。项目构建时必须启用选项[Generate per function stack usage information]，否则视图不能显示任何堆栈信息。

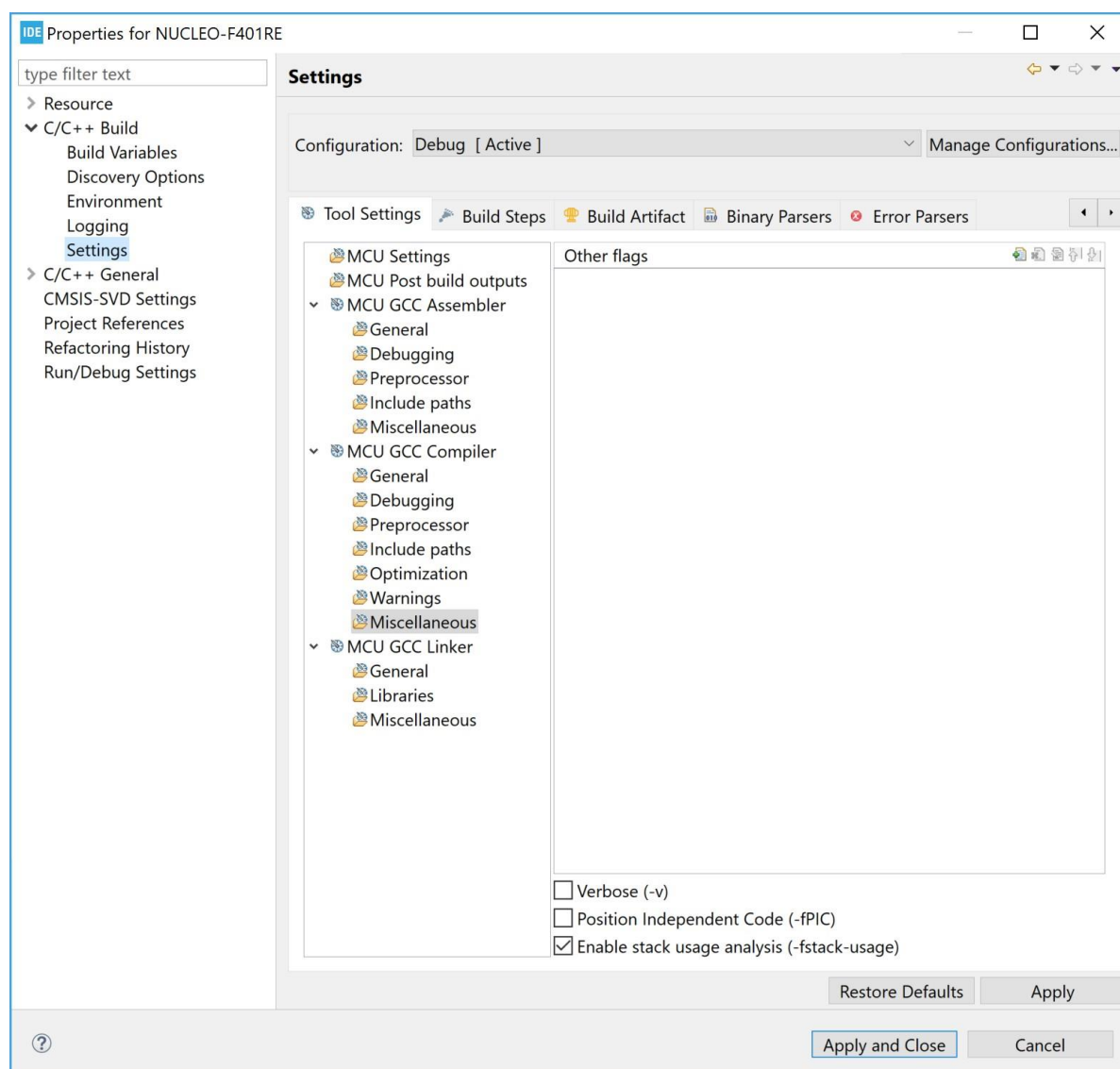
下一节将解释如何设置编译器来生成堆栈使用信息。

9.2.1 Enable stack usage information

如果视图顶部显示没有发现堆栈使用信息，请在编译器设置中启用，编译配置必须更新编译器以生成堆栈信息：

1. Open the project properties, for instance with a right-click on the project in the *Project Explorer* view
2. Select *Properties* and, in the dialog, select [C/C++ Build]>[Settings]
3. Select the *Tool Settings* tab
4. Select [MCU GCC Compiler]>[Miscellaneous]
5. Select [Enable stack usage information (-fstack-usage)] as shown in Figure 244
6. Save the setting and rebuild the program

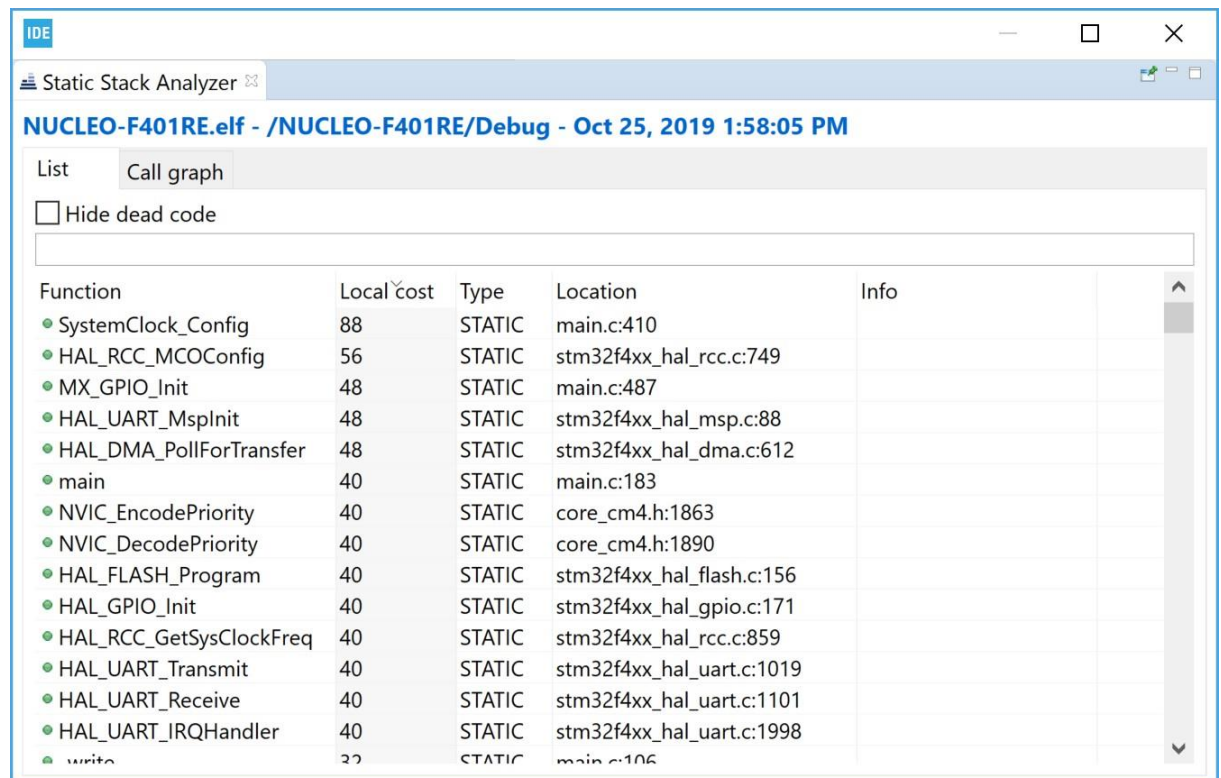
Figure 244. Enable generate per function stack usage information



9.2.2 List tab

*List*选项卡包含所选程序中包含的所有函数的列表，选项包括[Hide dead code]函数和[Filter]可见函数。使用[Hide dead code]选择启用或禁用死代码函数列表。如果使用，[Filter]字段限制显示与它包含的字符匹配的函数。

Figure 245. Static Stack Analyzer List tab



List选项卡中的列信息如表26所示。

Table 26. Static Stack Analyzer List tab details

Name	Description
Function	Function name.
Local cost	The number displays how many bytes of stack the function uses.
Type	Tells if the function uses a STATIC or DYNAMIC stack allocation. When DYNAMIC allocation is used the actual stack size is run-time dependent and the the <i>Local cost</i> value is uncertain due to the dynamic size of stack.
Location	Indicates where the function is declared. It is possible to double-click on a line and open the file with the defined function in the editor.
Info	Additional information about the calculation.

可以通过单击列名更改List选项卡排序顺序。

Note: 通过双击List选项卡中显示文件位置和行号的行，将在Editor视图中打开该函数。

9.2.3 Call Graph tab

Call Graph选项卡在树视图中包含详细的程序信息。程序中包含但没有被任何其他函数调用的每个函数都在顶层显示。可以展开树来查看被调用的函数。只有elf文件中可用的函数才能在选项卡中可见。

使用时，[Search...]按钮将触发显示与搜索栏中字符匹配的函数。搜索可以区分大小写，这取决于复选框中的选择[Case sensitive]。

Figure 246. Static Stack Analyzer Call Graph tab

Function	Depth	Max cost	Local cost	Type	Location	Info
ADC_IRQHandler	?	?	0			Max cost uncertain. Recursive. No stack usage information available for this...
Reset_Handler	16	248	0			Max cost uncertain. No stack usage information available for this function
_write	3	48	0			Max cost uncertain. No stack usage information available for this function
_sread	2	32	0			Max cost uncertain. No stack usage information available for this function
_sseek	2	24	0			Max cost uncertain. No stack usage information available for this function
_sclose	2	16	0			Max cost uncertain. No stack usage information available for this function
SysTick_Handler	1	12	8	STATIC	stm32f4xx_it.c:182	
NMI_Handler	0	4	4	STATIC	stm32f4xx_it.c:70	
DebugMon_Handler	0	4	4	STATIC	stm32f4xx_it.c:156	
MemManage_Handler	0	4	4	STATIC	stm32f4xx_it.c:98	
UsageFault_Handler	0	4	4	STATIC	stm32f4xx_it.c:128	
PendSV_Handler	0	4	4	STATIC	stm32f4xx_it.c:169	
BusFault_Handler	0	4	4	STATIC	stm32f4xx_it.c:113	
HardFault_Handler	0	4	4	STATIC	stm32f4xx_it.c:83	
SVC_Handler	0	4	4	STATIC	stm32f4xx_it.c:143	
frame_dummy	0	0	0			Max cost uncertain. No stack usage information available for this function

Call Graph选项卡中的列信息如表27所示。

Table 27. Static Stack Analyzer Call Graph tab details

Name	Description
Function	Function name.
Depth	Specifies the call stack depth this function uses: 0: the function does not call any other functions - Number ≥ 1: the function calls other functions ?: the function makes recursive calls or the depth cannot be calculated
Max cost	Specifies how many bytes of stack the function uses including stack needed for called functions.
Local cost	Specifies how many bytes of stack the function uses. This column does not take into account any stack that may be needed by the functions it may call.
Type	Specifies if the function uses a STATIC or DYNAMIC stack allocation. - STATIC: the function uses a fixed stack - DYNAMIC: the function uses a run-time dependent stack - Empty field: no stack usage information available for the function
Location	Indicates where the function is declared. It is possible to double-click on a line and open the file with the defined function in the editor.
Info	Contains specific information about the stack usage calculation. For instance, it can hold a combination of the following messages: - Max cost uncertain: the reason can be that the function makes a call to some sub-function where the stack information is not known, the function makes recursive calls, or others - Recursive: the function makes recursive calls - No stack usage information available for this function: no stack usage information available for this function - Local cost uncertain due to dynamic size, verify at run-time: the function allocates stack dynamically, for instance depending on a parameter

可以通过单击列名更改Call Graph选项卡排序顺序。

通过双击在选项卡中显示文件位置和行号的行，将在Editor视图中打开该函数。

Note: main函数通常由Reset_Handler调用，在这种情况下，在展开Reset_Handler节点时可以看到main函数。

如果该选项卡中列出了未使用的函数，请检查链接器选项[死代码删除]是否启用以从程序中删除未使用的代码。请在2.5.2节中阅读更多关于丢弃未使用的部分的内容。

“函数列”中函数名左侧的小图标表示：

- Green dot: the function uses STATIC stack allocation (fixed stack). 该函数使用静态堆栈分配(固定堆栈)。
- Blue square: the function uses DYNAMIC stack allocation (run-time dependent). 函数使用动态堆栈分配(依赖于运行时)。
- 010 icon: used if the stack information is not known. This can be the case for library functions or assembler functions. 当不知道堆栈信息时使用。这可以是库函数或汇编函数的情况。
- Three arrows in a circle: used in the *Call Graph* tab when the function makes recursive calls. 当函数进行递归调用时，在Call Graph选项卡中使用。

Figure 247. Function symbols in Static Stack Analyzer

Function	Depth
ADC_IRQHandler	?
Reset_Handler	16
LoopCopyDataInit	15
LoopFillZerobss	14
main	13
SystemClock_Config	5
MX_USART2_UART_Init	4
MX_GPIO_Init	1
SystemCoreClockUpdate	0
iprintf	12
readTemp	0
readSpeed	0
writeSpeed	0
writeTemp	0
SystemInit	0
FillZerobss	0

9.2.4 Using the filter and search field

List和Call Graph选项卡包含一个过滤器/搜索字段，可用于搜索与字段中输入的字符匹配的一个或多个特定函数。

图248显示了List选项卡，其中[Filter]字段用于查找名称中包含“read”字符串的函数。

Figure 248. Static Stack Analyzer List tab using search

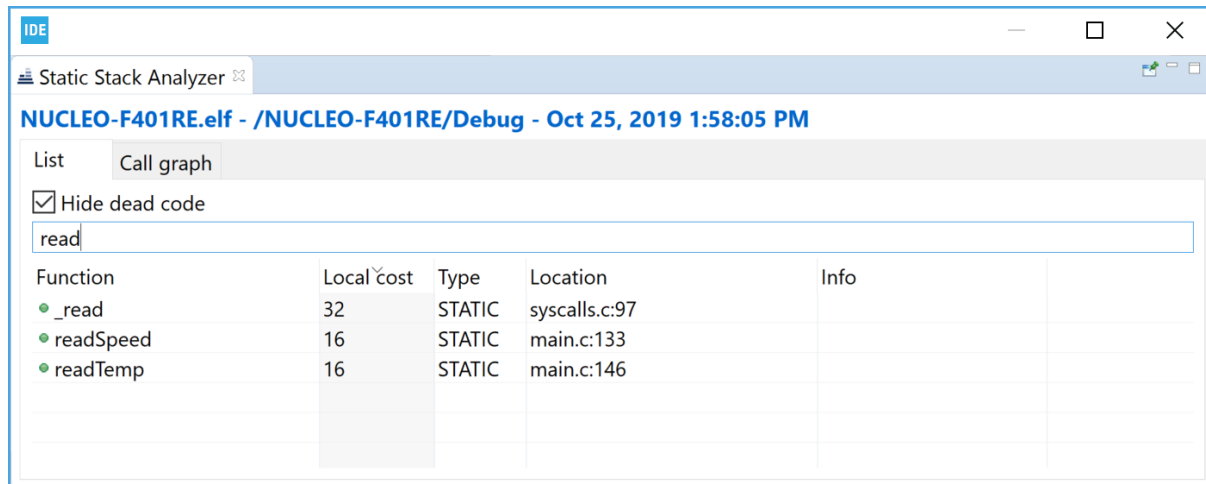
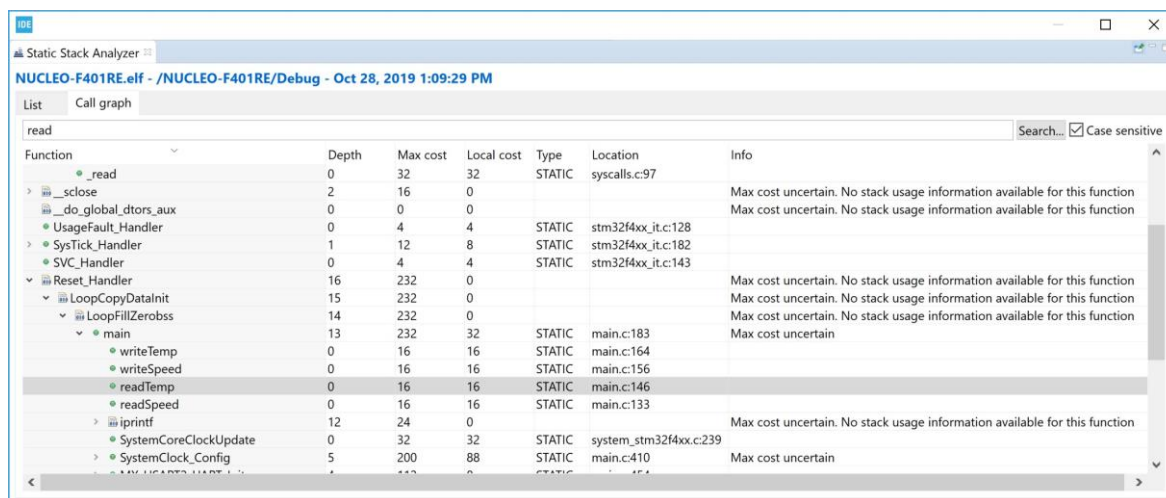


图249显示了[Search...]字段，用于过滤名称与“read”字符串匹配的函数。

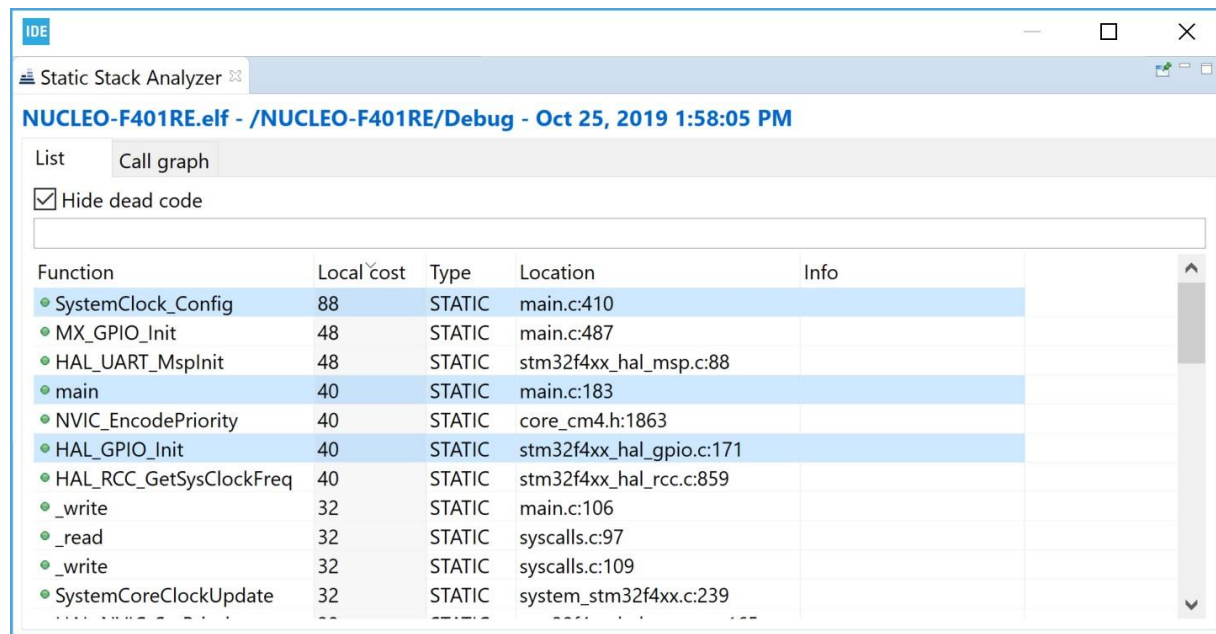
Figure 249. Static Stack Analyzer Call Graph using search



9.2.5 Copy and paste

通过选择要复制的行并按Ctrl+C，可以将List选项卡中的数据以CSV格式复制到其他应用程序。可以使用Ctrl+V命令将复制的数据粘贴到另一个应用程序中。

Figure 250. Copy and paste



用Ctrl+C复制图250中所选的行，得到如下的Ctrl+V结果:

```
"SystemClock_Config";"88";"STATIC";"main.c:410";""
"main";"40";"STATIC";"main.c:183";""
"HAL_GPIO_Init";"40";"STATIC";"stm32f4xx_hal_gpio.c:171";""
```

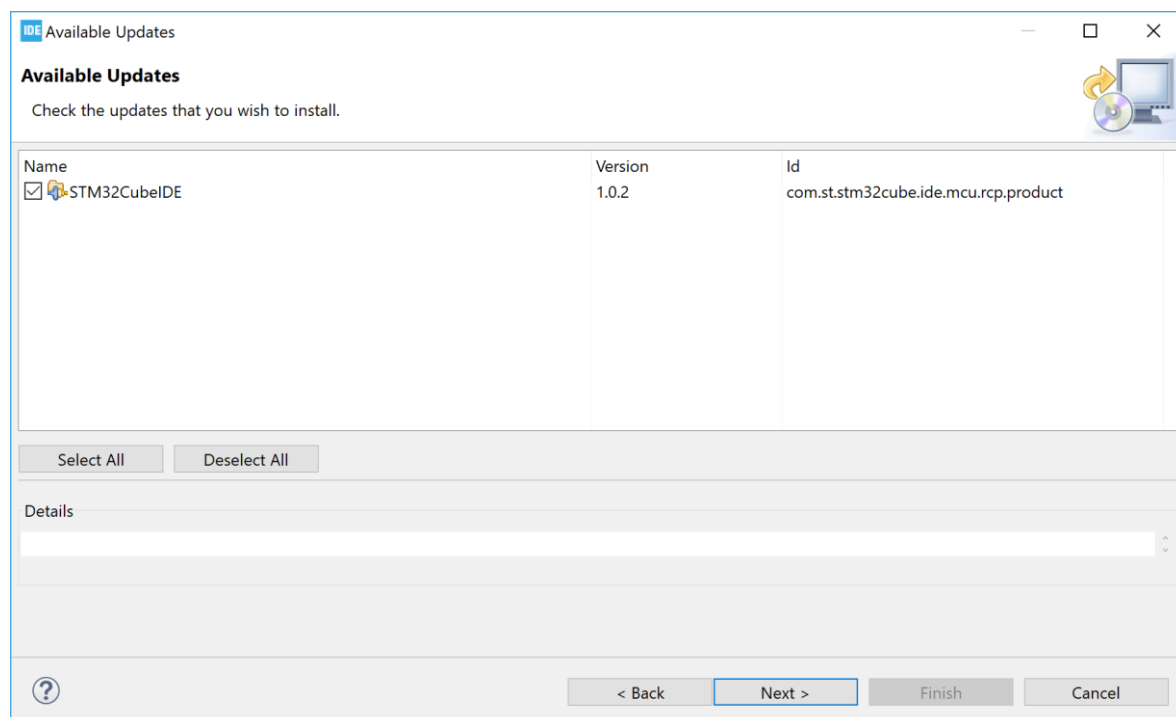
10 Installing updates and additional Eclipse® plugins

10.1 Check for updates

STM32CubeIDE定期检查可用更新，并在检测到新更新时打开可用更新对话框。也可以手动检查更新。使用菜单[Help]>[Check for Updates]检查是否有新软件可用。

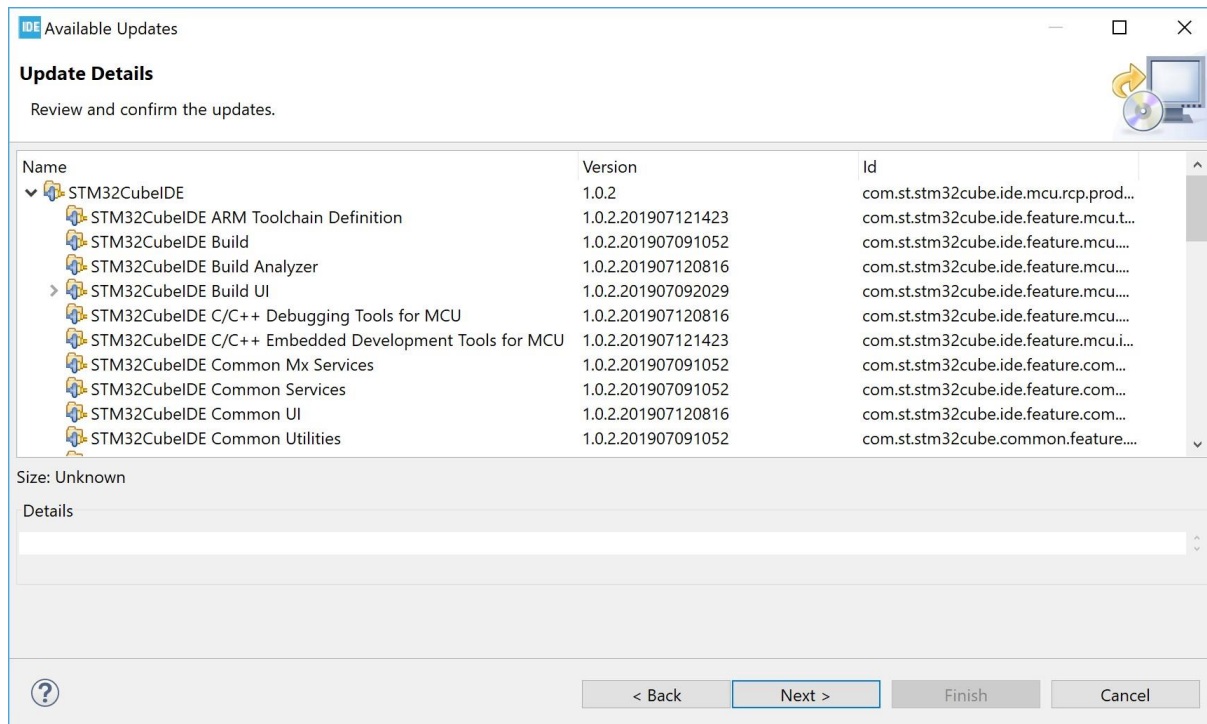
当找到更新时，选择要安装的更新并按[Next]。

Figure 251. STM32CubeIDE available updates



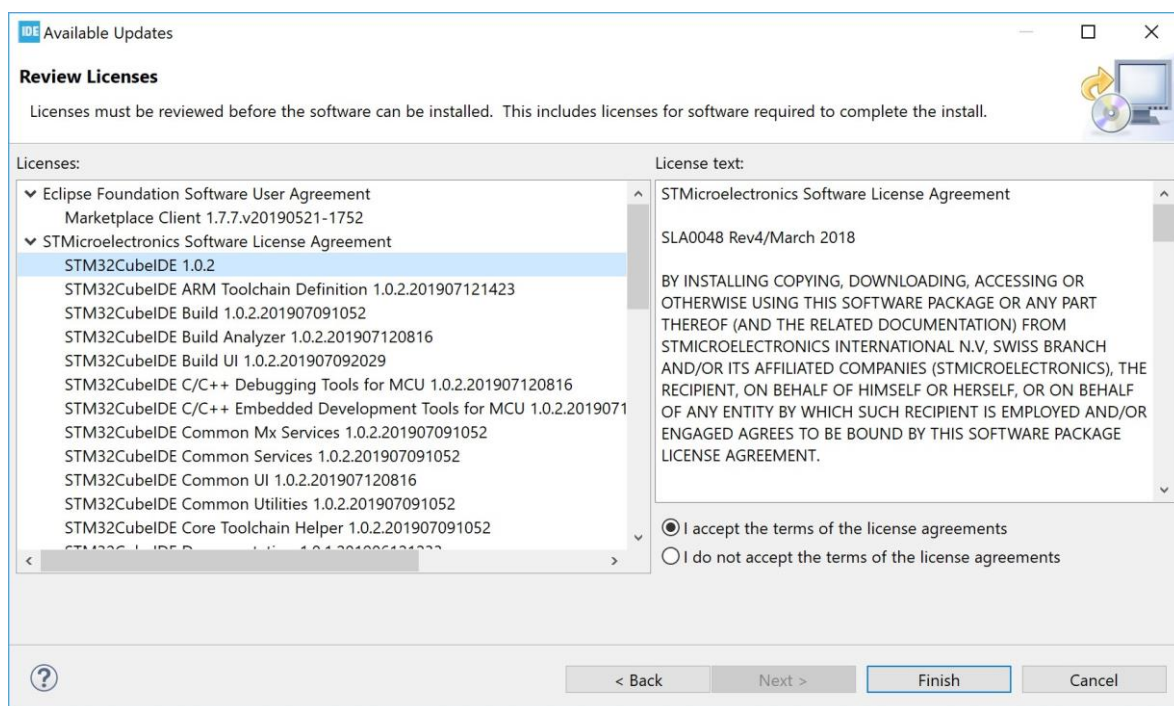
显示更新详情。检查并确认更新。按Next。

Figure 252. STM32CubeIDE update details



显示“Review Licenses”详细信息。检查许可，选择[我接受许可协议的条款]，然后按[完成]安装更新。

Figure 253. STM32CubeIDE update review licenses

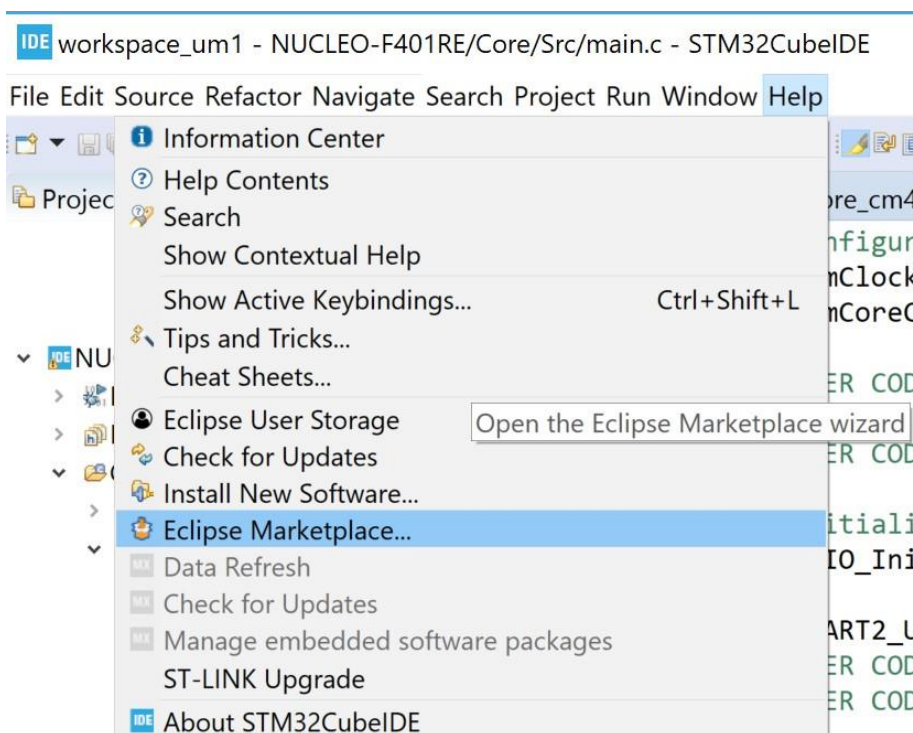


STM32CubeIDE窗口底部显示的进度条显示了安装完成率。更新完成后重新启动STM32CubeIDE。

10.2 Install from the Eclipse® market place

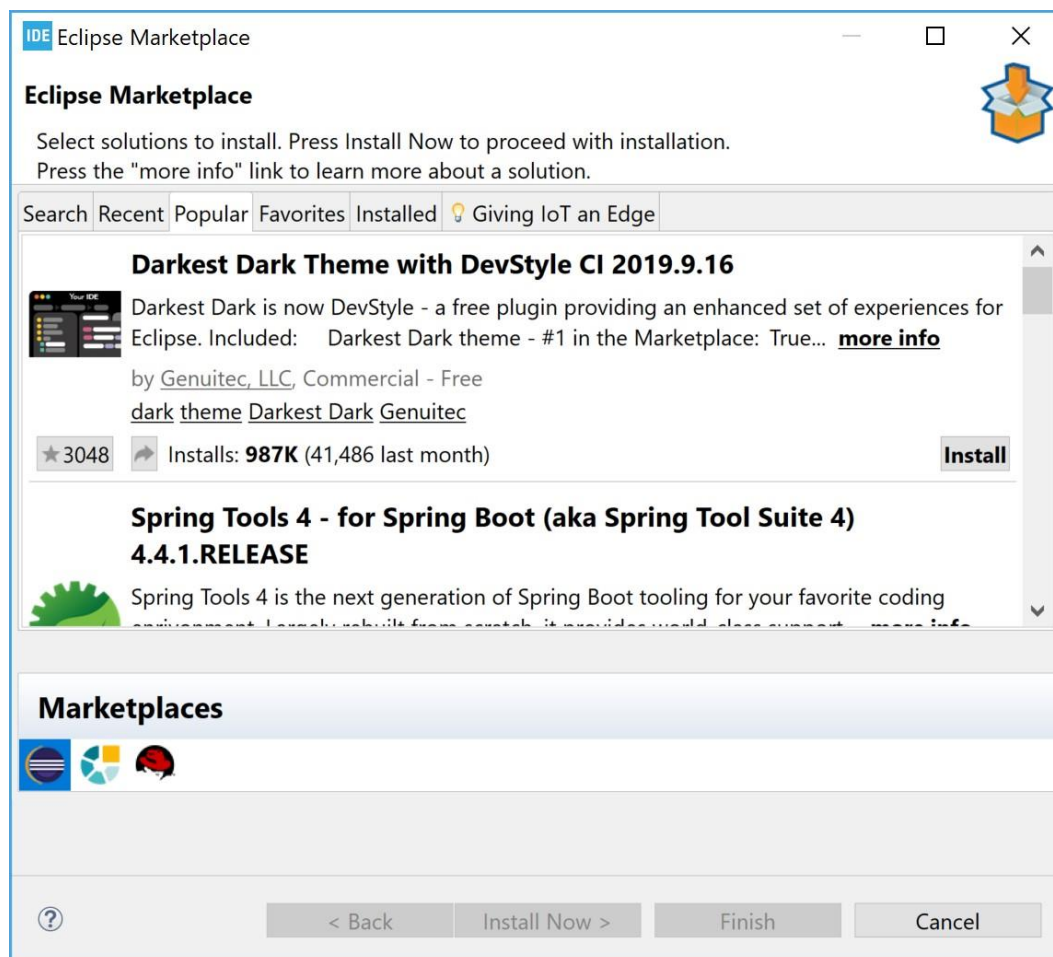
可以使用Eclipse市场在STM32CubeIDE中安装额外的第三方Eclipse®插件。要从Eclipse Marketplace安装，请选择菜单[Help]>[Eclipse Marketplace...]

Figure 254. Eclipse Marketplace menu



打开Eclipse Marketplace对话框。搜索插件或使用选项卡(最近, 流行, 收藏)找到想要的软件并安装它。

Figure 255. Eclipse marketplace



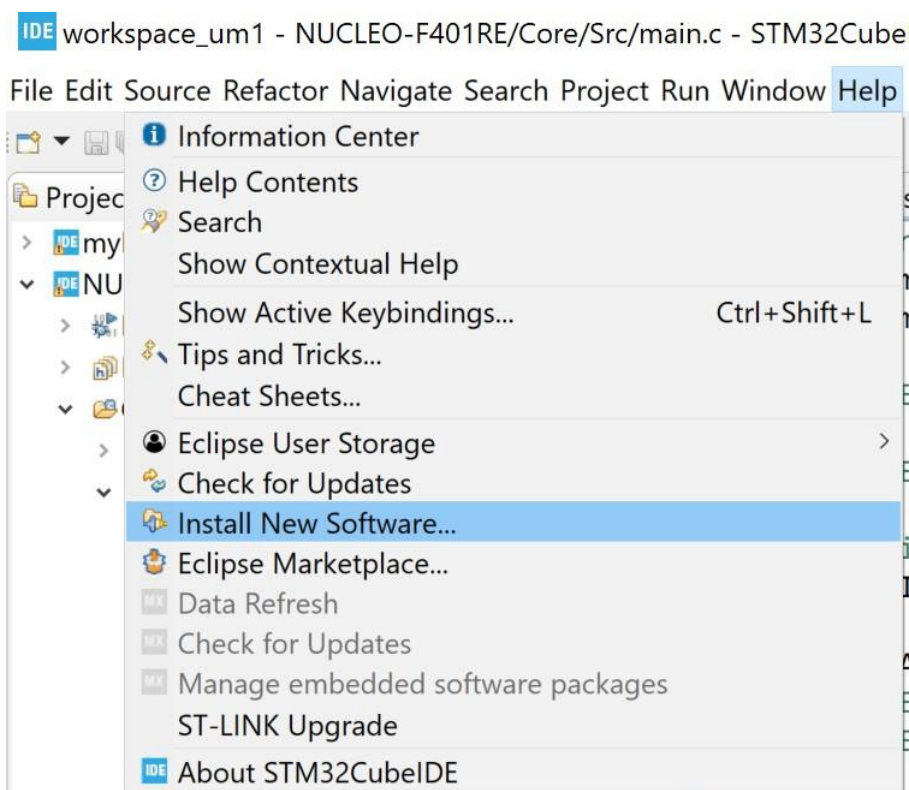
等待安装完成, 重新启动STM32CubeIDE。

10.3 Install using [Install new software...]

安装新软件的另一种方法是使用菜单[Help]>[Install New Software...]

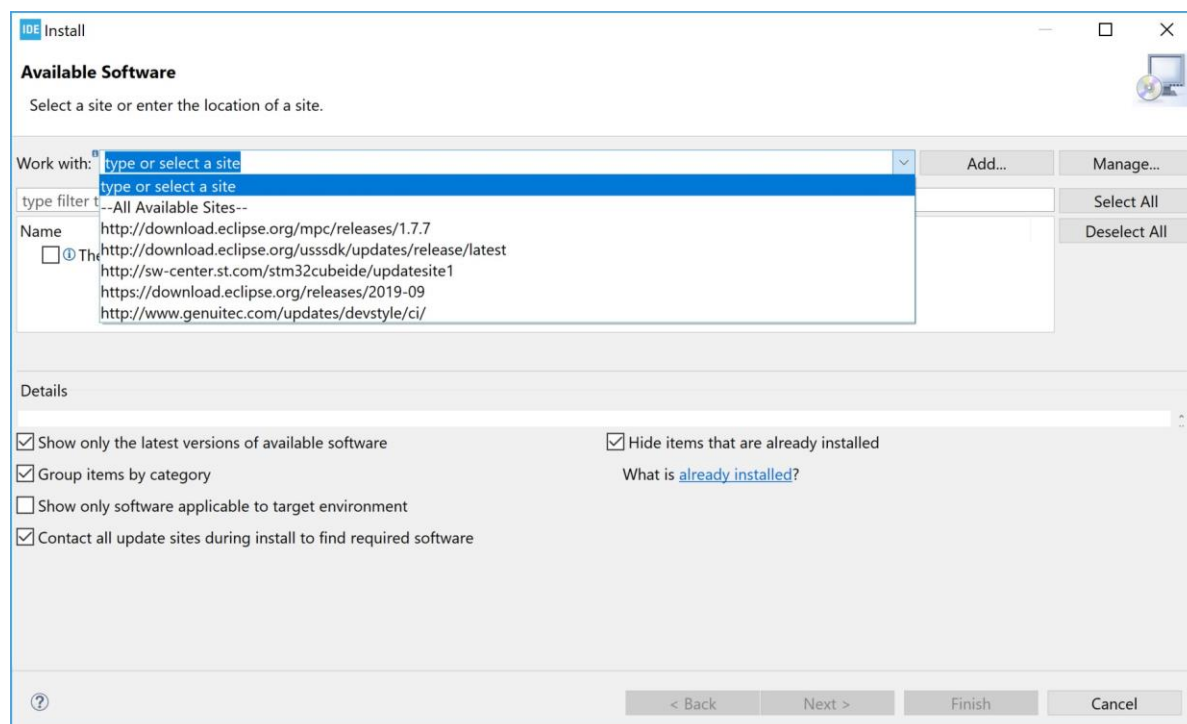
Note: 安装新的工具链时, 建议使用2.11节工具链管理器中描述的工具链管理器。

Figure 256. Install new software menu



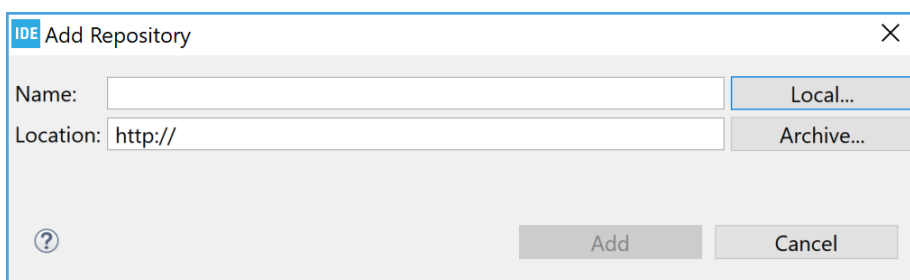
安装对话框打开。输入插件更新站点URL。如果URL未知，请使用[**--All Available Sites--**]。

Figure 257. Install new software



如果没有直接的Internet连接，可以将插件下载到Internet连接的计算机上的存档中，然后手动传输到安装了STM32CubeIDE的计算机上。点击[Add...]按钮添加存档文件，然后选择[Archive and select the downloaded file]。

Figure 258. Install new software from computer



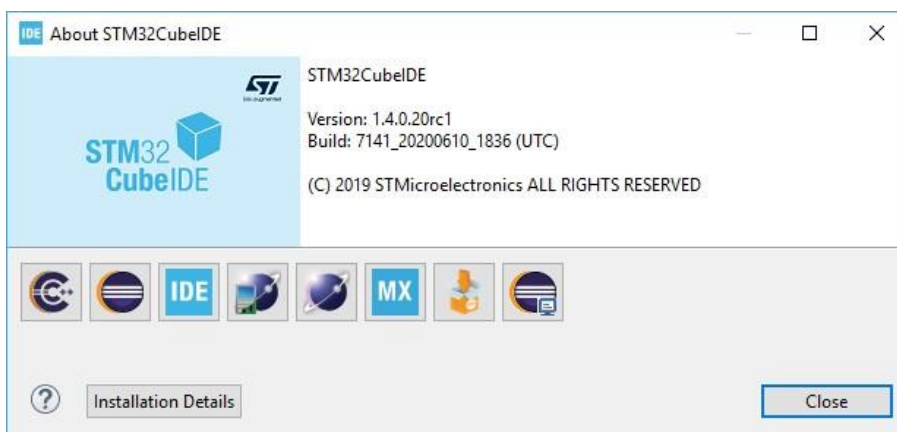
选择适当的插件并安装软件。安装完成后重新启动STM32CubeIDE。

Remember: 并非所有Eclipse®插件都与STM32CubeIDE兼容。

10.4 Uninstalling installed additional Eclipse® plugins

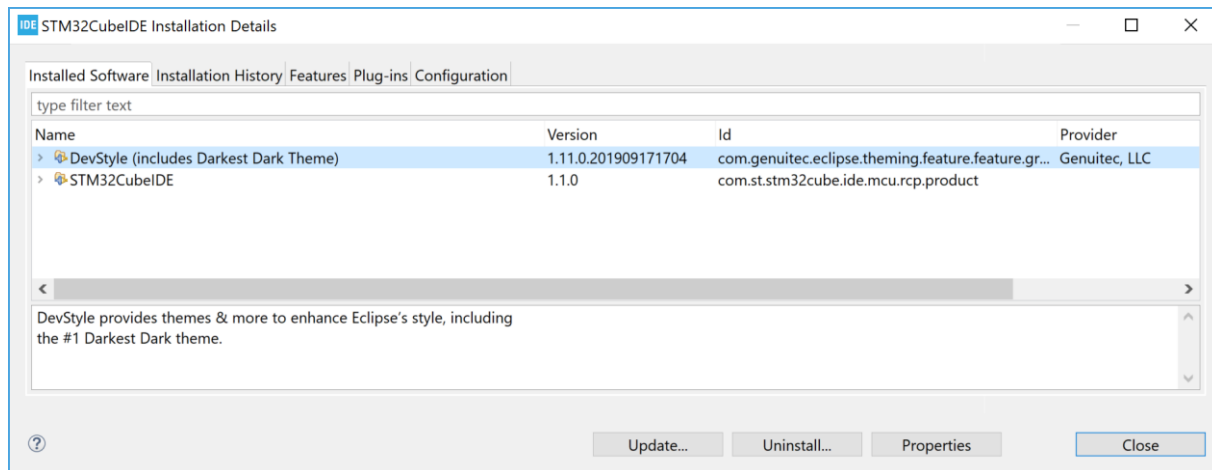
卸载不再需要的插件，选择菜单[Help]>[About STM32CubeIDE]。

Figure 259. About STM32CubeIDE



按[Installation Details]按钮打开STM32CubeIDE安装详细信息对话框。

Figure 260. Installation details

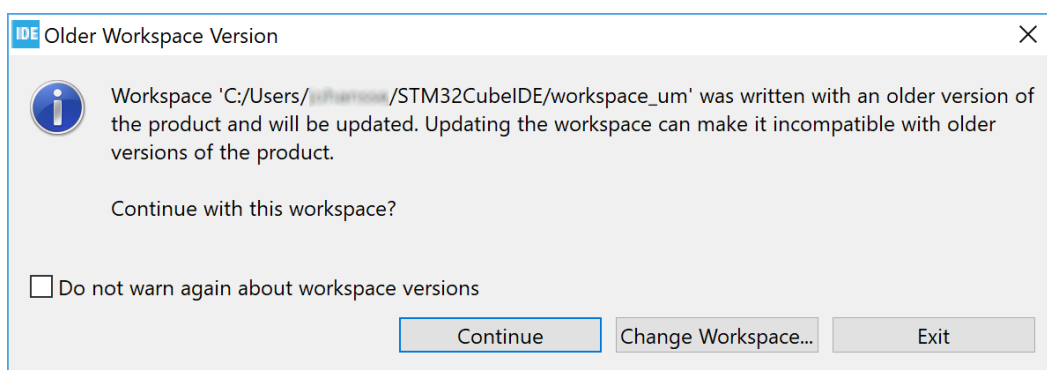


在“Installed Software”页签中选择要卸载的插件，按[uninstall...]. 卸载完成后重新启动STM32CubeIDE。

10.5 Update to new CDT™

当基于新版本的Eclipse®、CDT™或两者安装了新版本的STM32CubeIDE时，建议创建新的工作空间，而不是使用以前的工作空间。当尝试使用新的STM32CubeIDE使用旧的工作空间时，会显示以下警告。

Figure 261. Older workspace version warning



11 References

Table 28. STMicroelectronics reference documents

Reference	Document short name	Description	Document source
[ST-01]	DB3871	STM32CubeIDE data brief	www.st.com
[ST-02]	RN0114	STM32CubeIDE release note	
[ST-03]	UM2553	STM32CubeIDE quick start guide	
[ST-04]	UM2563	STM32CubeIDE installation guide	
[ST-05]	UM2578	Migration guide from TrueSTUDIO® to STM32CubeIDE	
[ST-06]	UM2579	Migration guide from System Workbench to STM32CubeIDE	
[ST-07]	UM2576	STM32CubeIDE ST-LINK GDB server	
[ST-08]	Getting started with projects based on the STM32MP1 Series in STM32CubeIDE ⁽¹⁾		Refer to STM32CubeIDE in the “Tools” section of wiki.st.com/stm32mpu
[ST-09]	AN5361	Getting started with projects based on dual-core STM32H7 microcontrollers in STM32CubeIDE	www.st.com
[ST-10]	AN5394	Getting started with projects based on the STM32L5 Series in STM32CubeIDE	
[ST-11]	AN5564	Getting started with projects based on dual-core STM32WL microcontrollers in STM32CubeIDE	
[ST-12]	AN4296	Use STM32F3/STM32G4 CCM SRAM with IAR Embedded Workbench®, Keil® MDK-ARM, STMicroelectronics STM32CubeIDE and other GNU-based toolchains	
[ST-13]	SLA0048	License agreement applicable to STM32CubeIDE	
[ST-14]	UM1718	STM32CubeMX for STM32 configuration and initialization C code generation	

1. Legacy application note [AN5360](#) remains available on www.st.com.

Table 29. External reference documents

Reference	Description	Document source
[EXT-01]	GNU Assembler	GNU tool suite ⁽¹⁾
[EXT-02]	GNU Compiler Collection	
[EXT-03]	GNU C Library	
[EXT-04]	GNU C Preprocessor	
[EXT-05]	GNU Linker	
[EXT-06]	GNU Binary Utilities	
[EXT-07]	Red Hat Newlib C Library	
[EXT-08]	Red Hat Newlib C Math Library	
[EXT-09]	Newlib nano readme	
[EXT-10]	Debugging with GDB	
[EXT-11]	GDB Quick Reference Card	
[EXT-12]	GNU Tools for STM32 Patch list	Information Center

1. For GNU documentation principles, refer to www.gnu.org.

Revision history

Table 30. Document revision history

Date	Revision	Changes
24-Jul-2020	1	Initial release.
2-Nov-2020	2	Document updated for STM32CubeIDE v1.5.0: - Only one toolchain installed by default - The <i>SFRs</i> view displays Arm® Cortex® core registers node - Debug with OpenOCD supports SWV and live expressions - Added <i>Preferences - Build variables</i> - Added <i>Toolchain Manager</i> - Added <i>RTOS-aware debugging</i> with FreeRTOS™ information - Added <i>General debug and run launch flow</i> - Added <i>Post-build with makefile targets</i>
18-Feb-2021	3	Document updated for STM32CubeIDE v1.6.0: - Added the <i>Azure RTOS ThreadX</i> section into chapter <i>RTOS-aware debugging</i>. Reorganized the <i>FreeRTOS</i> section - Updated the <i>Toolchain Manager</i> section for the support of local toolchains - Updated the <i>Project C/C++ build settings</i> section, MCU toolchain selection moved - Updated <i>Information Center</i> - Updated the entire document for the “SWV packet” terminology - Updated <i>References</i> - Removed <i>Section 4.3.3 SWV Exception Timeline Graph</i>
5-Jul-2021	4	Document updated for STM32CubeIDE v1.7.0: - Added <i>Section 2.7 Thread-safe wizard for empty projects and CDT projects</i> - Added <i>Section 3.8 Import STM32 Cortex-M executable</i> - Added <i>Section 6.3 RTOS-kernel-aware debug</i> - Updated <i>Information Center – Home page</i> - Updated <i>Headless build</i> description - Updated <i>Section 2.5.6 Linker script</i> with new memory map layout figure and additional description - Updated <i>Position-independent code</i> description - Updated debug configuration descriptions for ST-LINK GDB server, OpenOCD and SEGGER in <i>Debug using different GDB servers</i> - Updated <i>FreeRTOS Task List</i> view
17-Nov-2021	5	Document updated for STM32CubeIDE v1.8.0: - Added Section 1.3.3 Videos - Added Section 6.1.10 Azure RTOS TraceX tool - Updated Section 2.2.2 Creating a new STM32 static library project - Removed <i>Section 1.3.3 Technical documentation</i> and <i>Section 1.3.4 Closing the Information Center</i>

Contents

1	Getting started.....	2
1.1	Product information	2
1.1.1	System requirements.....	3
1.1.2	Downloading the latest STM32CubeIDE version.....	3
1.1.3	Installing STM32CubeIDE.....	3
1.1.4	License.....	4
1.1.5	Support	4
1.2	Using STM32CubeIDE.....	4
1.2.1	Basic concept and terminology	4
1.2.2	Starting STM32CubeIDE.....	5
1.2.3	Help system.....	6
1.3	Information Center.....	6
1.3.1	Accessing the <i>Information Center</i>	7
1.3.2	Home page.....	8
1.3.3	Videos	9
1.4	Perspectives, editors and views	9
1.4.1	Perspectives.....	10
1.4.2	Editors.....	14
1.4.3	Views.....	14
1.4.4	<i>Quick Access</i> edit field	16
1.5	Configuration - Preferences	17
1.5.1	Preferences - Editors.....	18
1.5.2	Preferences - Code style formatter.....	19
1.5.3	Preferences - Network proxy settings	21
1.5.4	Preferences - Build variables.....	22
1.6	Workspaces and projects	23
1.7	Managing existing workspaces.....	23
1.7.1	Backup of preferences for a workspace.....	24
1.7.2	Copy preferences between workspaces	24
1.7.3	Keeping track of Java heap space.....	24
1.7.4	Unavailable workspace.....	24

1.8	STM32CubeIDE and Eclipse® basics	25
1.8.1	Keyboard shortcuts	25
1.8.2	Editor zoom in and zoom out.....	27
1.8.3	Quickly find and open a file.....	27
1.8.4	Branch folding.....	28
1.8.5	Block selection mode.....	28
1.8.6	Compare files	31
1.8.7	Local file history.....	33
2	Creating and building C/C++ projects	38
2.1	Introduction to projects.....	38
2.2	Creating a new STM32 project.....	38
2.2.1	Creating a new STM32 executable project.....	38
2.2.2	Creating a new STM32 static library project.....	43
2.3	Configure the project build setting	49
2.3.1	Project build configuration	50
2.3.2	Project C/C++ build settings	55
2.4	Building the project.....	62
2.4.1	Building all projects	63
2.4.2	Build all build configurations.....	63
2.4.3	Headless build	64
2.4.4	Temporary assembly file and preprocessed C code.....	65
2.4.5	Build logging.....	65
2.4.6	Parallel build and build behaviour	65
2.4.7	Post-build with makefile targets	66
2.5	Linking the project.....	66
2.5.1	Run time library	67
2.5.2	Discard unused sections	69
2.5.3	Page size allocation for malloc.....	70
2.5.4	Include additional object files.....	71
2.5.5	Treat linker warnings and errors	71
2.5.6	Linker script.....	73
2.5.7	Modify the linker script.....	80

2.5.8	Include libraries.....	87
2.5.9	Referring to projects.....	89
2.6	I/O redirection.....	90
2.6.1	<code>printf()</code> redirection.....	90
2.7	Thread-safe wizard for empty projects and CDT™ projects.....	92
2.8	Position-independent code.....	99
2.8.1	Adding the <code>-fPIE</code> option.....	100
2.8.2	Run time library	101
2.8.3	Stack pointer configuration.....	101
2.8.4	Interrupt vector table.....	101
2.8.5	Global offset table	101
2.8.6	Interrupt vector table and symbols.....	102
2.8.7	Debugging position-independent code.....	102
2.9	Exporting projects.....	104
2.10	Importing existing projects	106
2.10.1	Importing an STM32CubeIDE project	106
2.10.2	Importing System Workbench and TrueSTUDIO® projects.....	108
2.10.3	Importing using project files association	111
2.10.4	Prevent “GCC not found in path” error	111
2.11	<i>Toolchain Manager</i>	111
2.11.1	Install new toolchain.....	114
2.11.2	Manage default toolchain	117
2.11.3	Uninstall toolchain	118
2.11.4	Using local toolchain	119
2.11.5	Network error	123
3	Debugging	124
3.1	Introduction to debugging.....	124
3.1.1	General debug and run launch flow.....	125
3.2	Debug configurations	126
3.2.1	Debug configuration	127
3.2.2	<i>Main</i> tab	127
3.2.3	<i>Debugger</i> tab	128

3.2.4	Startup tab	130
3.3	Manage debug configurations.....	133
3.4	Debug using different GDB servers	134
3.4.1	Debug using the ST-LINK GDB server.....	134
3.4.2	Debug using OpenOCD and ST-LINK.....	137
3.4.3	Debug using SEGGER J-Link	139
3.5	Start and stop debugging	141
3.5.1	Start debugging	141
3.5.2	Debug perspective and views.....	143
3.5.3	Main controls for debugging	144
3.5.4	Run, start and stop a program.....	146
3.5.5	Set breakpoints	146
3.5.6	Attach to running target.....	148
3.5.7	Restart or terminate debugging.....	150
3.6	Debug features.....	153
3.6.1	<i>Live Expressions</i> view	153
3.6.2	Shared ST-LINK	153
3.6.3	Debug multiple boards.....	154
3.6.4	STM32H7 multicore debugging.....	154
3.6.5	STM32MP1 debugging.....	154
3.6.6	STM32L5 debugging.....	154
3.7	Run configurations	155
3.8	Import STM32 Cortex [®] -M executable.....	156
4	Debug with Serial Wire Viewer tracing (SWV).....	161
4.1	Introduction to SWV and ITM.....	161
4.2	SWV debugging	161
4.2.1	SWV debug configuration.....	161
4.2.2	SWV settings configuration.....	164
4.2.3	SWV tracing	166
4.3	SWV views.....	167
4.3.1	<i>SWV Trace Log</i>	168
4.3.2	<i>SWV Exception Trace Log</i>	168

4.3.3	SWV Data Trace.....	170
4.3.4	SWV Data Trace Timeline Graph.....	172
4.3.5	SWV ITM Data Console and <code>printf</code> redirection	172
4.3.6	SWV Statistical Profiling.....	174
4.4	Change the SWV trace buffer size.....	176
4.5	Common SWV problems	177
5	Special Function Registers (SFRs).....	179
5.1	Introduction to SFRs.....	179
5.2	Using the <i>SFRs</i> view	179
5.3	Updating CMSIS-SVD settings.....	181
6	RTOS-aware debugging	183
6.1	Azure® RTOS ThreadX.....	183
6.1.1	Finding the views.....	183
6.1.2	<i>ThreadX Thread List</i> view	183
6.1.3	<i>ThreadX Semaphores</i> view.....	185
6.1.4	<i>ThreadX Mutexes</i> view	186
6.1.5	<i>ThreadX Message Queues</i> view.....	187
6.1.6	<i>ThreadX Event Flags</i> view	187
6.1.7	<i>ThreadX Timers</i> view	188
6.1.8	<i>ThreadX Memory Block Pools</i> view.....	189
6.1.9	<i>ThreadX Memory Byte Pools</i> view.....	189
6.1.10	Azure® RTOS TraceX tool.....	191
6.2	FreeRTOS™	194
6.2.1	Requirements.....	194
6.2.2	Finding the views.....	196
6.2.3	<i>FreeRTOS Task List</i> view	196
6.2.4	<i>FreeRTOS Timers</i> view	198
6.2.5	<i>FreeRTOS Semaphores</i> view.....	199
6.2.6	<i>FreeRTOS Queues</i> view	200
6.3	RTOS-kernel-aware debug	200
7	Fault Analyzer.....	204

7.1	Introduction to the Fault Analyzer	204
7.2	Using the <i>Fault Analyzer</i> view	205
8	<i>Build Analyzer</i>	208
8.1	Introduction to the <i>Build Analyzer</i>	208
8.2	Using the <i>Build Analyzer</i>	208
8.2.1	<i>Memory Regions</i> tab	208
8.2.2	<i>Memory Details</i> tab	209
9	<i>Static Stack Analyzer</i>	216
9.1	Introduction to the <i>Static Stack Analyzer</i>	216
9.2	Using the <i>Static Stack Analyzer</i>	217
9.2.1	Enable stack usage information	218
9.2.2	<i>List</i> tab	219
9.2.3	<i>Call Graph</i> tab	220
9.2.4	Using the filter and search field	221
9.2.5	Copy and paste	223
10	Installing updates and additional Eclipse® plugins	224
10.1	Check for updates	224
10.2	Install from the Eclipse® market place	226
10.3	Install using [Install new software...]	227
10.4	Uninstalling installed additional Eclipse® plugins	229
10.5	Update to new CDT™	230
11	References	231
	Revision history	233
	Contents	234
	List of tables	240
	List of figures	241

List of tables

Table 1.	Examples of toolchain build variables	22
Table 2.	Key shortcut examples	26
Table 3.	Memory map layout	74
Table 4.	<i>Toolchain Manager</i> column details.....	113
Table 5.	<i>Toolchain Manager</i> button information.....	113
Table 6.	SWV Trace Log columns details	168
Table 7.	SWV Exception Trace Log – Data columns details	169
Table 8.	SWV Exception Trace Log – Statistics columns details.....	170
Table 9.	SWV Data Trace columns details.....	172
Table 10.	SWV Statistical Profiling columns details	176
Table 11.	ThreadX Thread List details.....	185
Table 12.	ThreadX Semaphores details	186
Table 13.	ThreadX Mutexes details	186
Table 14.	ThreadX Message Queues details	187
Table 15.	ThreadX Event Flags details.....	188
Table 16.	ThreadX Timers details	188
Table 17.	ThreadX Memory Block Pools details	189
Table 18.	ThreadX Memory Byte Pools details	190
Table 19.	FreeRTOS Task List details	198
Table 20.	FreeRTOS Timers details.....	199
Table 21.	FreeRTOS Semaphores details	200
Table 22.	FreeRTOS Queues details.....	200
Table 23.	Memory Regions tab information	209
Table 24.	Memory Regions usage color.....	209
Table 25.	Memory Details tab information.....	210
Table 26.	Static Stack Analyzer List tab details.....	219
Table 27.	Static Stack Analyzer Call Graph tab details.....	220
Table 28.	STMicroelectronics reference documents.....	231
Table 29.	External reference documents.....	232
Table 30.	Document revision history.....	233

List of figures

Figure 1.	STM32CubeIDE key features	3
Figure 2.	STM32CubeIDE window	5
Figure 3.	STM32CubeIDE Launcher – Workspace selection.....	6
Figure 4.	Help menu	6
Figure 5.	Help - Information Center menu	7
Figure 6.	Information Center – Home page	8
Figure 7.	Information Center – Video browser page.....	9
Figure 8.	Reset perspective	10
Figure 9.	Toolbar buttons for switching perspective	10
Figure 10.	C/C++ perspective	11
Figure 11.	Debug perspective	11
Figure 12.	Device Configuration Tool perspective.....	12
Figure 13.	Remote System Explorer perspective.....	13
Figure 14.	New connection	13
Figure 15.	[Show View] menu.....	14
Figure 16.	Show View dialog	15
Figure 17.	Quick access.....	16
Figure 18.	Preferences	17
Figure 19.	Preferences - Text Editors	18
Figure 20.	Preferences - Formatter.....	19
Figure 21.	Preferences - Code style edit.....	20
Figure 22.	Preferences - Network Connections	21
Figure 23.	Preferences – Build variables	22
Figure 24.	Pre-build step using build variables.....	22
Figure 25.	Preferences - Workspaces.....	23
Figure 26.	Display of Java heap space status.....	24
Figure 27.	Workspace unavailable	25
Figure 28.	Shortcut keys	25
Figure 29.	Shortcut preferences.....	26
Figure 30.	Editor with text zoomed in	27
Figure 31.	Editor folding	28
Figure 32.	Editor block selection	29
Figure 33.	Editor text block addition.....	29
Figure 34.	Editor column block selection	30
Figure 35.	Editor column block paste	31
Figure 36.	Editor - Compare files	32
Figure 37.	Editor - File differences	32
Figure 38.	Local history	33
Figure 39.	Show local history.....	34
Figure 40.	File history	35
Figure 41.	Compare current history with local history.....	36
Figure 42.	Compare local file differences.....	37
Figure 43.	STM32 target selection.....	39
Figure 44.	STM32 board selection.....	39
Figure 45.	Project setup.....	40
Figure 46.	Firmware library package setup.....	41
Figure 47.	Initialization of all peripherals.....	41
Figure 48.	STM32CubeMX perspective opening	42
Figure 49.	Project creation started.....	42
Figure 50.	STM32CubeMX	43
Figure 51.	New C/C++ project	44
Figure 52.	Project type.....	45

Figure 53.	Project configuration selection.....	45
Figure 54.	Project default target selector	46
Figure 55.	Project MCU/MPU selector	47
Figure 56.	Project target selection	48
Figure 57.	Project target selection (advanced)	48
Figure 58.	Project target change.....	49
Figure 59.	Set the active build configuration using the toolbar.....	50
Figure 60.	Set active build configuration using right-click.....	51
Figure 61.	Set active build configuration using menu.....	52
Figure 62.	<i>Manage Configurations</i> dialog	52
Figure 63.	Create a new build configuration	53
Figure 64.	Updated <i>Manage Configurations</i> dialog	53
Figure 65.	Configuration deletion dialog.....	54
Figure 66.	Configuration renaming dialog.....	54
Figure 67.	Properties tabs.....	55
Figure 68.	Properties configurations	55
Figure 69.	Properties toolchain version.....	56
Figure 70.	Properties toolchain selection	56
Figure 71.	Properties tool MCU settings	57
Figure 72.	Properties tool MCU post-build settings	58
Figure 73.	Properties tool GCC assembler settings	59
Figure 74.	Properties tool GCC compiler settings	60
Figure 75.	Properties tool GCC linker settings	61
Figure 76.	Properties build steps settings.....	62
Figure 77.	Project build toolbar.....	62
Figure 78.	Project build console.....	63
Figure 79.	Project build all.....	63
Figure 80.	Project build-all configurations	64
Figure 81.	Headless build.....	65
Figure 82.	Parallel build.....	66
Figure 83.	Linker documentation	67
Figure 84.	Linker run time library.....	68
Figure 85.	Linker newlib-nano library and floating-point numbers.....	69
Figure 86.	Linker discard unused sections.....	70
Figure 87.	Linker include additional object files.....	71
Figure 88.	Linker fatal warnings	73
Figure 89.	Linker memory output	85
Figure 90.	Linker memory output specified order	85
Figure 91.	Linker memory displaying file <i>readme</i>	86
Figure 92.	Include a library.....	88
Figure 93.	Add library header files to the include paths.....	89
Figure 94.	Set project references	90
Figure 95.	Select a wizard.....	93
Figure 96.	<i>Thread-Safe Solution</i> wizard.....	94
Figure 97.	Thread-safe source folder location	95
Figure 98.	Thread-safe strategy selection	97
Figure 99.	Thread-safe properties.....	98
Figure 100.	Thread-safe files	98
Figure 101.	Thread-safe error dialog	99
Figure 102.	Position independent code, <i>-fPIE</i>	100
Figure 103.	Debugging position independent code.....	103
Figure 104.	Export project	104
Figure 105.	Export dialog	105
Figure 106.	Export archive	106

Figure 107.	Import project	107
Figure 108.	Import dialog.....	107
Figure 109.	Import projects	108
Figure 110.	Import System Workbench projects (1 of 3).....	109
Figure 111.	Import System Workbench projects (2 of 3).....	110
Figure 112.	Import System Workbench projects (3 of 3).....	110
Figure 113.	Import using project files association	111
Figure 114.	Open <i>Toolchain Manager</i>	112
Figure 115.	<i>Toolchain Manager</i>	112
Figure 116.	Install toolchain	114
Figure 117.	Check items to install	114
Figure 118.	Review items to install.....	115
Figure 119.	Review and accept licenses	115
Figure 120.	Security warning	116
Figure 121.	Restart to apply software update.....	116
Figure 122.	Toolchain installed	116
Figure 123.	Default toolchain	117
Figure 124.	Default toolchain updated.....	117
Figure 125.	Uninstall toolchain.....	118
Figure 126.	Uninstall details	118
Figure 127.	Software updates.....	119
Figure 128.	Toolchain uninstalled	119
Figure 129.	Add local toolchain	120
Figure 130.	Specify local toolchain location.....	121
Figure 131.	Specify local toolchain prefix.....	121
Figure 132.	Local toolchain added	122
Figure 133.	Edit local toolchain.....	122
Figure 134.	Toolchain network error.....	123
Figure 135.	General debug and run launch flowchart.....	125
Figure 136.	Debug as STM32 MCU.....	126
Figure 137.	Debug as STM32 MCU menu.....	127
Figure 138.	Debug configuration main tab	128
Figure 139.	Debug configuration debugger tab	129
Figure 140.	GDB server command line dialog.....	130
Figure 141.	Debug configuration startup tab	131
Figure 142.	Add/Edit item.....	132
Figure 143.	Manage debug configurations.....	133
Figure 144.	Manage debug configurations toolbar	133
Figure 145.	ST-LINK GDB server debugger tab.....	135
Figure 146.	OpenOCD debugger tab.....	138
Figure 147.	SEGGER debugger tab	140
Figure 148.	Debug configurations.....	142
Figure 149.	Confirm perspective switch.....	143
Figure 150.	Debug perspective	143
Figure 151.	[Run] menu.....	145
Figure 152.	<i>Debug</i> toolbar	145
Figure 153.	Debug breakpoint.....	147
Figure 154.	Breakpoint properties	147
Figure 155.	Conditional breakpoint.....	148
Figure 156.	Startup tab attach.....	149
Figure 157.	Reset the chip toolbar	150
Figure 158.	Restart configurations selection.....	150
Figure 159.	Restart configurations dialog.....	151
Figure 160.	Restart configurations dialog with additional command	152

Figure 161.	Select restart configuration.....	152
Figure 162.	Live Expressions.....	153
Figure 163.	Live expressions number format.....	153
Figure 164.	Run configurations startup tab.....	155
Figure 165.	Cortex®-M executable import dialog.....	156
Figure 166.	STM32 Cortex®-M executable dialog.....	157
Figure 167.	STM32 Cortex®-M executable MCU/MPU selection.....	157
Figure 168.	STM32 Cortex®-M CPU and core.....	158
Figure 169.	Cortex®-M debug configuration for imported project.....	159
Figure 170.	Project explorer view with imported project.....	160
Figure 171.	SWV core clock.....	162
Figure 172.	SWV debug configuration.....	162
Figure 173.	SWV show view.....	163
Figure 174.	SWV Trace log view.....	164
Figure 175.	SWV [Configure Trace] toolbar button.....	164
Figure 176.	SWV settings dialog.....	164
Figure 177.	SWV [Start/Stop Trace] toolbar button.....	166
Figure 178.	SWV Trace Log PC sampling.....	166
Figure 179.	[Remove all collected SWV data] toolbar button.....	167
Figure 180.	SWV views selectable from the menu.....	167
Figure 181.	SWV views common toolbar.....	167
Figure 182.	SWV graph views extra toolbar.....	168
Figure 183.	SWV Trace Log PC sampling and exceptions.....	168
Figure 184.	SWV Exception Trace Log – Data tab.....	169
Figure 185.	SWV Exception Trace Log – Statistics tab.....	169
Figure 186.	SWV Data Trace configuration.....	171
Figure 187.	SWV Data Trace.....	171
Figure 188.	SWV Data Trace Timeline Graph.....	172
Figure 189.	SWV settings.....	173
Figure 190.	SWV ITM Data Console.....	174
Figure 191.	SWV ITM port configuration.....	174
Figure 192.	SWV PC sampling enable.....	175
Figure 193.	SWV Statistical Profiling.....	176
Figure 194.	SWV Preferences.....	177
Figure 195.	Open the SFRs view using the [Quick Access] field.....	179
Figure 196.	SFRs view.....	180
Figure 197.	SFRs view toolbar buttons.....	181
Figure 198.	SFRs CMSIS-SVD Settings.....	181
Figure 199.	ThreadX views selectable from the menu.....	183
Figure 200.	ThreadX Thread List view (default).....	184
Figure 201.	ThreadX Thread List view (Stack Usage enabled).....	184
Figure 202.	ThreadX Semaphores view.....	186
Figure 203.	ThreadX Mutexes view.....	186
Figure 204.	ThreadX Message Queues view.....	187
Figure 205.	ThreadX Event Flags view.....	188
Figure 206.	ThreadX Timers view.....	188
Figure 207.	ThreadX Memory Block Pools view.....	189
Figure 208.	ThreadX Memory Byte Pools view.....	190
Figure 209.	File associations.....	191
Figure 210.	RAM buffer export (1 of 2).....	192
Figure 211.	RAM buffer export (2 of 2).....	192
Figure 212.	Existing trace overwrite.....	193
Figure 213.	TraceX analysis.....	193
Figure 214.	FreeRTOS™-related views selectable from the menu.....	196

Figure 215.	FreeRTOS Task List (default).....	196
Figure 216.	FreeRTOS™ Toggle Stack Checking	197
Figure 217.	FreeRTOS Task List (Min Free Stack enabled).....	197
Figure 218.	FreeRTOS Task List with ConfigRECORD_STACK_HIGH_ADDRESS enabled	197
Figure 219.	FreeRTOS Timers	198
Figure 220.	FreeRTOS Semaphores	199
Figure 221.	FreeRTOS Queues	200
Figure 222.	RTOS-kernel-aware debug	201
Figure 223.	RTOS-kernel-awareness debug configuration.....	202
Figure 224.	ThreadX-kernel-awareness debug configuration	202
Figure 225.	ThreadX port configuration	203
Figure 226.	FreeRTOS™ port configuration	203
Figure 227.	Open the Fault Analyzer view	204
Figure 228.	Fault Analyzer view	206
Figure 229.	Fault Analyzer toolbar	207
Figure 230.	Fault analyzer open editor on fault	207
Figure 231.	Fault Analyzer open disassembly on fault.....	207
Figure 232.	Build analyzer.....	208
Figure 233.	Memory Regions tab	209
Figure 234.	Memory Details tab.....	210
Figure 235.	Memory Details sorted by size.....	212
Figure 236.	Memory Details search and filter	212
Figure 237.	Sum of sizes.....	213
Figure 238.	Show byte count	213
Figure 239.	Show hex count.....	214
Figure 240.	Copy and paste.....	214
Figure 241.	Static Stack Analyzer List tab	216
Figure 242.	Static Stack Analyzer Call Graph tab.....	217
Figure 243.	Open the Static Stack Analyzer view.....	217
Figure 244.	Enable generate per function stack usage information.....	218
Figure 245.	Static Stack Analyzer List tab	219
Figure 246.	Static Stack Analyzer Call Graph tab.....	220
Figure 247.	Function symbols in Static Stack Analyzer	221
Figure 248.	Static Stack Analyzer List tab using search.....	222
Figure 249.	Static Stack Analyzer Call Graph using search.....	222
Figure 250.	Copy and paste.....	223
Figure 251.	STM32CubeIDE available updates	224
Figure 252.	STM32CubeIDE update details	225
Figure 253.	STM32CubeIDE update review licenses.....	225
Figure 254.	Eclipse Marketplace menu	226
Figure 255.	Eclipse marketplace	227
Figure 256.	Install new software menu	228
Figure 257.	Install new software.....	228
Figure 258.	Install new software from computer	229
Figure 259.	About STM32CubeIDE.....	229
Figure 260.	Installation details.....	230
Figure 261.	Older workspace version warning	230

IMPORTANT NOTICE – PLEASE READ CAREFULLY

STMicroelectronics NV and its subsidiaries ("ST") reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice. Purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST's terms and conditions of sale in place at the time of order acknowledgement.

Purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of Purchasers' products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, please refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2021 STMicroelectronics – All rights reserved